Windows BAT 批处理完全教程

从零基础入门到高级精通

覆盖 cmd.exe 全部核心特性 · 系统管理实战 · 黑科技应用

2026

目 录

第一部分 基础入门篇

第1章 认识 cmd.exe 与 BAT 批处理

第2章 第一个 BAT 程序与基础输出

第3章 批处理的执行机制

第二部分 核心语法篇

第4章 变量系统全解析

第5章 流程控制语句

第6章 重定向、管道与输入输出

第三部分 文件与系统操作篇

第7章 文件操作实战

第8章 目录与路径操作

第9章 系统管理命令

第10章 网络操作命令

第四部分 高级技巧与实战篇

第11章 高级语法与黑科技

第12章 实战项目一：系统维护工具集

第13章 实战项目二：网络运维工具集

第14章 实战项目三：进阶黑科技工具集

第五部分 附录与参考篇

附录A Windows 批处理核心命令速查表

附录B cmd.exe 环境变量大全

附录C 批处理常见问题与疑难解答

附录D BAT 开发工具与资源

附录E 示例代码全集索引

第一部分 基础入门篇

第1章 认识 cmd.exe 与 BAT 批处理

1.1 cmd.exe 是什么

cmd.exe 是 Windows 操作系统自带的命令行解释器，它的作用是接收用户输入的命令并执行。从 MS-DOS 时代开始，命令行就是 Windows 系统管理的核心工具之一，而 BAT 批处理文件则是命令行的脚本化形式——把一系列命令写入一个纯文本文件，让 cmd.exe 自动按顺序执行。

cmd.exe 的历史可以追溯到 1981 年的 MS-DOS 1.0。三十多年来，它始终保持着向后兼容的设计原则，这意味着今天的 BAT 脚本仍然可以在最新的 Windows 11 上运行。这既是 BAT 的优势（稳定性），也带来了一些限制（语法古老、功能有限）。

> **NOTE**
>
> Windows NT 内核（Windows NT/2000/XP/Vista/7/8/10/11）上的 cmd.exe 虽然外观和 DOS 时代相似，但内部已经完全重写。它是一个 32 位/64 位的 Windows 原生程序，能够访问 Win32 API，支持长文件名、环境变量延迟扩展等高级特性。

1.2 BAT 批处理的本质

BAT 文件的本质是一个 **纯文本文件**，后缀名为 `.bat` 或 `.cmd`。当你双击它时，Windows 会自动用 cmd.exe 打开并逐行执行。

让我们创建第一个 BAT 文件来感受一下。打开记事本（Notepad），输入以下内容，然后保存为 `hello.bat`：

```
@echo off
echo Hello, BAT World!
pause
```

保存时注意：在"另存为"对话框中，将"保存类型"改为"所有文件 (*.*)"，编码选择 **ANSI** 或 **UTF-8**（Windows 10 推荐）。然后双击 hello.bat，你会看到一个黑色命令行窗口弹出，显示 "Hello, BAT World!"，按任意键后窗口关闭。

TIP

.bat 和 .cmd 的区别：.bat 是 DOS 时代就有的后缀，.cmd 从 Windows NT 开始引入。两者功能几乎完全相同，唯一细微的差别在于 .cmd 执行出错时 %errorlevel% 的行为略有不同。实际使用中可以任选其一，本书统一使用 .bat 后缀。

1.3 开发环境搭建

写 BAT 脚本不需要任何 IDE，但一个好的编辑器能让你的开发效率翻倍。以下是推荐组合：

编辑器推荐

- **VS Code**（强烈推荐）：免费开源，通过插件支持 BAT 语法高亮、代码提示、片段
- **Notepad++**：经典轻量编辑器，自带 BAT 语法高亮
- **Sublime Text**：优雅的商业编辑器，安装 Batch Script 插件后支持完善
- **记事本**（Notepad）：系统自带，仅适合临时编写，不支持语法高亮

VS Code 推荐插件

- **Batch Script Support**（作者：felixfbecker）：完整的 BAT 语法高亮、片段、格式化
- **vscode-batch-runner**：在 VS Code 内直接运行 BAT 脚本并查看输出

cmd 的启动方式

- **Win + R**，输入 `cmd`，回车
- 在任意文件夹地址栏输入 `cmd`，回车（直接进入当前目录）
- Shift + 右键文件夹空白处 → 在此处打开命令窗口
- 开始菜单搜索 "cmd" 或 "命令提示符"
- 管理员身份：右键 → 以管理员身份运行

1.4 cmd vs PowerShell

很多读者会问：Windows 都有 PowerShell 了，为什么还要学 BAT？这是一个好问题。两者各有分工：

特性	cmd.exe (BAT)	PowerShell
历史	1981年起，DOS时代	2006年首次发布
脚本后缀	.bat / .cmd	.ps1
默认启用	所有 Windows 版本	Windows 7+
学习曲线	平缓，语法简洁	较陡，语法复杂
对象模型	文本流处理	.NET 对象管道
跨平台	仅 Windows	PowerShell Core 跨平台
系统兼容性	100% 兼容所有 Windows	部分旧系统不支持
适合场景	快速脚本、旧系统运维	复杂自动化、.NET 操作

实际上，PowerShell 和 BAT 并不是竞争关系，而是互补关系。BAT 仍然是最广泛使用的 Windows 脚本格式——因为它在所有 Windows 版本上都能直接运行，无需任何额外配置。在很多企业运维场景中，BAT 仍是首选的自动化工具。

> **NOTE**
>
> 本书专注于 cmd.exe / BAT 批处理。但在第 11 章"混合编程"中，我们会介绍 BAT 如何调用 PowerShell 来完成BAT 自身不擅长的任务（如加密、HTTP 请求、复杂文本处理）。

1.5 本书代码规范

为方便阅读，本书所有代码示例遵循以下规范：

- 所有代码开头都有 `@echo off`，关闭命令回显
- 变量名使用小写下划线风格，如 `%file_count%`
- 注释使用 `rem` 或 `::`，功能关键处必有注释
- 文件路径统一用反斜杠 `\`，含空格的路径加双引号
- 每个可运行的脚本都标注了预期输出
- 涉及编码时默认使用 UTF-8，配合 `chcp 65001`

— 第1章 完 —

进阶练习

练习1：确认系统信息。打开 cmd.exe 执行以下命令：

```
@echo off
ver
echo %computername%
echo %os%
echo %systemroot%
systeminfo | findstr /c:"OS"
```

练习2：创建第一个 BAT。记事本写三行，保存为 hello.bat 双击运行：

```
@echo off
echo 你好，这是我的第一个 BAT!
pause
```

练习3：探索 cmd.exe 帮助系统：

```
help
help | more
help set
help if
```

疑难解答

Q1: 双击 .bat 后闪退?

A: 脚本最后加 pause，或从 cmd 窗口手动运行。

Q2: 中文变乱码?

A: 记事本默认 ANSI/GBK 编码正确。VS Code 等工具保存时选 GBK，或脚本开头加 chcp 65001 并保存为 UTF-8 BOM。

Q3: 如何开机自启动?

A: 放快捷方式到 shell:startup 目录（Win+R 输入回车），或用任务计划程序。

进阶主题1：认识 cmd.exe 的所有启动参数

cmd.exe 有很多启动参数可以改变它的行为。用 cmd /? 查看完整列表，下面演示最常用的几个。/d 参数特别有用——它会跳过注册表中 AutoRun 里设置的命令，避免某些恶意脚本自动运行。

```
@echo off
echo === cmd.exe 启动参数全解 ===
echo /c  "cmd /c dir"  执行 dir 后立即退出
echo /k  "cmd /k dir"  执行 dir 后保持窗口
echo /q  关闭回显（等同 @echo off）
echo /s  忽略字符串里的引号
echo /u  输出用 Unicode (UCS-2)
echo /t:fg 设置前景/背景颜色
echo /d  不执行 AutoRun 注册表命令
echo /a  输出用 ANSI 编码
echo /f:on  文件和目录名自动补全 (^= 下翻)
echo /f:off 关闭自动补全
echo.
echo 示例: cmd /k "cd /d C:\ &amp;amp;&amp;amp; dir /w"
echo 示例: cmd /t:0A /k ver
echo 示例: cmd /d /k reg query HKLM\SOFTWARE\Microsoft\Command Processor
echo.
echo === 当前 cmd 版本 ===
cmd /? | findstr /c:"版本" /c:"Version"
pause
```

进阶主题2：配置 cmd 提示符 PGLG

prompt 命令可以自定义命令行提示符。$P 显示路径、$G 是大于号、$L 是小于号，组合起来就是经典的 C:\Users\YOU> 效果。在 Windows 10+ 还可以用 ANSI 转义码给提示符上色。

```
@echo off
echo === 配置 cmd 提示符 ===
echo 当前提示符:
prompt
```

```
echo.
echo 常用提示符符号:
echo   $P  当前驱动器和路径
echo   $G  大于号 &gt;
echo   $L  小于号 &lt;
echo   $B  管道符 ^|
echo   $Q  等号 =
echo   $T  当前时间
echo   $D  当前日期
echo   $V  Windows 版本号
echo   $N  当前驱动器
echo   $E  ESC 字符 (ANSI 转义)
echo   $H  退格（删除前一字符）
echo   $_  回车换行
echo.
echo 设置经典提示符 $P$G:
prompt $P$G
echo 试试吧！现在提示符会显示路径。
echo.
echo 设置带时间的提示符 $T$ $P$G:
prompt $T$ $P$G
echo 设置漂亮提示符 $E[1;32m$P$G$E[0m (ANSI):
prompt $E[1;32m$P$G$E[0m
pause
```

进阶主题3：设置命令历史和补全

cmd.exe 默认只保留 50 条历史命令，在注册表里可以调到 200。doskey 宏系统可以创建命令别名，大大提高效率。把宏保存到文件后，每次启动可以自动加载。

```
@echo off
echo === 命令历史和补全配置 ===
echo 当前 cmd 设置:
reg query "HKCU\Console" 2&gt;nul | findstr /c:"LineSize" /c:"HistoryBufferSize"
echo.
```

```
echo 历史缓冲区大小（默认 50，建议 200）：
reg add "HKCU\Console" /v HistoryBufferSize /t REG_DWORD /d 200 /f &gt;nul
echo 历史缓冲区数量:
reg add "HKCU\Console" /v NumberOfHistoryBuffers /t REG_DWORD /d 4 /f &gt;nul
echo 丢弃旧历史不警告:
reg add "HKCU\Console" /v HistoryNoDup /t REG_DWORD /d 1 /f &gt;nul
echo.
echo === doskey 宏 ===
doskey ls=dir /b $*
doskey ..=cd ..
doskey ...=cd ..\..
doskey c=cls
doskey h=doskey /history
echo 已定义别名: ls, .., ..., c, h
echo 试试输入 ls
echo.
echo 保存宏到文件:
doskey /macrofile=macros.txt &gt;nul 2&gt;&amp;1
if exist macros.txt echo 宏已保存到 macros.txt
echo 下次启动用: doskey /macrofile=macros.txt
pause
```

表 1-1 常用 cmd.exe 命令一览

命令	说明	常用示例
ver	显示 Windows 版本号	ver
cls	清屏	cls
date	显示/设置日期	date /t
time	显示/设置时间	time /t
echo	输出文本/变量	echo Hello %%username%%
set	查看/设置环境变量	set PATH
cd	改变当前目录	cd /d D:\Work

dir	列出目录内容	dir /b /s *.txt
md / mkdir	创建目录	mkdir new_folder
rd / rmdir	删除空目录	rd empty_dir
copy	复制文件	copy a.txt b.txt
xcopy	增强版复制（支持目录树）	xcopy /e /i src dst
move	移动/重命名	move old.txt new.txt
del	删除文件	del /q *.tmp
type	查看文件内容	type config.ini
ren / rename	重命名	ren old_name new_name
attrib	修改文件属性（只读/隐藏）	attrib +h secret.txt
find / findstr	在文本中查找	findstr "error" log.txt
sort	排序文本行	dir /b | sort
more	分屏显示	help | more
prompt	自定义提示符	prompt PG
title	设置窗口标题	title 我的BAT
color	设置前景/背景色	color 0A
shutdown	关机/重启/注销	shutdown /r /t 0
tasklist	列出运行中的进程	tasklist /fi "memusage gt 100000"
taskkill	终止进程	taskkill /f /im notepad.exe
wmic	管理 Windows 组件	wmic process list
net	网络/共享管理	net share / net user
ipconfig	网络配置	ipconfig /all
ping	网络连通测试	ping -n 4 www.baidu.com
systeminfo	详细系统信息	systeminfo
chcp	切换代码页	chcp 65001

表 1-2 cmd.exe 快捷键汇总

快捷键	功能	适用场景
Ctrl + C	强制中断当前命令	死循环或长时间运行
Ctrl + V	粘贴（Win10+）	快速输入长路径/命令
Ctrl + Shift + V	粘贴不带格式化	粘贴代码到 BAT
Ctrl + A	选中当前行全部文本	整行复制
Ctrl + F	查找历史命令	Win10+ F7 的替代

Ctrl + M	进入/退出"标记"模式	用鼠标选中复制
Ctrl + + / -	窗口字号放大/缩小	看不清时
Ctrl + Scroll Wheel	窗口字号缩放	快速调字号
Ctrl + Shift + 点击任务栏图标	管理员权限打开 cmd	最快捷的管理员方式
Shift + Tab	补全上一个文件名	反向选择补全
F1	逐字符重输上一条命令	自动补单字符
F2	复制上条命令到指定字符	输入字符后复制到该处
F3	完整重输上一条命令	最常用的历史回滚
F4	删除到指定字符	清屏式重打
F5	循环浏览历史命令（前进）	浏览下一条
F6	插入 EOF (Z) 字符	重定向文件结束符
F7	命令历史列表弹窗	可视化选择历史命令
F8	按前缀搜索历史	输入 cmd 后 F8 循环匹配
F9	按编号选历史命令	配合 F7 显示的编号
Tab	自动补全文件/路径	路径输入利器
↑ / ↓	浏览历史命令（上下条）	最常用的历史浏览
← / →	光标左右移动	编辑当前行
Home / End	光标移到行首/行尾	快速定位
Insert	插入/覆盖模式切换	编辑长命令行
Esc	清空当前输入行	打错了重打
Pause / Break	暂停屏幕输出	配合 type 大文件
Alt + Enter	全屏/窗口切换	旧式快捷键

第 1 章小结

本章搭建了 BAT 脚本最基础的舞台——cmd.exe。了解它的启动参数、提示符定制和帮助系统，是后续所有章节的地基。掌握"在 cmd 中用 help 自学习"这个能力，比死记命令更重要。

系统信息命令 ver、systeminfo、wmic 组合起来能拿到几乎一切硬件和 OS 细节；文件操作命令 cd、dir、copy、xcopy 每天都要用。学会用管道把多个命令串起来（dir /b | findstr "txt"），会立刻感受到 BAT 的威力。

快捷键表里 F1 到 F9 的历史命令功能、Tab 自动补全、Ctrl+C 强制中断，这三个技巧足以让日常操作效率翻倍。新手最容易忽略的"坑"是中文编码：记事本 ANSI/GBK 最省心，VS Code 保存必须选 GBK 或 UTF-8 BOM。

进阶部分的"系统工具箱"脚本把本章所有核心命令串成了菜单——这就是 BAT 脚本的真实形态：把零散命令打包成可复用工具。建议读者把这个工具箱保存下来，后续每学一个新命令就加一个菜单项。

开机加速项目是典型的"清理类 BAT"：删除临时文件、清空 Prefetch、清理回收站。注意脚本开头 setlocal enabledelayedexpansion 的使用——有计数循环就必须开启它。del /f /q、rd /s /q 的静默参数也要熟记。

下一章进入 echo 和输出控制，把"会写命令"升级到"会做界面"。BAT 没有 GUI，但用 echo 画菜单、用 color 上色、用 ANSI 转义码甚至能做彩色进度条。准备好了吗？

第 1 章大代码块 1：系统工具箱

```
@echo off
setlocal enabledelayedexpansion
echo ======================================================
echo   第1章综合实战：系统工具箱 v1.0
echo ======================================================
echo.
:menu
cls
echo   [1] 显示详细系统信息
echo   [2] 列出所有环境变量
echo   [3] 查找文件（全盘搜索）
echo   [4] 网络连接快速诊断
echo   [5] 查看本机 IP 配置
echo   [6] 任务列表与内存排行
echo   [7] 磁盘空间汇总
echo   [8] 列出已安装程序（简易）
echo   [0] 退出
echo.
```

```
set /p "choice=请选择功能编号: "
if "%choice%"=="" goto menu
if "%choice%"=="0" goto end
if "%choice%"=="1" goto sysinfo
if "%choice%"=="2" goto envlist
if "%choice%"=="3" goto findfile
if "%choice%"=="4" goto netdiag
if "%choice%"=="5" goto ipconfig
if "%choice%"=="6" goto taskmem
if "%choice%"=="7" goto diskspace
if "%choice%"=="8" goto installed
echo 无效选择，请重新输入
timeout /t 2 &gt;nul
goto menu
:sysinfo
echo === 系统详细信息 ===
systeminfo | findstr /i /c:"OS" /c:"系统" /c:"内存" /c:"处理器" /c:"网络"
echo.
pause
goto menu
:envlist
echo === 所有环境变量 ===
set | sort
echo.
pause
goto menu
:findfile
set /p "pattern=请输入文件名(支持通配符): "
echo 正在全盘搜索 %pattern% ...
for %%d in (C D E F G H) do (
    if exist %%d:\ (
        echo 搜索 %%d:\ ...
```

```
        dir /s /b %%d:\%pattern% 2&gt;nul
    )
)
echo 搜索完成
pause
goto menu
:netdiag
echo === 网络诊断 ===
ping -n 2 127.0.0.1 &amp;&amp; echo [OK] 本地回环正常 || echo [FAIL] 网卡异常
ping -n 2 www.baidu.com &amp;&amp; echo [OK] 外网可达 || echo [FAIL] DNS或网关异常
nslookup www.baidu.com 2&gt;nul | findstr "Address"
route print 2&gt;nul | findstr /i "0.0.0.0"
pause
goto menu
:ipconfig
ipconfig /all
echo.
pause
goto menu
:taskmem
echo === 任务列表（按内存排序 Top 10）===
tasklist /fo csv | sort /r /+44 | head -11
echo.
echo === 总进程数 ===
tasklist | find /c ":"
pause
goto menu
:diskspace
echo === 磁盘空间汇总 ===
wmic logicaldisk get size,freespace,caption,volumename 2&gt;nul
echo.
pause
```

```
goto menu
:installed
echo === 已安装程序（注册表读取）===
reg query "HKLM\Software\Microsoft\Windows\CurrentVersion\Uninstall" /s 2&gt;nul ^|
    findstr /i "DisplayName DisplayVersion" | findstr /v "HKEY_"
echo.
pause
goto menu
:end
echo 感谢使用，再见！
timeout /t 2 &gt;nul
exit /b 0
```

第 1 章大代码块 2：批量生成系统报告

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ============================================
echo   第1章进阶项目：批量生成系统报告
echo ============================================
echo.
set "reportdir=%~dp0reports"
if not exist "!reportdir!" mkdir "!reportdir!"
set "timestamp=%date:~0,4%%date:~5,2%%date:~8,2%_%time:~0,2%%time:~3,2%%time:~6,2%"
set "timestamp=!timestamp: =0!"
set "report=!reportdir!\sysreport_!timestamp!.txt"
echo 报告将保存到: !report!
echo.
echo [1/8] 写入文件头...
(
```

```
    echo ============================================
    echo  系统信息报告
    echo  生成时间: !date! !time!
    echo  计算机名: %computername%
    echo  用户名: %username%
    echo ============================================
    echo.
) &gt; "!report!"
echo [2/8] 写入操作系统信息...
echo === 操作系统 === &gt;&gt; "!report!"
systeminfo &gt;&gt; "!report!" 2&gt;&amp;1
echo. &gt;&gt; "!report!"
echo [3/8] 写入环境变量...
echo === 环境变量 === &gt;&gt; "!report!"
set | sort &gt;&gt; "!report!"
echo. &gt;&gt; "!report!"
echo [4/8] 写入磁盘信息...
echo === 磁盘空间 === &gt;&gt; "!report!"
wmic logicaldisk get caption,description,freespace,size,volumename &gt;&gt; "!report!" 2&gt;&amp;1
echo. &gt;&gt; "!report!"
echo [5/8] 写入网络配置...
echo === 网络配置 === &gt;&gt; "!report!"
ipconfig /all &gt;&gt; "!report!" 2&gt;&amp;1
echo. &gt;&gt; "!report!"
echo [6/8] 写入当前进程...
echo === 运行进程 === &gt;&gt; "!report!"
tasklist /fo table &gt;&gt; "!report!" 2&gt;&amp;1
echo. &gt;&gt; "!report!"
echo [7/8] 写入已安装服务...
echo === Windows 服务 === &gt;&gt; "!report!"
net start &gt;&gt; "!report!" 2&gt;&amp;1
echo. &gt;&gt; "!report!"
```

```
echo [8/8] 写入已安装程序...
echo === 已安装程序 === &gt;&gt; "!report!"
wmic product get name,version,vendor &gt;&gt; "!report!" 2&gt;&amp;1
echo.
echo ============================================
echo  报告生成完毕!
echo  文件位置: !report!
echo  文件大小:
for %%f in ("!report!") do echo    %%~zf 字节
echo ============================================
pause
```

第 1 章进阶项目：一键开机加速脚本

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo  第1章项目：一键开机加速脚本
echo  原理：清理临时文件 + 禁用不必要启动项
echo ============================================
echo.
set "cleanup_count=0"
set "skip_count=0"
echo [1/4] 清理用户临时文件...
if exist "%temp%" (
    for /f "delims=" %%f in ('dir /a-d /b /s "%temp%" 2^&gt;nul') do (
        del /f /q "%%f" 2&gt;nul &amp;&amp; set /a cleanup_count+=1 || set /a skip_count+=1
    )
    for /f "delims=" %%d in ('dir /ad /b /s "%temp%" 2^&gt;nul ^| sort /r') do (
        rd /s /q "%%d" 2&gt;nul
    )
```

```
)
echo    用户临时: 清理 !cleanup_count! 个，跳过 !skip_count! 个
echo [2/4] 清理 Windows 临时目录...
set "winclean=0"
set "winskip=0"
if exist "C:\Windows\Temp" (
    for /f "delims=" %%f in ('dir /a-d /b /s "C:\Windows\Temp" 2^&gt;nul') do (
        del /f /q "%%f" 2&gt;nul &amp;&amp; set /a winclean+=1 || set /a winskip+=1
    )
)
echo    系统临时: 清理 !winclean! 个
echo [3/4] 预读取文件夹（Prefetch）...
set "pfclean=0"
if exist "C:\Windows\Prefetch" (
    for /f "delims=" %%f in ('dir /a-d /b "C:\Windows\Prefetch" 2^&gt;nul') do (
        del /f /q "C:\Windows\Prefetch\%%f" 2&gt;nul &amp;&amp; set /a pfclean+=1
    )
)
echo    Prefetch: 清理 !pfclean! 个
echo [4/4] 回收站清理...
rd /s /q C:\$Recycle.Bin 2&gt;nul
echo    回收站清空
echo.
echo ============================================
echo  清理完成!
echo  总计清理文件: !cleanup_count! 个
echo  建议重启电脑以获得最佳效果
echo ============================================
pause
```

进阶项目1：完整的系统环境诊断脚本

这个 100 行的 BAT 脚本涵盖了系统诊断的 7 大维度：OS
版本、补丁列表、启动时间、CPU
参数、内存详情、磁盘信息、网络适配器。脚本不仅把结果打印到控制台，还自动保存为 diagnose_report_YYYYMMDD.txt。这种"控制台 + 文件"双输出模式是运维脚本的标准做法。

```
@echo off
setlocal enabledelayedexpansion
title 系统环境完整诊断脚本 v1.0
color 0B

rem ============================================================
rem  系统环境完整诊断脚本
rem  功能：检测 OS 版本、架构、已安装补丁、启动时间、硬件摘要
rem  输出：控制台打印 + 保存到 diagnose_report.txt
rem  作者：BAT 实战教程配套脚本
rem ============================================================

set "REPORT=%~dp0diagnose_report_%date:~0,4%%date:~5,2%%date:~8,2%.txt"
echo === 系统环境诊断报告 === &gt; "%REPORT%"
echo 生成时间: %date% %time% &gt;&gt; "%REPORT%"
echo. &gt;&gt; "%REPORT%"

echo [1/7] 操作系统基本信息...
echo === 操作系统 ===
echo 计算机名: %COMPUTERNAME%
echo 用户名:   %USERNAME%
echo 域:       %USERDOMAIN%
echo OS 描述:
wmic os get Caption /format:list | findstr Caption
echo OS 版本:
wmic os get Version /format:list | findstr Version
echo 系统架构: %PROCESSOR_ARCHITECTURE%
echo Windows 目录: %SystemRoot%
echo 当前目录:  %CD%
echo OS 描述: %OS%
```

```
ver
echo. &gt;&gt; "%REPORT%"
echo === 操作系统 === &gt;&gt; "%REPORT%"
wmic os get Caption,Version,BuildNumber,OSArchitecture /format:list &gt;&gt; "%REPORT%"

echo [2/7] 已安装补丁列表（最近 10 条）...
echo === 已安装补丁 (Top 10) ===
wmic qfe get HotFixID,InstalledOn,Description /format:list | findstr /v "^$" | head -30
echo. &gt;&gt; "%REPORT%"
echo === 已安装补丁 === &gt;&gt; "%REPORT%"
wmic qfe get HotFixID,InstalledOn,Description /format:csv &gt;&gt; "%REPORT%"

echo [3/7] 系统启动与运行时长...
echo === 启动信息 ===
for /f "tokens=2 delims==" %%t in ('wmic os get LastBootUpTime /format:list ^| findstr LastBoot') do (
    set "boot_raw=%%t"
    set "boot_date=!boot_raw:~0,4!-!boot_raw:~4,2!-!boot_raw:~6,2!"
    set "boot_time=!boot_raw:~8,2!:!boot_raw:~10,2!:!boot_raw:~12,2!"
    echo 上次启动: !boot_date! !boot_time!
)
set /a uptime_days=0
for /f "tokens=1" %%d in ('net statistics workstation ^| findstr "统计"') do set "stat_time=%%d"
echo 当前时间: %date% %time%
echo. &gt;&gt; "%REPORT%"
echo === 启动信息 === &gt;&gt; "%REPORT%"
wmic os get LastBootUpTime,SystemUpTime /format:list &gt;&gt; "%REPORT%"

echo [4/7] CPU 信息...
echo === CPU ===
wmic cpu get Name,NumberOfCores,NumberOfLogicalProcessors,MaxClockSpeed /format:list
echo 逻辑处理器数: %NUMBER_OF_PROCESSORS%
echo. &gt;&gt; "%REPORT%"
echo === CPU === &gt;&gt; "%REPORT%"
wmic cpu get Name,NumberOfCores,NumberOfLogicalProcessors,MaxClockSpeed /format:list &gt;&gt; "%REPORT%"
```

```
echo [5/7] 内存信息...
echo === 物理内存 ===
for /f "tokens=2 delims==" %%a in ('wmic computersystem get TotalPhysicalMemory /format:list ^| findstr Total') do (
    set /a gb=%%a / 1073741824
    echo 总内存: !gb! GB
)
echo 内存插槽:
wmic memorychip get Capacity,Speed,Manufacturer,PartNumber /format:list
echo. &gt;&gt; "%REPORT%"
echo === 内存 === &gt;&gt; "%REPORT%"
wmic memorychip get Capacity,Speed,Manufacturer /format:list &gt;&gt; "%REPORT%"

echo [6/7] 磁盘信息...
echo === 磁盘 ===
wmic logicaldisk get Caption,Description,Freespace,Size,FileSystem /format:table
echo === 物理磁盘 ===
wmic diskdrive get Model,Size,InterfaceType /format:list
echo. &gt;&gt; "%REPORT%"
echo === 磁盘 === &gt;&gt; "%REPORT%"
wmic logicaldisk get Caption,Freespace,Size,FileSystem /format:list &gt;&gt; "%REPORT%"

echo [7/7] 网络适配器...
echo === 网络适配器 ===
ipconfig /all | findstr /c:"适配器" /c:"IPv4" /c:"IPv6" /c:"网关" /c:"DNS"
echo MAC 地址:
getmac /fo table
echo. &gt;&gt; "%REPORT%"
echo === 网络 === &gt;&gt; "%REPORT%"
ipconfig /all &gt;&gt; "%REPORT%"

echo ============================================================
echo 诊断完成!
echo 详细报告已保存到: %REPORT%
```

```
echo ============================================================
echo.
echo 是否打开报告？(Y/N)
set /p openit=选择:
if /i "!openit!"=="Y" start "" notepad "%REPORT%"
endlocal
pause
exit /b
```

系统诊断指标汇总表（25 项）

序号	诊断类别	命令/API	输出字段	正常值/范围	异常时说明	严重程度
1	OS 描述	wmic os get Caption	Caption	Windows 10/11	版本过旧	低
2	OS 架构	%PROCESSOR_ARCHITECTURE%	AMD64/x86	AMD64	32位系统	中
3	OS 构建号	wmic os get BuildNumber	BuildNumber	>=19041	版本太老	中
4	上次启动	wmic os get LastBootUpTime	LastBootUpTime	<7天前	长期运行未重启	低
5	运行时长	wmic os get SystemUpTime	SystemUpTime	<720小时	系统未更新重启	低
6	补丁数量	wmic qfe get count	Count	>=10	补丁太少	高
7	最新补丁	wmic qfe get InstalledOn	InstalledOn	<30天	长期没补丁	高
8	CPU 名称	wmic cpu get Name	Name	>=4核	单核心老旧CPU	中
9	CPU 核心数	wmic cpu get NumberOfCores	NumberOfCores	>=2	单核系统	中
10	CPU 逻辑核	wmic cpu get NumberOfLogicalProcessors	NumberOfLogicalProcessors	>=4	过慢	低
11	CPU 主频	wmic cpu get MaxClockSpeed	MaxClockSpeed	>=2000	低性能	低
12	总内存	wmic computersystem TotalPhysicalMemory	TotalPhysicalMemory	>=4GB	内存不足	中
13	内存插槽数	wmic memorychip get count	Count	1-4	硬件信息	低
14	内存频率	wmic memorychip get Speed	Speed	>=1600	DDR3及以下	低
15	系统盘空间	wmic logicaldisk get Freespace	Freespace(C:)	>=10GB	系统盘满	高
16	磁盘文件系统	wmic logicaldisk get FileSystem	FileSystem	NTFS	FAT32不支持大文件	中
17	磁盘接口	wmic diskdrive get InterfaceType	InterfaceType	SATA/SSD/NVMe	老IDE	低
18	SSD 检测	wmic diskdrive get MediaType	MediaType	Fixed hard disk	无SSD	低
19	IP 地址	ipconfig /all	IPv4	内网段 10.x/192.168.x	公网IP直暴露	高
20	MAC 地址	getmac	MAC Address	正常6段	全0表示适配器异常	中
21	默认网关	ipconfig	Default Gateway	非空	无法上网	高
22	DNS 服务器	ipconfig	DNS Servers	>=1	无法解析域名	中

23	已安装杀软	Get-MpComputerStatus	RealTimeProtection	True	实时保护关闭	高
24	UAC 状态	reg query HKLM\...EnableLUA	EnableLUA	1	关闭=高风险	高
25	系统服务	sc query state= all	RUNNING	wuauserv等核心运行	关键服务停止	高

写系统诊断脚本时最重要的三个点：第一是容错——wmic 在某些精简版 Windows 上可能失败，要把错误抑制掉（2>nul）；第二是输出格式统一，上面用了 /format:list 和 /format:table 两种，list 适合脚本解析，table 适合人看；第三是延迟扩展，像本脚本里计算内存 GB 数就需要 !var! 而不是 %var%。

Windows 的系统信息通过四种渠道获取：环境变量（%COMPUTERNAME%、%NUMBER_OF_PROCESSORS%）、wmic WMI 查询、ipconfig/getmac 工具、注册表。环境变量最快但信息少，wmic 最全但稍慢，注册表最底层但最灵活。大型诊断脚本通常组合使用。

输出到文件时建议用引号包裹路径以防空格，同时在括号块内用 >> 追加重定向比单行 echo 重定向效率高。生成的报告可以定期对比——比如每周跑一次，观察磁盘空间是否持续下降、补丁是否及时安装、CPU 温度是否异常。这些趋势信息比单次检查更有价值。

严重程度列（高/中/低）是给运维看的快速 triage 信号："高"意味着可能有安全漏洞或系统即将出问题，需要立即处理；"中"意味着性能或兼容性有风险；"低"是提示性质。写任何监控脚本时，这种分级输出都比纯文字描述有用得多。

第2章 第一个 BAT 程序与基础输出

2.1 @echo off：关闭命令回显

在 BAT 中，`echo` 是最常用的命令之一，它的作用是开启或关闭"命令回显"。什么是命令回显？就是 cmd.exe 在执行每条命令之前，先把这条命令本身打印出来的行为。

比如，如果没有 @echo off：

```
REM test.bat

echo Hello

pause
```

运行后你会看到：

```
C:\&gt;REM test.bat

C:\&gt;echo Hello
Hello

C:\&gt;pause
请按任意键继续. . .
```

很明显，那些 `C:\>echo Hello` 这样的"命令本身"被打印出来了，这就是回显。使用 `@echo off` 后，所有命令都不会被回显——脚本输出会干净得多。

> **TIP**
>
> 开头的 @ 符号是什么？它的作用是让 echo off 这条命令本身也不被回显。如果只写 echo off，你会看到 "C:\>echo off" 这一行先被打印出来，然后回显才关闭。加上 @ 后，连 echo off 这条命令也被静默了。

`@echo off` 几乎是所有 BAT 脚本的第一行。唯一的例外是当你需要调试脚本时——临时去掉它可以看到每条命令的实际执行情况。

2.2 echo：输出文本

当 echo 后面跟有内容时，它就不再是控制回显，而是**输出文本**到控制台。

基本用法：

```
@echo off
echo 你好，世界!
echo Hello World
echo
```

注意第三条 `echo` 不带任何参数，它输出一个空行——用来分隔不同部分的输出。

输出特殊字符

echo 输出时有几个特殊字符需要注意，因为它们在 cmd 中有特殊含义：

- `%` —— 变量引用符号
- `&` —— 命令连接符
- `|` —— 管道符
- `< >` —— 重定向符
- `^` —— 转义符

如果你想在 echo 输出中包含这些特殊字符，需要用 ^（脱字符）转义：

```
@echo off
echo 100%% CPU
echo A ^| B
echo C ^&amp;amp; D
echo 5 ^&amp;gt; 3
```

输出结果：

```
100% CPU
A | B
C &amp;amp; D
5 &amp;gt; 3
```

WARNING

只有在 echo 的上下文中，% 才会被解析为变量引用。如果在括号块 (...) 或 for 循环内部，则需要用两个 %%（%%）来表示一个字面百分号。这一点我们在讲变量和 for 循环时会详细解释。

2.3 pause：暂停与交互

`pause` 命令会暂停脚本执行，并显示提示：

```
请按任意键继续. . .
```

这是 BAT 中最简单的"用户交互"方式。当你双击运行 BAT 文件时，窗口执行完命令后会立即关闭——用 pause 可以让窗口停住，让你看到输出结果。

进阶：自定义 pause 提示文字

```
@echo off
echo 脚本执行完毕，按任意键退出...
pause &amp;gt; nul
```

`pause > nul` 表示把 pause 的默认提示重定向到空设备（丢弃），只显示我们自己 echo 的提示。

2.4 rem 与 ::：注释的两种写法

BAT 中有两种写注释的方式：

```
rem 这是一行注释

:: 这也是一行注释

echo 上面两行不会被执行
```

两者的区别：

- **rem** —— 正规的注释命令，在 cmd 解析器中有正式定义，兼容性 100%
- **::** —— 利用标签（:label）语法的 hack，不是真正的注释命令，但写起来更简洁

> **WARNING**
> :: 的原理：以 : 开头的行被 cmd 当作标签（goto 的跳转目标），但 ::
> 不是合法的标签名（标签名不能包含冒号），所以 cmd
> 会跳过它。某些极端情况下（如括号块内或 for 的 () 内）:: 可能导致解析错误，此时必须用 rem。稳妥起见，**本书统一使用 rem**。

2.5 输出重定向初体验

有时我们不希望脚本输出到屏幕，而是写入文件。这就需要"重定向"。

```
@echo off

echo 这行将写入文件 &gt; output.txt

dir C:\Windows &gt; dir_list.txt
```

`>` 会覆盖文件内容，`>>` 则是**追加**到文件末尾。这是重定向最基本的用法。第 6 章会深入讲解完整的重定向体系。

2.6 编码入门：解决中文乱码

BAT 脚本中最常见的问题之一就是**中文乱码**。这是因为文件保存编码和 cmd 期望的编码不匹配。

Windows 的中文 cmd 默认使用 **GBK**（代码页 936）。

- 如果你的 BAT 文件用 ANSI/GBK 保存 → 直接双击运行正常
- 如果你的 BAT 文件用 UTF-8 保存 → 需要先执行 `chcp 65001`

```
@echo off

chcp 65001 &gt; nul

echo 这是 UTF-8 编码的中文

echo 不会再乱码了
```

TIP

最佳实践：使用 VS Code 或 Notepad++，将文件保存为 **UTF-8 with BOM**，脚本开头加上 `chcp 65001 > nul`。这样在 Windows 10/11 上中文显示最稳定。更彻底的编码方案见第 11.7 节。

2.7 第一个完整脚本

把学过的知识组合起来，写一个真正有用的脚本——系统信息小工具：

```
@echo off
chcp 65001 &gt; nul
title 系统信息查询工具

rem ========== 系统信息查询 ==========
echo ========================================
echo       欢迎使用系统信息查询工具
echo ========================================
echo.
echo 当前时间: %date% %time%
echo 计算机名: %computername%
echo 当前目录: %cd%
echo 操作系统: %os%
echo 处理器数: %number_of_processors%
echo.
echo ========================================
pause
```

保存为 `sysinfo.bat` 并运行，你会看到：

```
========================================
    欢迎使用系统信息查询工具
========================================

当前时间: 2026/11/10 二 14:30:00.00
计算机名: DESKTOP-HARRY
当前目录: C:\Users\Harry\Desktop
```

```
操作系统: Windows_NT

处理器数: 8


========================================

请按任意键继续. . .
```

脚本用到了多个"内置环境变量"（%date%、%computername% 等），这些我们在第 4 章变量系统中会详细讲解。现在你只需要知道：BAT 已经可以帮我们做真正有用的事情了！

— 第2章 完 —

进阶练习

练习1：echo 的各种用法演示：

```
@echo off
echo === echo 演示 ===
echo.
set name=张三
echo 你好，%name%
echo 这行写入文件 &gt; output.txt
echo 这行追加 &gt;&gt; output.txt
&lt;nul set /p "=不换行输出"
echo 完成!
echo 百分号: %%
echo 管道符: ^|
pause
```

练习2：带颜色的欢迎界面：

```
@echo off
color 0A
cls
echo.
echo   [1] 系统信息  [2] 网络诊断
echo   [0] 退出
set /p choice=选择:
echo 你选了 %choice%
pause
```

练习3：pause 变体和 choice 命令：

```
@echo off
echo 正在删除...
pause &gt; nul
echo 正在清理...
```

```
pause &gt; nul
echo 完成!
choice /c YN /m "确认退出"
if errorlevel 2 echo 选了 No
if errorlevel 1 echo 选了 Yes
```

疑难解答

Q1: echo 输出中文乱码?

A: 确认脚本编码和代码页一致。默认 chcp 936 = GBK。

Q2: echo %var% 是空的?

A: 括号块内需延迟扩展：setlocal enabledelayedexpansion 并用 !var!。

Q3: echo 如何输出空行?

A: echo. / echo(/ echo+ 都行。echo. 后面不能有空格。

进阶主题1：set /p 密码输入不回显技巧

set /p 默认会回显用户输入，对于密码场景这很危险。我们可以调用 PowerShell 的 Read-Host -AsSecureString，或者自己实现一个逐字符读取的方案。注意管道启动子进程的变量丢失问题。

```
@echo off
setlocal enabledelayedexpansion
echo === set /p 密码输入不回显技巧 ===
echo.
echo 标准 set /p 会回显输入内容:
set /p "pwd1=请输入密码(可见): "
echo 你输入了: !pwd1!
echo.
echo 用 PowerShell Read-Host -AsSecureString 实现真正的密码输入:
for /f "delims=" %%p in (
  powershell -Command "Add-Type -AssemblyName System.Windows.Forms; [System.Windows.Forms.TextBox]::new() | Out-Null; $s=Read-Host '请输入密码(隐藏)' -AsSecureString; [Runtime.InteropServices.Marshal]::PtrToStringAuto([Runtime.InteropServices.Marshal]::SecureStringToBSTR($s))"
```

```
) do set "pwd2=%%p"
echo 你输入的密码长度: !pwd2:len=!
for /l %%i in (1,1,0) do rem trick
set "len=0"
set "t=!pwd2!"
:count
if "!t!"=="" goto done_count
set /a len+=1
set "t=!t:~1!"
goto count
:done_count
echo 密码长度: !len!
echo.
echo 另一种简易方案(用 *) :
powershell -Command "$p=''; while(($c=$Host.UI.RawUI.ReadKey('NoEcho,IncludeKeyDown')).Character -ne 13){if($c.Character -eq 8){$p=$p.Substring(0,[Math]::Max(0,$p.Length-1));Write-Host -NoNewline "\b \b"}else{$p+=$c.Character;Write-Host -NoNewline "*"}}; echo; echo '输入完成'"
pause
```

进阶主题2：进度条动画（set /p + 延迟扩展）

用 <nul set /p 的回车技巧，配合延迟扩展的 for 循环，可以做出漂亮的 ASCII 进度条和旋转 spinner。ping -n 127.0.0.1 是最简单的 1 秒延迟 hack。

```
@echo off
setlocal enabledelayedexpansion
echo === 进度条动画 ===
set /a total=100
set /a bar_width=30
echo.
echo 正在处理...
for /L %%i in (0,1,!total!) do (
    set /a filled=%%i * bar_width / total
    set /a empty=bar_width - filled
    set "bar="
```

```
    for /L %%j in (1,1,!filled!) do set "bar=!bar!"
    for /L %%j in (1,1,!empty!) do set "bar=!bar! "
    &lt;nul set /p "= [!bar!] %%i%% "
    timeout /t 0 &gt;nul
    ping -n 2 127.0.0.1 &gt;nul
)
echo.
echo 完成!
echo.
echo === 简单 spinner ===
set "chars=| / - \"
for /L %%i in (1,1,20) do (
    for /f "tokens=%%i" %%c in ("!chars!") do (
        &lt;nul set /p "= 处理中 %%c "
    )
    ping -n 2 127.0.0.1 &gt;nul
)
&lt;nul set /p "= 处理中完成 "
echo.
pause
```

进阶主题3：多行文本输出（<nul set /p）

BAT 有 5 种方式输出多行文本：逐行 echo、^ 续行、<nul set /p 拼接、括号块写入文件、复杂脚本生成。括号块 + 重定向是生成配置文件的最佳方案。

```
@echo off
echo === 多行文本输出技巧 ===
echo.
echo --- 方法1: echo 逐行 ---
echo 第一行
echo 第二行
echo 第三行
echo.
echo --- 方法2: 用 ^ 续行符 ---
```

```
echo 这是一行很长的文本，^
它可以跨多行书写，^
但输出时是一行。
echo.
echo --- 方法3: 用 &lt;nul set /p 拼接 ---
&lt;nul set /p "=你好, "
&lt;nul set /p "=世界! "
&lt;nul set /p "=这是一行无换行的文本"
echo.
echo.
echo --- 方法4: 写入多行文件 ---
(
    echo 配置文件内容开始
    echo ====================
    echo Host=localhost
    echo Port=8080
    echo Debug=true
    echo Timeout=30
    echo ====================
    echo 配置文件内容结束
) &gt; config.ini
echo 已写入 config.ini:
type config.ini
del config.ini
echo.
echo --- 方法5: 复杂多行含特殊字符 ---
(
    @echo off
    echo %%SystemRoot%% = %SystemRoot%
    echo 管道符 ^| 和重定向 ^&gt; 都正常
    echo 百分号 %%%%
    if exist C:\Windows echo C:\Windows 存在
```

```
) &gt; script.bat
echo 已生成 script.bat:
type script.bat
del script.bat
pause
```

表 2-1 echo 的完整参数表

语法	说明	转义要点	示例
echo	输出换行文本	特殊字符 ^|^>^<^& 需转义	echo Hello
echo.	输出空行	点后不能有空格	echo.
echo/	输出空行（另一种）	更稳妥的空行写法	echo/
echo(	输出空行（最稳）	括号永远是空行	echo(
echo text>file	重定向到文件（覆盖）	不加空格直接连	echo hi>out.txt
echo text>>file	重定向到文件（追加）	双大于号	echo hi>>out.txt
^<nul set /p "=text"	不换行输出	管道符前加脱字符	<nul set /p "=Hello"
echo %var%	输出变量（预处理阶段）	百分号直接写	echo %%username%%
echo %%	输出字面 %	两个 %% 转义	echo 100%%
echo ^| ^> ^< ^&	输出特殊字符本身	每一个都要转义	echo a^|b
echo on	开启命令回显	调试时有用	echo on
echo off	关闭命令回显	脚本开头必用	@echo off
echo 【多行块】	批量写入多行文件	用括号括起 echo 组	(echo a & echo b)>f.txt
echo %date% %time%	输出日期时间	动态变量直接展开	echo %%date%%
echo !var!	延迟展开输出变量	需 setlocal enabledelayedexpansion	echo !counter!

表 2-2 颜色代码对照表（color 命令）

颜色名	代码	Hex	用途	备注
黑色 (Black)	0	#000000	默认背景色	color 07

蓝色 (Blue)	1	#0000AA	提示信息	
绿色 (Green)	2	#00AA00	成功消息	
青色 (Aqua)	3	#00AAAA	菜单标题	
红色 (Red)	4	#AA0000	错误警告	最常用的警示色
紫色 (Purple)	5	#AA00AA	高级/调试	
黄色 (Yellow)	6	#AA5500	注意事项	
白色 (White)	7	#AAAAAA	默认前景色	
灰色 (Gray)	8	#555555	次要文本	
浅蓝 (Light Blue)	9	#5555FF	链接样式	
浅绿 (Light Green)	A	#55FF55	成功高亮	
浅青 (Light Aqua)	B	#55FFFF	信息高亮	
浅红 (Light Red)	C	#FF5555	紧急错误	
浅紫 (Light Purple)	D	#FF55FF	高亮选择	
浅黄 (Light Yellow)	E	#FFFF55	警告高亮	
亮白 (Bright White)	F	#FFFFFF	强调文字	

第 2 章小结

本章深入到了 BAT 输出的方方面面——从最基础的 echo 到不换行输出、文件重定向、再到 ANSI 颜色码和 ASCII 艺术。echo 看似简单，但涉及特殊字符转义、变量展开时机时的坑比任何其他命令都多。

必须建立的直觉是：BAT 脚本解析分两阶段——预处理（变量展开）和执行。set /a 里百分号立即展开，!var! 延迟展开。理解了这一点，"为什么循环里变量不更新"这种困扰新手的问题立刻就通了。

颜色对照表给出了 color 命令的 16 种前景 + 16 种背景 = 256 种组合。实践中 color 0A（黑底亮绿）和 color 0C（黑底亮红）是最经典的搭配。进阶的 ANSI 转义码（ESC[31m ESC[0m）在 Windows 10 1511+ 也原生支持。

日志系统示例是 echo 的工业级用法：call 调用的 :log 子程序 + 时间戳格式化 + 级别过滤。BAT 里做函数就是用 call 和 goto 标签，用 call :func arg1 arg2 就能传参。这种模式贯穿整个 BAT 高级编程。

Banner 生成器和 UI 框架演示了用纯 ASCII 字符画边框、菜单、进度条的技巧。 ╔ ╚ ═ ╟ 这些 Unicode 框线字符需要 chcp 65001 配合正确字体才能显示。echo 输出 Unicode 的坑是：必须保存为 UTF-8 BOM 或用 GBK 字符集。

下一章进入 call/goto 和子程序。那是让脚本从"单一路径命令串"进化为"可维护模块化程序"的关键。准备好了吗？

第 2 章大代码块 1：屏幕绘制与 UI 框架

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ============================================
echo  第2章综合实战：屏幕绘制与UI框架
echo  用纯 BAT 实现菜单、窗口、进度条、仪表盘
echo ============================================
echo.
rem === 1. 画边框 ===
echo --- 1. 纯 BAT 窗口边框 ---
set "tl=╔" &amp; set "tr=╗" &amp; set "bl=╚" &amp; set "br=╝"
set "h=═" &amp; set "v=║"
set "width=46"
&lt;nul set /p "=!tl!"
for /L %%i in (1,1,!width!) do &lt;nul set /p "=!h!"
echo !tr!
&lt;nul set /p "=!v!"
for /L %%i in (1,1,!width!) do &lt;nul set /p "= "
echo !v!
&lt;nul set /p "=!v!"
&lt;nul set /p "=  BAT UI Framework Demo"
for /L %%i in (1,1,26) do &lt;nul set /p "= "
echo !v!
&lt;nul set /p "=!v!"
for /L %%i in (1,1,!width!) do &lt;nul set /p "= "
echo !v!
```

```
&lt;nul set /p "=!bl!"
for /L %%i in (1,1,!width!) do &lt;nul set /p "=!h!"
echo !br!
echo.
rem === 2. 模拟仪表盘 ===
echo --- 2. CPU使用率仪表盘（用 typeperf）---
typeperf "\Processor(_Total)\% Processor Time" -si 1 -sc 5 2&gt;nul | findstr /v "_Total" | findstr /v "^$"
echo.
rem === 3. 多行菜单框架 ===
echo --- 3. 功能菜单框架 ---
echo [A] 数据库备份    [B] 日志清理
echo [C] 磁盘检查      [D] 用户管理
echo [E] 网络测试      [F] 版本查询
echo [X] 退出
echo.
rem === 4. 使用 choice 做选择 ===
choice /c ABCDEFX /n /m "请选择 [A/B/C/D/E/F/X]: "
set "choice=%errorlevel%"
if !choice!==1 echo 你选了 A - 数据库备份
if !choice!==2 echo 你选了 B - 日志清理
if !choice!==3 echo 你选了 C - 磁盘检查
if !choice!==4 echo 你选了 D - 用户管理
if !choice!==5 echo 你选了 E - 网络测试
if !choice!==6 echo 你选了 F - 版本查询
if !choice!==7 echo 退出
echo.
rem === 5. 暂停效果 ===
echo --- 4. 动态等待动画 ---
set "anim=                                    "
set /a idx=0
for /L %%i in (1,1,30) do (
    set /a "idx=%%i %% 10"
```

```
    for /f "delims=" %%c in ('echo !anim:~!idx!,1!') do (
        &amp;lt;nul set /p "=  加载中 %%c\r"
    )
    ping -n 2 127.0.0.1 &amp;gt;nul
)
&amp;lt;nul set /p "=  加载完成      \r"
echo.
pause
```

第 2 章大代码块 2：带分级的日志系统

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &amp;gt;nul 2&amp;gt;&amp;amp;1
echo ============================================
echo   第2章进阶：日志系统（带时间戳和分级）
echo ============================================
echo.
set "LOGFILE=%~dp0app.log"
set "LOGLEVEL=2"
rem 日志级别：0=DEBUG 1=INFO 2=WARN 3=ERROR 4=FATAL
set "level_name[0]=DEBUG"
set "level_name[1]=INFO "
set "level_name[2]=WARN "
set "level_name[3]=ERROR"
set "level_name[4]=FATAL"
echo === 正在测试日志系统 ===
echo.
rem === 函数式日志（call 方案） ===
call :log 0 "这是一条 DEBUG 消息——内部调试用"
call :log 1 "这是一条 INFO 消息——正常流程记录"
```

```
call :log 2 "这是一条 WARN 消息——需要注意的情况"
call :log 3 "这是一条 ERROR 消息——发生了错误"
call :log 4 "这是一条 FATAL 消息——致命错误"
echo.
echo === 日志文件内容 ===
type "!LOGFILE!"
echo.
echo === 日志文件统计 ===
echo 总行数: &amp;amp; find /c /v "" "!LOGFILE!"
echo DEBUG: &amp;amp; find /c "DEBUG" "!LOGFILE!"
echo INFO : &amp;amp; find /c "INFO " "!LOGFILE!"
echo WARN : &amp;amp; find /c "WARN " "!LOGFILE!"
echo ERROR: &amp;amp; find /c "ERROR" "!LOGFILE!"
echo FATAL: &amp;amp; find /c "FATAL" "!LOGFILE!"
pause
exit /b 0
:log
set "lvl=%~1"
set "msg=%~2"
if !lvl! lss !LOGLEVEL! exit /b 0
rem 获取时间戳
for /f "tokens=1-2 delims= " %%a in ("%date% %time%") do (
    set "timestamp=%%a %%b"
)
set "timestamp=!timestamp: =0!"
set "lname=!level_name[%lvl%]!"
echo [!timestamp!] [!lname!] !msg! &amp;gt;&amp;gt; "!LOGFILE!"
if !lvl! geq 3 (
    echo [!timestamp!] [!lname!] !msg!
)
exit /b 0
```

第 2 章进阶项目：ASCII Banner 生成器

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ============================================
echo   第2章项目：带 ASCII 艺术的 Banner 生成器
echo   将简单文字转换为大字符 Banner
echo ============================================
echo.
set /p "text=请输入要转换的英文文本: "
if "!text!"=="" set "text=Hello"
echo.
echo === 简单点矩阵风格 Banner ===
call :banner_simple "!text!"
echo.
echo === 7x5 像素风格 Banner ===
call :banner_pixel "!text!"
echo.
pause
exit /b 0
:banner_simple
setlocal enabledelayedexpansion
set "input=%~1"
set "output="
for %%L in (A B C D E F G H I J K L M N O P Q R S T U V W X Y Z 0 1 2 3 4 5 6 7 8 9) do (
    set "letter_%%L=████████       "
)
set "letter_ =                    "
set "letter_A=██          ██  ██  ██  ██  ██████  ██          ██  ██      ██  ██      ██"
set "letter_B=███████     ██          ██  ██████    ██          ██          ██  ███████"
set "letter_C=████████  ██            ██                ██            ██        ████████"
```

```
set "letter_D=███████    ██         ██  ██        ██  ██        ██        ██  ███████"
set "letter_E=█████████  ██           ██        ████          ██        ██  ████████"
set "letter_F=████████   ██           ██        ████          ██                    "
echo （用简化 5 行 ASCII banner 演示）
for %%L in (H E L O _ _ _ _ W O R L D) do (
    if "%%L"=="_" (
        echo.
    ) else (
        echo ███  ███ ████████ ███   ███   ████████ ███  ███ ████████
        echo ███  ███ ███   ███   ███   ███ ███ ███ ███ ███ ███
        echo ████████ ██████  ███   ███   ████████ ███  ███ ████████
        echo ███  ███ ███   ███   ███   ███ ███ ███ ███ ███ ███
        echo ███  ███ ████████ ████████ ████████  ███  ███ ████████ ███  ███
    )
)
endlocal
exit /b 0
:banner_pixel
echo --- 像素风格（5行）---
echo  _  _    _ _     __     __      _   _
echo ^| ^| ^| ^| ___^| ^| ^| ___  \ \   / /__ _ __^| ^| __^| ^|
echo ^| ^|_^| ^|/ _ \ ^| ^|/ _ \  \ \ /\ / / _ \^|  __^| ^|/ _` ^|
echo ^|  _  ^|  __/ ^| ^| (_) ^|  \ V  V / (_) ^| ^|  ^| ^| (_^| ^|
echo ^|_^| ^|_^|\___^|_^|_^|\___/    \_/\_/ \___/^|_^|  ^|_^|\__,_^|
exit /b 0
```

进阶项目2：交互式安装向导模板

这个完整的安装向导涵盖了商业安装程序的所有标准阶段：欢迎页、许可协议、目录选择、权限检查、旧版备份、文件复制、快捷方式、注册表写入、PATH 更新、完成页。脚本带完整的错误处理和回滚机制——任何一步失败都不会让系统处于半安装状态。

```
@echo off
setlocal enabledelayedexpansion
title 交互式安装向导 v2.0
color 0E

rem ============================================================
rem  交互式安装向导模板
rem  用 set /p + choice 实现 yes/no 安装流程
rem  带进度显示、错误处理、回滚支持
rem ============================================================

set "APP_NAME=我的 BAT 工具包"
set "APP_VERSION=1.2.0"
set "INSTALL_DIR=C:\Program Files\MyBATTool"
set "BACKUP_DIR=%~dp0backup_%random%"
set "LOG_FILE=%~dp0install_log.txt"
set /a step=0

echo. &gt; "%LOG_FILE%"
echo [%date% %time%] ============== 安装开始 ============== &gt;&gt; "%LOG_FILE%"

:WELCOME
cls
echo.
echo    ================================================
echo.       欢迎使用 !APP_NAME! v!APP_VERSION!
echo    ================================================
echo.
echo    本向导将引导您完成安装过程。
echo    安装前请关闭其他正在运行的程序。
echo.
echo.       [1] 开始安装
echo.       [2] 查看许可协议
echo.       [3] 退出安装
```

```
echo.
set /p "wel=请选择: "
if "!wel!"=="1" goto :LICENSE
if "!wel!"=="2" goto :SHOW_LICENSE
if "!wel!"=="3" goto :CANCEL
goto :WELCOME

:SHOW_LICENSE
cls
echo.
echo === !APP_NAME! 许可协议 ===
echo 本软件按 MIT 许可发布。
echo 你可以自由使用、修改、分发。
echo 作者不承担使用本软件造成的任何损失。
echo.
echo 按任意键返回...
pause &gt;nul
goto :WELCOME

:LICENSE
cls
echo.
echo 是否接受以上许可协议？
choice /C YN /M "接受并继续安装(Y) 或 取消(N)"
if errorlevel 2 goto :CANCEL
if errorlevel 1 goto :CHOOSE_DIR

:CHOOSE_DIR
cls
echo.
echo.        默认安装目录: !INSTALL_DIR!
echo.
choice /C YNC /M "使用默认目录(Y) 自定义(C) 取消(N)"
if errorlevel 3 goto :CANCEL
```

```
if errorlevel 2 (
    echo.
    set /p "INSTALL_DIR=请输入安装目录: "
    if "!INSTALL_DIR!"=="" (set "INSTALL_DIR=C:\Program Files\MyBATTool")
)
echo 选择的目录: !INSTALL_DIR! &gt;&gt; "%LOG_FILE%"

rem 权限检查
if not exist "!INSTALL_DIR!" (
    echo 正在创建安装目录...
    mkdir "!INSTALL_DIR!" 2&gt;nul
    if errorlevel 1 (
        echo.
        echo [错误] 无法创建目录 !INSTALL_DIR!
        echo        请以管理员身份运行本脚本。
        choice /C AR /M "管理员权限运行(A) 返回修改目录(R)"
        if errorlevel 2 goto :CHOOSE_DIR
        powershell -Command "Start-Process cmd -ArgumentList '/c %~f0' -Verb RunAs"
        exit /b
    )
)

rem 备份已安装版本
if exist "!INSTALL_DIR!\*" (
    echo 检测到已有版本，正在备份...
    mkdir "!BACKUP_DIR!" 2&gt;nul
    xcopy /E /Y /Q "!INSTALL_DIR!" "!BACKUP_DIR!\" &gt;nul 2&gt;&amp;1
    echo 已备份到 !BACKUP_DIR! &gt;&gt; "%LOG_FILE%"
    echo 备份完成。
)

goto :CONFIRM

:CONFIRM
```

```
cls
echo.
echo    === 安装摘要 ===
echo    应用名称:  !APP_NAME! v!APP_VERSION!
echo    安装目录:  !INSTALL_DIR!
echo    目标用户:  %USERNAME%
echo    计算机:    %COMPUTERNAME%
echo.
choice /C YN /M "确认开始安装"
if errorlevel 2 goto :CHOOSE_DIR
if errorlevel 1 goto :DO_INSTALL

:DO_INSTALL
cls
echo.
echo    正在安装 !APP_NAME! v!APP_VERSION!
echo.

set /a total_steps=5
rem Step 1: 清理旧文件
set /a step+=1
echo [!step!/!total_steps!] 清理旧文件...
if exist "!INSTALL_DIR!\*.bak" del /q "!INSTALL_DIR!\*.bak" 2&gt;nul
call :ShowProgress !step! !total_steps!

rem Step 2: 复制主文件
set /a step+=1
echo [!step!/!total_steps!] 复制主程序文件...
mkdir "!INSTALL_DIR!\bin" 2&gt;nul
mkdir "!INSTALL_DIR!\modules" 2&gt;nul
mkdir "!INSTALL_DIR!\docs" 2&gt;nul
copy /y "%~dp0main.bat" "!INSTALL_DIR!\bin\" &gt;nul 2&gt;&amp;1
copy /y "%~dp0modules\*.bat" "!INSTALL_DIR!\modules\" &gt;nul 2&gt;&amp;1
copy /y "%~dp0README.txt" "!INSTALL_DIR!\docs\" &gt;nul 2&gt;&amp;1
```

```
echo  文件复制完成 &gt;&gt; "%LOG_FILE%"
call :ShowProgress !step! !total_steps!

rem Step 3: 创建快捷方式
set /a step+=1
echo [!step!/!total_steps!] 创建开始菜单快捷方式...
set "START_MENU=%APPDATA%\Microsoft\Windows\Start Menu\Programs\MyBATTool"
mkdir "!START_MENU!" 2&gt;nul
powershell -Command "$s=New-Object -ComObject WScript.Shell; $c=$s.CreateShortcut('!START_MENU!\MyBATTool.lnk'); $c.TargetPath='!INSTALL_DIR!\bin\main.bat'; $c.WorkingDirectory='!INSTALL_DIR!'; $c.Save()"
call :ShowProgress !step! !total_steps!

rem Step 4: 注册表写入
set /a step+=1
echo [!step!/!total_steps!] 注册到系统...
reg add "HKCU\Software\MyBATTool" /v InstallDir /t REG_SZ /d "!INSTALL_DIR!" /f &gt;nul
reg add "HKCU\Software\MyBATTool" /v Version /t REG_SZ /d "!APP_VERSION!" /f &gt;nul
reg add "HKCU\Software\MyBATTool" /v InstallDate /t REG_SZ /d "%date%" /f &gt;nul
call :ShowProgress !step! !total_steps!

rem Step 5: PATH 更新
set /a step+=1
echo [!step!/!total_steps!] 添加到 PATH...
call :AddToPath "!INSTALL_DIR!\bin"
call :ShowProgress !step! !total_steps!

echo.
echo 安装完成!
echo.
choice /C YN /M "是否立即启动 !APP_NAME!"
if errorlevel 1 start "" "!INSTALL_DIR!\bin\main.bat"
goto :FINISH

:CANCEL
echo.
```

```
echo 安装已取消。已备份文件保存在 !BACKUP_DIR!
echo 如需清理备份，手动删除该文件夹即可。
echo [%date% %time%] 安装已取消 &gt;&gt; "%LOG_FILE%"
pause
exit /b

:FINISH
echo [%date% %time%] 安装完成 &gt;&gt; "%LOG_FILE%"
echo.
echo 感谢使用 !APP_NAME!!
echo 安装目录: !INSTALL_DIR!
echo 日志文件: !LOG_FILE!
pause
exit /b

:ShowProgress
set /a cur=%1*30/%2
set /a max=30
set "bar="
for /L %%i in (1,1,!cur!) do set "bar=!bar!■"
set /a empty=!max!-!cur!
for /L %%i in (1,1,!empty!) do set "bar=!bar!  "
&lt;nul set /p "=  [!bar!] %1/%2   "
ping -n 2 127.0.0.1 &gt;nul
exit /b

:AddToPath
set "new=%~1"
set "cur_path="
for /f "tokens=2 delims==" %%p in ('reg query HKCU\Environment /v PATH 2^&gt;nul ^| findstr PATH') do set "cur_path=%%p"
echo !cur_path! | findstr /i "!new!" &gt;nul
if errorlevel 1 (
    reg add "HKCU\Environment" /v PATH /t REG_EXPAND_SZ /d "!cur_path!;!new!" /f &gt;nul 2&gt;&amp;1
    echo  已添加到用户 PATH &gt;&gt; "%LOG_FILE%"
```

```
)
exit /b
```

安装向导各阶段实现清单（25 项）

安装阶段	典型任务	BAT 实现方式	错误处理	回滚策略	时间估算	关键注意
欢迎页	显示 Logo + 版本	cls + echo 多边框	choice 输入校验	退出不写文件	<1秒	不要用 ASCII art 过大
许可协议	显示 MIT/GPL 文本	type license.txt | more	用户必须 accept	拒绝=直接退出	<1秒	choice /C YN 最可靠
目录选择	默认 vs 自定义	set /p + 权限检查	mkdir 失败提 UAC	回退到上次路径	2秒	路径含空格必须双引号
权限检查	管理员验证	net session >nul 2>nul	非管理员弹 UAC	等待用户重试	3秒	HKLM 操作必须 admin
备份旧版	xcopy /E /Y 备份	xcopy 或 robocopy	磁盘不足警告	跳过备份继续	5-10 秒	备份目录用 %random% 唯一
文件复制	主程序+库文件	xcopy /E /Y /Q	copy 失败重试3次	恢复备份	10-30 秒	robocopy 更稳（可重试）
快捷方式	开始菜单入口	WScript.Shell COM	COM 调用失败	跳过（非关键）	2秒	PowerShell 调用更可靠
注册表写入	安装信息持久化	reg add /v /t /d /f	reg add 访问拒绝	跳过（非关键）	1秒	HKCU 比 HKLM 权限松
PATH 更新	加入命令搜索路径	reg query + 拼接	PATH 过长警告	放弃添加	1秒	用 reg query 读当前值
DLL 注册	COM 组件注册	regsvr32 /s	regsvr32 失败	跳过	3秒	64位系统区分 syswow64
服务注册	后台服务安装	sc create + sc config	sc 失败需 admin	sc delete 回滚	5秒	sc 命令比 net 服务名精确
防火墙	放行应用端口	netsh advfirewall	netsh 失败	跳过	3秒	需管理员权限
计划任务	开机/定时启动	schtasks /create /sc	schtasks 失败	schtasks /delete	5秒	/ru /rp 存储凭据
文件关联	扩展名绑定应用	assoc + ftype	assoc 失败	恢复默认	2秒	assoc .ext=ftype_name

卸载入口	控制面板显示	HKLM\Uninstall\Key	写注册表失败	跳过	1秒	DisplayIcon 指向 exe
日志记录	全流程审计	>> logfile.txt	磁盘只读提示	不影响安装	<1秒	每步写一条方便排查
进度显示	ASCII 进度条	set /p + 延迟扩展	不适用	不适用	实时	ping -n 做伪延迟
异常中断	Ctrl+C 处理	if errorlevel 1 goto :CANCEL	提示已取消	已备份可恢复	即时	trap Ctrl+C 靠 if errorlevel
清理	临时文件删除	del /q /f /s	删除失败跳过	不影响	1秒	用 2>nul 抑制错误
完成页	总结 + 启动选项	choice /C YN	choice 校验	启动前再确认	<1秒	可以显示"查看日志"
静默安装	无人值守参数	if "%~1"=="/S"	参数校验	不适用	跳过UI	必须支持 /S 参数
卸载脚本	反向安装流程	保存 backup 信息	卸载失败提示	提示手动卸载	10-30秒	记录 uninstall.log
升级检测	版本判断	wmic qfe / version	无法读取则跳过	手动升级	3秒	注册表 Version 字段
回滚	从 backup 恢复	xcopy /E /Y 反向	恢复失败警告	提示手动恢复	10-30秒	只回滚已成功的步骤
签名验证	文件完整性校验	certutil -hashfile + 比对	哈希不匹配	拒绝安装	2-5秒	SHA256 比 MD5 安全

BAT 写安装程序最大的挑战是权限问题。写到 HKLM 注册表、安装到 Program Files、注册系统服务——这些都需要管理员权限。标准做法是用 net session >nul 2>nul 检测当前是否 admin，如果不是就调用 powershell Start-Process -Verb RunAs 提权重启自身。注意参数传递要用 /c %~f0 %* 保留原参数。

PATH 更新不能直接 set PATH=xxx——那只在当前进程有效。持久化 PATH 要写注册表 HKCU\Environment\PATH。读取时也要用 reg query 而不是 %PATH%，否则你读到的是旧值。还要注意 Windows 的 PATH 长度限制是 1024 字符，超过的话 trunc 会丢条目。

安装脚本一定要做旧版备份！用 backup_%random% 作为目录名可以保证唯一不冲突。回滚时只要把备份目录反向 xcopy 回安装目录就行。商业安装程序（如 Inno Setup、NSIS）也都是这个思路——先备份旧文件，再写新文件，失败则恢复。

静默安装（/S 参数）是企业部署的标配。BAT 脚本开头用 if "%~1"=="/S" goto :SILENT 检测，然后跳过所有 choice 和 set /p，自动采用默认值。这种设计让同一个安装包既能交互式安装，又能被 SCCM/GPO/批处理批量部署。

安装日志是出问题后的救命稻草。每一步都用 >> logfile.txt 追加写入，时间戳用[%date% %time%]。不要用 > 覆盖——一旦中途崩溃就失去所有上下文。日志里要记录"开始某步"、"某步成功"、"某步失败原因"三级信息。

第3章 批处理的执行机制

3.1 cmd.exe 的解析流程

理解 cmd.exe 如何解析 BAT 文件，是写出高级脚本的前提。cmd 的解析分为**两个阶段**:

第一阶段：预处理（预处理扫描）

- 识别括号块 (...)、for 循环、if 语句的整体结构
- 在括号块内进行**百分号变量展开**（%var% 替换成实际值）
- 处理转义字符（^）

第二阶段：执行

- 按顺序执行每条命令
- 在执行阶段进行**延迟扩展**（!var!）

> **WARNING**
> 这两个阶段的存在，导致了 BAT 中最经典的"陷阱"：在括号块内用 %var% 看不到变量的更新。这就是"延迟扩展"（enabledelayedexpansion）存在的原因。我们会在第 4 章详细展开。

3.2 exit 的两种形式

`exit` 有两种用法，效果天差地别:

形式	作用域	效果
exit	关闭整个 cmd.exe 进程	所有 cmd 窗口都会关闭
exit /b	仅退出当前 BAT 脚本	返回调用处，不关闭 cmd 窗口

```
@echo off
```

```
call sub.bat
echo sub.bat 已返回
pause
```

在 sub.bat 中：

```
@echo off
echo 我是子脚本
exit /b
echo 这行不会被执行
```

如果 sub.bat 里写的是不带 /b 的 `exit`，那么整个 cmd 窗口都会关闭，主脚本里 echo 那行永远不会运行。**写 BAT 时 99% 的情况应该用 exit /b**。

3.3 退出码与 errorlevel

每条命令执行完毕后，会返回一个"退出码"（exit code），保存在 `%errorlevel%` 中。• 0 表示成功

• 非 0 表示失败或不同的状态

```
@echo off
copy file1.txt file2.txt
if errorlevel 1 (
   echo 复制失败了!
) else (
   echo 复制成功
)
```

`if errorlevel 1` 的含义是"如果 errorlevel >= 1"，即任何非零值。这是 BAT 中检查命令是否成功的标准模式。

TIP

技巧：可以在脚本中用 `set errorlevel=` 手动清除 errorlevel，或者用 `exit /b N` 返回自定义的退出码（N 是数字），供调用方检查。

3.4 命令的搜索优先级

当你在 BAT 中写一条命令（比如 `echo`）时，cmd 按以下顺序查找这条命令：

① **别名** —— doskey 定义的命令别名（默认没有）

② **内部命令** —— cmd.exe 内置的命令（echo、set、if、for、goto、call 等）

③ **.COM** —— 当前目录下的 .com 文件

④ **.EXE** —— 当前目录下的 .exe 文件

⑤ **.BAT / .CMD** —— 当前目录下的 .bat / .cmd 文件

⑥ **PATH 目录** —— 按 path 环境变量列出的目录，依次重复 ③~⑤

> **WARNING**
>
> 这就是为什么你**不能**把自己的 BAT 文件命名为 echo.bat 或 if.bat——因为 echo 是内部命令，会被优先匹配，你的脚本永远不会被调用。同理，不要让你的脚本名和 path 中的 .exe 重名。

3.5 call / goto 的底层逻辑

goto :label —— 改变脚本的执行顺序，跳转到标签位置继续执行。

call :label —— 调用"内部子程序"，跳转到标签并把调用位置压入栈，遇到 `exit /b` 或 `goto :eof` 后回到调用处。

```
@echo off
echo 开始
call :sub1
echo 返回了
call :sub2
echo 全部完成
pause

:sub1
echo 我是子程序1
call :sub_sub
echo 从 sub_sub 返回
goto :eof

:sub_sub
echo 嵌套调用
goto :eof
```

```
:sub2
echo 我是子程序2
exit /b
```

`goto :eof` 是一个特殊标签，意思是"跳到文件末尾"——等价于 exit /b。

3.6 常见执行陷阱

路径中包含空格：路径必须用双引号包裹，否则会被当作多个参数。

```
rem 错误写法
copy C:\Program Files\a.txt D:\

rem 正确写法
copy "C:\Program Files\a.txt" D:\
```

权限不足：某些操作（写 C:\Program Files、改注册表 HKLM 分支）需要管理员权限。右键 cmd → 以管理员身份运行，或用第 11 章介绍的 UAC 提权代码。

命令被同名文件覆盖：如果当前目录有一个 echo.bat，cmd 在搜索内部命令之前不会先找它。但如果有 dir.bat，而你想调用系统的 dir 命令，可以用 `%SystemRoot%\System32\dir.exe` 显式指定路径。

编码不匹配：UTF-8 文件没加 chcp 65001、GBK 文件被 UTF-8 cmd 打开 → 中文乱码。解决方案见第 2.6 节和第 11.7 节。

— 第3章 完 —

进阶练习

练习1：验证逐行执行机制：

```
@echo off
echo 第1行正常
nonexistent_cmd
echo 第3行能到吗？
echo errorlevel=%errorlevel%
pause
```

练习2：call vs goto 区别：

```
@echo off
echo 主程序开始
call :Sub1
echo 从 Sub1 回来了
goto :End
:Sub1
echo Sub1 里
exit /b
:End
echo 结束
pause
```

练习3：exit /b 返回值控制：

```
@echo off
call :CheckFile "C:\Windows\notepad.exe"
echo 返回码: %errorlevel%
call :CheckFile "C:\不存在.txt"
echo 返回码: %errorlevel%
pause
:CheckFile
if exist "%~1" (
```

```
    echo 存在: %~1
    exit /b 0
) else (
    echo 不存在
    exit /b 1
)
```

疑难解答

Q1: call 和 goto 什么区别?

A: call 调用子程序可返回（类函数）； goto 跳转后回不来。

Q2: 脚本跑到一半停了?

A: 检查括号块是否闭合、errorlevel 判断、或某个命令失败导致后续被跳过。

Q3: 出错时让脚本停止?

A: 每步后加 if errorlevel 1 exit /b，或 command || exit /b。

进阶主题1：子程序调用链（A->B->C 嵌套）

call 可以无限嵌套（栈深度有限），每一层都可以用 setlocal/endlocal 创建独立变量作用域。exit /b 只退出当前层，回到上一层 call 之后继续执行。这是 BAT 实现"函数"的核心机制。

```
@echo off
echo === 子程序调用链 A-^&gt;B-^&gt;C ===
call :SubA
echo 主程序结束
pause
exit /b
:SubA
echo  SubA 开始
call :SubB
echo  SubA 继续（SubB 返回后）
exit /b
```

```
:SubB
echo    SubB 开始
call :SubC
echo    SubB 继续（SubC 返回后）
exit /b
:SubC
echo      SubC 开始
echo      SubC 做一些事...
setlocal
set "var=仅在 SubC 内可见"
echo      SubC 里 var=!var!
endlocal
echo      SubC 结束
exit /b
echo 这行永远不会执行（exit /b 之前）
```

进阶主题2：条件执行的短路行为

&& 和 || 在 BAT 里是短路运算符：&& 前命令成功才执行后命令，|| 前命令失败才执行后命令。利用这个特性可以写出优雅的错误处理链，比如 build && test && deploy。

```
@echo off
echo === 条件执行的短路行为 ===
echo.
echo --- 1. ^&amp;^&amp; 短路 (前失败则后不执行) ---
dir notexist 2&gt;nul ^&amp;^&amp; echo 前面成功了（这行不应出现）
echo 上面的 echo 没有执行，因为 dir 失败了
echo.
echo --- 2. ^|^| 短路 (前成功则后不执行) ---
dir "%SystemRoot%\notepad.exe" 2&gt;nul ^|^| echo 前面失败了（这行不应出现）
echo 上面的 echo 没有执行，因为 dir 成功了
echo.
echo --- 3. 复合错误处理 ---
echo 尝试创建目录...
```

```
mkdir testdir 2&gt;nul ^&amp;^&amp; (
    echo 目录创建成功
    cd testdir ^&amp;^&amp; echo 进入目录
) ^|^| (
    echo 目录创建失败或已存在
    cd testdir 2&gt;nul ^&amp;^&amp; echo 进入已有目录
)
echo 清理...
cd ..
rmdir testdir 2&gt;nul
echo.
echo --- 4. 实际应用: 链式构建 ---
echo git pull ^&amp;^&amp; npm install ^&amp;^&amp; npm test ^&amp;^&amp; npm run build
echo 这就是为什么写 ^&amp;^&amp; 链：任一步失败就停下
pause
```

进阶主题3：goto 和 call 的混合使用

goto 负责粗粒度流程（菜单跳转、循环控制），call 负责细粒度功能（日志、校验、转换）。这是 BAT 大型脚本的标准架构模式。不要把 goto 用在子程序上——跳出去就回不来了。

```
@echo off
echo === goto 和 call 的混合使用 ===
echo.
echo --- 场景: 主循环 + 通用子程序 ---
:MainLoop
cls
echo [1] 选项A  [2] 选项B  [3] 选项C  [0] 退出
set /p "choice=选择: "
if "%choice%"=="0" goto :End
if "%choice%"=="1" goto :DoA
if "%choice%"=="2" goto :DoB
if "%choice%"=="3" goto :DoC
echo 无效选择 &amp; pause &amp; goto :MainLoop
```

```
:DoA
echo 执行A...
call :Log "执行了A"
pause
goto :MainLoop
:DoB
echo 执行B...
call :Log "执行了B"
pause
goto :MainLoop
:DoC
echo 执行C...
call :Log "执行了C"
pause
goto :MainLoop
:Log
echo [%date% %time%] %~1 &gt;&gt; activity.log
exit /b
:End
echo 退出程序
echo.
echo === goto vs call 总结 ===
echo goto: 跳转，不返回，适合菜单、循环、错误处理
echo call: 调用，返回，适合通用子程序、模块化
echo 混用: goto 做流程控制，call 做功能模块
if exist activity.log del activity.log
pause
```

表 3-1 call vs goto 对比表

对比项	call :label	goto :label	说明

执行后是否返回	是，返回原位置	否，跳转不返回	call 会记住返回点
传参支持	是，%~1 %~2	否	goto 不带参数
返回值	exit /b N 设置 errorlevel	无独立返回	call 后可取 errorlevel
调用深度	可多层嵌套（栈）	单向跳转	call 有栈深度限制
局部变量	配合 setlocal/endlocal	无效	call 内 setlocal 可隔离
递归调用	支持但需小心栈溢出	自然尾递归	call 递归深度 ~100
退出子流程	exit /b 0	goto :eof 或 goto 下一标签	exit /b 等价 goto :eof
标签名命名	call :func_name	goto :line_marker	标签可以用 : 或不用
性能开销	略高（栈操作）	低（直接跳转）	现代系统差异不大
栈内存	每次 call 消耗少量栈	不消耗	不要在循环里 call 一万次
调试追踪	call 栈层次清楚	goto 可能成意大利面条	大型脚本用 call 替代 goto
错误处理	call 后检查 errorlevel	需要自己用变量标记	if errorlevel 2 echo failed
批处理中 goto :eof	从 call 返回	结束整个脚本	！关键区别!
循环控制	循环体内可 call 子程序	循环体内 goto 会跳出循环	call 不影响 for 循环

表 3-2 子程序最佳实践清单

最佳实践	推荐写法	反面示例	原因
子程序用 call	call :backup	goto :backup; :backup ... goto :eof	call 自动返回
传参用 %~N	call :func %var1% %var2%	call :func "text with spaces"	~ 自动去引号
返回值用 exit /b	:func ... exit /b 1	set "ret=val"; goto :eof	errorlevel 更标准
局部变量隔离	:func setlocal set x=1 ... endlocal exit /b 0	:func set x=1	防止污染外层变量
大脚本分模块	主文件 + call include\module.bat	一个 1000 行巨型 BAT	可维护性大幅提升
主流程末尾用 goto :eof	主流程末尾 goto :eof :func ... exit /b 0	主流程直接 fall-through 到子程序	避免意外执行子程序
避免深层嵌套 call	call :a → call :b → call :c	10 层以上嵌套	BAT 栈有限，过深会崩溃

递归要有终止条件	:factorial %n% if %n%==1 exit /b 1 call :factorial %n-1%	没有终止条件	栈溢出崩溃
goto 只用于简单跳转	goto loop :loop ...	goto 跨多层 call	可读性差
参数校验	:func if "%~1"=="" exit /b 9	:func set val=%~1	避免空参数导致语法错误
返回码统一	0=成功 1=参数错 2=文件不存在	随意返回数字	便于调用方判断
日志子程序封装	call :log INFO "msg"	echo ...	统一日志格式
setlocal/endlocal 成对	beginlocal ... endlocal	只写 setlocal	泄漏环境
call 后检查 errorlevel	call :func if errorlevel 1 goto fail	call :func	及时发现错误
标签名见名知意	:backup_files :check_connect	:a :b :sub	调试时知道跳到哪

第 3 章小结

本章解决了 BAT 脚本"从一堆命令"升级到"结构化程序"的关键问题——call、goto 和 exit /b 的配合。新手最核心的认知突破是：call :func 后 func 里的 exit /b 只退出子程序，而 goto :eof 从 call 返回；如果没有 call 直接 goto :eof 则结束整个脚本。

表 3-1 总结了 call 和 goto 在 15 个维度上的差异。最重要的三点：call 会记住返回点，goto 不会；call 能传参用 %~N 接收；call 后 errorlevel 可作为返回值。记住这三条就能写清晰的模块化脚本了。

表 3-2 给出了 15 条子程序最佳实践，其中最容易被忽略的是"主流程末尾用 goto :eof"——如果不写 goto :eof，脚本会 fall-through 到下面的子程序标签，第一次运行还好，call 返回后就会接着执行，造成意料外的行为。

模块化框架示例展示了大型脚本的典型组织方式：call :init 初始化 → call :show_menu 菜单循环 → 每个功能一个 call 子程序。日志、备份、恢复、清理、诊断全都拆成独立子程序，改一处不影响别处。

call vs goto 测试脚本对比了 goto 不返回、call 会返回、传参、变量作用域和 errorlevel 返回值五个典型场景。建议读者亲手跑这个脚本并观察输出，比读十页书印象更深。

下一章进入变量的修饰符和环境变量。%~dp0、%~nx1 这些路径修饰符是 BAT 脚本最常用的技巧之一，必须烂熟于心。

第 3 章大代码块 1：模块化 BAT 框架

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ==============================================
echo   第3章综合实战：模块化 BAT 框架 v1.0
echo   用 call/goto 构建可维护的大型脚本
echo ==============================================
echo.
rem === 主入口 ===
call :init
call :show_menu
goto :eof
:init
set "SCRIPT_NAME=Modular Framework"
set "SCRIPT_VERSION=1.0"
set "SCRIPT_DATE=2026-11-10"
set "workdir=%~dp0workspace"
if not exist "!workdir!" mkdir "!workdir!"
call :log INFO "脚本初始化完成"
exit /b 0
:show_menu
cls
echo  ============================================
echo    !SCRIPT_NAME! v!SCRIPT_VERSION!
echo    工作目录: !workdir!
echo  ============================================
echo.
echo    [1] 创建备份
echo    [2] 清理旧备份
echo    [3] 查看备份列表
```

```
echo    [4] 恢复备份
echo    [5] 系统诊断
echo    [0] 退出
echo.
set /p "choice=请选择: "
if "!choice!"=="1" call :do_backup
if "!choice!"=="2" call :do_cleanup
if "!choice!"=="3" call :do_list
if "!choice!"=="4" call :do_restore
if "!choice!"=="5" call :do_diagnose
if "!choice!"=="0" goto :eof
call :show_menu
exit /b 0
:do_backup
set "backup_name=backup_%date:~0,4%%date:~5,2%%date:~8,2%_%time:~0,2%%time:~3,2%%time:~6,2%"
set "backup_name=!backup_name: =0!"
set "backup_dir=!workdir!\!backup_name!"
mkdir "!backup_dir!"
echo  正在备份 C:\Users\Public\Documents ...
xcopy /e /i /q "C:\Users\Public\Documents" "!backup_dir!" 2&gt;nul
call :log INFO "备份完成: !backup_name!"
echo  备份完成: !backup_dir!
pause
exit /b 0
:do_cleanup
echo  正在清理 7 天前的旧备份...
forfiles /p "!workdir!" /m backup_* /d -7 /c "cmd /c rd /s /q @path" 2&gt;nul
call :log INFO "旧备份清理完成"
echo  清理完成
pause
exit /b 0
:do_list
```

```
echo  === 当前备份列表 ===
dir /b /ad "!workdir!" 2&gt;nul
echo.
echo  总大小:
dir /s /b "!workdir!" 2&gt;nul | find /c /v ""
pause
exit /b 0
:do_restore
call :do_list
set /p "target=请输入要恢复的备份名: "
if not exist "!workdir!\!target!" (
    echo  备份不存在: !target!
    exit /b 1
)
echo  恢复备份 !target! 到 C:\Users\Public\Documents ...
xcopy /e /i /q /y "!workdir!\!target!" "C:\Users\Public\Documents"
call :log INFO "已恢复: !target!"
echo  恢复完成
pause
exit /b 0
:do_diagnose
echo  === 系统诊断 ===
call :check_disk
call :check_mem
call :check_net
call :check_services
echo  === 诊断完毕 ===
pause
exit /b 0
:check_disk
echo  --- 磁盘检查 ---
wmic logicaldisk where "freespace &lt; 1073741824" get caption,freespace 2&gt;nul
```

```
exit /b 0
:check_mem
echo  --- 内存 Top 5 ---
tasklist /sort "memusage" /fo table 2&gt;nul | findstr /n "." | findstr "^[1-5]:"
exit /b 0
:check_net
echo  --- 网络连通性 ---
ping -n 2 8.8.8.8 &gt;nul 2&gt;&amp;1 &amp;&amp; echo    [OK] 外网可达 || echo    [FAIL] 网络异常
exit /b 0
:check_services
echo  --- 关键服务状态 ---
for %%s in (W32Time wuauserv Schedule) do (
    sc query %%s | findstr "STATE"
)
exit /b 0
:log
set "lvl=%~1" &amp; set "msg=%~2"
set "ts=%date:~0,4%-%date:~5,2%-%date:~8,2% %time:~0,8%"
set "ts=!ts: =0!"
echo [!ts!] [!lvl!] !msg!
exit /b 0
```

第 3 章大代码块 2：call vs goto 深入对比测试

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第3章进阶：call vs goto 深入对比测试
echo   理解两者在执行流和返回上的差异
echo ============================================
echo.
```

```
rem === 测试 1: goto 会改变执行流，不返回 ===
echo [测试 1] goto 不返回:
echo  主流程开始
goto skip_line
echo  这行不会被执行
:skip_line
echo  主流程继续
echo.
rem === 测试 2: call 调用后会返回 ===
echo [测试 2] call 返回:
echo  主流程开始
call :sub_test
echo  主流程继续（call 返回后执行）
echo.
rem === 测试 3: call 可以传参 ===
echo [测试 3] call 传参:
call :add_numbers 10 20
call :add_numbers 100 50
echo.
rem === 测试 4: 变量作用域 ===
echo [测试 4] 变量作用域（setlocal）:
set "var=外层值"
echo  进入前 var=%var%
call :scope_test
echo  调用后 var=!var!
echo.
rem === 测试 5: errorlevel 返回值 ===
echo [测试 5] 返回值测试:
call :return_test 1
echo  errorlevel=!errorlevel!
call :return_test 5
echo  errorlevel=!errorlevel!
```

```
call :return_test 99
echo  errorlevel=!errorlevel!
pause
exit /b 0
:sub_test
echo    子程序执行中...
exit /b 0
:add_numbers
set /a result=%~1 + %~2
echo    %~1 + %~2 = %result%
exit /b 0
:scope_test
setlocal
set "var=内层值"
echo    内层 var=%var%
endlocal
exit /b 0
:return_test
echo    收到参数: %~1
exit /b %~1
```

第 3 章进阶项目：通用批处理工具箱

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第3章项目：通用批处理工具箱
echo   收集最常用的 BAT 子程序，开箱即用
echo ============================================
echo.
rem 使用说明: call :函数名 参数1 参数2
```

```
rem 所有函数都用 exit /b 返回
call :box_draw "工具箱已加载" 50
echo.
call :progress_bar 60
echo.
call :spinner_test
echo.
call :file_size "C:\Windows\System32\cmd.exe"
echo.
call :dir_size "%temp%"
echo.
call :base64_test
echo.
pause
exit /b 0
:box_draw
set "text=%~1" &amp;amp; set /a width=%~2
set /a inner=width - 2
&amp;lt;nul set /p "=+"
for /L %%i in (1,1,!inner!) do &amp;lt;nul set /p "=-"
echo +
&amp;lt;nul set /p "=|"
for /L %%i in (1,1,!inner!) do &amp;lt;nul set /p " "
echo |
set /a pad_left=(inner - -1) / 2
&amp;lt;nul set /p "=|  !text!"
set /a remaining=inner - 2 - -1
for /L %%i in (1,1,!remaining!) do &amp;lt;nul set /p " "
echo |
&amp;lt;nul set /p "=|"
for /L %%i in (1,1,!inner!) do &amp;lt;nul set /p " "
echo |
```

```
&lt;nul set /p "=+"
for /L %%i in (1,1,!inner!) do &lt;nul set /p "=-"
echo +
exit /b 0
:progress_bar
set /a percent=%~1
set /a filled=percent / 2
set /a empty=50 - filled
&lt;nul set /p "=  ["
for /L %%i in (1,1,!filled!) do &lt;nul set /p "=#"
for /L %%i in (1,1,!empty!) do &lt;nul set /p "-"
echo ] !percent!%%
exit /b 0
:spinner_test
echo --- Spinner 演示 ---
set "chars=| / - \"
for /L %%i in (1,1,20) do (
    for /f "tokens=%%i" %%c in ("!chars!") do (
        &lt;nul set /p "=  处理中 %%c\r"
    )
    ping -n 2 127.0.0.1 &gt;nul
)
&lt;nul set /p "=  完成!         \r"
echo.
exit /b 0
:file_size
set "f=%~1"
if not exist "!f!" (
    echo  文件不存在: !f!
    exit /b 1
)
for %%F in ("!f!") do (
```

```
    echo  文件: %%~nxF
    echo  大小: %%~zF 字节
    set /a "kb=%%~zF / 1024"
    echo       !kb! KB
)
exit /b 0
:dir_size
set "d=%~1"
if not exist "!d!" (
    echo  目录不存在: !d!
    exit /b 1
)
set /a total=0
for /f "delims=" %%f in ('dir /s /a-d /b "!d!" 2^&gt;nul') do (
    set /a total+=(%%~zf / 1024)
)
echo  目录 !d! 总大小: !total! KB
exit /b 0
:base64_test
echo  --- 用 certutil 做 Base64 编解码 ---
echo Hello World &gt; "%temp%\test_input.txt"
certutil -f -encodehex "%temp%\test_input.txt" "%temp%\test_hex.txt" 4 &gt;nul 2&gt;&amp;1
echo  原始内容:
type "%temp%\test_input.txt"
echo  HEX 编码:
type "%temp%\test_hex.txt"
del /q "%temp%\test_input.txt" "%temp%\test_hex.txt"
exit /b 0
```

进阶项目3：子程序库 - 10 个常用工具函数

这是一份可以直接放进你的每个 BAT 脚本的公共子程序库。10 个函数覆盖了 BAT 开发中最高频的操作：检测管理员、获取系统版本、日志输出、安全复制（带重试）、文件大小、子串检测、字符串处理、时间戳、随机数、Base64 编码。每个函数都设计成 endlocal & set 的经典传值模式。

```
@echo off
setlocal enabledelayedexpansion
rem ============================================================
rem  BAT 公共子程序库 - 10 个常用工具函数
rem  用法: 在你的脚本开头 call "%~f0" /LIBRARY 跳过主函数
rem       或者直接把这些 :FunctionName 复制到你的脚本里
rem ============================================================

rem ========== 主程序入口 ==========
if /i "%~1"=="/LIBRARY" goto :EOF

echo === BAT 公共子程序库 ===
echo 正在测试 10 个工具函数...
echo.

call :check_admin result
echo check_admin - 管理员权限: !result!

call :get_os_ver ver_name ver_num ver_arch
echo get_os_ver - 名称=!ver_name! 版本=!ver_num! 架构=!ver_arch!

call :log_msg INFO "这是一条 INFO 消息"
call :log_msg WARN "这是一条 WARN 消息"
call :log_msg ERROR "这是一条 ERROR 消息"

call :safe_copy "%~f0" "%temp%\test_copy.bat"
echo safe_copy - result=%errorlevel%

call :file_size "%~f0" size_kb
echo file_size - !size_kb! KB

call :str_contains "Hello World" "World" contains
echo str_contains - "Hello World" 包含 "World": !contains!
```

```
call :str_trim "   hello   " trimmed
echo str_trim - 原= "   hello   " 去空格后= "!trimmed!"

call :timestamp ts
echo timestamp - !ts!

call :rand_int 1 100 rand_val
echo rand_int - 1-100 随机数: !rand_val!

call :base64_encode "test input" encoded
echo base64_encode - !encoded!

del /q "%temp%\test_copy.bat" 2&gt;nul
echo.
echo === 所有函数测试完毕 ===
pause
exit /b

rem ============================================================
rem 函数 1: check_admin - 检测管理员权限
rem 用法: call :check_admin var_name
rem 返回: 1=管理员, 0=非管理员
rem ============================================================
:check_admin
setlocal
net session &gt;nul 2&gt;&amp;1
set /a admin=0
if !errorlevel! equ 0 set /a admin=1
endlocal &amp; set "%~1=%admin%"
exit /b

rem ============================================================
rem 函数 2: get_os_ver - 获取操作系统版本信息
rem 用法: call :get_os_ver out_name out_version out_arch
rem ============================================================
:get_os_ver
```

```
setlocal
set "_name=Unknown"
set "_ver=Unknown"
set "_arch=%PROCESSOR_ARCHITECTURE%"
for /f "tokens=2 delims==" %%a in ('wmic os get Caption /format:list 2^&gt;nul ^| findstr Caption') do set "_name=%%a"
for /f "tokens=2 delims==" %%b in ('wmic os get Version /format:list 2^&gt;nul ^| findstr Version') do set "_ver=%%b"
endlocal &amp; set "%~1=%_name%" &amp; set "%~2=%_ver%" &amp; set "%~3=%_arch%"
exit /b

rem ============================================================
rem 函数 3: log_msg - 带时间戳的日志输出
rem 用法: call :log_msg LEVEL "消息内容"
rem 日志同时输出到控制台和 %LOG_FILE%
rem ============================================================
:log_msg
setlocal
set "_level=%~1"
set "_msg=%~2"
for /f "tokens=*" %%t in ('time /t') do set "_time=%%t"
set "_line=[%date% !_time!] [!_level!] !_msg!"
if /i "!_level!"=="ERROR" echo. &amp; echo ^&gt;^&gt;^&gt; !_line! &amp; echo.
if /i not "!_level!"=="ERROR" echo !_line!
if defined LOG_FILE echo !_line! &gt;&gt; "%LOG_FILE%"
endlocal
exit /b

rem ============================================================
rem 函数 4: safe_copy - 带重试的安全文件复制
rem 用法: call :safe_copy "源路径" "目标路径"
rem 返回: errorlevel 0=成功, 1=失败
rem ============================================================
:safe_copy
setlocal
```

```
set "_src=%~1"
set "_dst=%~2"
set /a _retries=3
set /a _delay=2
set /a _attempt=0
:safe_copy_loop
set /a _attempt+=1
if not exist "!_src!" (
    echo [safe_copy] 源文件不存在: !_src!
    endlocal &amp; exit /b 1
)
copy /y "!_src!" "!_dst!" &gt;nul 2&gt;&amp;1
if !errorlevel! equ 0 (
    endlocal &amp; exit /b 0
)
if !_attempt! lss !_retries! (
    echo [safe_copy] 第 !_attempt! 次失败，等待 !_delay! 秒重试...
    ping -n !_delay! 127.0.0.1 &gt;nul
    goto :safe_copy_loop
)
echo [safe_copy] 3 次重试均失败
endlocal &amp; exit /b 1

rem ============================================================
rem 函数 5: file_size - 获取文件大小
rem 用法: call :file_size "文件路径" out_var
rem 返回: KB 为单位的整数
rem ============================================================
:file_size
setlocal
set "_kb=0"
if exist "%~1" (
```

```
    for %%a in ("%~1") do set /a "_kb=%%~za/1024"
)
endlocal &amp;amp; set "%~2=%_kb%"
exit /b

rem ============================================================
rem 函数 6: str_contains - 子串检测
rem 用法: call :str_contains "原串" "查找" out_var
rem 返回: 1=包含, 0=不包含
rem ============================================================
:str_contains
setlocal
set "_hay=%~1"
set "_needle=%~2"
set "_result=0"
echo "!_hay!" | find /i "!_needle!" &amp;gt;nul 2&amp;gt;&amp;amp;1
if !errorlevel! equ 0 set "_result=1"
endlocal &amp;amp; set "%~3=%_result%"
exit /b

rem ============================================================
rem 函数 7: str_trim - 去除首尾空格
rem 用法: call :str_trim "  hello  " out_var
rem ============================================================
:str_trim
setlocal
set "_s=%~1"
:trim_left
if "!_s:~0,1!"==" " set "_s=!_s:~1!" &amp;amp; goto :trim_left
:trim_right
if "!_s:~-1!"==" " set "_s=!_s:~0,-1!" &amp;amp; goto :trim_right
endlocal &amp;amp; set "%~2=%_s%"
exit /b
```

```
rem ============================================================
rem 函数 8: timestamp - 生成时间戳字符串
rem 用法: call :timestamp out_var
rem 返回: 20261110_143025 格式
rem ============================================================
:timestamp
setlocal
set "_ts=%date:~0,4%%date:~5,2%%date:~8,2%_%time:~0,2%%time:~3,2%%time:~6,2%"
set "_ts=!_ts: =0!"
endlocal &amp;amp; set "%~1=%_ts%"
exit /b

rem ============================================================
rem 函数 9: rand_int - 生成范围内随机整数
rem 用法: call :rand_int min max out_var
rem ============================================================
:rand_int
setlocal
set /a _min=%~1
set /a _max=%~2
set /a _range=_max-_min+1
set /a _val=%random% %% _range + _min
endlocal &amp;amp; set "%~3=%_val%"
exit /b

rem ============================================================
rem 函数 10: base64_encode - Base64 编码字符串
rem 用法: call :base64_encode "内容" out_var
rem ============================================================
:base64_encode
setlocal
set "_tmp=%temp%\b64_in_%random%.txt"
echo|set /p "=%~1" &amp;gt; "!_tmp!"
```

```
set "_encoded="
for /f "usebackq delims=" %%l in (`certutil -encode "!_tmp!" ^| findstr /v /c:"-----"`) do (
    set "_encoded=!_encoded!%%l"
)
del /q "!_tmp!"
endlocal &amp;amp; set "%~2=%_encoded%"
exit /b
```

可复用子程序清单（25 个扩展函数）

函数名	用途	参数	返回值	实现原理	依赖	典型调用	注意事项
check_admin	检测管理员权限	(out_var)	1=admin 0=no	net session >nul 返回码	无	call :check_admin is_admin	HKLM 操作前必调
get_os_ver	获取 OS 版本详情	(out_name out_ver out_arch)	三个变量	wmic os get 三个字段	wmic	call :get_os_ver n v a	wmic 可能被安全软件禁用
log_msg	带时间戳的日志	LEVEL "msg"	无（写 stdout+文件）	echo + 时间拼接	无	call :log_msg INFO "ok"	需先 set LOG_FILE
safe_copy	带重试的复制	"src" "dst"	errorlevel	copy + for 循环重试	copy	call :safe_copy a.txt b.txt	3次重试，ping 延迟
file_size	文件大小(KB)	"path" out_var	KB 整数	%%~za / 1024	for %%, dir	call :file_size a.txt size	%%~za 返回字节
str_contains	子串检测	"hay" "needle" out	1/0	echo | find /i	find	call :str_contains abcd bc r	find 忽略大小写用 /i
str_trim	去首尾空格	" s " out_var	去空格后的串	循环 :~0,1! 逐个弹掉	延迟扩展	call :str_trim " x " r	只能去半角空格
timestamp	生成时间戳	out_var	YYYYMMDD_HHMMSS	date/time 子串拼接	无	call :timestamp ts	date 格式依赖系统区域
rand_int	范围随机整数	min max out_var	范围内整数	%random% %% range	%random%	call :rand_int 1 10 r	模运算小偏差可接受
base64_encode	Base64 编码	"str" out_var	编码串	certutil -encode 临时文件	certutil	call :base64_encode hi r	certutil 是原生工具
base64_decode	Base64 解码	"b64" out_var	原串	certutil -decode 临时文件	certutil	call :base64_decode aGVsbG8= r	先写临时文件再解

str_replace	子串替换	"s" "old" "new" out	替换后字符串	!s:old=new! 内置替换	延迟扩展	call :str_replace ab a x r	BAT 原生支持，最安全
file_ext	提取文件扩展名	"path" out_var	扩展名含点	%%~x 修饰符	for %%, cmd	call :file_ext file.zip e	%%~x 包括点号
file_dir	提取目录部分	"path" out_var	目录含 \	%%~dp 修饰符	for %%, cmd	call :file_dir C:\a\b d	%%~dp 末尾有反斜杠
path_join	拼接路径段	"a" "b" out_var	a\b	简单字符串拼接	无	call :path_join C:\ a\b r	注意双反斜杠
is_dir	判断目录存在	"path" out_var	1=存在 0=不存在	if exist "path\"	cmd	call :is_dir C:\ a r	目录末尾加 \
is_file	判断文件存在	"path" out_var	1=存在 0=不存在	if exist "path"	cmd	call :is_file a.txt r	不加尾部斜杠
sleep	指定秒数等待	seconds	无（暂停）	ping -n N 127.0.0.1	ping	call :sleep 5	精度约 ±0.5 秒
get_free_port	找空闲端口	start_port out_var	可用端口号	for /L + netstat findstr	netstat	call :get_free_port 10000 p	逐个测试直到空
sha256	计算文件哈希	"path" out_var	SHA256 串	certutil -hashfile	certutil	call :sha256 a.txt h	输出需 findstr /v certutil
dir_size	目录总大小	"path" out_var	MB 整数	for /R + 累加 %%~za	延迟扩展	call :dir_size C:\Dir s	大目录可能慢
grep_count	统计匹配行数	"pattern" "file" out_var	行数	find /c /v "" + find	find	call :grep_count error log.txt c	/c /v "" 是行数技巧
ini_read	读 INI 配置	"file" section key out	值	for /f 解析 ini 格式	for /f	call :ini_read c.ini DB host r	用 [section] 过滤
clip_set	写入剪贴板	"text"	无	echo | clip	clip	call :clip_set "hi"	clip 是 Windows 自带
http_get	HTTP GET 请求	URL out_var	响应体	curl -s -o -	curl 或 PS	call :http_get url body	Windows 10+ 自带 curl

写 BAT 子程序最重要的是参数传递和返回值。BAT 没有真正的函数参数，靠 %~1、%~2 位置取值；返回值靠 endlocal & set trick——因为 BAT 解析器会先把整行的 %var% 展开，这时 endlocal 还没执行，所以值能"漏出"作用域。这是 BAT 唯一安全的传值办法。

safe_copy 里的 ping 延迟技巧（ping -n 5 127.0.0.1 >nul）是 BAT 实现 sleep 的标准 hack。timeout /t 也能用但会显示倒计时，ping 更静默。精度大约 ±0.5 秒，足够重试场景用。真正精确的延迟要用 PowerShell Start-Sleep -Milliseconds。

这个函数库还可以扩展：目录递归大小计算、INI 文件读写、HTTP 下载（用 PowerShell Invoke-WebRequest）、剪贴板操作（clip）、压缩解压（certutil + 临时文件）。但有些场景用 PowerShell 反而比 BAT 本身更简洁——BAT + PowerShell 混合调用才是王道。

函数库的组织方式：主脚本开头加 if /i "%~1"=="/LIBRARY" goto :EOF，这样其他脚本可以 call library.bat /LIBRARY 来加载所有子程序定义而不执行主程序。这是 BAT 实现 import/模块化的经典手法。

第二部分 核心语法篇

第4章 变量系统全解析

BAT 的变量系统是其最独特也最容易让人困惑的部分。本章将从基础到高级，彻底讲透 BAT 变量的方方面面。

4.1 set 命令：变量的定义、赋值与删除

`set` 是 BAT 变量的核心命令。定义和赋值的基本语法：

```
@echo off
set name=Harry
set age=25
echo 姓名: %name%
echo 年龄: %age%
echo 明年: %age%+1
```

运行结果：

```
姓名: Harry
年龄: 25
明年: 25+1
```

几个要点：

• **等号两边不能有空格**：`set name = Harry` 会把变量名当成 "name "（带空格），值为 " Harry"（带空格）

• 变量值可以包含空格，此时最好用引号：`set "name=Harry Potter"`

• 引用变量用 `%  %` 包裹

• 变量名不区分大小写：`%NAME%` 和 `%name%` 等价

> **TIP**
> 推荐写法：始终用 `set "var=value"` 的形式——双引号内不会受尾部空格影响，且等号右侧的特殊字符（&、| 等）不需要单独转义。

查看变量

`set` 不带任何参数会列出当前所有环境变量；`set name` 会列出所有以 "name" 开头的变量。

删除变量

```
set name=
```

等号后面什么都不写，就是删除变量。

4.2 本地/全局变量隔离

cmd.exe 的变量是"进程级"的——默认情况下，你在一个 cmd 窗口里设置的变量，在该窗口存活期间始终有效。`setlocal` / `endlocal` 可以创建一个"局部作用域"。

```
@echo off
set "var=全局值"
echo 进入 setlocal 前: %var%

setlocal
set "var=局部值"
echo setlocal 内: %var%
endlocal
```

```
echo endlocal 后: %var%
```

输出：

```
进入 setlocal 前: 全局值
setlocal 内: 局部值
endlocal 后: 全局值
```

endlocal 会把 setlocal 之后所有变量的改动"回滚"——这在写子程序时非常有用，可以避免子程序的变量污染主脚本。

4.3 延迟扩展：解决括号块变量更新难题

这是 BAT 最经典也最重要的知识点。请看下面的代码：

```
@echo off
set count=0
for /L %%i in (1,1,5) do (
    set /a count+=1
    echo 当前计数: %count%
)
echo 最终: %count%
```

你期望输出 1~5，但实际输出是：

```
当前计数: 0
当前计数: 0
当前计数: 0
当前计数: 0
当前计数: 0
最终: 5
```

为什么 for 循环里 %count% 始终是 0？因为 **cmd 在执行 for 循环之前，已经把括号块里所有的 %变量% 都展开成了当时的值（0）**。之后 set /a count+=1 虽然改变了 count 的值，但 %count% 不会重新展开。

> **NOTE**
>
> 回忆第 3.1 节的"两阶段解析"：预处理阶段展开 %var%，执行阶段才做变量更新。for (...) 括号块在预处理阶段就把所有 %count% 替换成了 0，之后的执行里自然看到的都是 0。

解决方案——`setlocal enabledelayedexpansion` + `!  !`：

```
@echo off
setlocal enabledelayedexpansion
set count=0
for /L %%i in (1,1,5) do (
    set /a count+=1
    echo 当前计数: !count!
)
echo 最终: !count!
endlocal
```

这次输出正确了：

```
当前计数: 1
当前计数: 2
当前计数: 3
当前计数: 4
当前计数: 5
最终: 5
```

!var! 与 **%var%** 的区别：

语法	展开时机	适用场景
%var%	预处理阶段（执行前）	括号块外部、for/if 外部
!var!	执行阶段（运行时）	setlocal enabledelayedexpansion 后的括号块内

TIP

何时启用延迟扩展？**只要在 for (...) 或 if (...) 括号块内需要读取自己刚改过的变量，就必须启用。**建议养成习惯：所有非 trivial 的 BAT 脚本开头都加上 setlocal enabledelayedexpansion。

4.4 参数修饰符：%~dp0 等

当 BAT 文件被调用时，命令行参数通过 `%1`、`%2` ... 来引用。但这些参数修饰符（%~x1）是 BAT 最强大也最难记的特性之一。

基础参数

```
@echo off
echo 参数1: %1
echo 参数2: %2
echo 参数总数: %*
echo 脚本自身: %0
```

运行：`test.bat hello world`

```
参数1: hello
参数2: world
参数总数: hello world
脚本自身: test.bat
```

修饰符详解

假设脚本路径为：`C:\Scripts\test.bat`，调用参数 `%1 = "C:\Data\input.txt"`

修饰符	含义	示例值
%~1	去掉外层双引号	C:\Data\input.txt
%~f1	完整路径 + 文件名	C:\Data\input.txt
%~d1	驱动器号	C:
%~p1	目录路径（不含驱动器号）	\\Data\\
%~n1	文件名（不含后缀）	input
%~x1	文件扩展名	.txt
%~s1	短路径（8.3格式）	C:\DATA\INPUT.TXT
%~a1	文件属性	--a------
%~t1	修改时间	2026/11/10 14:30
%~z1	文件大小（字节）	1024
%~dp0	脚本自身所在目录	C:\\Scripts\\\
%~nx0	脚本自身文件名+后缀	test.bat

TIP

最常用的几个：**%~dp0**（脚本所在目录）、**%~nx0**（脚本文件名）、**%~f1**（参数的完整路径）。组合写法：**%~dpnx0** = 驱动器 + 目录 + 文件名 + 后缀，即完整路径。

4.5 set /a：数值计算

BAT 没有原生数值类型，所有变量都是字符串。但 `set /a` 可以进行整数运算：

```
@echo off
set /a a=10+5
echo 10+5 = %a%

set /a b=a*2
echo %a% * 2 = %b%

set /a c=b/3
echo %b% / 3 = %c%

set /a d=b%%3
echo %b% %% 3 = %d%

set /a e=12 &amp;amp; 10
echo 12 AND 10 = %e%

set /a f=1 || 0
echo 1 OR 0 = %f%

set /a g+=5
echo g+=5 -&amp;gt; %g%
```

输出：

```
10+5 = 15
15 * 2 = 30
30 / 3 = 10
30 % 3 = 0
12 AND 10 = 8
1 OR 0 = 1
g+=5 -&amp;gt; 5
```

set /a 支持的运算符（按优先级）：

优先级	运算符	含义
1（最高）	( )	括号分组
2	! ~ -	逻辑非、按位取反、负号

3	* / %%	乘、除、取模
4	+ -	加、减
5	<< >>	左移、右移
6	&	按位与
7	^	按位异或
8	|	按位或
9	== != < <= > >=	比较（返回 0 或 1）
10	= += -= *= /= %%= ^= |= &= <<= >>=	赋值运算
11（最低）	,	表达式分隔

WARNING

set /a 仅支持 32 位有符号整数（-2147483648 ~ 2147483647）。不支持小数运算。如果需要小数运算，要么用 PowerShell/VBScript，要么自己模拟（如先乘后除）。

4.6 字符串操作

BAT 没有内置字符串函数，但可以通过 **变量替换语法** 实现替换和截取。

字符串替换

```
@echo off
set "text=Hello World Hello BAT"
echo 原字符串: %text%
echo 替换后: %text:Hello=嗨%
echo 去掉所有空格: %text: =%
```

语法：`%var: = %`，旧字符串替换成新字符串。新为空则删除。

字符串截取

```
@echo off
set "text=0123456789"
echo 原字符串: %text%
echo 从第5个开始取: %text:~4%
echo 从0开始取前5个: %text:~0,5%
```

```
echo 从第3个开始取4个: %text:~2,4%
echo 从末尾取3个: %text:~-3%
echo 从第2个开始,到末尾前3个: %text:~2,-3%
```

语法：`%var:~ , %`

计算字符串长度

BAT 没有 len() 函数，但可以用 for 循环 + 替换技巧：

```
@echo off
setlocal enabledelayedexpansion
set "str=Hello World"
set len=0
for /L %%i in (0,1,100) do (
   if "!str:~%%i,1!" neq "" set /a len+=1
)
echo "%str%" 的长度是: !len!
endlocal
```

4.7 内置特殊变量

cmd.exe 预定义了很多特殊变量，随时可以使用：

变量	含义	示例值
%cd%	当前工作目录	C:\Users\Harry
%date%	当前日期（格式依赖系统区域设置）	2026/11/10 二
%time%	当前时间	14:30:00.00
%random%	0~32767 之间的随机数	12345
%errorlevel%	上一条命令的退出码	0
%cmdextversion%	命令扩展版本号	2
%_cmdvarset%	命令行参数（shift 后变化）	参数1 参数2

TIP

%date% 和 %time%
的格式依赖系统的区域设置，不同电脑上可能不一样。如果需要统一格式，用 PowerShell 辅助，或用 wmic 命令：`for /f "tokens=2 delims==" %i in ('wmic os get localdatetime /value') do @set datetime=%i`

4.8 环境变量详解

除了 cmd 内置的特殊变量，Windows 还预定义了大量系统环境变量：

变量	含义
%path%	命令搜索路径（PATH 环境变量）
%pathext%	可执行文件扩展名列表
%windir% / %systemroot%	Windows 安装目录（如 C:\Windows）
%systemdrive%	系统驱动器号（如 C:）
%system%	System32 目录
%userprofile%	当前用户主目录
%appdata%	当前用户 AppData\Roaming
%localappdata%	当前用户 AppData\Local
%temp% / %tmp%	临时文件目录
%computername%	计算机名
%username%	当前用户名
%number_of_processors%	CPU 逻辑核心数
%os%	操作系统（值固定为 Windows_NT）

NOTE

path 环境变量是最重要的之一。当 cmd 找不到某条命令时，它会按 path 里列出的目录依次搜索。你可以用 `set "path=C:\MyTools;%path%"` 在脚本开头临时添加目录，用完后记得用 setlocal/endlocal 恢复。

— 第4章 完 —

进阶练习

练习1：参数修饰符全解：

```
@echo off
echo === 参数修饰符 ===
echo 脚本自身: %0
echo 完整路径: %~f0
echo 所在目录: %~dp0
echo 文件名:  %~nx0
echo 扩展名:  %~x0
echo 驱动器:  %~d0
echo 所有参数: %*
pause
```

练习2：字符串操作大集合：

```
@echo off
setlocal enabledelayedexpansion
set "str=Hello World BAT 批处理"
echo 原: !str!
echo 替换 BAT=cmd: !str:BAT=cmd!
echo 前5字符: !str:~0,5!
echo 从第6个: !str:~5!
echo 末尾5个: !str:~-5!
set len=0
for /L %%i in (0,1,100) do if "!str:~%%i,1!" neq "" set /a len+=1
echo 长度: !len!
endlocal
pause
```

练习3：set /a 位运算：

```
@echo off
set /a a=255 ^&amp;amp; 15
```

```
echo 255 AND 15 = %a%
set /a b=1 ^&lt;^&lt; 8
echo 1 左移8位 = %b%
set /a c=100
set /a "c *= (2+3)"
echo 100*5 = %c%
pause
```

练习4：环境变量完整探索：

```
@echo off
echo === 常用环境变量 ===
echo COMPUTERNAME: %computername%
echo USERNAME:     %username%
echo HOMEDRIVE:    %homedrive%
echo APPDATA:      %appdata%
echo TEMP:         %temp%
echo WINDIR:       %windir%
echo NUMBER_OF_PROCESSORS: %number_of_processors%
echo === 所有环境变量 ===
set
pause
```

疑难解答

Q1: 什么时候必须 setlocal enabledelayedexpansion?

A: for/if 括号块内需要读取刚改过的变量值时。建议只要有括号块+修改变量就加上。

Q2: 为什么 set "var=value" 比 set var=value 好?

A: 双引号防末尾空格变成变量名的一部分。set var=value 后面有空格会创建叫"var "的变量。

Q3: if errorlevel 1 有什么坑?

A: 意思是"大于等于1"不是"等于1"！精确匹配要写 if "%errorlevel%"=="1"。

进阶主题1：变量的作用域传递

BAT 默认所有变量都是全局的，子程序会直接修改主程序变量。用 setlocal 创建作用域隔离，子程序内的修改在 endlocal 后消失。经典传值技巧是 endlocal & set var=%val%，利用解析顺序把值传出作用域。

```
@echo off
setlocal enabledelayedexpansion
echo === 变量作用域传递 ===
echo.
echo --- 默认: 全局变量 ---
set "g_var=全局值"
echo 主程序: g_var=!g_var!
call :TestScope
echo 主程序返回后: g_var=!g_var! (被改了!)
echo.
echo --- setlocal: 隔离变量 ---
echo 调用带 setlocal 的子程序...
call :TestLocalScope
echo 主程序返回后: g_var=!g_var! (保持原值!)
echo.
echo --- 在 setlocal 内用 endlocal &amp;amp; set 传出 ---
echo 调用需要返回值的子程序...
call :CalcSquare 5 result
echo 主程序里: result=!result!
pause
exit /b
:TestScope
set "g_var=被子程序修改了"
exit /b
:TestLocalScope
setlocal
set "g_var=在子程序内修改"
echo 子程序内: g_var=!g_var!
```

```
endlocal
exit /b
:CalcSquare
setlocal
set /a n=%~1 * %~1
endlocal &amp;amp; set "%~2=%n%"
exit /b
```

进阶主题2：数组实现（set array[0]=x）

BAT 没有真正的数组，但用 set array[0]=xxx 的"伪数组"方案非常实用。配合 for /L 遍历，甚至能模拟关联数组 dict[key]=value。call 配合 shift 可以把数组作为参数传递。

```
@echo off
setlocal enabledelayedexpansion
echo === 数组实现 ===
echo.
echo --- 方法1: set array[n]=v ---
set "fruit[0]=苹果"
set "fruit[1]=香蕉"
set "fruit[2]=橙子"
set "fruit[3]=葡萄"
set "fruit[4]=西瓜"
echo fruit[0]=!fruit[0]!
echo fruit[3]=!fruit[3]!
echo.
echo --- 遍历数组 ---
for /L %%i in (0,1,4) do echo  fruit[%%i]=!fruit[%%i]!
echo.
echo --- 方法2: 带索引的字符串 ---
set "colors=红 橙 黄 绿 蓝 靛 紫"
set "idx=0"
for %%c in (!colors!) do (
    set "color_list[!idx!]=%%c"
    set /a idx+=1
```

```
)
echo 彩虹颜色数: !idx!
for /L %%i in (0,1,6) do echo  color_list[%%i]=!color_list[%%i]!
echo.
echo --- 方法3: 关联数组(模拟) ---
set "dict[name]=张三"
set "dict[age]=25"
set "dict[city]=北京"
echo dict[name]=!dict[name]!
echo dict[age]=!dict[age]!
echo.
echo --- 方法4: 数组作为命令行参数 ---
set "myarr=10 20 30 40 50"
call :PrintElements !myarr!
pause
exit /b
:PrintElements
setlocal
set "sum=0"
:loop
if "%~1"=="" goto :done
echo  元素: %~1
set /a sum+=%~1
shift
goto loop
:done
echo 总和: !sum!
endlocal
exit /b
```

进阶主题3：字符串拼接优化

BAT 字符串拼接有 5 种方法：直接 set、循环累加、^ 跨行续行、替换技巧、延迟扩展。循环内累加要用延迟扩展 !var!，而普通位置用 %var%

更快——解析发生在执行前。

```
@echo off
setlocal enabledelayedexpansion
echo === 字符串拼接优化 ===
echo.
echo --- 方法1: 直接用 set ---
set "hello=Hello"
set "world=World"
set "greeting=!hello! !world!"
echo "!greeting!"
echo.
echo --- 方法2: 循环拼接 (高效) ---
set "long_str="
for /L %%i in (1,1,10) do (
    set "long_str=!long_str!line%%i; "
)
echo "!long_str!"
echo.
echo --- 方法3: 用 ^ 跨行拼接 ---
set "multiline=这是一个很长的字符串，^
它可以跨越多行书写，^
用 ^ 作为续行符，^
但最终是一个连续的值。"
echo "!multiline!"
echo.
echo --- 方法4: 字符串替换做拼接 ---
set "s=abcdef"
echo 原始: !s!
echo 前加 X: !s!s!=X!
set "s=!s!012345"
set "s=X!s!"
echo 前加 X 后加 012345: "!s!"
```

```
echo.
echo --- 方法5: !var! 性能注意 ---
echo set var=%%var%% 比 set var=!var! 更快
echo 但括号块内必须用 !var!
set "a=123"
set "b=456"
set "c=!a!!b!"
echo "!c!"
pause
```

表 4-1 变量修饰符速查表

修饰符	返回值	示例	说明
%~f1	完整路径	C:\Users\Me\test.bat	文件完全路径
%~d1	驱动器号	C:	只取盘符
%~p1	目录路径	\Users\Me\	含末尾反斜杠
%~n1	文件名（无后缀）	test	主文件名
%~x1	扩展名	.bat	含点号
%~s1	短路径 (8.3)	C:\USERS\ME\TEST~1.BAT	兼容旧程序
%~a1	文件属性	--a------	r只读 h隐藏 s系统 a归档
%~t1	修改日期时间	05/20/2026 14:30	系统本地化格式
%~z1	文件大小	1024	字节数
%~$PATH:1	在 PATH 中搜索到的完整路径	C:\Windows\System32\ping.exe	搜索 PATH
%~dp1	驱动器 + 目录	C:\Users\Me\	组合修饰符
%~nx1	文件名 + 扩展名	test.bat	组合修饰符
%~ftza1	完整+日期+属性+大小	--a------ 1024 05/20/2026 C:\...	多修饰符组合
%~f0	脚本自身完整路径	即 %~f1 %~f2 的 0 号参数	脚本自引用
%~dp0	脚本所在目录	最常用！脚本同目录读文件	几乎每脚本必用
%~nx0	脚本自身文件名	my_script.bat	用于日志标题
%~1	去引号后的参数	用在 call 传递时	避免引号嵌套问题

表 4-2 常见环境变量一览表

变量	典型值	含义	来源
%SystemRoot%	C:\Windows	Windows 安装目录	系统
%windir%	C:\Windows	同上（旧名）	系统
%SystemDrive%	C:	系统所在驱动器	系统
%System%	C:\Windows\System32	系统目录	cmd 动态
%ProgramFiles%	C:\Program Files	64 位程序目录	系统
%ProgramFiles(x86)%	C:\Program Files (x86)	32 位程序目录	系统
%USERPROFILE%	C:\Users\Harry	当前用户主目录	系统
%APPDATA%	C:\Users\Harry\AppData\Roaming	用户应用数据	系统
%LOCALAPPDATA%	C:\Users\Harry\AppData\Local	本地应用数据	系统
%TEMP% / %TMP%	C:\Users\Harry\AppData\Local\Temp	临时文件目录	系统
%COMPUTERNAME%	DESKTOP-HARRY	计算机名	系统
%USERNAME%	Harry	当前用户名	系统
%USERDOMAIN%	DESKTOP-HARRY	用户域	系统
%NUMBER_OF_PROCESSORS%	8	CPU 逻辑核数	系统
%PROCESSOR_ARCHITECTURE%	AMD64	CPU 架构	系统
%OS%	Windows_NT	操作系统类型（固定值）	系统
%PATH%	C:\Windows\System32;...	命令搜索路径	系统+用户
%PATHEXT%	.COM;.EXE;.BAT;.CMD;...	可执行文件后缀列表	系统
%cd%	C:\Users\Harry	当前工作目录	cmd 动态
%date%	2026/11/10	当前日期（系统格式）	cmd 动态
%time%	15:30:45.12	当前时间（系统格式）	cmd 动态
%random%	0~32767 随机	随机数	cmd 动态
%errorlevel%	上条命令退出码	0=成功 非零=失败	cmd 动态
%cmdextversion%	2	命令扩展版本号	cmd 常量
%=C:%	C:\某个目录	C 盘当前目录	cmd 动态

第 4 章小结

变量修饰符是 BAT 世界里最实用的"捷径"。%~dp0
几乎出现在每一个像样的脚本开头，%~nx1 用来安全处理文件名，%~f1
把相对路径转为绝对路径。记不住全部 17 个修饰符没关系，但 dp0 和 nx1
必须条件反射式地写。

修饰符不是函数调用——它是 cmd
内部在参数展开阶段就处理好的，没有性能开销。而且修饰符可以任意组合：%~ftza1 一口气拿到完整路径、日期、属性和大小。当你需要路径信息时，先想"有没有修饰符能直接给出"，而不是自己用字符串截取。

环境变量分系统级、用户级、cmd 动态三类。系统级在注册表
HKLM\SYSTEM\...\Environment，用户级在 HKCU\Environment。%date%、%time%、%random%
是每次展开都重新计算的动态变量——千万不要在预处理阶段就"缓存"它们！

环境变量快照脚本演示了 set > file 快速保存全部变量、findstr /v /g:file
做文本差分的技巧。这是 BAT
里"对比两次状态差异"最常用的方法——配置变更、注册表变化、环境污染排查都靠它。

智能路径解析器把所有修饰符串到一起演示，还包括路径拼接和存在性检查。BAT
里"判断路径存在且是目录"的标准写法是：if exist "path*" echo
是目录。这个技巧必须知道。

下一章进入 if 和 for——条件判断和循环，BAT 脚本的"逻辑骨架"。准备好了吗？

第 4 章大代码块 1：变量与路径处理大全

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ================================================
echo   第4章综合实战：变量与路径处理大全
echo   演示所有变量修饰符的用法
echo ================================================
echo.
rem === 设置一个示例路径参数 ===
set "sample=%~dp0docs\readme.txt"
echo 示例路径: !sample!
```

```
echo 脚本自身: %~f0
echo.
echo === 所有修饰符演示 ===
echo 完整路径 %%~f0: %~f0
echo 驱动器  %%~d0: %~d0
echo 路径    %%~p0: %~p0
echo 文件名  %%~n0: %~n0
echo 扩展名  %%~x0: %~x0
echo 短名字  %%~s0: %~s0
echo 属性    %%~a0: %~a0
echo 日期    %%~t0: %~t0
echo 大小    %%~z0: %~z0
echo.
echo === 组合修饰符 ===
echo 目录   %%~dp0: %~dp0
echo 名称+扩展 %%~nx0: %~nx0
echo 根+路径 %%~dpx0: %~dpx0
echo 短路径  %%~fs0: %~fs0
echo 完整+日期 %%~ftza0: %~ftza0
echo.
echo === ~ 去除引号 ===
set "quoted="C:\Program Files\test.bat""
echo 带引号: !quoted!
echo %%~1 去引号: !quoted:"=!
echo 用 call :func "!quoted!" 后 %%~1 的效果
call :tilde_test "C:\Program Files\test.bat"
echo.
echo === 环境变量遍历 ===
echo --- 所有以 C 开头的变量 ---
for /f "tokens=1* delims==" %%a in ('set') do (
    echo %%a=%%b
)
```

```
echo.
echo === 数组模拟 ===
set "fruits[0]=Apple"
set "fruits[1]=Banana"
set "fruits[2]=Cherry"
set "fruits[3]=Date"
set "fruits[4]=Elderberry"
for /L %%i in (0,1,4) do (
    echo   fruits[%%i] = !fruits[%%i]!
)
echo.
echo === 字符串替换 ===
set "text=Hello World Hello BAT"
echo 原文: !text!
echo 替换 Hello 为 Hi: !text:Hello=Hi!
echo 去掉空格: !text: =!
echo 转小写: !text!
set "lower=!text:A=a! !text:B=b! !text:H=h!"
echo 部分小写: !lower!
pause
exit /b 0
:tilde_test
echo  带引号参数: [%1]
echo  用 %%~1 去除: [%~1]
exit /b 0
```

第 4 章大代码块 2：环境变量快照与对比

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
```

```
echo   第4章进阶：环境变量监控与快照工具
echo   对比两次环境变量的差异
echo ============================================
echo.
set "SNAP_DIR=%~dp0snapshots"
if not exist "!SNAP_DIR!" mkdir "!SNAP_DIR!"
call :snapshot before
echo.
echo === 模拟环境变化 ===
set "NEW_VAR=I_am_a_new_variable"
set "PATH=%PATH%;C:\MyCustomTool"
set "TEMP=C:\NewTempLocation"
echo 新增 NEW_VAR
echo 修改 PATH
echo 修改 TEMP
echo.
call :snapshot after
echo.
echo === 对比差异 ===
echo --- 新增的变量 ---
findstr /v /g:"!SNAP_DIR!\before.txt" "!SNAP_DIR!\after.txt" ^| findstr /v "^$"
echo --- 被删除的变量 ---
findstr /v /g:"!SNAP_DIR!\after.txt" "!SNAP_DIR!\before.txt" ^| findstr /v "^$"
echo.
echo === 注册表环境变量 ===
echo --- 用户环境变量 ---
reg query "HKCU\Environment" 2&gt;nul
echo.
echo --- 系统环境变量（前 20 条）---
reg query "HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Environment" 2&gt;nul ^| head -25
pause
exit /b 0
```

```
:snapshot
setlocal enabledelayedexpansion
set "name=%~1"
set &gt; "!SNAP_DIR!\!name!.txt"
echo [!date! !time!] 快照 !name! 已保存
echo  变量数量: &amp;amp; type "!SNAP_DIR!\!name!.txt" | find /c "="
endlocal
exit /b 0
```

第 4 章进阶项目：智能路径解析器

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第4章项目：智能路径解析器
echo   支持相对路径、绝对路径、~%dp0 等各种格式
echo ============================================
echo.
set /p "input_path=请输入任意路径: "
if "!input_path!"=="" set "input_path=%~dp0test.txt"
echo.
call :parse_path "!input_path!"
echo.
pause
exit /b 0
:parse_path
set "raw=%~1"
echo === 路径解析结果 ===
echo 原始输入:   !raw!
rem 规范化为绝对路径
for %%P in ("!raw!") do (
```

```
    set "full=%%~fP"
    set "drive=%%~dP"
    set "dir=%%~dpP"
    set "name=%%~nP"
    set "ext=%%~xP"
    set "short=%%~sP"
    set "attr=%%~aP"
    set "date=%%~tP"
    set "size=%%~zP"
)
echo.
echo --- 修饰符分解 ---
echo 完整绝对路径 (!full!
echo 驱动器号  !drive!
echo 目录路径  !dir!
echo 文件主名  !name!
echo 扩展名    !ext!
echo 短路径    !short!
echo 文件属性  !attr!
echo 修改时间  !date!
echo 文件大小  !size! 字节
echo.
echo --- 组合路径 ---
echo 父目录:   !dir!
echo 完整文件名: !name!!ext!
echo URL风格:  file:///C:/example/path
echo 命令风格: "!full!"
echo.
echo --- 路径拼接演示 ---
set "base=C:\Users\Harry"
set "relative=Desktop\test.txt"
for %%P in ("!base!\!relative!") do echo 拼接结果: %%~fP
```

```
echo.
echo --- 存在性检查 ---
if exist "!full!" (
    echo [OK] 路径存在
    if exist "!full!"\* (
        echo     类型: 目录
    ) else (
        echo     类型: 文件
    )
) else (
    echo [X] 路径不存在
)
exit /b 0
```

进阶项目4：配置文件解析器

这个完整的 INI 解析器实现了所有标准功能：section 分组、key=value 读取、默认值、注释跳过（; 和 #）、布尔值解析、大小写不敏感匹配。整个解析器只用 50 行 BAT 实现，核心是 for /f + usebackq + 延迟扩展的经典组合。

```
@echo off
setlocal enabledelayedexpansion
title 完整的 INI 配置文件解析器

rem ============================================================
rem  完整的 INI 配置文件解析器
rem  从 ini 文件读取 key=value 对，支持默认值、类型转换
rem  支持 [section] 分组、注释行、空行跳过
rem  用法: call :ini_read "config.ini" section key out_var default
rem ============================================================

rem 创建示例 INI 文件
(
```

```
    echo ; 这是注释行
    echo # 也是注释
    echo [database]
    echo host=localhost
    echo port=3306
    echo username=root
    echo password=secret123
    echo database=myapp
    echo.
    echo [server]
    echo bind=0.0.0.0
    echo port=8080
    echo max_connections=100
    echo debug=false
    echo timeout=30
    echo.
    echo [paths]
    echo data_dir=C:\ProgramData\MyApp
    echo log_dir=C:\Logs\MyApp
    echo temp_dir=%%TEMP%%\MyApp
    echo.
    echo [features]
    echo enable_ssl=true
    echo enable_cache=yes
    echo compression=on
    echo max_retries=5
) &gt; sample.ini

echo === 解析示例 INI 文件 ===
echo.

call :ini_read "sample.ini" database host db_host
call :ini_read "sample.ini" database port db_port 3306
```

```
call :ini_read "sample.ini" database username db_user
call :ini_read "sample.ini" database password db_pwd
call :ini_read "sample.ini" database database db_name
echo [database] section:
echo   host=!db_host!
echo   port=!db_port!
echo   username=!db_user!
echo   password=!db_pwd!
echo   database=!db_name!
echo.

call :ini_read "sample.ini" server bind srv_bind
call :ini_read "sample.ini" server port srv_port
call :ini_read "sample.ini" server max_connections srv_maxconn
call :ini_read "sample.ini" server debug srv_debug
call :ini_read "sample.ini" server timeout srv_timeout
echo [server] section:
echo   bind=!srv_bind!
echo   port=!srv_port!
echo   max_connections=!srv_maxconn!
echo   debug=!srv_debug!
echo   timeout=!srv_timeout!
echo.

call :ini_read "sample.ini" features enable_ssl feat_ssl
call :ini_read "sample.ini" features enable_cache feat_cache
call :ini_read "sample.ini" features missing_key feat_missing 这里是默认值
echo [features] section:
echo   enable_ssl=!feat_ssl!
echo   enable_cache=!feat_cache!
echo   missing_key=!feat_missing!  ^(应该是默认值^)
echo.

echo === 验证布尔值解析 ===
```

```
call :ini_read_bool "sample.ini" server debug is_debug
echo server.debug 布尔值: !is_debug!
call :ini_read_bool "sample.ini" features enable_ssl has_ssl
echo features.enable_ssl 布尔值: !has_ssl!
echo.

echo === 遍历整个 INI 文件 ===
call :ini_dump "sample.ini"

del sample.ini
echo.
pause
exit /b

rem ============================================================
rem :ini_read - 读取单个配置值
rem 参数: "%1"=文件路径 %2=section %3=key %4=out_var %5=默认值(可选)
rem ============================================================
:ini_read
setlocal
set "_file=%~1"
set "_section=%~2"
set "_key=%~3"
set "_result="
if "%~5" neq "" set "_result=%~5"
set "_cur_section="
if not exist "!_file!" goto :ini_read_done
for /f "usebackq tokens=1,* delims==" %%a in ("!_file!") do (
    set "_line=%%a"
    rem 跳过空行
    if not "!_line!"=="" (
        rem 跳过注释行
        if "!_line:~0,1!"==";" goto :ini_read_continue
        if "!_line:~0,1!"=="#" goto :ini_read_continue
```

```
        rem 检测 section 头
        if "!_line:~0,1!"=="[" (
            set "_cur_section=!_line:~1,-1!"
        ) else (
            rem 匹配 section 和 key
            if /i "!_cur_section!"=="!_section!" (
                if /i "%%a"=="!_key!" (
                    set "_result=%%b"
                )
            )
        )
    )
    :ini_read_continue
)
:ini_read_done
endlocal &amp;amp; set "%~4=%_result%"
exit /b

rem ============================================================
rem :ini_read_bool - 读取并解析布尔值
rem ============================================================
:ini_read_bool
setlocal
call :ini_read "%~1" "%~2" "%~3" _raw_val
set "_bool=0"
if /i "!_raw_val!"=="true" set "_bool=1"
if /i "!_raw_val!"=="yes" set "_bool=1"
if /i "!_raw_val!"=="on" set "_bool=1"
if "!_raw_val!"=="1" set "_bool=1"
endlocal &amp;amp; set "%~4=%_bool%"
exit /b

rem ============================================================
```

```
rem :ini_dump - 遍历并打印整个 INI 文件
rem ============================================================
:ini_dump
setlocal
set "_file=%~1"
set "_cur_section="
echo 配置文件内容:
for /f "usebackq tokens=1,* delims==" %%a in ("!_file!") do (
    set "_line=%%a"
    if not "!_line!"=="" (
        if "!_line:~0,1!"=="[" (
            echo.
            echo [!_line:~1,-1!]
        ) else (
            if not "!_line:~0,1!"==";" (
                if not "!_line:~0,1!"=="#" (
                    echo   %%a = %%b
                )
            )
        )
    )
)
endlocal
exit /b
```

INI 解析实现要点汇总（25 项）

INI 解析要点	问题	解决方案	BAT 实现方式	可靠性	性能
注释行跳过	; 和 # 两种注释	两种前缀都检测	if "!line:~0,1!"==";"	高	O(n)
Section 分组	[section] 格式	切 [和] 取中间	!_line:~1,-1!	高	O(1)
key=value 拆分	等号可能含多个	用 delims== 拆，tokens=1,*	for /f delims==	高	O(1)

value 含等号	a=b=c=d	tokens=1,* 把后面全算 value	tokens=1,* delims==	高	O(1)
空行处理	ini 文件有空行	if not "!_line!"===""	空字符串判断	高	O(1)
默认值支持	key 不存在时	函数第 5 参数作为默认值	if "%~5" neq ""	高	O(1)
大小写不敏感	Host vs host	if /i 比较 section 和 key	if /i "!cur!"=="!target!"	高	O(1)
布尔值解析	true/yes/on/1	四种真值统一映射	四个 if /i 判断	高	O(1)
数字校验	port 必须数字	echo !val!|findstr /r "^[0-9]*$"	正则 findstr	中	O(n)
文件编码	UTF-8 vs ANSI	统一保存为 ANSI(GBK)	chcp 936	高	O(1)
变量展开	%TEMP% 这类	BAT 默认会展开	setlocal + !var!	高	O(1)
路径中的反斜杠	C:\Dir	直接读入不转义	保留原值	高	O(1)
等号周围空格	key = value	手动 trim 空格	循环去首尾空格	中	O(k)
嵌套引号	"quoted value"	读入后去掉首尾引号	if "!v:~0,1!"==""""	中	O(1)
多值列表	ports=80,443	用 for 循环拆分	for /f delims=,	高	O(n)
缺省 Section	key 在任何 section 外	先匹配无 section 情况	cur_section="" 特殊判断	中	O(1)
重复 key	同一个 key 多次出现	取最后一次出现	顺序覆盖赋值	高	O(n)
文件不存在	配置文件丢失	ini_read 返回默认值	if not exist 先判断	高	O(1)
大文件性能	ini 超过几百行	BAT for /f 逐行扫描	一次性扫描 O(n)	中	O(n)
热更新	运行时改 ini	需要重新 call ini_read	每次读都扫文件	低	O(n)
多 Section 同 key	不同 section 同名 key	用 section+key 定位	两重判断	高	O(1)
ini_write 写回	修改配置	BAT 原生不支持，需重建文件	读-改-写 三阶段	低	O(n)
INI vs 注册表	配置放哪	ini 方便查看/备份	HKCU\Software\App 也可以	中	看场景
JSON 替代	更复杂配置	用 PowerShell ConvertFrom-Json	powershell -Command	高	中
YAML 替代	层级配置	BAT 难解析，建议用 PS/Python	powershell 第三方模块	高	中

BAT 解析 ini 文件的核心难点在于 for /f 的 usebackq 模式——没有它双引号文件名会被当成字符串字面量。tokens=1,* delims== 这个写法是关键：tokens=1 取 key，* 把剩余所有（包括后续的等号和值）合并成

%%b。这样值里有多个等号也不会被截断。

注释跳过用前缀匹配 !_line:~0,1!，先检查 ; 和 #，然后检测 section 头 [。这种"逐行扫描+状态变量追踪当前 section"的模式是 BAT 解析所有结构化文本的通用套路——无论是 ini、csv、log 还是自定义格式，都是同一套框架。

写回 ini（ini_write）是 BAT 不擅长的事。因为 BAT 没有 insert/delete 行的能力，要修改就必须整文件读进来、改数组、再整文件写出去。建议复杂配置用 PowerShell 直接操作 JSON 或注册表，BAT 里只管读配置然后调用对应的工具。

第5章 流程控制语句

5.1 if 语句的4种模式

BAT 中 if 语句有且仅有4种用法，分别用于不同场景：

模式	语法	用途	示例
1. 字符串比较	if "a"=="b"	比较两个字符串	if "%name%"=="Harry"
2. errorlevel	if errorlevel N	检查退出码 >= N	if errorlevel 1 echo 失败
3. exist	if exist 路径	检查文件/目录是否存在	if exist C:\data.txt
4. defined	if defined var	检查变量是否已定义	if defined debug echo debug mode

模式1：字符串比较

```
@echo off
set "password=123456"
set /p input=请输入密码:
if "%input%"=="%password%" (
    echo 密码正确，欢迎!
) else (
    echo 密码错误!
)
```

注意：比较运算符是 `==`（双等号），不是单等号。比较时两侧最好都加双引号，防止其中一侧为空时出现语法错误。

> **WARNING**
> 大小写不敏感：`if "ABC"=="abc"` 结果为真。如果需要大小写敏感比较，用 if 加上 /I 开关的反义... 不对，BAT 没有直接的大小写敏感比较。一个办法是把两边都转成大写后比较。

模式2：if errorlevel

```
@echo off
copy important.txt backup.txt
if errorlevel 1 (
    echo [错误] 复制失败!
) else (
    echo [成功] 备份完成
)
```

重要：`if errorlevel N` 检查的是 **errorlevel >= N**，不是等于！如果你想只检查 errorlevel == 0，正确写法是：

```
if errorlevel 1 (
    echo 非零
) else (
    echo 等于零
)
```

模式3：if exist

```
@echo off
if exist "C:\Windows\System32\cmd.exe" (
    echo cmd.exe 存在
) else (
    echo cmd.exe 不存在
)

if exist "D:\data" (
```

```
    echo D:\data 是文件或目录
)

if not exist "%temp%\cache.tmp" (
    echo 缓存文件不存在，需要重新生成
)
```

模式4：if defined

```
@echo off
set "debug=1"
if defined debug (
    echo [DEBUG] 调试模式已开启
    echo [DEBUG] 当前目录: %cd%
)
set debug=
if defined debug (echo 不该出现) else (echo debug 变量已清除)
```

5.2 if 与 else：多分支嵌套

BAT 没有 else if，多分支只能靠嵌套：

```
@echo off
set /p score=请输入分数(0-100):
if %score% geq 90 (
    echo 等级: 优秀
) else if %score% geq 80 (
    echo 等级: 良好
) else if %score% geq 60 (
    echo 等级: 及格
) else (
    echo 等级: 不及格
)
```

注意这里用到了比较运算符 `geq`（大于等于）。BAT 支持的比较运算符：

运算符	含义	数学符号
EQU	等于	==
NEQ	不等于	!=
LSS	小于	<
LEQ	小于等于	<=
GTR	大于	>
GEQ	大于等于	>=

NOTE

比较运算符用于 if 的数值比较（if %a% gtr %b%）。如果比较字符串，用 == 和 !=。

5.3 goto 与 call

我们在第 3.5 节已经讲过 goto 和 call 的基本用法。这里看一个更完整的菜单程序:

```
@echo off
setlocal enabledelayedexpansion
:menu
cls
echo ========================
echo    工具箱菜单
echo ========================
echo [1] 查看系统信息
echo [2] 清理临时文件
echo [3] 网络诊断
echo [0] 退出
echo ========================
set /p choice=请选择:

if "%choice%"=="1" goto :sysinfo
if "%choice%"=="2" goto :cleanup
if "%choice%"=="3" goto :netcheck
if "%choice%"=="0" goto :eof
```

```
goto :menu

:sysinfo
echo.
echo --- 系统信息 ---
systeminfo | more
pause
goto :menu

:cleanup
echo.
echo --- 清理临时文件 ---
echo 清理 %temp% ...
del /q /s "%temp%\*.*" 2&gt;nul
echo 完成!
pause
goto :menu

:netcheck
echo.
echo --- 网络诊断 ---
ipconfig /all
pause
goto :menu
```

5.4 for 循环的4种模式

for 是 BAT 中最强大的命令，也是语法最复杂的。它有 4 种模式，分别由 /D、/R、/L、/F 开关激活。

模式1：for /D —— 仅目录

遍历指定目录下的**子目录名**：

```
@echo off
for /D %%d in (C:\Users\*) do (
```

```
    echo 目录: %%d
)
```

输出：

```
目录: C:\Users\Administrator
目录: C:\Users\Harry
目录: C:\Users\Public
...
```

模式2：for /R —— 递归遍历

递归遍历目录树，匹配文件或目录：

```
@echo off
rem 递归列出 C:\Windows 下所有 .log 文件
for /R "C:\Windows" %%f in (*.log) do (
    echo %%f
)
```

TIP

for /R 的括号里可以写模式（如 *.log），也可以写固定的文件名（如 %%f in (test.txt)），后者会返回每个子目录下是否存在 test.txt。

模式3：for /L —— 数值区间循环

类似 C 语言的 for (i=start; i<end; i+=step)：

```
@echo off
rem 从 1 数到 10，步长为 1
for /L %%i in (1,1,10) do echo %%i
echo.
rem 从 10 倒数到 0，步长为 -1
for /L %%i in (10,-1,0) do (
    set /a square=%%i*%%i
    echo %%i 的平方是 !square!
)
```

语法：`for /L %%i in (  ,  ,  ) do`

模式4：for /F —— 文本/命令解析（最强大）

这是 for 循环中最复杂也最有用的模式。它能逐行读取文本或命令输出，然后按 tokens 拆分成变量。

读取文件：

```
@echo off
rem 假设 data.txt 内容为: "张三,25,北京" 一行一条
for /F "tokens=1,2,3 delims=," %%a in (data.txt) do (
    echo 姓名: %%a 年龄: %%b 城市: %%c
)
```

解析命令输出：

```
@echo off
rem 列出所有运行中的进程名
echo 当前运行的进程:
for /F "tokens=1" %%a in ('tasklist /NH /fo csv') do (
    echo %%a
)
```

for /F 的参数详解：

参数	含义	示例
"tokens=N"	取第 N 列（从1开始）	tokens=1,2,3
"tokens=N-M"	取第 N 到 M 列	tokens=1-3
"tokens=*"	取整行	tokens=*
"delims=x"	列分隔符（默认空格/Tab）	delims=,
"skip=N"	跳过前 N 行	skip=1
"usebackq"	高级引用模式	usebackq
"eol=x"	注释行起始符（默认 ;）	eol=#

TIP

usebackq 的作用：让 for /F 能正确处理带空格的文件名和特殊字符。启用后，**`命令`**（反引号）表示执行命令，**"文件"**（双引号）表示文件名，单引号用于字面量。现代 BAT 脚本中几乎总是应该加 usebackq。

5.5 for 循环高级变体

嵌套循环 + 变量延迟扩展：

```
@echo off
setlocal enabledelayedexpansion
rem 双重循环：打印九九乘法表
for /L %%i in (1,1,9) do (
    for /L %%j in (1,1,%%i) do (
        set /a prod=%%i*%%j
        &lt;nul set /p "=%%i x %%j = !prod!  "
    )
    echo.
)
endlocal
```

`<nul set /p` 是一个经典技巧——用于不换行输出（set /p 默认不带换行符）。`<nul` 是为了提供一个空的输入流。

— 第5章 完 —

进阶练习

练习1：if 的各种写法：

```
@echo off
set /p num=输入一个数字:
if %num% gtr 100 (echo 大于100) else if %num% gtr 50 (echo 51-100) else (echo 50或以下)
set "color=red"
if "%color%"=="red" echo 是红色
if /i "%color%"=="RED" echo 忽略大小写匹配
if exist "C:\Windows\notepad.exe" echo notepad 存在
if defined username echo username 已定义
pause
```

练习2：for 的所有变体：

```
@echo off
echo === for /L ===
for /L %%i in (1,1,5) do echo 序号 %%i
echo === for %%f ===
for %%f in (*.bat) do echo BAT: %%f
echo === for /F 解析 ===
echo a=10 &gt; test.txt
echo b=20 &gt;&gt; test.txt
for /F "tokens=1,2 delims==" %%a in (test.txt) do echo %%a = %%b
del test.txt
echo === for /D ===
for /D %%d in ("%windir%\*") do echo 目录: %%d
pause
```

练习3：choice 菜单系统：

```
@echo off
:menu
cls
```

```
echo [1] 系统信息  [2] 进程列表  [3] 磁盘
echo [Q] 退出
choice /C 123Q /M "选择"
if errorlevel 4 goto :quit
if errorlevel 3 goto :disk
if errorlevel 2 goto :proc
if errorlevel 1 goto :sys
goto :menu
:sys
systeminfo | findstr /c:"OS"
pause &amp;amp; goto :menu
:disk
wmic logicaldisk get caption,freespace
pause &amp;amp; goto :menu
:proc
tasklist /fo table
pause &amp;amp; goto :menu
:quit
exit /b
```

疑难解答

Q1: for 循环为什么用 %%i 不是 %i?

A: BAT 文件里 %%i，命令行直接执行 %i。BAT 里 % 本身需 %% 转义。

Q2: for /F 单引号和双引号区别?

A: usebackq 模式下单引号包裹命令会执行并解析；双引号是字符串；不加引号是文件名。

进阶主题1：for /F 多标记解析高级技巧

for /F 是 BAT 里最强大的解析工具。tokens 可以取 1-31 个标记，delims 指定分隔符，skip 跳过开头行，eol 指定注释行。usebackq 模式下可以反引号包裹命令执行，双引号包裹字符串。

```
@echo off
setlocal enabledelayedexpansion
echo === for /F 多标记解析高级技巧 ===
echo.
echo --- 准备测试数据 ---
(echo 张三,25,北京,工程师
 echo 李四,30,上海,设计师
 echo 王五,28,广州,产品经理
 echo 赵六,35,深圳,CTO) &gt; people.csv
echo 测试文件内容:
type people.csv
echo.
echo --- 1. 按逗号分割 tokens=1-4 ---
for /F "tokens=1-4 delims=," %%a in (people.csv) do (
    echo 姓名=%%a  年龄=%%b  城市=%%c  职业=%%d
)
echo.
echo --- 2. 跳过标题行 skip=1 ---
(echo Name,Age,City,Job
 echo 张三,25,北京,工程师) &gt; people2.csv
for /F "skip=1 tokens=1,3 delims=," %%a in (people2.csv) do (
    echo %%a 住在 %%c
)
echo.
echo --- 3. 注释行处理 eol=# ---
(echo # 这是注释
 echo 数据行1
 echo # 另一个注释
 echo 数据行2) &gt; data.txt
for /F "eol=# tokens=*" %%l in (data.txt) do (
    echo 数据: %%l
)
```

```
echo.
echo --- 4. usebackq + 命令执行 ---
for /F "usebackq tokens=1,3" %%a in (`netstat -ano ^| findstr LISTENING`) do (
    echo 监听: %%a PID=%%c
)
del people.csv people2.csv data.txt 2&gt;nul
pause
```

进阶主题2：if exist 多文件检查

if exist 只能检查单个路径。检查多个文件需要嵌套 if 或 for 循环。配合 %%~zA 可以判断文件大小，用 () 包裹多个检查可以形成清晰的状态输出表格。

```
@echo off
echo === if exist 多文件检查 ===
echo.
echo --- 1. 基础: 检查文件是否存在 ---
if exist "%SystemRoot%\notepad.exe" echo ^[OK^] notepad.exe 存在
if not exist "C:\不存在.txt" echo ^[OK^] 不存在.txt 确实不存在
echo.
echo --- 2. 检查目录是否存在 ---
if exist "%APPDATA%\Microsoft" (
    echo ^[OK^] APPDATA\Microsoft 存在
) else (
    echo ^[MISS^] 目录缺失
)
echo.
echo --- 3. 检查多个文件(用 ^&amp;^&amp;) ---
if exist "%SystemRoot%\system32\cmd.exe" ^&amp;^&amp; exist "%SystemRoot%\notepad.exe" (
    echo ^[OK^] 两个文件都存在
)
rem 上面不对, 应该用嵌套 if
if exist "%SystemRoot%\system32\cmd.exe" if exist "%SystemRoot%\notepad.exe" (
    echo ^[OK^] 嵌套 if: 两个文件都存在
)
```

```
echo.
echo --- 4. 批量检查配置文件 ---
setlocal
set "missing="
for %%f in (config.ini database.db settings.json) do (
    if not exist "%%f" (
        echo ^[MISS^] %%f
        set "missing=1"
    ) else (
        echo ^[OK^]   %%f
    )
)
if defined missing echo.
if defined missing echo 有缺失文件!
endlocal
echo.
echo --- 5. 检查文件是否有内容 ---
echo test &gt; small.txt
if exist small.txt (
    for %%A in ("small.txt") do (
        if %%~zA gtr 0 (
            echo ^[OK^] small.txt 有内容, %%~zA 字节
        ) else (
            echo ^[WARN^] small.txt 是空文件
    )
)
del small.txt
pause
```

进阶主题3：for 循环里修改变量的陷阱

BAT 在执行前先解析整个括号块，所以 %var% 在 for/if 内部永远是旧值。解决方案是 setlocal enabledelayedexpansion + !var!。call 可以突破延迟扩展的限制，因为 call 重新解析。

```
@echo off
setlocal enabledelayedexpansion
echo === for 循环里修改变量的陷阱 ===
echo.
echo --- 陷阱1: 解析时机 ---
echo 不加 enabledelayedexpansion:
set "count=0"
for /L %%i in (1,1,5) do (
    set /a count+=1
    echo  不加延迟扩展: count=!count! 但这里用的是 !count!
)
echo 注意: for 块内用 !var! 才能实时读取
echo.
echo --- 陷阱2: 括号块内 echo %var% ---
echo 错误示范 (所有 echo 在执行时看到的都是初始值):
set "msg=初始值"
for /L %%i in (1,1,3) do (
    set "msg=第 %%i 次"
    echo  echo %%msg%% 写法: %%msg%%
)
echo 正确示范 (用延迟扩展):
set "msg=初始值"
for /L %%i in (1,1,3) do (
    set "msg=第 %%i 次"
    echo 正确写法: ^!msg^! = !msg!
)
echo.
echo --- 陷阱3: for /F 内部 shift ---
set "data=a b c d"
for %%w in (!data!) do (
    echo  元素: %%w
)
```

```
echo.
echo --- 陷阱4: 在 for 内调用 call 改变变量 ---
set "total=0"
for /L %%i in (1,1,5) do (
    call :AddTotal %%i
    echo  %%i: total 现在是 !total!
)
echo 最终: !total!
pause
exit /b
:AddTotal
set /a total+=%~1
exit /b
```

表 5-1 if 条件类型总结

条件类型	语法	示例	返回/说明
字符串相等	if "a"=="b"	if "!name!"=="BAT"	区分大小写
字符串忽略大小写	if /i "a"=="B"	if /i "HELLO"=="hello"	/i 参数
数值等于	if %a% equ %b%	if !x!==10	equ 或直接 ==
数值不等于	if %a% neq %b%	if !x! neq 0	neq
数值大于	if %a% gtr %b%	if !x! gtr 5	greater than
数值大于等于	if %a% geq %b%	if !score! geq 60	greater or equal
数值小于	if %a% lss %b%	if !temp! lss 0	less than
数值小于等于	if %a% leq %b%	if !size! leq 1024	less or equal
文件存在	if exist "path"	if exist "file.txt"	文件或目录均可
目录存在	if exist "path*"	if exist "C:\Win*"	末尾 * 区分目录
errorlevel 比较	if errorlevel N	if errorlevel 1	大于等于 N 为真
errorlevel 等于	if !errorlevel!==N	if !errorlevel!==0	延迟展开精确比较
变量已定义	if defined var	if defined PATH	非空字符串为真
命令成功	cmd && ...	ping -n 1 host && echo OK	&& 链式执行
命令失败	cmd || ...	ping host || echo FAIL	|| 失败时执行

否定条件	if not 条件	if not exist file	not 关键字
括号块	if 条件 (...) else (...)	if exist file (type file) else (echo missing)	多行复合
if 条件 /v:on	if /v:on %var%==val	开启变量显示（调试）	调试用
if /c:ignorecase	if /c:i %a%==B	/c:i 等于 /i	另一种写法

表 5-2 for 五种变体对比

for 变体	语法	用途	迭代对象
for %%v in (list)	for %%v in (a b c) do ...	固定列表遍历	空格/逗号分隔的元素
for %%v in (*.txt)	for %%v in (C:*.txt) do ...	文件通配符遍历	匹配的文件
for /L	for /L %%i in (start,step,end)	数字范围循环	整数序列
for /F	for /F "options" %%v in (...)	解析文本/命令输出	每行/每个 token
for /R	for /R "root" %%f in (*.ext)	递归目录遍历	匹配的文件
for /D	for /D %%d in ("path*")	仅遍历目录	子目录名
for /F "file"	for /F "usebackq" %%v in ("file.txt")	读文件每一行	文本文件
for /F "cmd"	for /F "delims=" %%v in ('dir /b')	读命令输出	命令 stdout
for 变量修饰	%%~dpf %%~nxf %%~z...	参数修饰符在 for 中同样可用	%%~NX
嵌套 for	for %%a in (...) do for %%b in (...) do	双层循环	两个维度
延迟展开	!var! 在 for () 内	循环内动态更新变量	必须 enabledelayedexpansion
goto 跳出	在 for () 内 goto :label	中断循环（会完全跳出）	简单场景
exit /b 跳出	call :sub 后 exit /b	子程序内跳出循环	推荐方式
skip / tokens	/F 专用选项	/F "skip=5 tokens=2,3"	跳过行 / 列
usebackq	/F "usebackq"	使用反引号或双引号	避免引号冲突
eol	/F "eol=#"	定义注释行起始字符	默认 ;
findstr 配合	dir /b | findstr pattern	for /F 先过滤再遍历	性能更好

第 5 章小结

if 和 for 是 BAT 脚本的"逻辑骨架"。表 5-1 把 if 的 19 种用法分门别类列出来，其中最容易出错的是 errorlevel——if errorlevel 1 是"大于等于 1"，不是"等于 1"！精确等于必须用 !errorlevel!==0 延迟展开写法。

表 5-2 把 for 的五种核心变体 + 12 个周边细节全部展开。for /F 是最强也最复杂的：解析文本、解析命令输出、指定 delims/tokens/eol/skip/usebackq 五大选项。掌握 for /F，BAT 能处理的事情会翻十倍。

冒泡排序示例展示了 BAT 里的数组模拟（set "data[i]=value"）和双层循环。BAT 没有真正的数组，靠的是延迟展开 + 数组风格的变量名。这种技巧虽然不优雅，但在 BAT 的限制下足够用。

数据过滤 pipeline 演示了 for /F + findstr + find /c 的组合拳：生成日志 → 按级别过滤 → 统计各级别数量 → 提取特定列。这就是 BAT 处理文本数据的典型流水线。

文件批量重命名项目把 if 和 for 用到了极致：for /R 遍历文件、if exist 检查、if 字符串替换、call :tolower 子程序、for /L 序号填充。这个项目是检验 BAT 逻辑能力的好考题。

下一章进入重定向和管道——BAT 里最容易误解、但也最强大的数据流动机制。准备好了吗？

第 5 章大代码块 1：条件与循环全场景演示

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo  第5章综合实战：条件与循环全场景演示
echo ============================================
echo.
echo === 一、if 的 8 种条件 ===
echo.
rem 1. 字符串比较
set "name=BAT"
if "!name!"=="BAT" echo [1] 字符串相等: OK
if not "!name!"=="Python" echo [1b] 字符串不等: OK
rem 2. 存在性检查
```

```
if exist "%systemroot%\system32\cmd.exe" echo [2] 文件存在: OK
if exist "%systemroot%" echo [2b] 目录存在: OK
if not exist "C:\NonExistent" echo [2c] 不存在检测: OK
rem 3. errorlevel
ver &gt;nul
if !errorlevel!==0 echo [3] errorlevel 等于 0: OK
rem 4. 数值比较
set /a x=10 &amp; set /a y=20
if !x! lss !y! echo [4] x lss y (小于): OK
if !y! gtr !x! echo [4b] y gtr x (大于): OK
if !x!==10 echo [4c] 数值相等: OK
rem 5. 否定检查
set "var=hello"
if defined var echo [5] defined var: OK
set "empty="
if not defined empty echo [5b] not defined empty: OK
rem 6. 命令成功检查（&amp;&amp; ||）
ver &gt;nul 2&gt;&amp;1 &amp;&amp; echo [6] 命令成功: OK || echo [6] 命令失败
no_such_cmd 2&gt;nul || echo [6b] 命令不存在 || 捕获: OK
rem 7. /i 忽略大小写
if /i "HELLO"=="hello" echo [7] /i 忽略大小写: OK
rem 8. 复合条件（嵌套 if）
set "score=85"
if !score! gtr 0 (
    if !score! lss 60 (echo [8] 成绩: 不及格) else (
        if !score! lss 80 (echo [8] 成绩: 及格) else (
            echo [8] 成绩: 优秀
        )
    )
)
echo.
echo === 二、for 的 5 种变体 ===
```

```
echo.
rem 变体 1: for %%v in (a b c) —— 列表循环
echo [for-in] 列表循环:
for %%v in (apple banana cherry) do echo   - %%v
rem 变体 2: for /L 范围循环
echo [for-L] 范围循环 (1,2,9):
for /L %%i in (1,2,9) do echo   %%i
rem 变体 3: for /F 解析文本文件
echo line1 &gt; "%temp%\test_for.txt"
echo line2 &gt;&gt; "%temp%\test_for.txt"
echo [for-F] 解析文件:
for /f "usebackq delims=" %%l in ("%temp%\test_for.txt") do echo   %%l
del "%temp%\test_for.txt"
rem 变体 4: for /R 递归目录
echo [for-R] 递归搜索 .exe (System32 下前 5 个):
set /a cnt=0
for /r "%systemroot%\System32" %%f in (*.exe) do (
    set /a cnt+=1
    if !cnt! leq 5 echo   %%~nxf
)
rem 变体 5: for /D 仅目录
echo [for-D] 仅列出目录 (C:\Windows 下前 3 个):
set /a cnt=0
for /d %%d in ("%systemroot%\*") do (
    set /a cnt+=1
    if !cnt! leq 3 echo   %%~nd
)
pause
```

第 5 章大代码块 2：冒泡排序 + 数据过滤 pipeline

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第5章进阶：冒泡排序 + 数据过滤 pipeline
echo ============================================
echo.
rem === 冒泡排序 ===
echo --- 冒泡排序 ---
set "arr=5 3 8 1 9 2 7 4 6"
echo 原数组: !arr!
rem 拆分到数组中
set /a n=0
for %%v in (!arr!) do (
    set /a n+=1
    set "data[!n!]=%%v"
)
set /a n_minus_one=n-1
for /L %%i in (1,1,!n_minus_one!) do (
    for /L %%j in (1,1,!n_minus_one!) do (
        set /a next=%%j+1
        if !data[%%j]! gtr !data[!next!]! (
            set "tmp=!data[%%j]!"
            set "data[%%j]=!data[!next!]!"
            set "data[!next!]=!tmp!"
        )
    )
)
set "sorted="
for /L %%i in (1,1,!n!) do set "sorted=!sorted! !data[%%i]!"
echo 排序后: !sorted!
echo.
rem === 数据过滤 pipeline ===
```

```
echo --- 数据过滤 pipeline ---
rem 1. 生成模拟日志
(
    echo 2026-11-10 10:00:01 INFO  Server started
    echo 2026-11-10 10:00:02 WARN  Disk 80%% full
    echo 2026-11-10 10:00:03 ERROR Database connection failed
    echo 2026-11-10 10:00:04 INFO  Retrying...
    echo 2026-11-10 10:00:05 ERROR Timeout after 30s
    echo 2026-11-10 10:00:06 WARN  Memory usage high
    echo 2026-11-10 10:00:07 INFO  Recovery complete
) &gt; "%temp%\sample.log"
echo 生成模拟日志: %temp%\sample.log
echo.
echo [Step 1] 过滤 ERROR:
type "%temp%\sample.log" | findstr "ERROR"
echo.
echo [Step 2] 统计各级别数量:
echo   INFO : &amp; type "%temp%\sample.log" | find /c "INFO"
echo   WARN : &amp; type "%temp%\sample.log" | find /c "WARN"
echo   ERROR: &amp; type "%temp%\sample.log" | find /c "ERROR"
echo.
echo [Step 3] for /F 提取时间戳:
for /f "tokens=1,2,3" %%a in ('type "%temp%\sample.log"') do (
    echo   [%%a %%b] %%c
)
del "%temp%\sample.log"
pause
```

第 5 章进阶项目：文件批量重命名工具

```
@echo off
```

```
setlocal enabledelayedexpansion
echo ============================================
echo   第5章项目：文件批量重命名工具
echo   三种模式: 序号 / 前缀 / 日期替换
echo ============================================
echo.
set /p "dir=请输入目录路径: "
if not exist "!dir!" echo 目录不存在 &amp;amp; exit /b 1
echo.
echo [1] 添加序号前缀 (001_ 002_ ...)
echo [2] 添加自定义前缀
echo [3] 替换文件名中的特定字符
echo [4] 转为小写
echo [5] 批量修改扩展名
set /p "mode=请选择模式 [1-5]: "
echo.
if "!mode!"=="1" goto mode1
if "!mode!"=="2" goto mode2
if "!mode!"=="3" goto mode3
if "!mode!"=="4" goto mode4
if "!mode!"=="5" goto mode5
goto end
:mode1
set /a idx=0
for %%f in ("!dir!\*.txt") do (
    set /a idx+=1
    set /a padded=1000+!idx!
    set "num=!padded:~1!"
    ren "%%f" "!num!_%%~nxf"
    echo   %%~nxf -^&gt; !num!_%%~nxf
)
echo 完成 !idx! 个文件
```

```
goto end
:mode2
set /p "prefix=请输入前缀: "
set /a cnt=0
for %%f in ("!dir!\*.txt") do (
    ren "%%f" "!prefix!_%%~nxf"
    echo   %%~nxf -^&gt; !prefix!_%%~nxf
    set /a cnt+=1
)
echo 完成 !cnt! 个文件
goto end
:mode3
set /p "search=请输入要替换的字符: "
set /p "replace=请输入替换为: "
set /a cnt=0
for %%f in ("!dir!\*!search!*") do (
    set "oldname=%%~nxf"
    set "newname=!oldname:!search!=!replace!!"
    if not "!oldname!"=="!newname!" (
        ren "%%f" "!newname!"
        echo   !oldname! -^&gt; !newname!
        set /a cnt+=1
    )
)
echo 完成 !cnt! 个文件
goto end
:mode4
set /a cnt=0
for %%f in ("!dir!\*.*") do (
    set "name=%%~nxf"
    call :tolower name newname
    if not "!name!"=="!newname!" (
```

```
        ren "%%f" "!newname!"
        set /a cnt+=1
    )
)
echo 完成 !cnt! 个文件
goto end
:mode5
set /p "old_ext=请输入旧扩展名(如 .txt): "
set /p "new_ext=请输入新扩展名(如 .log): "
set /a cnt=0
for %%f in ("!dir!\*!old_ext!") do (
    ren "%%f" "%%~nf!new_ext!"
    set /a cnt+=1
    echo   %%~nxf -^&gt; %%~nf!new_ext!
)
echo 完成 !cnt! 个文件
goto end
:tolower
setlocal enabledelayedexpansion
set "s=!%~1!"
for %%A in (A=a B=b C=c D=d E=f F=f G=g H=h I=i J=j K=k L=l M=m N=n O=o P=p Q=q R=r S=s T=t U=u V=v W=w X=x Y=y Z=z
set "s=!s:%%A!"
endlocal &amp; set "%~2=%s%"
exit /b 0
:end
pause
```

进阶项目5：文件搜索器 - 递归遍历

这个递归搜索器实现了 BAT
能做的所有文件搜索功能：按扩展名通配符、按大小范围过滤（KB
为单位）、按扩展名列表排除目录、CSV 导出结果。核心逻辑是 ScanDir 递归子目录 +

CheckFile 匹配大小过滤，经典的"递归+回调"结构。

```
@echo off
setlocal enabledelayedexpansion
title 递归文件搜索器 v1.0

rem ============================================================
rem  递归文件搜索器
rem  按扩展名、大小、日期过滤，支持排除目录
rem  支持搜索结果导出 CSV / TXT
rem ============================================================

rem ========== 参数解析 ==========
set "SEARCH_DIR=%~1"
if "!SEARCH_DIR!"=="" set "SEARCH_DIR=."
set "EXT_FILTER=%~2"
if "!EXT_FILTER!"=="" set "EXT_FILTER=*"
set "MIN_SIZE=%~3"
if "!MIN_SIZE!"=="" set "MIN_SIZE=0"
set "MAX_SIZE=%~4"
if "!MAX_SIZE!"=="" set "MAX_SIZE=999999999"
set "OUTPUT=search_result_%date:~0,4%%date:~5,2%%date:~8,2%.csv"

rem ========== 排除目录列表（可在这里追加） ==========
set "EXCLUDE_DIRS=node_modules .git .svn __pycache__ temp tmp dist build bin obj"

echo === 递归文件搜索器 ===
echo 搜索目录: !SEARCH_DIR!
echo 扩展名:   !EXT_FILTER!
echo 最小大小: !MIN_SIZE! KB
echo 最大大小: !MAX_SIZE! KB
echo 排除目录: !EXCLUDE_DIRS!
echo 输出文件: !OUTPUT!
echo.

rem ========== 检查目录 ==========
```

```
if not exist "!SEARCH_DIR!" (
    echo [错误] 目录不存在: !SEARCH_DIR!
    pause
    exit /b 1
)

rem ========== 写 CSV 表头 ==========
echo "文件名","完整路径","大小(KB)","修改时间" &gt; "!OUTPUT!"

set /a total=0
set /a matched=0
set /a total_size=0

echo 正在扫描...
call :ScanDir "!SEARCH_DIR!"

echo.
echo ========== 搜索完成 ==========
echo 扫描文件总数: !total!
echo 匹配文件数:   !matched!
echo 匹配总大小:   !total_size! KB
echo 结果已保存到: !OUTPUT!
echo.
choice /C YN /M "是否打开结果文件"
if errorlevel 1 start "" notepad "!OUTPUT!"
endlocal
pause
exit /b

rem ============================================================
:ScanDir
set "_cur=%~1"
rem 先列出当前目录下的文件
for %%f in ("!_cur!\!EXT_FILTER!") do (
    set /a total+=1
```

```
    if !errorlevel! equ 0 (
        call :CheckFile "%%f"
    )
)
rem 再递归子目录
for /d %%d in ("!_cur!\*") do (
    set "_skip=0"
    set "_name=%%~nd"
    for %%e in (!EXCLUDE_DIRS!) do (
        if /i "!_name!"=="%%e" set "_skip=1"
    )
    if !_skip! equ 0 call :ScanDir "%%d"
)
exit /b

rem ============================================================
:CheckFile
set "_f=%~1"
rem 获取大小
for %%a in ("!_f!") do (
    set /a "_kb=%%~za/1024" 2&gt;nul
    set "_name=%%~nxa"
    set "_path=%%~fa"
    set "_mtime=%%~ta"
)
if "!_kb!"=="" set /a _kb=0
rem 大小过滤
if !_kb! lss !MIN_SIZE! exit /b
if !_kb! gtr !MAX_SIZE! exit /b
rem 匹配成功
set /a matched+=1
set /a total_size+=!_kb!
```

```
echo "!_name!","!_path!","!_kb!","!_mtime!" &gt;&gt; "!OUTPUT!"
exit /b
```

文件搜索 25 个维度实现方案

搜索维度	说明	BAT 实现	速度	精度	灵活度	备注
按文件名	通配符匹配 *.py	for /R 配合 *	快	高	高	最常用的搜索
按扩展名	*.bat / *.txt	for /R * 再 %%~x 判断	快	高	高	直接写通配符
按大小(精确)	恰好多少字节	%%~za 直接取	快	高	中	BAT 原生支持
按大小(范围)	100KB-1MB	if /lss /gtr %%~za	快	高	高	数学比较
按修改时间	最近 7 天	用 forfiles /d -7	中	高	中	Windows 自带
按创建时间	创建时间过滤	forfiles 不直接支持	中	中	低	需 PowerShell
按内容(文本)	包含某个字符串	findstr /s /m	慢	中	中	二进制文件会乱码
按内容(正则)	高级内容匹配	findstr /r /s	慢	高	高	支持有限正则
按属性	隐藏/系统/只读	attrib + for	中	高	低	需 attrib 配合
按所有者	用户/组过滤	icacls 或 dir /q	慢	高	低	需要额外工具
递归深度限制	只搜 3 层	手动计数或 robocopy	中	高	中	BAT 原生不支持
排除目录	跳过 node_modules	for /d + if /i 跳过	中	高	高	最实用的优化
排除隐藏文件	.开头或 H 属性	dir /a:-h	快	高	中	/a:h 只显示隐藏
多扩展名	bat,cmd,ps1	多个 for 或 *.b*	慢	高	低	BAT 不支持 OR 过滤
正则文件名	a.*c$	findstr /r 配合 dir /b	慢	中	高	迂回实现
UTF-8 文件名	中文/日文	chcp 65001	中	中	高	编码问题小心
长路径支持	>260 字符	\?\ 前缀	快	高	低	Windows 10+ 默认开
符号链接	junction/symlink	dir /a:L /b	快	中	中	会递归死循环
压缩文件内部	zip/rar 内容	需要 7z / tar	慢	高	低	BAT 原生不支持
空文件	0 字节检测	if %%~za==0	快	高	高	容易漏的检查
临时文件清理	*.tmp / *.bak	多通配符搜索+del	快	高	高	常见清理脚本
重复文件	哈希比对去重	certutil -hashfile + 比对	极慢	高	低	大目录慎用
文件计数统计	某类型有多少个	for /R + set /a count+=1	中	高	中	配合延迟扩展

导出 CSV	供 Excel 分析	echo "a","b","c" > file.csv	快	高	高	加双引号防逗号
实时监控	文件变化通知	BAT 难实现，用 PowerShell IO.FileWatcher	慢	低	极低	BAT 不适合实时

BAT 做文件搜索的核心是 for /R 递归遍历，配合 %%~xa 取扩展名、%%~za 取字节大小、%%~ta 取时间戳。for /R 比手动用 for /d 递归子目录简单得多，但缺少排除目录能力——如果项目里有 node_modules 或 .git 这种大目录必须手动跳过，否则性能暴跌。

按内容搜索的利器是 findstr /s /m：/s 递归子目录、/m 只输出文件名（不显示匹配行）。findstr 还支持正则 /r，但语法很简陋（不支持 +、?、分组等）。内容搜索速度比文件名过滤慢 10-100 倍，大目录慎用。

CSV 导出要记得字段加双引号——如果路径里有逗号（罕见但可能），不加引号 Excel 会拆列。同时用 %%~fa 获取完整路径（包括驱动器号），而不是 %%~dpna。时间戳 %%~ta 格式依赖系统区域，导出后可能需要 PowerShell 统一成 ISO 格式。

第6章 重定向、管道与输入输出

6.1 标准流概念

每个 Windows 进程启动时，都有三个预定义的"标准流"。cmd.exe 和它执行的命令也不例外：

流	编号	描述	默认连接
stdin（标准输入）	0	程序读取用户输入	键盘
stdout（标准输出）	1	程序输出正常结果	屏幕（控制台）
stderr（标准错误）	2	程序输出错误/警告信息	屏幕（控制台）

6.2 重定向符号全解

重定向可以把这些标准流从默认位置（屏幕/键盘）改到文件或其他命令。

语法	含义	说明

命令 > 文件	stdout → 文件（覆盖）	最常用，写日志、导出
命令 >> 文件	stdout → 文件（追加）	不会覆盖旧内容
命令 < 文件	stdin ← 文件	从文件读取输入
命令 2> 文件	stderr → 文件（覆盖）	单独捕获错误
命令 2>> 文件	stderr → 文件（追加）	单独追加错误
命令 2>&1	stderr → 与 stdout 相同位置	合并错误到正常输出
命令 1>&2	stdout → 与 stderr 相同位置	把正常输出当作错误
命令 > nul	stdout → 丢弃	静默正常输出
命令 2> nul	stderr → 丢弃	静默错误输出
命令 > nul 2>&1	stdout 和 stderr 都丢弃	完全静默

注意顺序！ `command > out.txt 2>&1` 和 `command 2>&1 > out.txt` 效果完全不同：

```
rem 正确：先把 stdout 指向文件，再把 stderr 也指向那个文件
command &gt; out.txt 2&gt;&amp;1

rem 错误：先把 stderr 指向 stdout（此时是屏幕），然后才改 stdout 指向文件
rem 结果：错误仍然输出到屏幕
command 2&gt;&amp;1 &gt; out.txt
```

TIP

记忆口诀：**数字在前表示"从某流来"，&加数字表示"指向某流"。**2>&1 = 把 stderr(2) 指向 stdout(1) 当前的位置。

6.3 管道符 |

管道符把左边命令的 stdout 直接作为右边命令的 stdin。这是 Unix 哲学的核心——每个命令只做一件事，用管道组合出强大功能。

```
rem 统计当前目录下有多少个 .txt 文件
dir /b *.txt | find /c /v ""

rem 列出含 "error" 的日志行
type server.log | findstr "error"

rem 从系统进程里过滤出 Python
 tasklist | findstr /I python
```

WARNING

管道在 cmd 中会创建新的 cmd.exe 实例来运行两端的命令，所以管道两端的命令在**独立的进程**中运行。这意味着管道内对变量的修改不会反映到主脚本中——如果需要在管道内改变量后读取，得用别的办法。

6.4 find / findstr / sort / more

这些是配合管道最常用的过滤和格式化工具：

find —— 简单文本搜索

```
rem 在文件中搜索 "error" 字符串
type log.txt | find "error"

rem 统计匹配行数
type log.txt | find /c "error"

rem 不区分大小写 + 显示行号
type log.txt | find /I /N "error"
```

findstr —— 支持正则表达式

```
rem 搜索所有匹配正则的行
dir /b | findstr "\.docx$"

rem 搜索多个关键词（空格分隔，任意一个匹配即可）
findstr "error warning" log.txt

rem 正则：error 后面跟数字
findstr "error [0-9][0-9]*" log.txt
```

NOTE

findstr 的正则是**简化版**，不支持？、\w 、\d 等现代正则特性。如果需要完整正则，考虑用 PowerShell 或 Python。

sort —— 排序

```
rem 按字母排序
 type list.txt | sort
```

```
rem 逆序排序
type list.txt | sort /R

rem 按第2列排序（以逗号分隔）
type data.csv | sort /, /+2
```

more —— 分页显示

```
rem 一页一页地看长输出
dir C:\Windows /s | more

rem 从第 20 行开始显示
type longfile.txt | more +20
```

6.5 set /p 交互输入

除了 pause 等待按键，set /p 是 BAT 中获取用户输入的主要方式：

```
@echo off
set /p name=请输入你的名字:
set /p age=请输入年龄:
echo 你好 %name%，你 %age% 岁了
```

WARNING

set /p 的限制：**不能超过 1024 字符**、**不能处理特殊字符**（如 & | %）、如果用户什么都不输入直接回车，变量保持原来的值（不会变空）。如果需要强制输入，用 "if "%var%"" 判断后用 goto 跳回。

6.6 重定向的高级坑

空文件陷阱：

如果命令没有产生任何输出，`command > file.txt`
会创建一个**0字节的空文件**。如果原文件已存在，会被清空。这在写日志时要小心。

重定向与延迟扩展：

在 for 循环中用 `> file.txt`，每次迭代都会**覆盖**文件。如果想追加，用 >>；但更好的做法是在循环结束后一次性写入：

```
@echo off
setlocal enabledelayedexpansion
set output=
for /L %%i in (1,1,5) do (
    set output=!output!line%%i!
)
echo !output! &gt; result.txt
```

权限冲突：

写入 C:\、C:\Program Files 等受保护目录需要管理员权限，否则会报"拒绝访问"。解决：用管理员身份运行，或让用户把文件写入 %temp% 或 %userprofile%。

— 第6章 完 —

进阶练习

练习1：重定向所有组合：

```
@echo off
echo 1. 覆盖: echo hello &gt; file.txt
echo hello &gt; demo.txt
echo 2. 追加: echo world &gt;&gt; demo.txt
echo world &gt;&gt; demo.txt
echo 3. 错误输出: dir notexist 2&gt;error.log
dir notexist 2&gt;error.log
echo 4. 静默: dir notexist 2&gt;nul
dir notexist 2&gt;nul
echo 5. 合并: dir notexist &gt;all.txt 2&gt;&amp;1
dir notexist &gt;all.txt 2&gt;&amp;1
type demo.txt
del demo.txt error.log all.txt
pause
```

练习2：管道的妙用：

```
@echo off
echo 管道妙用:
echo 只显示 LISTENING:
netstat -ano | findstr LISTENING
echo BAT 文件数:
dir /b *.bat | find /c /v ""
pause
```

疑难解答

Q1: > nul 2>&1 和 2>&1 > nul 区别?

A: >nul 2>&1 是两个都丢弃（先改 stdout，再让 stderr 跟随）。2>&1 > nul 只丢 stdout，stderr 还在屏幕上。顺序很重要!

Q2: echo | set /p 为什么不工作?

A: 管道启动新 cmd 实例，变量在子进程里设置完就丢了。直接 set var=text。

进阶主题1：重定向与管道的性能影响

管道会启动子进程，而且缓冲区满了会阻塞。避免不必要的管道（如 type file | findstr 其实可以直接 findstr
file）。批量处理大量数据时，了解管道的工作原理对写出高性能脚本很重要。

```
@echo off
echo === 重定向与管道的性能影响 ===
echo.
echo --- 1. 重定向到文件比重定向到 nul 稍慢 ---
echo 对比测试 (ping 100次):
echo 输出到 nul:
powershell -Command "$sw=[Diagnostics.Stopwatch]::StartNew(); 1..100|%%{ping -n 1 127.0.0.1 ^&gt;nul 2^&gt;^&amp;1}; $sw.Stop(); echo $sw.ElapsedMilliseconds ms"
echo 输出到文件:
powershell -Command "$sw=[Diagnostics.Stopwatch]::StartNew(); 1..100|%%{ping -n 1 127.0.0.1 ^&gt;nul 2^&gt;^&amp;1}; $sw.Stop(); echo $sw.ElapsedMilliseconds ms"
echo.
echo --- 2. 管道会启动子进程 ---
echo "dir ^| find /c /v """ 启动 2 个 cmd:
echo   cmd1 执行 dir ^| 把 stdout 给 cmd2 的 stdin
echo   cmd2 执行 find 把结果输出到屏幕
echo.
echo --- 3. 避免不必要的管道 ---
echo 慢: type file.txt ^| findstr "pattern"
echo 快: findstr "pattern" file.txt
echo.
echo --- 4. 批量处理时的管道陷阱 ---
echo 错误: echo data ^| set var=x  (管道在子进程, 变量丢失)
echo 正确: set var=x
```

```
echo.
echo --- 5. 管道的缓冲区 ---
echo 管道默认缓冲区 4KB, 满了写入者会阻塞
echo 大输出用 tee 分块处理
echo.
echo --- 6. 实际性能对比 ---
echo (以下命令可手动运行体会)
echo 统计行数: dir /s /b C:\Windows ^| find /c /v ""
echo 过滤进程: tasklist ^| findstr /i svchost
echo 混合: dir /s /b C:\*.exe ^| findstr /i setup ^| findstr /i ".exe$" ^| more
pause
```

进阶主题2：文件描述符 3-9 的高级用法

BAT 预留 fd 0-2 做 stdin/stdout/stderr，fd 3-9 是你可以自由使用的。在括号块里把 fd3 绑定到日志文件，块内所有 echo >&3
都自动写日志，块外恢复正常。这是实现日志系统的优雅方案。

```
@echo off
echo === 文件描述符 3-9 的高级用法 ===
echo.
echo --- 基础回顾: 0=stdin 1=stdout 2=stderr ---
echo fd3-fd9 是用户自定义描述符，可以持久化重定向目标
echo.
echo --- 1. 把 fd3 绑定到文件 ---
echo 正常输出:
echo 这行到屏幕
echo fd3 重定向: fd3^&gt;debug.log
echo ^&gt;^&amp;3 这行到文件 ^(通过 fd3^)^&gt;^&amp;3
echo 用 cmd 测试:
cmd /c "echo 屏幕信息 &amp; echo 调试信息 ^&gt;^&amp;3" 3&gt;debug.log
echo 屏幕可见: 屏幕信息
echo 文件内容: type debug.log
echo.
echo --- 2. 在括号块内持久化重定向 ---
```

```
(
    echo 这个块的所有输出都到文件
    echo 包括这行
    echo 以及这行
) 1&gt;all_output.log
echo 恢复正常
echo.
echo --- 3. 同时重定向 fd3 和 fd4 ---
(
    echo 普通输出
    echo 调试信息1 ^&gt;^&amp;3
    echo 调试信息2 ^&gt;^&amp;3
    echo 警告信息1 ^&gt;^&amp;4
) 1&gt;normal.log 3&gt;debug.log 4&gt;warnings.log
echo 清理...
del debug.log normal.log warnings.log all_output.log 2&gt;nul
echo.
echo --- 4. 恢复被重定向的 fd ---
echo 重定向前先保存, 执行后恢复:
( echo 临时重定向 ) 1&gt;temp.txt
echo 之后又正常到屏幕了
echo.
echo --- 5. 用 fd 实现日志系统 ---
echo 创建带日志的函数调用模式:
(
    echo 开始日志
    call :MyFunction 参数1
    echo 结束日志
) &gt; run.log
type run.log
del run.log
pause
```

进阶主题3：tee 命令实现（PowerShell 配合）

BAT 原生没有 tee（同时显示和保存）。最实用的方案是 PowerShell 的 Tee-Object，一行代码就能用。纯 BAT 的 tee 实现起来又慢又丑，不值得折腾。

```
@echo off
echo === tee 命令实现 ===
echo.
echo BAT 本身没有 tee, 但可以配合 PowerShell:
echo.
echo --- 方法1: PowerShell Tee-Object ---
echo 模拟: ping -t 然后 tee 到文件
echo 演示: dir 同时显示和保存
dir | powershell -Command "$input | Tee-Object -FilePath dir_list.txt"
echo 已保存到 dir_list.txt:
type dir_list.txt
del dir_list.txt
echo.
echo --- 方法2: BAT + PowerShell 封装 ---
echo 创建一个 tee.bat:
(
    @echo off
    powershell -Command "$input | Tee-Object -FilePath '%~1'"
) &gt; tee_test.bat
echo 用法: dir ^| tee_test.bat output.txt
echo.
echo --- 方法3: 纯 BAT tee (简单版) ---
echo 纯 BAT tee.bat:
(
    @echo off
    setlocal
    set "f=%~1"
    if not defined f set "f=tee_output.txt"
    for /f "delims=" %%l in ('more') do (
```

```
        echo %%l
        echo %%l ^&gt;^&gt; "%f"
    )
    endlocal
) &gt; tee_simple.bat
echo 这种纯 BAT 版其实不好用，推荐用 PowerShell
del tee_test.bat tee_simple.bat 2&gt;nul
echo.
echo --- 方法4: 真正好用的方案 ---
echo 直接用 PowerShell 脚本做 tee:
echo Write-Host "日志同时输出和保存" -ForegroundColor Cyan
echo Write-Host 的颜色会在 bat 里正常显示
echo.
echo 最终推荐: PowerShell Tee-Object 配合 BAT
echo some_command ^| powershell Tee-Object -FilePath log.txt
pause
```

表 6-1 重定向符号一览表

语法	名称	功能	等价于
>	重定向覆盖	stdout → 文件（覆盖）	1>
>>	重定向追加	stdout → 文件（追加）	1>>
2>	错误重定向覆盖	stderr → 文件	2>
2>>	错误重定向追加	stderr → 文件（追加）	2>>
&>	合并重定向（cmd 默认）	stdout+stderr → 文件	>file 2>&1
2>&1	错误合并到输出	stderr → stdout 当前目标	句柄克隆
1>&2	输出合并到错误	stdout → stderr 当前目标	句柄克隆
<	输入重定向	stdin ← 文件	0<
|	管道	cmd1 stdout → cmd2 stdin	中间临时管道
<<	HereDoc（cmd 不支持）	内联输入	用 type 替代
1>nul	丢弃 stdout	静默成功输出	

2>nul	丢弃 stderr	静默错误输出	
>nul 2>&1	全静默	丢弃所有输出	最常用的静默组合
>a 2>&1	合并到同一文件	stdout+stderr → a	
>a 2>b	分到不同文件	stdout → a, stderr → b	
(echo a & echo b) > f	代码块重定向	整个块的输出	一次重定向多行
>>f 2>&1	追加合并	stdout+stderr 追加到 f	日志常用
type f | cmd	文件内容管道	文件内容作为输入	cat file 风格
for /f "usebackq" %%v in ("file")	文件直接解析	for 读文件内容	替代 type | for
for /f %%v in ('cmd')	命令输出解析	for 读命令输出	替代 cmd | for

表 6-2 管道 vs 临时文件对比

对比项	管道 (|)	临时文件	说明
简洁性			管道一行搞定
可调试性			临时文件可看中间结果
大文件性能	较好（流式）	略差（磁盘 I/O）	管道不写盘
多步中间结果	不方便	方便（每步存一个文件）	管道无法检查中间状态
变量传递	不直接支持	方便	管道在子 shell 中，变量不传回
多次使用结果	每次都要重新算	一次算好多次读	缓存效应
命令退出码	最后一条命令	主命令	管道的退出码可能不准
并行处理	串行（阻塞）	可后处理并行	管道天然串行
错误处理	管道断开难捕获	每步独立错误处理	临时文件方式更强壮
代码可读性	短管道好，长管道差	每步注释清晰	超过 3 级管道可读性断崖
磁盘空间	不消耗临时空间	消耗临时空间	临时文件要手动清理
跨脚本复用	困难	方便（输出文件直接传）	临时文件是天然接口
安全考虑	管道不落地	落地文件要权限	敏感数据别写盘

推荐场景	简单 2-3 步过滤	复杂 ETL / 可调试脚本	不要纠结，用哪种方便哪种
典型示例	type log | find ERR | sort	step1.cmd; step2.cmd; step3.cmd	

第 6 章小结

重定向和管道是 BAT 里最容易"说不清"但又最重要的东西。表 6-1 覆盖了从基础的 > >> 2> 到高级的 2>&1 句柄克隆、代码块重定向、for /F 两种模式共 20 种写法。记住 "1>nul 2>&1 全静默" 和 "2>&1 是把 stderr 跟到当前 stdout 目标" 这两条铁律。

表 6-2 把管道和临时文件方案做了 15 个维度的对比。最核心的差异是：管道在子 shell 里执行，变量不会传回父脚本；临时文件每步独立，可以检查中间结果。BAT 里超过 3 级管道后可读性骤降，复杂脚本推荐用临时文件分步。

重定向全测试脚本涵盖了 10 种典型用法：覆盖、追加、stderr、合并、丢弃、输入、分离句柄、跳过回显、代码块重定向、句柄克隆。每段都有可观察的输出变化，是练习重定向的绝佳材料。

日志分析器项目展示了 find /c + findstr + for /F 的组合拳，跨文件 ERROR/WARN 统计和排序。生成模拟日志的 :gen_log 子程序里用到了 set /a level=!random! %% 10 做随机分级，是 BAT 里简单随机的标准写法。

管道的一个常见陷阱：在管道左边用 set 设置变量，右边读不到。因为管道两侧是独立的 cmd 实例。解决方案是用临时文件传递数据，或者用 for /F 替代。

下一章进入文件操作——copy/xcopy/robocopy 的对比，以及如何处理各种文件扩展名。这是 BAT 脚本里每天都要写的内容。

第 6 章大代码块 1：重定向与管道全测试

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第6章综合实战：重定向与管道全测试
echo ============================================
```

```
echo.
echo === 一、重定向 10 种用法 ===
echo.
rem 1. 标准输出覆盖
echo [1] ^&gt; 覆盖写入文件:
echo hello &gt; "%temp%\redir_test.txt"
type "%temp%\redir_test.txt"
rem 2. 标准输出追加
echo [2] ^^^&gt;^&gt; 追加写入:
echo world &gt;&gt; "%temp%\redir_test.txt"
type "%temp%\redir_test.txt"
rem 3. 标准错误覆盖
echo [3] 2^&gt; 重定向错误:
no_such_cmd 2&gt; "%temp%\stderr.txt"
echo  错误内容: &amp; type "%temp%\stderr.txt"
rem 4. 所有输出合并
echo [4] ^&gt;file 2^&gt;^&amp;1 合并 stdout+stderr:
(echo 正常输出 &amp; no_such_cmd) &gt; "%temp%\all.txt" 2&gt;&amp;1
echo  合并内容:
type "%temp%\all.txt"
rem 5. 丢弃输出
echo [5] ^&gt;nul 2^&gt;nul 丢弃:
dir /s /b C:\Windows 2&gt;nul &gt;nul &amp; echo  执行完毕（输出已丢弃）
rem 6. 输入重定向
echo [6] ^&lt; 从文件读输入:
echo y &gt; "%temp%\answer.txt"
del /p "%temp%\answer.txt" &lt; "%temp%\answer.txt" 2&gt;nul
del /q "%temp%\answer.txt"
echo  已完成
rem 7. &amp;&gt; 或 &gt;a 2&gt;&amp;b 到不同文件
echo [7] stdout 和 stderr 到不同文件:
(echo ok &amp; dir nofile.txt 2^&gt;^&amp;1) &gt; "%temp%\out.txt" 2&gt; "%temp%\err.txt"
```

```
echo   stdout: &amp;amp; type "%temp%\out.txt"
echo   stderr: &amp;amp; type "%temp%\err.txt"
rem 8. 跳过回显 (^&gt;nul)
echo [8] call 不回显:
@echo 这行不显示 &gt;nul
echo   已跳过
rem 9. 立即重定向（命令块）
echo [9] 整块命令重定向:
(echo line1 &amp;amp; echo line2 &amp;amp; echo line3) &amp;gt; "%temp%\block.txt"
type "%temp%\block.txt"
rem 10. 克隆句柄到不同地方
echo [10] 高级句柄操作:
(for /l %%i in (1,1,5) do echo line %%i) 3&amp;gt; "%temp%\fd3.txt" 2&amp;gt;nul &amp;gt; "%temp%\final.txt"
type "%temp%\final.txt"
echo.
rem 清理
del /q "%temp%\redir_test.txt" "%temp%\stderr.txt" "%temp%\all.txt" "%temp%\out.txt" "%temp%\err.txt"
"%temp%\block.txt" "%temp%\fd3.txt" "%temp%\final.txt" 2&amp;gt;nul
echo === 重定向全部测试完成 ===
pause
```

第 6 章大代码块 2：复杂管道 chain + 临时文件技术

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第6章进阶：复杂管道 chain + 临时文件技术
echo ============================================
echo.
echo === 1. 长管道 chain（4 级以上）===
echo --- 最常修改的系统目录 Top 5 ---
dir "%systemroot%" /s /a-d /o-d /t:w 2&amp;gt;nul | findstr "个文件"
```

```
echo.
echo --- 最大文件 Top 5 ---
dir "%systemroot%" /s /a-d /b 2&gt;nul | sort /r /+40 | head -5
echo.
echo --- 进程内存排序 Top 3 ---
tasklist /fo csv | findstr /v "PID" | sort /r /+35
echo.
echo === 2. 临时文件方案（更可控）===
set "tmpdir=%temp%\pipeline_demo"
mkdir "!tmpdir!" 2&gt;nul
echo --- Step 1: 列出所有 .exe ---
dir "%systemroot%\System32\*.exe" /b &gt; "!tmpdir!\01_exes.txt" 2&gt;nul
echo   数量: &amp;amp; find /c /v "" "!tmpdir!\01_exes.txt"
echo --- Step 2: 过滤长度 ---
for /f "delims=" %%f in ('dir /b "%systemroot%\System32\*.exe" 2^&gt;nul') do (
    echo %%~zf %%f
) | sort -n &gt; "!tmpdir!\02_sorted.txt"
echo   已按大小排序
echo --- Step 3: 取最大 5 个 ---
sort /r "!tmpdir!\02_sorted.txt" | head -5 &gt; "!tmpdir!\03_top5.txt"
echo.
echo --- 最终结果（临时文件方式）---
type "!tmpdir!\03_top5.txt"
echo.
echo === 对比 ===
echo 管道: 简洁但难以调试
echo 临时文件: 可分步验证、可缓存结果
echo.
echo === 清理 ===
rd /s /q "!tmpdir!"
pause
```

第 6 章进阶项目：日志分析器（pipeline 版）

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo  第6章项目：日志分析器（pipeline 版）
echo  用 findstr + sort + for /F 构建完整流水线
echo ============================================
echo.
set "logdir=%~dp0sample_logs"
mkdir "!logdir!" 2&gt;nul
rem 生成多种日志文件
call :gen_log "server.log" 200
call :gen_log "db.log" 150
call :gen_log "access.log" 300
echo 生成示例日志:
dir /b "!logdir!"
echo.
rem === 分析 ===
echo === 分析开始 ===
set "total_errors=0"
set "total_warns=0"
for %%f in ("!logdir!\*.log") do (
    for /f %%c in ('type "%%f" ^| find /c "ERROR"') do set /a total_errors+=%%c
    for /f %%c in ('type "%%f" ^| find /c "WARN"') do set /a total_warns+=%%c
)
echo.
echo === 汇总统计 ===
echo 总 ERROR 数: !total_errors!
echo 总 WARN  数: !total_warns!
echo.
echo === 每个文件的 ERROR ===
```

```
for %%f in ("!logdir!\*.log") do (
    echo --- %%~nxf ---
    type "%%f" | findstr "ERROR"
)
echo.
echo === 按时间排序（最新 10 条 ERROR/WARN）===
type "!logdir!\*.log" | findstr /i "error warn" | sort /r | head -10
echo.
echo === 清理 ===
rd /s /q "!logdir!"
pause
exit /b 0
:gen_log
set "fname=%~1"
set /a n=%~2
set "fpath=!logdir!\!fname!"
if exist "!fpath!" del "!fpath!"
for /L %%i in (1,1,!n!) do (
    set /a hour=!random! %% 24
    set /a min=!random! %% 60
    set /a sec=!random! %% 60
    set /a level=!random! %% 10
    if !level!==0 (
        echo 2026-11-10 !hour!:!min!:!sec! ERROR   !fname! 错误发生 %%i 次 &gt;&gt; "!fpath!"
    ) else if !level! leq 2 (
        echo 2026-11-10 !hour!:!min!:!sec! WARN    !fname! 警告 %%i &gt;&gt; "!fpath!"
    ) else (
        echo 2026-11-10 !hour!:!min!:!sec! INFO    !fname! 正常 %%i &gt;&gt; "!fpath!"
    )
)
exit /b 0
```

进阶项目6：日志收集器

日志收集器是故障排查的第一利器。这个脚本一次性收集系统、应用、安全三大核心事件日志，再追加 systeminfo、驱动、补丁、启动项、计划任务、网络连接这 6 大维度信息，最后自动生成汇总报告。BAT 用 wmic ntevent 做主力收集，PowerShell Get-WinEvent 做安全日志补充。

```
@echo off
setlocal enabledelayedexpansion
title 系统日志收集与分析器

rem ============================================================
rem  日志收集器 - 收集系统/应用/安全事件日志
rem  按级别过滤（Critical/Error/Warning/Info）
rem  输出：控制台摘要 + CSV 文件 + 错误报告
rem ============================================================

set "OUT_DIR=%~dp0log_report_%date:~0,4%%date:~5,2%%date:~8,2%"
mkdir "!OUT_DIR!" 2&gt;nul
set "SUMMARY=!OUT_DIR!\summary.txt"
set "ERROR_LOG=!OUT_DIR!\errors.csv"
set "WARN_LOG=!OUT_DIR!\warnings.csv"

echo === 系统日志收集与分析 ===
echo 输出目录: !OUT_DIR!
echo.

echo === 1. 系统事件日志 ===
echo 正在收集 System 日志（最近 200 条）...
echo "时间","来源","ID","级别","消息" &gt; "!ERROR_LOG!"
wmic ntevent where "Logfile='System'" get TimeGenerated,SourceName,EventIdentifier,EventType,Message /format:csv &gt; "!OUT_DIR!\system_raw.csv" 2&gt;nul
set /a sys_count=0
set /a sys_error=0
set /a sys_warn=0
set /a sys_info=0
```

```
for /f "usebackq tokens=1,2,3,4,5,* delims=," %%a in ("!OUT_DIR!\system_raw.csv") do (
    if "%%a" neq "Node" (
        set /a sys_count+=1
        set "_level=%%d"
        if "!_level!"=="2" (set /a sys_error+=1)
        if "!_level!"=="3" (set /a sys_warn+=1)
        if "!_level!"=="4" (set /a sys_info+=1)
    )
)
echo System 日志: 总计 !sys_count! 条, Error=!sys_error!, Warning=!sys_warn!, Info=!sys_info!
echo.

echo === 2. 应用程序事件日志 ===
wmic ntevent where "Logfile='Application'" get TimeGenerated,SourceName,EventIdentifier,EventType,Message /format:csv &gt; "!OUT_DIR!\app_raw.csv" 2&gt;nul
set /a app_count=0
set /a app_error=0
set /a app_warn=0
for /f "usebackq tokens=1 delims=," %%a in ("!OUT_DIR!\app_raw.csv") do (
    if "%%a" neq "Node" set /a app_count+=1
)
echo Application 日志: 总计 !app_count! 条
echo.

echo === 3. 安全事件日志 ===
rem 安全日志需要管理员权限，用 PowerShell Get-WinEvent 更可靠
powershell -Command "Get-WinEvent -LogName Security -MaxEvents 100 -ErrorAction SilentlyContinue | Select-Object TimeCreated,Id,LevelDisplayName,Message | Export-Csv -Path '!OUT_DIR!\security.csv' -NoTypeInformation"
if exist "!OUT_DIR!\security.csv" (
    echo Security 日志: 已导出（需要管理员权限才能完整读取）
) else (
    echo Security 日志: 非管理员权限，跳过
)
echo.
```

```
echo === 4. 收集系统信息日志 ===
echo 系统信息:
systeminfo &gt; "!OUT_DIR!\systeminfo.txt" 2&gt;&amp;1
echo 已导出 systeminfo.txt

echo 驱动列表:
driverquery /v /fo csv &gt; "!OUT_DIR!\drivers.csv" 2&gt;&amp;1
echo 已导出 drivers.csv

echo 已安装补丁:
wmic qfe get /format:csv &gt; "!OUT_DIR!\hotfixes.csv" 2&gt;&amp;1
echo 已导出 hotfixes.csv

echo 最近启动程序:
reg query "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" &gt; "!OUT_DIR!\runkeys.txt" 2&gt;&amp;1
echo 已导出 runkeys.txt

echo 计划任务:
schtasks /query /fo csv &gt; "!OUT_DIR!\scheduled_tasks.csv" 2&gt;&amp;1
echo 已导出 scheduled_tasks.csv

echo 网络连接:
netstat -ano &gt; "!OUT_DIR!\netstat.txt" 2&gt;&amp;1
echo 已导出 netstat.txt

echo === 5. 生成汇总报告 ===
(
    echo ================ 系统日志分析汇总 ================
    echo 生成时间: %date% %time%
    echo 计算机:   %COMPUTERNAME%
    echo 用户:     %USERNAME%
    echo.
    echo --- Event Log 统计 ---
    echo System  日志: !sys_count! 条 (Error=!sys_error!, Warn=!sys_warn!, Info=!sys_info!)
    echo App     日志: !app_count! 条
```

```
    echo.
    echo --- 建议关注 ---
    if !sys_error! gtr 0 (
        echo [!] System 日志中有 !sys_error! 条 Error，请检查
    ) else (
        echo [√] System 无 Error 级别事件
    )
    if !sys_warn! gtr 10 (
        echo [!] System 日志中 Warning 数量较多(!sys_warn!)，建议关注
    )
    echo.
    echo --- 导出文件列表 ---
    dir /b "!OUT_DIR!"
) &gt; "!SUMMARY!"

echo.
echo ============ 收集完成 ============
echo 结果目录: !OUT_DIR!
echo 汇总报告: !SUMMARY!
echo.
choice /C YN /M "打开结果目录"
if errorlevel 1 start "" explorer "!OUT_DIR!"
endlocal
pause
exit /b
```

25 类 Windows 系统日志来源清单

日志类型	来源	收集方式	格式	权限	典型内容	体积	关键级别	清理建议
System	Windows 内核/驱动	wmic ntevent	CSV/TXT	普通用户	蓝屏、服务崩溃、驱动错误	1-10MB	Error/Critical	7天后归档

Application	应用程序写入	wmic ntevent	CSV/TXT	普通用户	App 崩溃、DLL 加载失败	1-5 MB	Error/Warning	7天后归档
Security	安全审计	Get-WinEvent	CSV/XML	管理员	登录失败、权限提升、策略改变	5-50MB	FailureAudit	不能删，至少保留 90 天
Setup	Windows 安装	wevtutil qe Setup	XML	普通用户	补丁安装、功能启用	1-3 MB	Info	可清空
ForwardedEvents	远程转发	wevtutil qe	XML	普通用户	远程主机转发的事件	不定	全级别	转发完删
PowerShell	PowerShell 操作	Get-WinEvent	CSV	普通用户	脚本执行、命令历史	1-2 MB	Warning	按需
DbgPrint	内核调试	windbg/kd	TXT	Debug 模式	驱动调试输出	不定	Info	临时日志
IIS	Web 服务器	cscript adsutil.vbs	W3C	IIS Admin	HTTP 请求/错误 4xx/5xx	10-100 MB	Error	轮滚归档 30 天
SQL Server	数据库	SQL Server ErrorLog	TXT	SQL Admin	连接失败、死锁、备份	5-20MB	Error	30 天归档
Windows Update	自动更新	wuauserv log	ETL/TXT	管理员	补丁下载安装失败	1-5 MB	Error	可删或转储
BITS	后台传输	bitsadmin /list	XML	管理员	大文件下载/上传进度	不定	Info	下载完删
Firewall	防火墙日志	netsh advfirewall show	TXT	管理员	端口命中、拒绝连接	10-100 MB	Error(拒绝)	轮滚归档
Defender	杀软	Get-MpThreatDetection	CSV	普通用户	恶意软件检测记录	1-10MB	Critical/Error	90 天
DNS Client	DNS 解析	nltest /dsgetdc	TXT	普通用户	解析失败、超时	不定	Error	按需
RDP	远程桌面	wevtutil qe TerminalServices	XML	普通用户	登录成功/失败记录	1-5 MB	Failure	90 天
DHCP	IP 分配	wevtutil qe Microsoft-Windows-DHCP	XML	普通用户	租约分配/过期	1-3 MB	Error	30 天
PrintSpooler	打印服务	wevtutil qe Microsoft-Windows-PrintSpooler	XML	普通用户	打印机错误、卡纸	不定	Error	7 天
TaskScheduler	计划任务	schtasks /query /v	CSV	普通用户	任务执行/失败	2-10MB	Failure	30 天

W32Time	时间同步	w32tm /query /log	TXT	普通用户	NTP 同步失败	不定	Error	30 天
USBHUB	USB 设备	wevtutil qe Microsoft-Windows-DriverFrameworks	XML	普通用户	USB 设备插拔/故障	不定	Error	7 天
Battery	电源状态	wevtutil qe Microsoft-Windows-Kernel-Power	XML	普通用户	电池、充电状态	不定	Info	7 天
BitLocker	磁盘加密	manage-bde -status	TXT	管理员	加密/解密/恢复	不定	Error	永久
Hyper-V	虚拟机	Get-WinEvent -LogName Hyper-V-*	CSV	Hyper-V Admin	VM 启动/崩溃/迁移	5-20MB	Error	30 天
Cluster	故障转移	Get-WinEvent -LogName Microsoft-Windows-Failover Clustering	CSV	域 Admin	节点故障、角色转移	10-50MB	Critical/Error	90 天
AD/DNS	Active Directory	Get-WinEvent -LogName Directory Service	CSV	域 Admin	复制失败、认证异常	5-20MB	Error	90 天

Windows 事件日志系统分三层：传统 Event Log（System、Application、Security）用 wmic ntevent 可以读取；新的 Analytical/Debug 日志（如 Microsoft-Windows-*）用 PowerShell Get-WinEvent 更方便；ETW 实时追踪用 logman start/stop。BAT 最擅长前两层，第三层建议交给 PowerShell。

安全日志（Security Event Log）是最敏感的一类——默认只有本地 Administrator 和域 Domain Admins 能完整读取。普通用户调用 Get-WinEvent Security 会拿到"拒绝访问"。收集脚本里必须提前判断权限，失败就跳过而不是让脚本崩溃。

日志保留策略是运维核心：安全日志按合规要求至少保留 90 天，有些金融机构要求一年以上。Windows 事件查看器可以设置自动轮滚（max size + overwrite oldest），或用 wevtutil el + wevtutil epl 手动归档到 EVTX 文件——比 CSV 保留完整事件属性。

第三部分 文件与系统操作篇

第7章 文件操作实战

7.1 路径与通配符

在 BAT 中，路径是最容易出错的地方之一。理解通配符、相对路径和绝对路径能帮你避开大量陷阱。

通配符

- `*` —— 匹配任意长度字符（包含零个）
- `?` —— 匹配**恰好一个**字符

```
dir *.txt      &amp;amp; rem 所有 .txt 文件
dir *.log      &amp;amp; rem 所有 .log 文件
dir report?.txt  &amp;amp; rem report1.txt、reportA.txt 等（reportX.txt，X 为一个字符）
dir a?c.*      &amp;amp; rem abc.txt、a1c.log、aZc.doc 等
```

WARNING

在 BAT 的文件名处理中，通配符是"贪婪"的。例如 `*.txt` 会匹配 file.txt、backup.txt、甚至 file.txt.bak 吗？不会——cmd
在内部会把文件名按最后一个点分割，后缀只匹配最后一个后缀。但某些命令（如 copy）会有不同行为，注意测试。

相对路径 vs 绝对路径

- **绝对路径**：从驱动器号开始，如 `C:\Users\Harry\data.txt`——无论当前目录在哪，都指向同一个文件
- **相对路径**：相对于当前工作目录（%cd%），如 `data\input.txt`——在不同目录下运行会指向不同位置
- `.` —— 当前目录
- `..` —— 上一级目录

- `..\..` —— 上两级目录

7.2 copy 命令

`copy` 是最基础的文件复制命令。

```
rem 最简单的复制
copy file.txt backup.txt

rem 复制到另一个目录
copy file.txt D:\backup\

rem 批量复制（通配符）
copy *.txt archive\

rem 复制时合并文件
copy file1.txt + file2.txt merged.txt

rem 二进制模式（带 /b），不修改换行符
copy /b large_file.bin D:\storage\

rem 安静模式，不显示进度
copy /q source.txt dest.txt
```

TIP

/a 和 /b 的区别：/a 是 ASCII 文本模式（默认），会把 Unix 换行符 \n 转成 Windows 的 \r\n；/b 是二进制模式，原封不动复制。复制 .exe、图片、压缩包**必须**用 /b。

7.3 xcopy vs robocopy

copy 能做的事有限。复杂的复制场景需要 xcopy 或 robocopy。

特性	copy	xcopy	robocopy
文件复制			
目录递归		/E /S	/E
复制权限		/O /X	/COPYALL
镜像同步			/MIR
断点续传			/Z

多线程			/MT:n
排除文件		/EXCLUDE:file	/XF /XD
网络路径	有限	支持	原生优化
是否内建	是(cmd)	是（外部）	是（Vista+）

7.4 robocopy 深度解析

`robocopy`（Robust File Copy）是 Vista 引入的企业级复制工具。它不是普通命令——它是一个专业的备份/同步工具。

```
rem 基础镜像备份
robocopy "C:\Data" "D:\Backup\Data" /MIR

rem /MIR = /E（包含空子目录） + /PURGE（删除目标端多余文件）
rem 这意味着目标端会完全变成源端的镜像

rem 备份文档到网络盘，跳过已同步的文件
robocopy "%userprofile%\Documents" \\NAS\Backup\Docs /E /Z /MT:8 /R:3 /W:5

rem 参数说明:
rem /Z    重启模式（断点续传）
rem /MT:8  8线程加速
rem /R:3   失败后重试3次
rem /W:5   每次重试等待5秒

rem 排除临时文件和回收站
robocopy "C:\Project" "D:\Project_Backup" /E /XD "$RECYCLE.BIN" "System Volume Information" /XF "*.tmp" "*.log"
```

robocopy 退出码含义（非常重要！）：

退出码	含义	是否成功
0	没有文件被复制（源端=目标端）	成功
1	所有文件已成功复制	成功
2	额外文件或目录被检测到	成功
4	某些文件无法复制（权限或锁定）	警告
8+	严重错误	失败

WARNING

注意！robocopy 的退出码 0~7 都表示成功（只是不同程度的成功）。传统的 `if errorlevel 1` 会把退出码 1~7 当成失败——正确判断是 `if errorlevel 8`。

7.5 move 和 del

move —— 移动/重命名

```
rem 文件重命名
move old.txt new.txt

rem 跨驱动器移动（内部是复制+删除）
move "C:\file.txt" "D:\archive\file.txt"

rem 目录重命名（Windows 中改名 = 移动到同目录不同名）
move "old_folder" "new_folder"

rem 批量移动所有 .jpg 到指定目录
for %%f in (*.jpg) do move "%%f" "D:\Photos\"
```

del —— 删除文件

```
rem 删除单个文件
del file.txt

rem 强制删除只读文件
del /f readonly.txt

rem 批量删除
del *.tmp
del /q C:\Windows\Temp\*.*

rem 连同子目录删除
del /s /q "C:\Logs\*.log"

rem 删除多个指定文件
del file1.txt file2.txt file3.txt
```

WARNING

del 只删文件，不删目录。删除空目录用 rd，删除非空目录用 rd /s /q。在 del 前最好先 echo 出将要删除的文件列表，确认无误再删——BAT 的删除是**永久删除**，不会进回收站！

7.6 文件重命名与批量处理

`ren`（rename）是重命名命令。结合 for 循环，可以做强大的批量重命名：

```
@echo off
setlocal enabledelayedexpansion
rem 批量把当前目录所有 .log 改成 .bak
for %%f in (*.log) do ren "%%f" "%%~nf.bak"

rem 在文件名前加上日期前缀
for %%f in (*.jpg) do (
    ren "%%f" "20261110_%%f"
)

rem 用数字序列重命名
i=0
for %%f in (*.png) do (
    set /a i+=1
    set "name=00!i!"
    ren "%%f" "!name:~-3!.png"
)
```

7.7 文件比较：fc

```
rem 基本比较
fc file1.txt file2.txt

rem 忽略大小写和空白差异
fc /i /w file1.txt file2.txt

rem 二进制比较（图片、exe）
fc /b image1.png image2.png
```

```
rem 把比较结果保存到文件
fc log1.txt log2.txt &gt; diff.txt 2&gt;&amp;1
```

7.8 实战：批量备份脚本

综合本章所学，写一个实用的"每日自动备份"脚本：

```
@echo off
chcp 65001 &gt; nul
setlocal enabledelayedexpansion

rem ========== 每日自动备份脚本 ==========
set "BACKUP_DIR=D:\DailyBackup"
set "SOURCE_DIR=C:\Users\Harry\Documents"
set /a "KEEP_DAYS=30"

rem 创建备份目录
if not exist "%BACKUP_DIR%" mkdir "%BACKUP_DIR%"

rem 生成日期标签（统一格式 YYYYMMDD）
for /f "tokens=2 delims==" %%d in ('wmic os get localdatetime /value') do set "DT=%%d"
set "DATE_TAG=%DT:~0,8%"
set "BACKUP_PATH=%BACKUP_DIR%\%DATE_TAG%"

rem 用 robocopy 执行镜像备份
robocopy "%SOURCE_DIR%" "%BACKUP_PATH%" /MIR /Z /MT:4 /R:2 /W:3 &gt; "%BACKUP_PATH%_log.txt" 2&gt;&amp;1

rem robocopy 退出码 0-7 都是成功
if errorlevel 8 (
    echo [失败] 备份出现严重错误!
    exit /b 1
)

rem 清理超过 N 天的旧备份
for /f "delims=" %%d in ('dir /b /ad /o-n "%BACKUP_DIR%"') do (
    set /a count+=1
```

```
    if !count! gtr %KEEP_DAYS% (
        echo 删除旧备份: %%d
        rd /s /q "%BACKUP_DIR%\%%d"
    )
)

echo [完成] 备份成功: %BACKUP_PATH%
endlocal
```

— 第7章 完 —

进阶练习

练习1：文件备份脚本：

```
@echo off
setlocal enabledelayedexpansion
set "SRC=%~dp0data"
set "DST=%~dp0backup"
if not exist "%SRC%" (echo 源目录不存在 &amp;amp; exit /b 1)
if not exist "%DST%" mkdir "%DST%"
set count=0
for /R "%SRC%" %%f in (*) do (
    set /a count+=1
    copy /y "%%f" "%DST%\%%~nxf" &gt;nul
)
echo 备份 !count! 个文件到 %DST%
endlocal
pause
```

练习2：robocopy 高级用法：

```
@echo off
rem 多线程复制
robocopy C:\LargeFolder D:\Backup /E /MT:8 /R:3 /W:5
rem 镜像复制
robocopy C:\Source D:\Mirror /MIR
rem 只复制7天内的
robocopy C:\Recent D:\Backup /E /MAXAGE:7
rem 排除临时文件
robocopy C:\Project D:\Backup /E /XD node_modules .git /XF *.tmp
echo 退出码: %errorlevel%
pause
```

练习3：certutil 计算哈希：

```
@echo off
echo SHA1:
certutil -hashfile "%~f0" SHA1 | findstr /v certutil
echo MD5:
certutil -hashfile "%~f0" MD5 | findstr /v certutil
echo SHA256:
certutil -hashfile "%~f0" SHA256 | findstr /v certutil
pause
```

疑难解答

Q1: copy /b 是什么?

A: copy /b file1 + file2 output 按二进制拼接文件。经典：多视频片段拼成一个。/b 二进制模式遇 0x1A 不停止。

Q2: xcopy vs robocopy?

A: robocopy 完胜。支持多线程、断点续传、过滤、镜像、日志。xcopy 是古董。

进阶主题1：批量改名（for + ren）

批量改名有三种场景：统一加前缀/后缀、替换文件名中的字符、按序号重排。关键技巧是 for %%f in (*.ext) 遍历 + ren 改名 + 延迟扩展处理中间变量。注意 for 循环中变量名的 %%~nxf 修饰符。

```
@echo off
setlocal enabledelayedexpansion
echo === 批量改名 ===
echo.
echo --- 1. 给所有文件加前缀 ---
echo 创建测试文件...
echo 测试 &gt; 01.txt
echo 测试 &gt; 02.txt
echo 测试 &gt; 03.txt
echo 当前文件:
```

```
dir /b *.txt
echo.
echo 批量加前缀 "bak_"
for %%f in (*.txt) do (
    ren "%%f" "bak_%%f"
)
echo 改名后:
dir /b *.txt
echo.
echo --- 2. 替换文件名中的字符 ---
echo 把 bak_ 改成 final_
for %%f in (bak_*.txt) do (
    set "name=%%~nxf"
    set "name=!name:bak_=final_!"
    ren "%%f" "!name!"
)
echo 改名后:
dir /b *.txt
echo.
echo --- 3. 按序号重命名 ---
echo (重新准备)
del *.txt
echo a &gt; a.txt
echo b &gt; b.txt
echo c &gt; c.txt
set "idx=0"
for /f "delims=" %%f in ('dir /b /o:n *.txt') do (
    set /a idx+=1
    set "num=00!idx!"
    ren "%%f" "!num:~-3!_%%f"
)
echo 序号重命名后:
```

```
dir /b *.txt
del *.txt
pause
```

进阶主题2：临时文件安全处理（%TEMP% + 清理）

临时文件用 %TEMP% + 随机后缀创建唯一路径，脚本结束时务必清理。最安全的做法是把整个脚本逻辑放在括号块里，最后统一清理。别忘了 attrib -h -s -r 先取消文件属性。

```
@echo off
echo === 临时文件安全处理 ===
echo.
echo --- 1. 使用 %TEMP% 目录 ---
echo 用户临时目录: %TEMP%
echo 系统临时目录: %SystemRoot%\Temp
echo.
echo --- 2. 生成唯一临时文件名 ---
set "tmpfile=%TEMP%\mytemp_%random%_%date:~0,4%%date:~5,2%%date:~8,2%.txt"
echo 临时文件: %tmpfile%
echo 临时内容 &gt; "%tmpfile%"
echo 临时文件已创建
dir "%tmpfile%"
echo.
echo --- 3. 用 trap 确保清理 ---
echo 在 catch 块里处理异常退出:
(
    set "tmpdir=%TEMP%\work_%random%"
    mkdir "%tmpdir%"
    echo 在临时目录工作...
    cd "%tmpdir%"
    echo 生成一些文件 &gt; output.dat
    echo 处理中...
    cd /d "%~dp0"
    echo 清理临时目录:
    rmdir /s /q "%tmpdir%"
```

```
    echo 清理完成
) 2&gt;&amp;1
echo.
echo --- 4. 安全删除技巧 ---
echo 先用 attrib 取消只读再删:
attrib -h -s -r "%tmpfile%" 2&gt;nul
del /f /q "%tmpfile%" 2&gt;nul
echo.
echo --- 5. 跨会话持久化临时目录 ---
echo 如果脚本需要跨多次运行保留临时文件:
set "persistent=%TEMP%\myscript_data"
if not exist "%persistent%" mkdir "%persistent%"
echo 持久化临时目录: %persistent%
pause
```

进阶主题3：用 certutil 做 base64 加解密

certutil 不只是证书工具，还能做文件哈希和 base64 编解码。-encode 把文件变成 base64，-decode 反解回来。这是 BAT 里实现"加密"脚本或自解压的核心技术。

```
@echo off
echo === certutil 做 base64 加解密 ===
echo.
echo --- 1. 编码: 文件 ^&gt; base64 ---
echo Hello World 这是测试文本 &gt; original.txt
echo 编码前大小:
dir original.txt
certutil -encode original.txt encoded.b64 &gt;nul
echo base64 内容:
type encoded.b64
echo.
echo --- 2. 解码: base64 ^&gt; 文件 ---
certutil -decode encoded.b64 decoded.txt &gt;nul
echo 解码后内容:
type decoded.txt
```

```
echo.
echo --- 3. 验证文件相等 ---
certutil -hashfile original.txt MD5
certutil -hashfile decoded.txt MD5
echo 两个 MD5 应该完全一致
echo.
echo --- 4. 加密敏感脚本 ---
echo 创建一个 BAT 脚本:
echo @echo off &gt; secret.bat
echo echo 这是秘密脚本 &gt;&gt; secret.bat
certutil -encode secret.bat secret.b64 &gt;nul
rem 可以删除 secret.bat, 只保留 secret.b64
echo 已编码为 secret.b64
echo.
echo --- 5. 一键自解压 ---
echo 创建自解码脚本:
(
    @echo off
    certutil -decode "%~f0"_data output.bat &gt;nul
    call output.bat
    goto :eof
    -----BEGIN CERTIFICATE-----
) &gt; self_decode.bat
echo (完整自解压方案见实战项目)
echo.
del original.txt encoded.b64 decoded.txt secret.bat secret.b64 self_decode.bat 2&gt;nul
pause
```

表 7-1 文件操作命令对比

命令	能力级别	递归	多线程	断点续传	镜像模式	权限保留	推荐场景

copy	1 (基础)						单文件复制 / 简单脚本
xcopy	2 (中级)					部分	复制整个目录树
robocopy	5 (专业)		(MT:)		(/MIR)		生产级备份同步
move	1.5 (基础)						重命名 / 跨分区移动
del	1 (基础)	(需/s)					删除文件
rd	1 (基础)	(需/s)					删除目录
attrib	1 (基础)	(需/s)					修改属性
fc	1 (基础)						文件内容比较
replace	2 (中级)						替换已有文件
unzip (PowerShell)	3 (进阶)						解压 ZIP
tar	3 (进阶)						解压 tar.gz
certutil	2 (高级)						下载 / 校验 / 编码
expand	2 (高级)						解压 .cab 文件
icacls	3 (进阶)						修改 ACL 权限
cipher	3 (进阶)						EFS 加密

表 7-2 批处理文件常用扩展名

扩展名	命令类型	说明	示例程序	备注	
.bat	批处理（旧风格）	command.com 格式	cmd.exe	最通用	
.cmd	批处理（新风格）	cmd.exe 增强格式	cmd.exe	推荐新脚本用 .cmd	
.ps1	PowerShell 脚本	PowerShell 5.1+	powershell.exe	跨平台	
.vbs	VBScript	Visual Basic 脚本	wscript.exe	旧但仍可用	
.js	JScript	微软的 JavaScript	wscript.exe	少用	
.wsf	Windows Script File	可混合多种脚本	wscript.exe	企业内部脚本	
.exe	可执行文件	编译后的二进制	任何程序	BAT 可直接调用	

.com	旧式可执行	DOS 风格 COM 文件	command.com	同 .exe	
.msi	Windows Installer	安装包格式	msiexec.exe	静默安装	/qn 参数
.msix/.appx	UWP 安装包	现代应用安装	Add-AppxPackage	Win10+	
.reg	注册表文件	导入/导出注册表	regedit.exe	双击即导入	
.scr	屏幕保护程序	可执行文件类型	直接双击运行	其实就是 exe	
.hta	HTML Application	IE 渲染的 HTML 脚本	mshta.exe	做简易 GUI	
.cpl	控制面板程序	控制面板项	control.exe	rundll32 调用	
.inf	安装信息文件	驱动/安装配置	install.exe	驱动开发用	
.dll	动态链接库	可调用的库	rundll32.exe	不能单独运行	
.ocx	ActiveX 控件	COM 组件	regsvr32	注册后可用	
.msp	补丁包	Windows Installer 补丁	msiexec.exe	安装补丁	
.batx	（自定义）	无官方特殊含义	自己关联	仅在 BAT 内部有意义	

第 7 章小结

文件操作的"三驾马车"是 copy（简单）、xcopy（中级）、robocopy（专业）。表 7-1 用 15 个维度把它们和 move、del、rd、attrib、fc、replace、tar、certutil 等一网打尽。bat 新脚本一律用 robocopy，别再纠结 xcopy 了。

robocopy 的关键参数：/MIR 镜像同步（会删多余文件！）、/MT:8 8 线程加速、/R:3 /W:5 失败重试 3 次每次等 5 秒、/E 包含空子目录、/NFL /NDL 精简日志。它的退出码 0-7 都算成功（0=没变化，7=有变化且有警告），>=8 才是真正失败。

文件扩展名表列了 19 种脚本/可执行相关后缀。BAT 里最常见的是 .bat 和 .cmd，两者在现代 Windows 上几乎等价；.ps1 PowerShell 脚本可以用 powershell -File script.ps1 从 BAT 里调用；.msi 安装包可以用 msiexec /qn /i package.msi 静默安装。

增量备份脚本用 robocopy /MIR 做完整镜像 + 按日期命名目录 + 只保留最近 5 份。这是生产环境最常用的备份模板。robocopy 的 %errorlevel% 判断要记住：小于等于 7 都可以视为成功。

文件操作的安全意识：del /s /q *.tmp 会静默删除所有临时文件；rd /s /q 递归删除目录，不可恢复；xcopy /E /Y 复制时自动覆盖不提示。危险操作前务必加 pause 让用户确认，或者用 echo 显示将要做什么。

下一章进入路径修饰符的高级用法——拼接、规范化、UNC路径、相对/绝对互相转换。准备好了吗？

第 7 章大代码块 1：文件操作全家桶

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第7章综合实战：文件操作全家桶
echo   copy / xcopy / robocopy / move / del
echo ============================================
echo.
set "src=%temp%\file_ops_src"
set "dst=%temp%\file_ops_dst"
rmdir /s /q "!src!" "!dst!" 2&gt;nul
mkdir "!src!" 2&gt;nul
mkdir "!src!\sub1" 2&gt;nul
mkdir "!src!\sub2" 2&gt;nul
echo file1 &gt; "!src!\file1.txt"
echo file2 &gt; "!src!\file2.txt"
echo file3 &gt; "!src!\sub1\file3.txt"
echo file4 &gt; "!src!\sub2\file4.dat"
echo 准备好源目录:
dir /b /s "!src!"
echo.
rem === copy ===
echo [1] copy - 单/多文件复制:
copy "!src!\file1.txt" "!dst!\" /nfl /ndl /njh /njs /nc /ns 2&gt;nul &amp;amp; echo   copy 完成
echo   内容: &amp;amp; type "!dst!\file1.txt"
echo.
rem === xcopy ===
```

```
echo [2] xcopy - 递归复制整个目录:
xcopy "!src!" "!dst!\xcopy_copy\" /e /i /q /y 2&gt;nul &amp; echo   xcopy 完成
dir /b /s "!dst!\xcopy_copy"
echo.
rem === robocopy ===
echo [3] robocopy - 专业级复制（多线程、断点续传）:
robocopy "!src!" "!dst!\robocopy_copy" /E /NFL /NDL /NJH /NJS /NC /NS /NP &gt;nul 2&gt;&amp;1
echo   robocopy 完成 (exit code: !errorlevel!)
dir /b /s "!dst!\robocopy_copy"
echo.
rem === move ===
echo [4] move - 文件移动:
mkdir "!dst!\move_test" 2&gt;nul
copy "!src!\file2.txt" "!dst!\move_test\" 2&gt;nul
move "!dst!\move_test\file2.txt" "!dst!\moved.txt" &gt;nul 2&gt;&amp;1
echo   move 完成
dir /b "!dst!"
echo.
rem === del ===
echo [5] del - 批量删除:
copy "!src!\sub1\file3.txt" "!dst!\file3.txt" 2&gt;nul
echo   删除前: &amp; dir /b "!dst!\file3.txt" 2&gt;nul
del /q "!dst!\file3.txt" 2&gt;nul
echo   删除后: &amp; dir /b "!dst!\file3.txt" 2&gt;nul &amp; echo   (已删除)
echo.
rem === 清理 ===
rmdir /s /q "!src!" "!dst!"
echo === 文件操作全部演示完毕 ===
pause
```

第 7 章大代码块 2：robocopy 增量备份脚本

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo  第7章进阶：增量备份脚本（robocopy 版）
echo  只备份变化的文件，保留历史版本
echo ============================================
echo.
set "BACKUP_SRC=%~dp0data"
set "BACKUP_ROOT=%~dp0backups"
set "timestamp=%date:~0,4%%date:~5,2%%date:~8,2%_%time:~0,2%%time:~3,2%"
set "timestamp=!timestamp: =0!"
set "backup_name=backup_!timestamp!"
set "full_dest=!BACKUP_ROOT!\!backup_name!"
echo 备份源:  !BACKUP_SRC!
echo 备份目标: !full_dest!
mkdir "!BACKUP_SRC!" 2&gt;nul
echo test1 &gt; "!BACKUP_SRC!\sample.txt"
echo test2 &gt;&gt; "!BACKUP_SRC!\sample.txt"
echo line1 &gt; "!BACKUP_SRC!\config.ini"
mkdir "!BACKUP_SRC!\sub" 2&gt;nul
echo nested &gt; "!BACKUP_SRC!\sub\nested.txt"
echo 源文件列表:
dir /b /s "!BACKUP_SRC!"
echo.
robocopy "!BACKUP_SRC!" "!full_dest!" /E /MIR /R:3 /W:5 /MT:8 /NFL /NDL /NJH /NJS /NC /NS /NP
set "rc=!errorlevel!"
if !rc! leq 7 (
    echo.
    echo [OK] 备份成功 (robocopy exit: !rc!)
) else (
    echo.
    echo [FAIL] 备份失败 (robocopy exit: !rc!)
```

```
)
echo.
echo === 备份内容 ===
dir /b /s "!full_dest!"
echo.
echo === 保留最近 5 份，删除更早的 ===
set /a keep=5
for /f "skip=!keep! delims=" %%d in ('dir /ad /b /o-n "!BACKUP_ROOT!"') do (
    echo   删除旧备份: %%d
    rd /s /q "!BACKUP_ROOT!\%%d"
)
echo.
echo === 当前备份列表 ===
dir /ad /b "!BACKUP_ROOT!"
echo.
pause
```

第 7 章进阶项目：双向批量文件同步工具

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第7章项目：批量文件同步工具
echo   双向同步两个目录，支持镜像和合并
echo ============================================
echo.
set /p "source=源目录: "
set /p "target=目标目录: "
if not exist "!source!" echo 源目录不存在 &amp;amp; pause &amp;amp; exit /b 1
if not exist "!target!" mkdir "!target!"
echo.
```

```
echo [1] 镜像同步（目标变成源的精确副本）
echo [2] 单向合并（源覆盖目标，保留目标独有文件）
echo [3] 双向合并（两边都复制过去，冲突时覆盖）
set /p "mode=选择模式 [1-3]: "
echo.
if "!mode!"=="1" goto mirror
if "!mode!"=="2" goto oneway
if "!mode!"=="3" goto twoway
goto end
:mirror
echo === 镜像同步 ===
echo 目标将变成源的精确副本（多的删除）
pause
robocopy "!source!" "!target!" /E /MIR /R:2 /W:3 /MT:8
echo 完成 (exit: !errorlevel!)
goto end
:oneway
echo === 单向合并 ===
echo 源中新文件会复制过去，目标独有文件保留
robocopy "!source!" "!target!" /E /R:2 /W:3 /MT:8
echo 完成 (exit: !errorlevel!)
goto end
:twoway
echo === 双向合并 ===
echo 先从源到目标，再从目标到源
echo --- 源 -^&gt; 目标 ---
robocopy "!source!" "!target!" /E /R:2 /W:3 /MT:8
echo --- 目标 -^&gt; 源 ---
robocopy "!target!" "!source!" /E /R:2 /W:3 /MT:8
echo 完成
goto end
:end
```

```
echo.
echo === 同步结果对比 ===
echo 源目录文件数: &amp;amp; dir /s /b "!source!" 2&amp;gt;nul | find /c /v ""
echo 目标文件数:  &amp;amp; dir /s /b "!target!" 2&amp;gt;nul | find /c /v ""
pause
```

进阶项目7：一键文件夹打包工具

这个完整的打包脚本覆盖了压缩场景的所有常用需求：7z 和 zip 双格式、AES-256 密码保护、多卷分卷（-v100m、-v1g）、7z 完整性校验、时间戳自动命名、可选 7z 回退 PowerShell 模式。参数解析用 /src /out /fmt /7z /p /split 六个开关，支持 /? 帮助。

```
@echo off
setlocal enabledelayedexpansion
title 一键文件夹打包工具 v2.0

rem ============================================================
rem  一键文件夹打包工具
rem  7z/zip 压缩 + 密码保护 + 多卷分卷
rem  支持排除文件、时间戳命名、自动校验完整性
rem ============================================================

rem ========== 配置区 ==========
set "SOURCE=%~1"
if "!SOURCE!"=="" set "SOURCE=%~dp0data"
set "OUTPUT_DIR=%~dp0backup"
set "FMT=zip"
set /a USE_7Z=0
set /a USE_PASSWORD=0
set "PASSWORD="
set /a USE_SPLIT=0
set "VOLUME_SIZE=100m"
set /a CHECK_INTEGRITY=1
```

```
set /a SHOW_HELP=0

rem ========== 参数解析 ==========
:arg_loop
if "%~2"=="" goto :arg_done
if /i "%~2"=="/src" (set "SOURCE=%~3" &amp;amp; shift &amp;amp; shift &amp;amp; goto :arg_loop)
if /i "%~2"=="/out" (set "OUTPUT_DIR=%~3" &amp;amp; shift &amp;amp; shift &amp;amp; goto :arg_loop)
if /i "%~2"=="/fmt" (set "FMT=%~3" &amp;amp; shift &amp;amp; shift &amp;amp; goto :arg_loop)
if /i "%~2"=="/7z" (set /a USE_7Z=1 &amp;amp; shift &amp;amp; goto :arg_loop)
if /i "%~2"=="/p" (set /a USE_PASSWORD=1 &amp;amp; set "PASSWORD=%~3" &amp;amp; shift &amp;amp; shift &amp;amp; goto :arg_loop)
if /i "%~2"=="/split" (set /a USE_SPLIT=1 &amp;amp; set "VOLUME_SIZE=%~3" &amp;amp; shift &amp;amp; shift &amp;amp; goto :arg_loop)
if /i "%~2"=="/skip" (set /a CHECK_INTEGRITY=0 &amp;amp; shift &amp;amp; goto :arg_loop)
if /i "%~2"=="/?" (set /a SHOW_HELP=1 &amp;amp; shift &amp;amp; goto :arg_loop)
shift
goto :arg_loop
:arg_done

if !SHOW_HELP! equ 1 goto :show_help

rem ========== 检查源目录 ==========
if not exist "!SOURCE!" (
    echo [错误] 源目录不存在: !SOURCE!
    echo 用法: pack.bat /src C:\Data /out D:\Backup /fmt zip
    pause
    exit /b 1
)

rem ========== 7z 检测 ==========
where 7z &amp;gt;nul 2&amp;gt;&amp;amp;1
if !errorlevel! equ 0 set /a HAS_7Z=1 else set /a HAS_7Z=0

if !USE_7Z! equ 1 if !HAS_7Z! equ 0 (
    echo [警告] 未检测到 7z，改用 PowerShell Compress-Archive
    set /a USE_7Z=0
)
```

```
rem ========== 生成时间戳 ==========
set "TS=%date:~0,4%%date:~5,2%%date:~8,2%_%time:~0,2%%time:~3,2%%time:~6,2%"
set "TS=!TS: =0!"
for %%d in ("!SOURCE!") do set "NAME=%%~nd"
set "OUT_NAME=!NAME!_!TS!"

rem ========== 创建输出目录 ==========
if not exist "!OUTPUT_DIR!" mkdir "!OUTPUT_DIR!" 2&gt;nul

echo.
echo === 文件夹打包工具 ===
echo 源目录:     !SOURCE!
echo 输出目录:   !OUTPUT_DIR!
echo 压缩格式:   !FMT!  ^(7z=!USE_7Z!^)
echo 密码保护:   !USE_PASSWORD!
echo 多卷分卷:   !USE_SPLIT! ^(大小=!VOLUME_SIZE!^)
echo 完整性校验: !CHECK_INTEGRITY!
echo 输出名称:   !OUT_NAME!
echo.

rem ========== 开始压缩 ==========
echo 正在压缩...
echo.

set /a start_tick=%time:~6,2%+%time:~3,2%*60+%time:~0,2%*3600

if !USE_7Z! equ 1 goto :compress_7z

:compress_zip
set "OUT_FILE=!OUTPUT_DIR!\!OUT_NAME!.zip"
if !USE_PASSWORD! equ 1 (
    if !HAS_7Z! equ 1 (
        7z a -tzip -p"!PASSWORD!" "!OUT_FILE!" "!SOURCE!" -mx=5
    ) else (
        echo [提示] PowerShell Compress-Archive 不支持密码，改用 AES-256 ZIP 请安装 7-Zip
```

```
        powershell -Command "Compress-Archive -Path '!SOURCE!\*' -DestinationPath '!OUT_FILE!' -CompressionLevel Optimal"
    )
) else (
    if !USE_SPLIT! equ 1 (
        if !HAS_7Z! equ 1 (
            7z a -tzip -v!VOLUME_SIZE! "!OUT_FILE!" "!SOURCE!" -mx=5
        ) else (
            echo [提示] 分卷压缩需要 7-Zip，已跳过分卷
            powershell -Command "Compress-Archive -Path '!SOURCE!\*' -DestinationPath '!OUT_FILE!' -CompressionLevel Optimal"
        )
    ) else (
        powershell -Command "Compress-Archive -Path '!SOURCE!\*' -DestinationPath '!OUT_FILE!' -CompressionLevel Optimal"
    )
)
goto :compress_done

:compress_7z
set "OUT_FILE=!OUTPUT_DIR!\!OUT_NAME!.7z"
set "7z_args=a -t7z -mx=5"
if !USE_PASSWORD! equ 1 set "7z_args=!7z_args! -p!PASSWORD!"
if !USE_SPLIT! equ 1 set "7z_args=!7z_args! -v!VOLUME_SIZE!"
7z !7z_args! "!OUT_FILE!" "!SOURCE!"
goto :compress_done

:compress_done
set /a end_tick=%time:~6,2%+%time:~3,2%*60+%time:~0,2%*3600
set /a elapsed=end_tick-start_tick
if !elapsed! lss 0 set /a elapsed+=86400

echo.
echo 压缩完成！ 耗时 !elapsed! 秒
```

```
rem ========== 完整性校验 ==========
if !CHECK_INTEGRITY! equ 1 (
    echo.
    echo === 完整性校验 ===
    if !HAS_7Z! equ 1 (
        if !USE_SPLIT! equ 1 (
            7z t "!OUT_FILE!.001" 2&gt;nul || 7z t "!OUT_FILE!.part01" 2&gt;nul
        ) else (
            7z t "!OUT_FILE!"
        )
    ) else (
        echo 用 certutil 生成 SHA256:
        certutil -hashfile "!OUT_FILE!" SHA256 | findstr /v certutil | findstr /v ":"
    )
)

rem ========== 显示结果 ==========
echo.
echo === 输出文件 ===
dir /b "!OUTPUT_DIR!\!OUT_NAME!*" 2&gt;nul
echo.
choice /C YN /M "打开输出目录"
if errorlevel 1 start "" explorer "!OUTPUT_DIR!"
endlocal
exit /b

:show_help
echo.
echo 用法: pack.bat /src ^&lt;源目录^&gt; [选项]
echo.
echo 选项:
echo   /src ^&lt;path^&gt;      要打包的源目录
echo   /out ^&lt;path^&gt;      输出目录 (默认: backup)
```

```
echo   /fmt zip^|7z      压缩格式 (默认 zip)
echo   /7z               强制使用 7-Zip
echo   /p ^&lt;password^&gt;    设置压缩密码
echo   /split ^&lt;size^&gt;    分卷大小 (例: 100m, 1g)
echo   /skip             跳过完整性校验
echo   /?                显示帮助
echo.
echo 示例:
echo   pack.bat /src C:\Project /out D:\Backup
echo   pack.bat /src C:\Data /7z /p secret123
echo   pack.bat /src C:\BigFolder /split 500m
pause
exit /b
```

25 种压缩/打包工具 BAT 调用方案

压缩工具	格式	BAT 调用方式	密码支持	分卷支持	压缩率	速度	跨平台	备注
certutil	无	certutil -hashfile/encode	无	无	N/A	N/A	N/A	只做哈希/Base64
PowerShell	ZIP	Compress-Archive	不支持	不支持	中	中	Windows 10+	原生最方便
PowerShell	GZIP	IO.Compression.GZipStream	无	无	中	中	Win 10+	手动写代码
PowerShell	TAR	tar.exe (Win 10+)	无	无	高	快	Win 10 1803+	可调用 tar.exe
7-Zip CLI	7z	7z a -t7z file.7z dir	AES-256	支持 -v100m	极高	中	全平台	最强大
7-Zip CLI	ZIP	7z a -tzip -p file.zip dir	ZipCrypto/AES	支持	中	中	全平台	替代 PS Zip
WinRAR CLI	RAR	rar a -p file.rar dir	AES-256	支持 -v100m	极高	中	全平台	商业软件
RAR CLI	RAR5	rar a -ma5 -p file.rar dir	AES-256	支持	极高	中	全平台	推荐 RAR5

Bandizip	BZ	bz c -fmt bz -pwd file.bz	AES-256	支持	高	快	Win/Mac	GUI 免费
tar.exe	TAR	tar -cf file.tar dir	无	无	无(仅打包)	快	Win 10+	Windows 内置
tar.exe	TAR.GZ	tar -czf file.tar.gz dir	无	无	高	中	Win 10+	原生可用
tar.exe	TAR.XZ	tar -cJf file.tar.xz dir	无	无	极高	慢	Win 10+	极限压缩
MakeCab	CAB	makecab source.txt file.cab	无	无	中	快	Windows 内置	系统组件用
PowerShell	MSIX	打包工具.exe	无	无	中	中	Win 10+	UWP 应用格式
Docker	镜像	docker save image.tar	无	无	N/A	N/A	有 Docker	容器镜像
WIM	部署映像	dism /capture-image	无	无	高	中	Win 7+	系统部署用
ESD	极限压缩 WIM	dism /export-image	无	无	极高	慢	Win 8+	安装介质用
SSH/SCP	远程压缩	scp + remote tar	SSH 加密	无	高	中	全平台	传输时压缩
BITS	后台上传	bitsadmin /addfile /upload	HTTPS/TLS	无	无	慢	Win 7+	大文件断点续传
云存储 SDK	OSS/S3	aws s3 cp + zip	服务端加密	多 part	看原	看带宽	全平台	直接存云端
7-Zip 自解压	EXE	7z a -sfx installer.exe	AES-256	支持	高	中	Windows	不用解压工具
Inno Setup	安装包	ISCC compile.iss	无	无	高	中	Windows	可交互安装
NSIS	安装包	makensis script.nsi	无	无	高	中	Windows	脚本驱动
自写 EXE	C#/Python zipfile	写代码调用	AES 可加	可写	高	看实现	全平台	完全可控
分卷恢复记录	RAR/7z	-rr5%	N/A	恢复卷	N/A	N/A	N/A	数据损坏可恢复

打包场景选型要点：如果只需要 zip 格式且不用密码，PowerShell Compress-Archive 最方便（Windows 10 原生自带）；需要密码加密或分卷压缩就必须装 7-Zip 或 WinRAR；要做安装包就用 Inno Setup/NSIS，或 BAT + iexpress 生成自解压包。

密码压缩的安全细节：ZIP 的传统 ZipCrypto 算法很弱（可被已知明文攻击），一定要用 AES-256（7z a -tzip -p 使用的就是 AES-256）。但 AES-256 zip 用 Windows

资源管理器打开时会提示密码错误——因为系统自带的 zip 解压不支持 AES，必须用 7-Zip 或 WinRAR。

多卷分卷（split/volume）对 BAT 最有用的场景是把大目录备份到 FAT32 U 盘（单文件上限 4GB）或通过邮件发送。7z a -v100m 会生成 file.7z.001、file.7z.002…解压时只要对 .001 调用 7z x 就会自动串联所有分卷。

第8章 目录与路径操作

8.1 cd —— 切换目录

`cd`（change directory）是最基础的目录操作命令。

```
rem 切换到子目录
cd Documents

rem 切换到上一级
cd ..

rem 切换到根目录
cd

rem 跨盘符切换（必须加 /d！）
cd /d D:\Work

rem 查看当前目录
cd
```

WARNING

重要：**cd C:\Windows** 不会从 C 盘切到 D 盘，**cd /d D:\Work** 才能跨盘符。这是 cmd 的历史遗留设计——盘符和目录是分开管理的。

8.2 pushd / popd —— 路径栈

这对命令实现了一个"目录栈"。pushd 把当前目录压入栈并切换到新目录；popd 弹出栈顶目录并切换回去。

```
@echo off
cd C:\Users
pushd D:\Work\Project
echo 现在在: %cd%\npushd "E:\Data\Archive"
echo 现在在: %cd%\n
popd
echo 回到: %cd%\n
popd
echo 回到: %cd%
```

pushd/popd 的好处：你不需要记住之前在哪个目录，popd 自动回到上一个位置。它们在临时进入某个目录处理文件，然后返回原处时特别有用。

pushd 的一个隐藏特性：**可以直接 pushd 网络路径**，它会自动映射成临时盘符！

```
pushd \\NAS\Share\Docs
rem 现在会自动有一个 Z: 盘映射到 \\NAS\Share\Docs
popd
rem popd 后 Z: 盘自动断开
```

8.3 dir 和 tree —— 目录遍历

dir 的常用参数

```
rem 基本列表
dir

rem 简洁模式（只显示文件名）
dir /b

rem 递归列出所有文件
dir /s /b

rem 只显示目录名
dir /ad /b
```

```
rem 只显示 .txt 文件
dir /b *.txt

rem 按修改时间排序（从新到旧）
dir /o-d

rem 按大小排序（从大到小）
dir /o-s /s

rem 显示完整路径
dir /s /b
```

tree —— 可视化目录树

```
rem 显示当前目录树
tree

rem 带文件名的完整树
tree /f

rem 导出到文本文件
tree /f &gt; folder_structure.txt
```

8.4 md / rd —— 创建和删除目录

md —— 创建目录

```
rem 创建单个目录
md new_folder

rem 创建嵌套目录（一条命令搞定所有中间层）
md a\b\c\d

rem 批量创建项目结构
md src md tests md docs md build
```

TIP

md 可以一次性创建多级嵌套目录，不需要逐级先创建中间目录。这是 Windows 独有的便利特性。

rd —— 删除目录

```
rem 删除空目录
rd empty_folder

rem 强制删除非空目录及其内容
rd /s /q big_folder

rem /s 递归，/q 安静模式（不确认）
```

8.5 路径拼接与规范化

在 BAT 中处理路径时，经常需要把相对路径转成绝对路径。几个实用技巧：

```
rem 获取脚本自身所在目录
set "SCRIPT_DIR=%~dp0"

rem 基于脚本目录拼接相对路径
set "DATA_DIR=%SCRIPT_DIR%data"

rem 把相对路径转成绝对路径
for %%i in ("relative\path.txt") do set "ABS_PATH=%%~fi"

rem 获取文件所在目录
for %%i in ("%SCRIPT_DIR%..\config.ini") do set "CFG_DIR=%%~dpi"
```

8.6 实战：批量创建项目结构

```
@echo off
setlocal
rem ========== 创建新项目目录结构 ==========
set /p "proj_name=项目名称: "
if "%proj_name%"=="" exit /b
```

```
set "proj_dir=%cd%\%proj_name%"

mkdir "%proj_dir%"

mkdir "%proj_dir%\src"

mkdir "%proj_dir%\src\components"

mkdir "%proj_dir%\src\utils"

mkdir "%proj_dir%\tests"

mkdir "%proj_dir%\docs"

mkdir "%proj_dir%\build"

mkdir "%proj_dir%\deploy"

rem 创建 .gitkeep 占位文件（让空目录能被 git 跟踪）

for /d %%d in ("%proj_dir%\src\components" "%proj_dir%\src\utils" "%proj_dir%\tests" "%proj_dir%\build"
"%proj_dir%\deploy") do (

    echo. &gt; "%%~d\.gitkeep"

)

echo 项目 %proj_name% 创建完毕!

tree /f "%proj_dir%"

endlocal
```

— 第8章 完 —

进阶练习

练习1：PATH 和 cd 的区别：

```
@echo off
echo PATH 决定系统找可执行文件的位置
set "PATH=C:\MyTools;%PATH%"
echo 已添加 C:\MyTools 到 PATH
echo.
echo === cd ===
cd
cd /d D:\ 跨驱动器切换
cd \ 根目录
cd .. 上级
pause
```

练习2：pushd/popd 目录栈：

```
@echo off
echo 初始: %cd%
pushd C:\Windows
echo pushd C:\Windows -^&gt; %cd%
pushd C:\Users
echo pushd C:\Users -^&gt; %cd%
pushd %temp%
echo pushd %%temp%% -^&gt; %cd%
popd &amp;amp; echo popd -^&gt; %cd%
popd &amp;amp; echo popd -^&gt; %cd%
popd &amp;amp; echo popd -^&gt; %cd%
echo.
rem 创建临时目录
set "tmpdir=%temp%\work_%random%"
mkdir "%tmpdir%"
```

```
pushd "%tmpdir%"
echo 在 %cd% 工作...
popd
rmdir /s /q "%tmpdir%"
echo 清理完成，回到 %cd%
pause
```

疑难解答

Q1: %~dp0 末尾反斜杠?

A: %~dp0 总是带尾部反斜杠。建议保留拼接子目录。需要去掉用 %~dp0:~0,-1%。

Q2: cd C:\ 不生效?

A: BAT 默认在当前驱动器切换。跨驱动器必须用 cd /d D:\。

进阶主题1：路径拼接的 7 种方式

BAT 路径拼接的核心是理解 %~dp0 总是带尾部反斜杠。7 种场景覆盖了从简单到复杂的所有情况：直接拼接、参数修饰符、去尾反斜杠、join 风格、处理空格、相对转绝对、跨驱动器。

```
@echo off
echo === 路径拼接的 7 种方式 ===
echo.
echo 假设脚本在 C:\Project\build.bat
echo %~dp0 = C:\Project\  (带尾部反斜杠)
echo.
echo --- 1. 直接拼接 ---
set "full=%~dp0data\config.ini"
echo "%~dp0" + "data\config.ini" = "%full%"
echo.
echo --- 2. 用 %~f0 获取完整路径 ---
echo 脚本完整路径: %~f0
echo 脚本目录: %~dp0
```

```
echo 脚本名:  %~nx0
echo 无扩展名: %~n0
echo 扩展名:  %~x0
echo 驱动器:  %~d0
echo.
echo --- 3. 去掉尾部反斜杠 ---
set "d=%~dp0"
echo 原始: "%d%"
echo 去掉最后一位: "%d:~0,-1%"
echo 再拼接: "%d:~0,-1%\subdir"
echo.
echo --- 4. join 风格拼接 ---
set "root=C:\MyApp"
set "sub=bin\tools"
set "full=%root%\%sub%"
echo "%full%"
echo.
echo --- 5. 处理空格 ---
set "dir1=C:\Program Files\My Company"
set "file=config.ini"
set "full=%dir1%\%file%"
echo 必须用双引号: "%full%"
echo.
echo --- 6. 相对路径转绝对路径 ---
echo 相对: data\file.txt
for %%F in ("data\file.txt") do echo 绝对: %%~fF
echo.
echo --- 7. 跨驱动器安全拼接 ---
set "file=D:\other\script.bat"
for %%F in ("%file%") do echo 父目录: %%~dpF
pause
```

进阶主题2：遍历所有驱动器

遍历驱动器有三种方案：wmic logicaldisk 获取完整信息、for %%D in (A-Z) 逐个检查、fsutil 查询。wmic 最干净，for + exist 最通用。别忘了网络驱动器——用 net use 获取。

```
@echo off
echo === 遍历所有驱动器 ===
echo.
echo --- 1. 用 wmic 获取逻辑驱动器 ---
echo 可用驱动器:
wmic logicaldisk get caption,description,freespace,size /format:table
echo.
echo --- 2. 遍历所有驱动器号 ---
for %%D in (C D E F G H I J K L M N O P Q R S T U V W X Y Z) do (
    if exist %%D:\ (
        if exist %%D:\ (
            echo 驱动器 %%D: 存在
        )
    )
)
echo.
echo --- 3. 用 fsutil 查询驱动器信息 ---
for %%D in (C D E F G H) do (
    if exist %%D:\ fsutil volume diskfree %%D: 2&gt;nul | findstr /c:"总字节数" /c:"可用字节"
)
echo.
echo --- 4. 检查每个驱动器的文件系统 ---
for %%D in (C D E F G H) do (
    if exist %%D:\ (
        for /f "tokens=3" %%t in ('wmic logicaldisk where "DeviceID='%%D:'" get FileSystem ^| findstr /v FileSystem')
do (
            echo %%D: - 文件系统: %%t
        )
    )
```

```
)
echo.
echo --- 5. 网络驱动器 ---
net use 2&gt;nul | findstr /c:"映射" /c:"Microsoft" /c:"远程"
echo 用 fsutil 也能检测:
echo fsutil fsinfo drives
pause
```

进阶主题3：用 pushd 安全切换目录并自动恢复

pushd/popd 维护一个目录栈。pushd 切换同时入栈，popd 出栈并回去。这在临时目录工作、网络路径访问等场景特别有用——脚本结束时自动回到原目录，不会迷路。

```
@echo off
echo === 用 pushd 安全切换目录并自动恢复 ===
echo.
echo pushd 的栈机制:
pushd C:\
echo 切换到 C:\, 入栈当前目录
pushd %APPDATA%
echo 切换到 %APPDATA%, 再次入栈
pushd %TEMP%
echo 切换到 %TEMP%, 栈深度=3
echo.
echo popd 逐个出栈:
popd &amp;amp; echo popd - now at %cd%
popd &amp;amp; echo popd - now at %cd%
popd &amp;amp; echo popd - now at %cd%
echo.
echo --- 1. 安全工作区模式 ---
setlocal
set "work=%TEMP%\build_%random%"
mkdir "%work%"
pushd "%work%"
echo 开始在 "%cd%" 工作...
```

```
echo 创建文件 &gt; output.txt
echo 处理中...
popd
echo 回到 %cd%
echo 工作完成，清理...
rmdir /s /q "%work%"
endlocal
echo.
echo --- 2. 网络路径也能用 pushd ---
echo pushd \\server\share 会自动映射驱动器
echo pushd C:\Windows 切完自动 popd 回去
echo.
echo --- 3. pushd /? 帮助 ---
pushd /?
echo.
echo --- 4. 注意事项 ---
echo pushd 不能跨 cmd 实例，栈随 cmd 关闭而清空
echo 如果目标目录不存在 pushd 会报错且不入栈
pushd "C:\不存在的目录 2&gt;nul || echo pushd 失败不入栈
echo 当前 %cd% 没变: %cd%
pause
```

表 8-1 路径修饰符速查

修饰符	返回什么	示例输入	示例输出	关键特性
%~f1	完整绝对路径	file.txt	C:\WD\file.txt	最常用修饰符
%~d1	驱动器号	C:\file.txt	C:	只有盘符
%~p1	目录路径	C:\Users\f.txt	\Users\	含末尾 \
%~n1	主文件名	report.docx	report	不含后缀
%~x1	扩展名	report.docx	.docx	含点号
%~s1	8.3 短路径	C:\PROGRA1\...	C:\PROGRA1\...	兼容旧程序

%~a1	文件属性	report.txt	--a------	r/h/s/a
%~t1	修改日期时间	report.txt	11/10/26 15:30	本地化格式
%~z1	文件大小（字节）	report.txt	1024	整数
%~$PATH:1	在 PATH 中搜索	ping	C:\Win\System32\ping.exe	唯一要加 PATH:
%~dp1	驱动器+目录	C:\Users\f.txt	C:\Users\	组合修饰符
%~nx1	文件名+后缀	report.docx	report.docx	最常用组合
%~fs1	完整短路径	report.txt	C:\WD\REPORT~1.TXT	兼容旧 API
%~ftza1	全部信息	report.txt	a-- 1024 11/10/26 C:\...	大组合
%~dp0	脚本所在目录	(自动)	C:\script_dir\	几乎每脚本都用
%~nx0	脚本文件名	(自动)	my_script.bat	日志标题
%~f0	脚本完整路径	(自动)	C:\WD\my_script.bat	self-reference
%~1	去引号参数	"hello world"	hello world	call 传递时关键
%~a1 组合	全部属性位	--a------	r=只读 h=隐藏 s=系统 a=归档	8位
%~t1 解析	可再 %~t1:~0,10	11/10/2026 15:30	可以用字符串操作再加工	时间戳再解析

表 8-2 路径拼接 7 种方式对比

方案	复杂度	可靠性	处理 . / ..	处理重复 \\	处理引号	多段拼接	性能	推荐
直接字符串 + "\\"	1/10	3/10			手动	手动拼	极快	临时脚本
%%~fP 自动规范化	2/10	8/10			自动	需 %%~fP	快	BAT 首选
PowerShell [IO.Path]::Combine	6/10	10/10			自动		中（启动开销）	复杂生产脚本
pushd + for %%f	4/10	7/10			自动	不直接支持	中	需要解析相对路径

手动去斜杠拼接	5/10	9/10			手动	需要循环	快	完全可控但麻烦
certutil /fsutil	8/10	8/10	部分		不相关	不支持	中	不做路径拼接
WMI ExecMethod("Join")	9/10	10/10			自动		慢	过度工程化

第 8 章小结

路径修饰符表把 14 种基础 + 6 种组合全部列清。最核心的几个：%~dp0 脚本目录、%~nx1 文件名、%~f1 完整路径、%~1 去引号。BAT 写脚本的第一条纪律——任何涉及文件路径的操作，先想到能不能用修饰符。

路径拼接对比把 BAT 里常见的 7 种路径拼接方案按可靠性从低到高排序：直接拼（最简但易错）→ %%~fP（BAT 内置首选）→ PowerShell [IO.Path]::Combine（最标准）。记住 BAT 里 %%~fP 会自动规范化 . 和 \.\，但不会处理跨盘符相对路径。

路径工具箱项目把 10 个最常用的路径操作封装成 call 子程序：取父目录、取扩展名、取文件名、去扩展名、多段拼接、绝对化、相对化、规范化、取目录基名。可以整段复制到自己的脚本里当函数库用。

路径相等判断的坑：Windows 路径不区分大小写，所以不能直接 if "!a!"=="!b!" 比较。正确做法是 if /i "!np1!"=="!np2!" 或者先 %%~fP 规范化再比较。UNC 路径的修饰符行为不一样——%%~dpUNC 会把 \\server\share 拆到根级别。

长路径（>260 字符）在现代 Windows 上需要注册表开启 HKLM\SYSTEM\CurrentControlSet\Control\FileSystem\LongPathsEnabled=1，或者用 \?\ 前缀绕过。BAT 的 for %%P 在长路径下可能有问题，复杂情况交给 PowerShell。

下一章进入进程管理和注册表。tasklist/taskkill、wmic process、注册表 HKLM/HKCU/HKCR/HKU/HKCC 五大根键的结构和操作，准备好了吗？

第 8 章大代码块 1：路径处理深入演示

```
@echo off

setlocal enabledelayedexpansion
```

```
chcp 65001 &gt;nul 2&gt;&amp;1
echo ============================================
echo   第8章综合实战：路径处理深入演示
echo ============================================
echo.
set "sample=C:\Users\Harry\Documents\report.docx"
echo 示例: !sample!
echo.
rem === 1. 所有 14 个修饰符 ===
echo === 所有修饰符 ===
set "idx=0"
for %%P in ("!sample!") do (
    for %%A in (%%~dP %%~pP %%~nP %%~xP %%~fP %%~sP %%~aP %%~tP %%~zP %%~dpP %%~nxP %%~ftzaP %%~dp0 %%~nx0) do (
        set /a idx+=1
        echo   [!idx!] %%A
    )
)
echo.
rem === 2. UNC 路径 ===
echo === UNC 路径（网络共享） ===
set "unc_sample=\\server01\share\folder\file.txt"
echo UNC示例: !unc_sample!
for %%P in ("!unc_sample!") do (
    echo   %%~fP
    echo   %%~dpP
    echo   %%~nxP
)
echo.
rem === 3. 相对路径转绝对 ===
echo === 相对路径转绝对 ===
cd "%temp%"
set "rel=..\Windows\System32\cmd.exe"
```

```
echo 相对: !rel!
for %%P in ("!rel!") do echo 绝对: %%~fP
cd /d "%~dp0"
echo.
rem === 4. 长路径（&gt;260字符）===
echo === 长路径支持 ===
setlocal enableextensions
set "long_prefix=\?\"
echo   传统路径最大 260 字符
echo   长路径前缀: \?\ 可以突破限制
endlocal
echo.
rem === 5. 规范化路径 ===
echo === 规范化（去除冗余 \ 和 .）===
set "messy=C:\Users\..\Harry\Documents\.\report.txt"
echo 混乱: !messy!
for %%P in ("!messy!") do echo 规范: %%~fP
echo.
rem === 6. 父目录 ===
echo === 父目录提取 ===
set "fullpath=C:\Users\Harry\Documents\report.docx"
for %%P in ("!fullpath!") do (
    echo 完整路径: %%~fP
    echo 父目录:   %%~dpP
    rem 去掉末尾 \
    set "parent=%%~dpP"
    echo 父目录(无\): !parent:~0,-1!
)
echo.
rem === 7. 比较两个路径 ===
echo === 路径相等判断 ===
set "p1=C:\Windows\System32\cmd.exe"
```

```
set "p2=C:\Windows\SYSTEM32\cmd.exe"
for %%P in ("!p1!") do set "np1=%%~fP"
for %%P in ("!p2!") do set "np2=%%~fP"
echo  p1=!p1!
echo  p2=!p2!
echo  规范化后: !np1! == !np2! ?
if /i "!np1!"=="!np2!" echo  YES（Windows 路径不区分大小写）
pause
```

第 8 章大代码块 2：路径拼接 7 种方案对比

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo  第8章进阶：路径拼接 7 种方案对比
echo  比较各自的优缺点和边界情况
echo ============================================
echo.
set "base=C:\Users\Harry"
set "rel1=Documents"
set "rel2=\report.txt"
echo 基础参数:
echo  base = !base!
echo  rel1 = !rel1!
echo  rel2 = !rel2!
echo.
echo [方案 1] 直接字符串拼接:
set "r1=!base!\!rel1!\!rel2!"
echo  !r1!
echo  缺点: 可能出现双斜杠或缺少斜杠
echo.
```

```
echo [方案 2] 用 %%~fP 自动规范化:
for %%P in ("!base!\!rel1!\!rel2!") do set "r2=%%~fP"
echo   !r2!
echo   优点: 自动去除 . 和 \.\
echo.
echo [方案 3] PowerShell 处理:
for /f "delims=" %%P in ('powershell -NoProfile -Command "[IO.Path]::Combine('"!base!"', '"!rel1!"', '"!rel2!"')"') do set "r3=%%P"
echo   !r3!
echo   优点: 最标准的路径拼接
echo.
echo [方案 4] 用 cd 到 base 再 for %%f:
pushd "!base!"
for %%f in ("!rel1!\!rel2!") do set "r4=%%~ff"
popd
echo   !r4!
echo   优点: 同时解析相对路径
echo.
echo [方案 5] 手动去斜杠拼接:
set "base_clean=!base!
if "!base_clean:~-1!"=="\" set "base_clean=!base_clean:~0,-1!"
set "rel_clean=!rel2!"
if "!rel_clean:~0,1!"=="\" set "rel_clean=!rel_clean:~1!"
set "r5=!base_clean!\!rel1!\!rel_clean!"
echo   !r5!
echo   优点: 完全可控
echo.
echo [方案 6] certutil 配合:
echo  在 BAT 中 certutil 不做路径拼接，仅做文件操作
echo.
echo [方案 7] cmdline 转义技巧:
set "cmd="!base!\!rel1!\!rel2!""
```

```
echo   !cmd!
echo   用于命令行参数时记得加引号
echo.
pause
```

第 8 章进阶项目：路径工具箱 - 10 个路径处理函数

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ============================================
echo   第8章项目：路径工具箱 - 10 个路径处理函数
echo   可直接 call 调用
echo ============================================
echo.
set "sample=C:\Users\Harry\Documents\project\src\main.py"
echo 测试样本: !sample!
echo.
call :path_parent "!sample!"
call :path_extension "!sample!"
call :path_filename "!sample!"
call :path_stem "!sample!"
call :path_without_ext "!sample!"
call :path_join "C:\Users\Harry" "Documents" "project"
call :path_absolute "Documents"
call :path_relative "C:\Users\Harry\Documents" "C:\Users\Harry"
call :path_norm "C:\Users\Harry\..\Harry\.\Documents"
call :path_basename_dir "C:\Users\Harry\Documents"
echo.
echo （输出已在上方）
pause
```

```
exit /b 0
:path_parent
set "p=%~1"
for %%P in ("!p!") do (
    set "r=%%~dpP"
    if "!r:~-1!"=="\" set "r=!r:~0,-1!"
)
echo   :path_parent = !r!
exit /b 0
:path_extension
set "p=%~1"
for %%P in ("!p!") do echo   :path_extension = %%~xP
exit /b 0
:path_filename
set "p=%~1"
for %%P in ("!p!") do echo   :path_filename = %%~nxP
exit /b 0
:path_stem
set "p=%~1"
for %%P in ("!p!") do echo   :path_stem = %%~nP
exit /b 0
:path_without_ext
set "p=%~1"
for %%P in ("!p!") do (
    set "d=%%~dpP"
    set "n=%%~nP"
    echo   :path_without_ext = !d!!n!
)
exit /b 0
:path_join
set "r=%~1"
shift
```

```
:pj_loop
if "%~1"=="" goto pj_done
set "seg=%~1"
if "!r:~-1!"=="\" set "r=!r:~0,-1!"
if "!seg:~0,1!"=="\" set "seg=!seg:~1!"
set "r=!r!\!seg!"
shift
goto pj_loop
:pj_done
echo   :path_join = !r!
exit /b 0
:path_absolute
set "p=%~1"
cd "%~dp0"
for %%P in ("!p!") do echo   :path_absolute = %%~fP
exit /b 0
:path_relative
set "from=%~1"
set "to=%~2"
for /f "delims=" %%r in ('powershell -NoProfile -Command "$f='!from!'; $t='!to!'; [IO.Path]::GetRelativePath($t,$f)"')
do echo   :path_relative = %%r
exit /b 0
:path_norm
set "p=%~1"
for %%P in ("!p!") do echo   :path_norm = %%~fP
exit /b 0
:path_basename_dir
set "p=%~1"
for %%P in ("!p!") do echo   :path_basename_dir = %%~nP
exit /b 0
```

进阶项目8：智能路径别名管理

这个脚本实现了 BAT 开发者梦寐以求的功能：像 Linux 的 ~/.bash_aliases 一样管理常用路径。配置文件存 %USERPROFILE%\.pathalias\aliases.ini，每次 alias add 同时 setx 持久化 + set 当前会话生效。配合 doskey 宏和 cmd AutoRun 注册表，可以做到"新开一个 cmd 窗口就自动加载所有别名"。

```
@echo off
setlocal enabledelayedexpansion
title 智能路径别名管理器 v1.0

rem ============================================================
rem  智能路径别名管理器
rem  用 setx 持久化别名 + 配置文件 + 自动补全
rem  用法: pathalias add name "C:\Long\Path\To\Dir"
rem        pathalias use name
rem        pathalias list
rem        pathalias del name
rem ============================================================

set "ALIAS_DIR=%USERPROFILE%\.pathalias"
set "ALIAS_FILE=!ALIAS_DIR!\aliases.ini"

rem ========== 初始化 ==========
if not exist "!ALIAS_DIR!" mkdir "!ALIAS_DIR!" 2&gt;nul
if not exist "!ALIAS_FILE!" (
    echo ; PathAlias 配置文件 &gt; "!ALIAS_FILE!"
    echo ; 格式: alias_name=full_path &gt;&gt; "!ALIAS_FILE!"
    echo ; 修改后用 pathalias reload 生效 &gt;&gt; "!ALIAS_FILE!"
)

rem ========== 参数分发 ==========
set "CMD=%~1"
if "!CMD!"=="" goto :interactive
if /i "!CMD!"=="add" goto :cmd_add
if /i "!CMD!"=="set" goto :cmd_add
```

```
if /i "!CMD!"=="del" goto :cmd_del
if /i "!CMD!"=="remove" goto :cmd_del
if /i "!CMD!"=="use" goto :cmd_use
if /i "!CMD!"=="cd" goto :cmd_use
if /i "!CMD!"=="list" goto :cmd_list
if /i "!CMD!"=="ls" goto :cmd_list
if /i "!CMD!"=="show" goto :cmd_list
if /i "!CMD!"=="reload" goto :cmd_reload
if /i "!CMD!"=="init" goto :cmd_init
if /i "!CMD!"=="help" goto :cmd_help
if /i "!CMD!"=="/?" goto :cmd_help
echo 未知命令: !CMD!
goto :cmd_help

:interactive
cls
echo.
echo   === 路径别名管理器 ===
echo   配置文件: !ALIAS_FILE!
echo.
echo   [1] 添加别名   [2] 使用别名   [3] 列出全部
echo   [4] 删除别名   [5] 重新加载   [6] 初始化 doskey
echo   [0] 退出
echo.
set /p choice=选择:
if "!choice!"=="1" goto :cmd_add
if "!choice!"=="2" goto :cmd_use
if "!choice!"=="3" goto :cmd_list
if "!choice!"=="4" goto :cmd_del
if "!choice!"=="5" goto :cmd_reload
if "!choice!"=="6" goto :cmd_init
if "!choice!"=="0" exit /b
```

```
goto :interactive

:cmd_add
set "ANAME=%~2"
set "APATH=%~3"
if "!ANAME!"=="" (
    set /p "ANAME=别名名称(英文): "
)
if "!APATH!"=="" (
    set /p "APATH=目标路径: "
)
if "!ANAME!"=="" (echo [错误] 别名名不能为空 &amp;amp; pause &amp;amp; exit /b 1)
if "!APATH!"=="" (echo [错误] 路径不能为空 &amp;amp; pause &amp;amp; exit /b 1)

rem 规范化路径
for %%a in ("!APATH!") do set "APATH=%%~fa"

rem 写配置文件（先删旧的再追加）
findstr /v /i /c:"!ANAME!=" "!ALIAS_FILE!" &gt; "!ALIAS_FILE!.tmp" 2&gt;nul
echo !ANAME!=!APATH! &gt;&gt; "!ALIAS_FILE!.tmp"
move /y "!ALIAS_FILE!.tmp" "!ALIAS_FILE!" &gt;nul

rem setx 持久化环境变量
setx PA_!ANAME! "!APATH!" &gt;nul

rem 当前会话也立即生效
set "PA_!ANAME!=!APATH!"

echo.
echo 已添加别名:
echo   !ANAME! = !APATH!
echo.
echo 现在可以: cd /d %%PA_!ANAME!%%
echo 或: pathalias use !ANAME!
echo.
pause
```

```
exit /b

:cmd_use
set "ANAME=%~2"
if "!ANAME!"=="" set /p "ANAME=使用哪个别名: "
set "TARGET=!PA_!ANAME!!"
if "!TARGET!"=="" (
    rem 从配置文件读取
    for /f "tokens=2 delims==" %%a in ('findstr /i /c:"!ANAME!=" "!ALIAS_FILE!"') do set "TARGET=%%a"
)
if "!TARGET!"=="" (
    echo [错误] 别名 "!ANAME!" 未定义
    echo 用 pathalias add 先添加
    pause
    exit /b 1
)
cd /d "!TARGET!"
echo 已切换到: !TARGET!
exit /b

:cmd_list
echo.
echo ============ 已定义的别名 ============
for /f "usebackq tokens=1,2 delims==" %%a in ("!ALIAS_FILE!") do (
    if not "%%a"=="" (
        if "%%a:~0,1!"==";" goto :skip_comment
        echo   %%-15s = %%b
        :skip_comment
    )
)
echo ========================================
echo.
echo 环境变量前缀 PA_（可用 set PA_* 查看）
```

```
pause
exit /b

:cmd_del
set "ANAME=%~2"
if "!ANAME!"=="" set /p "ANAME=删除哪个别名: "
findstr /v /i /c:"!ANAME!=" "!ALIAS_FILE!" &gt; "!ALIAS_FILE!.tmp" 2&gt;nul
move /y "!ALIAS_FILE!.tmp" "!ALIAS_FILE!" &gt;nul 2&gt;&amp;1
rem 清除持久化变量
reg delete "HKCU\Environment" /v PA_!ANAME! /f &gt;nul 2&gt;&amp;1
echo 已删除别名 !ANAME!
pause
exit /b

:cmd_reload
echo 重新加载所有别名...
for /f "usebackq tokens=1,2 delims==" %%a in ("!ALIAS_FILE!") do (
    if not "%%a"=="" (
        if not "%%a:~0,1!"==";" (
            set "PA_%%a=%%b"
        )
    )
)
echo 已从 !ALIAS_FILE! 重新加载
pause
exit /b

:cmd_init
echo 正在初始化 doskey 自动补全...
doskey alias=pathalias $*
doskey pa=pathalias $*
doskey add=pathalias add $*
doskey del=pathalias del $*
doskey use=pathalias use $*
```

```
doskey ls=pathalias list
echo 已定义 doskey 别名: alias, pa, add, del, use, ls
echo 建议把下面这行加到 AutoRun 里:)
echo   reg add "HKCU\Software\Microsoft\Command Processor" /v AutoRun /t REG_SZ /d "pathalias reload" /f
pause
exit /b

:cmd_help
echo.
echo === 路径别名管理器 帮助 ===
echo.
echo pathalias add ^&lt;name^&gt; ^&lt;path^&gt;    添加别名
echo pathalias use ^&lt;name^&gt;          切换目录
echo pathalias del ^&lt;name^&gt;          删除别名
echo pathalias list                列出全部
echo pathalias reload              重新加载
echo pathalias init                初始化 doskey
echo pathalias help                本帮助
echo.
echo 配置文件: !ALIAS_FILE!
echo 持久化:  setx PA_^&lt;name^&gt;
pause
exit /b
```

25 种路径别名高频场景

路径操作	传统方法	缺点	别名管理	实现方式	持久化	自动补全	建议
cd 深目录	cd /d C:\Dev\Project\BigLongName	输入多易错	pa proj	cd /d %%PA_proj%%	setx PA_proj	doskey	最常用
常用工具目录	cd C:\Program Files\Git\cmd	含空格要引号	pa git	set PA_git=...	setx	doskey	必配

临时目录	cd %TEMP%	记 %TEMP% 还可以	pa tmp	setx PA_tmp=%TEMP%	setx	自动	可选
多环境切换	手动改 env 变量	容易改漏	pa py39/py312	两个 alias 并行	setx x2	doskey	Python 开发者
公司/家目录	手动切换 D:\Work vs E:\Home	经常忘	pa work/pa home	双 alias	setx	doskey	混合办公
项目快速访问	每次资源管理器拖路径	麻烦	pa p1~pa p10	批量定义	setx x10	doskey	多项目
网络路径	\\server\share\folder	记忆困难	pa srv	\srv\share\folder	setx	doskey	域环境
下载目录	cd %USERPROFILE%\Downloads	OK 但长	pa dl	setx PA_dl	setx	自动	日常
桌面目录	cd %USERPROFILE%\Desktop	OK 但长	pa desk	setx PA_desk	setx	自动	日常
注册表持久化	reg add HKCU…Run	复杂	pathalias auto-reload	AutoRun load ini	注册表	启动加载	建议配
doskey 宏	doskey ls=dir /b	仅当前窗口	pathalias init	保存宏到文件	doskey /macrofile	自动加载	效率倍增
PATH 追加	set PATH=%PATH%;C:\new	仅当前会话	pa bin	配置文件 PATH_LIST	启动时 for /f 加	自动	开发机
虚拟环境	activate.bat 路径长	pa venv	cd + call activate	setx	doskey	Python 必配	
SSH 隧道	手动 ssh -L	命令长参数多	pa tunnel	ssh config + alias	SSH config	自动	运维
Docker 容器	docker exec -it container	容器名长	pa db	docker exec	无	doskey	容器化场景
Git 仓库	cd C:\Repos\myproject	路径长	pa repo	setx PA_repo	setx	doskey	Git 开发者
虚拟机共享	\\vboxsvr\share	固定	pa vm	setx PA_vm	setx	doskey	VM 用户
FTP/SFTP	手动 ftp 命令	参数多	pa ftp	sftp config	.ssh/config	自动	运维

备份目录	D:\Backup\2026\week45	每周变	pa backup	脚本动态 set	setx + 定时	手动	备份
云盘目录	C:\Users\X\OneDrive\...	路径深	pa cloud	setx	setx	doskey	云同步
项目构建输出	cd build\debug	多层	pa debug	setx	setx	doskey	开发者
日志目录	C:\Logs\App1\2026-11	日期变	pa log	脚本动态	setx + 日定时	手动	运维
脚本目录	cd C:\Scripts\BAT	OK	pa bat	setx	setx	doskey	脚本作者
配置目录	cd C:\Windows\System32\drivers\etc	系统深层	pa etc	管理员才能进	setx	doskey	系统管理
用户配置导入	pathalias import file.ini	批量创建	import 命令	for /f 解析	文件	手动	新机器部署

setx 的坑：它写 HKCU\Environment 注册表，新 cmd 窗口才生效，当前窗口不生效。所以脚本里 add 完还要手动 set "PA_xxx=..." 让当前会话立即可用。还有 setx 不能直接展开 %USERPROFILE%——它会把 % 当成字面字符。必须用 set /p 先把 %USERPROFILE% 展开到变量再传 setx。

cmd AutoRun 注册表项是让别名自动加载的关键：reg add "HKCU\Software\Microsoft\Command Processor" /v AutoRun /t REG_SZ /d "pathalias reload" /f。这样每次启动 cmd.exe 都会自动执行 pathalias reload，把所有 ini 里的别名 set 到当前会话。但注意 AutoRun 也会在某些安全场景（如 regsvr32）执行，要小心。

doskey 宏可以让命令输入更短：doskey pa=pathalias $* 意味着输入 pa add proj "C:\Projects" 就等于输入完整命令。doskey /macrofile=macros.txt 可以把宏保存下来配合 AutoRun 加载。BAT 的自动化体验全靠这套"配置文件 + 环境变量 + doskey + AutoRun"组合拳。

第9章 系统管理命令

BAT 脚本真正强大的地方在于——它可以直接调用 Windows 庞大的系统管理命令集。本章讲解进程、服务、注册表、计划任务等核心系统操作。

9.1 进程管理：tasklist / taskkill

tasklist —— 查看进程

```
rem 列出所有进程（默认表格格式）
tasklist

rem CSV 格式（便于解析）
tasklist /fo csv

rem 只列出包含 "sql" 的进程
tasklist /fi "imagename eq sql*"

rem 显示进程关联的服务
tasklist /svc

rem 显示详细内存/CPU使用
tasklist /v

rem 远程机器上查看进程
tasklist /s 192.168.1.100 /u admin /p password
```

taskkill —— 结束进程

```
rem 按进程名结束（所有同名实例）
taskkill /im notepad.exe

rem 按 PID 结束
taskkill /pid 1234

rem 强制结束（不弹"是否结束"对话框）
taskkill /f /im notepad.exe

rem 结束进程树（包括子进程）
taskkill /f /im chrome.exe /t

rem 批量结束多个进程
taskkill /im notepad.exe /im calc.exe /im mspaint.exe

rem 远程结束进程
```

```
taskkill /s 192.168.1.100 /f /im notepad.exe
```

WARNING

taskkill /f 是暴力结束——进程没有机会保存数据就被杀了。如果可能，先用不带 /f 的 taskkill 给进程一个优雅退出的机会。

9.2 start —— 启动程序

```
rem 启动一个程序
start notepad.exe

rem 带参数启动
start "My Title" calc.exe

rem 等待程序结束后继续
start /wait setup.exe

rem 最小化启动
start /min cmd.exe

rem 后台静默运行（无窗口）
start /b /wait "" "silent_app.exe" /s

rem 在指定目录下启动
start /d "D:\Data" explorer.exe .

rem 打开文件（用默认关联程序）
start "" report.pdf
start "" https://www.bing.com
start "" mailto:test@example.com
```

WARNING

start 的第一个参数是**窗口标题**，可以为空但不能省略——所以经常看到 `start "" app.exe`，两个引号就是为了提供一个空标题参数。如果你直接写 start app.exe，app.exe 会被当成标题！

9.3 systeminfo 与 wmic

systeminfo

```
rem 查看完整系统信息

systeminfo

rem 导出到文件

systeminfo &gt; my_pc_info.txt 2&gt;&amp;1

rem 远程查询

systeminfo /s 192.168.1.100 /u admin /p pwd
```

wmic —— Windows Management Instrumentation

`wmic` 是 Windows 上最强大的系统查询工具，可以访问几乎所有管理信息。

```
rem 操作系统信息

wmic os get Caption,Version,BuildNumber,OSArchitecture

rem CPU 信息

wmic cpu get Name,NumberOfCores,NumberOfLogicalProcessors,MaxClockSpeed

rem 内存信息

wmic memorychip get Capacity,Speed,Manufacturer

rem 磁盘信息

wmic diskdrive get Model,Size,MediaType

rem 已安装的软件

wmic product get Name,Version,InstallDate

rem 网卡信息

wmic nic where "PhysicalAdapter=True" get Name,MACAddress,Speed

rem 启动时间

for /f "tokens=2 delims==" %i in ('wmic os get lastbootuptime /value ^| find "="') do @echo 上次启动: %i
```

9.4 reg —— 注册表操作

注册表是 Windows 的"总配置中心"。reg 命令可以增删查改注册表项。

```
rem 查询注册表
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion

rem 写入注册表
reg add "HKLM\SOFTWARE\MyApp" /v Version /t REG_SZ /d "1.0.0" /f

rem /v  值名（Value name）
rem /t  值类型（REG_SZ, REG_DWORD, REG_EXPAND_SZ, REG_MULTI_SZ, REG_BINARY）
rem /d  值数据（Data）
rem /f  强制覆盖不确认

rem 删除值
reg delete "HKLM\SOFTWARE\MyApp" /v Version /f

rem 删除整个键
reg delete "HKLM\SOFTWARE\MyApp" /f

rem 导出注册表
reg export HKLM\SOFTWARE\MyApp myapp_registry.reg

rem 导入注册表
reg import myapp_registry.reg
```

NOTE

注册表根键缩写：

缩写	完整名称	含义
HKLM	HKEY_LOCAL_MACHINE	系统级配置（所有用户）
HKCU	HKEY_CURRENT_USER	当前用户配置
HKCR	HKEY_CLASSES_ROOT	文件类型关联
HKU	HKEY_USERS	所有用户配置
HKCC	HKEY_CURRENT_CONFIG	当前硬件配置

9.5 schtasks —— 计划任务

创建计划任务

```
rem 每天凌晨2点运行备份脚本
schtasks /create /tn "DailyBackup" /tr "D:\Scripts\backup.bat" /sc daily /st 02:00 /f

rem 每周一早上8点运行
schtasks /create /tn "WeeklyReport" /tr "D:\Scripts\report.bat" /sc weekly /d MON /st 08:00 /f

rem 系统启动时运行
schtasks /create /tn "AutoSync" /tr "D:\Scripts\sync.bat" /sc onstart /rl HIGHEST /f

rem 用户登录时运行
schtasks /create /tn "WelcomeMsg" /tr "msg * 欢迎回来！" /sc onlogon /f
```

管理计划任务

```
rem 列出所有任务
schtasks /query /fo csv /v

rem 运行指定任务
schtasks /run /tn "DailyBackup"

rem 结束正在运行的任务
schtasks /end /tn "DailyBackup"

rem 删除任务
schtasks /delete /tn "DailyBackup" /f

rem 导出任务配置
schtasks /query /tn "DailyBackup" /xml &gt; task.xml

rem 从 XML 导入任务
schtasks /create /tn "DailyBackup" /xml task.xml /f
```

9.6 服务管理

sc（Service Control）是专业的服务管理工具，比 net start/stop 功能更全：

```
rem 查看所有服务
sc query type= service state= all
```

```
rem 启动/停止服务
sc start Spooler
sc stop Spooler

rem 查询单个服务详情
sc query W32Time
sc qc W32Time  &amp;amp; rem 显示配置

rem 创建服务
sc create MyService binPath= "D:\MyApp\service.exe" start= auto

rem 删除服务（先停止再删）
sc stop MyService
sc delete MyService

rem 改变服务启动类型
sc config Spooler start= disabled    &amp;amp; rem 禁用
sc config Spooler start= auto        &amp;amp; rem 自动
sc config Spooler start= demand      &amp;amp; rem 手动
```

WARNING

sc 的参数里 = 后面必须有一个空格！正确：`start= auto`，错误：`start=auto`。这是 sc 的一个非常"坑"的设计。

9.7 net user / net share

```
rem 创建新用户（需要管理员权限）
net user newuser Password123 /add

rem 设置用户密码
net user newuser NewPassword

rem 删除用户
net user newuser /del

rem 创建共享文件夹
net share Docs=D:\Documents /grant:everyone,FULL
```

```
rem 删除共享
net share Docs /del

rem 列出所有共享
net share

rem 查看会话
net session
```

9.8 netsh —— 网络配置

```
rem 设置静态 IP
netsh interface ip set address "以太网" static 192.168.1.100 255.255.255.0 192.168.1.1

rem 设置 DNS
netsh interface ip set dns "以太网" static 8.8.8.8 primary

rem 改为自动获取 DHCP
netsh interface ip set address "以太网" dhcp
netsh interface ip set dns "以太网" dhcp

rem 导出/导入网络配置
netsh dump &gt; network_config.txt
netsh exec network_config.txt

rem 配置防火墙规则
netsh advfirewall firewall add rule name="Allow SSH" dir=in action=allow protocol=TCP localport=22
```

9.9 实战：开机自启脚本

综合本章所学，写一个能让脚本"开机自动运行"的方案：

```
@echo off
rem ========== 方案一：放入启动文件夹 ==========
rem 对当前用户有效
set "STARTUP_DIR=%appdata%\Microsoft\Windows\Start Menu\Programs\Startup"
copy /y "%~f0" "%STARTUP_DIR%\%~nx0"
```

```
echo 已添加到启动文件夹（当前用户）

rem ========== 方案二：注册表 Run 键 ==========
rem HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run（当前用户）
rem HKLM\...\Run（所有用户，需要管理员权限）
reg add "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "MyTool" /t REG_SZ /d ""%~f0"" /f

rem ========== 方案三：计划任务 ==========
schtasks /create /tn "MyAutoTask" /tr ""%~f0"" /sc onlogon /f

echo 三种开机自启方式已配置
```

— 第9章 完 —

进阶练习

练习1：tasklist 和 taskkill：

```
@echo off
echo === 进程管理 ===
tasklist /fo table
echo.
echo notepad 进程:
tasklist /fi "imagename eq notepad.exe"
echo.
rem taskkill /f /im notepad.exe
pause
```

练习2：注册表操作：

```
@echo off
echo === 注册表操作 ===
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion"
reg add "HKCU\Software\MyBAT" /v Version /t REG_SZ /d "1.0" /f
reg add "HKCU\Software\MyBAT" /v Count  /t REG_DWORD /d 42 /f
for /f "tokens=2*" %%a in (
  reg query "HKCU\Software\MyBAT" /v Version
) do set "ver=%%b"
echo 版本: %ver%
reg delete "HKCU\Software\MyBAT" /f
pause
```

练习3：wmic 系统信息：

```
@echo off
echo --- CPU ---
wmic cpu get Name,NumberOfCores /format:list
echo --- 内存 ---
wmic memorychip get Capacity,Speed /format:list
```

```
echo --- 磁盘 ---
wmic diskdrive get Model,Size /format:list
echo --- 操作系统 ---
wmic os get Caption,Version /format:list
pause
```

疑难解答

Q1: "拒绝访问" 怎么办?

A: 需要管理员权限。HKLM 注册表、修改服务等都需要提升。

Q2: wmic 输出太丑?

A: 用 /format:list 比表格好看。PowerShell Get-CimInstance 更好。

进阶主题1: 用 tasklist 筛选特定进程并批量 kill

tasklist /fi 可以按进程名、PID、内存、会话等条件筛选，/fo csv 输出可以管道给 PowerShell 进一步处理排序。taskkill /f /im 批量杀进程很危险，先 tasklist 看清楚再 kill。/t 参数会连子进程一起杀。

```
@echo off
setlocal enabledelayedexpansion
echo === tasklist 筛选并批量 kill ===
echo.
echo --- 1. tasklist /fi 筛选 ---
echo 只看 notepad:
tasklist /fi "imagename eq notepad.exe"
echo.
echo 只看 PID 大于 1000 的:
tasklist /fi "pid gt 1000" /fo csv | head -5
echo.
echo 按内存排序 (找内存大户):
powershell -Command "tasklist /fo csv ^| ConvertFrom-Csv ^| Sort-Object WorkingSet64 -Descending ^| Select-Object -First 10 Name,@{N='Mem(MB)';E={[Math]::Round($_.WorkingSet64/1MB,1)}} ^| Format-Table -AutoSize"
echo.
```

```
echo --- 2. 批量 kill by imagename ---
echo 先启动几个 notepad...
start "" notepad.exe
start "" notepad.exe
start "" notepad.exe
timeout /t 1 &gt;nul
echo 现在有这些 notepad:
tasklist /fi "imagename eq notepad.exe" /fo csv
echo.
echo 批量 kill: taskkill /f /im notepad.exe
taskkill /f /im notepad.exe &gt;nul
echo 已 kill 所有 notepad
echo.
echo --- 3. 按内存占用 kill 大户 ---
echo 杀掉内存超过 500MB 的进程 (谨慎!):
echo for /f "tokens=5" %m in ('tasklist /fi "memusage gt 500000" ^| findstr /v K') do taskkill /f /pid %m
echo (需要在命令行手动跑)
echo.
echo --- 4. 优雅关闭 vs 强杀 ---
echo taskkill /im notepad.exe        优雅关闭 (发 WM_CLOSE)
echo taskkill /f /im notepad.exe     强制终止 (TerminateProcess)
echo taskkill /t /im cmd.exe         连同子进程一起杀
pause
```

进阶主题2：用 sc 创建自启动服务

sc 是服务管理命令行工具。原生 sc 只能把专门的服务程序注册成服务，普通 exe 需要用 nssm 或 srvany 包装。Service_AUTO_START 是开机自启动，Service_DISABLED 是禁用。修改后重启生效。

```
@echo off
echo === 用 sc 创建自启动服务 ===
echo (需要管理员权限)
echo.
echo --- 1. sc 命令基础 ---
```

```
echo 查询服务: sc query ServiceName
echo 创建服务: sc create MyService binPath= "C:\path\to\exe.exe"
echo 启动服务: sc start MyService
echo 停止服务: sc stop MyService
echo 删除服务: sc delete MyService
echo.
echo --- 2. 创建一个测试服务 ---
echo 先检查权限:
net session &gt;nul 2&gt;nul
if %errorlevel% neq 0 (
    echo [WARN] 需要管理员权限!
    echo 请右键 cmd 以管理员运行
    pause
    exit /b 1
)
echo [OK] 有管理员权限
echo.
echo --- 3. 创建服务 (示例) ---
echo 用 nssm 把任意 exe 包装成服务更简单:
echo nssm install MyApp "C:\apps\myapp.exe"
echo nssm set MyApp Start SERVICE_AUTO_START
echo nssm start MyApp
echo.
echo --- 4. 查看服务列表 ---
echo 自动启动的服务:
sc query state= all | findstr /c:"SERVICE_NAME" /c:"STATE"
echo.
echo --- 5. 自启动类型 ---
echo SERVICE_AUTO_START    自动 (开机即启)
echo SERVICE_DEMAND_START  手动 (需要时启动)
echo SERVICE_DISABLED      禁用
echo.
```

```
echo --- 6. 禁用某个服务 ---
echo sc config "Wuauserv" start= disabled
echo (禁用 Windows Update 服务, 仅作示例)
pause
```

进阶主题3：用 whoami / gpresult 检查权限

whoami 三剑客：whoami 看身份、whoami /groups 看组、whoami /priv 看权限。gpresult 检查域策略。net session 是最简单的管理员检测技巧——只有管理员能返回成功。

```
@echo off
echo === 用 whoami / gpresult 检查权限 ===
echo.
echo --- 1. whoami 基础 ---
echo 当前身份:
whoami
echo.
echo 所有组:
whoami /groups
echo.
echo 权限列表:
whoami /priv
echo.
echo --- 2. 检查是否管理员 ---
net session &gt;nul 2&gt;nul
if %errorlevel% equ 0 (
    echo [OK] 管理员权限
) else (
    echo [INFO] 非管理员, 但可能有特殊权限
)
echo.
echo --- 3. gpresult 组策略 ---
echo 域用户组成员关系:
gpresult /r | findstr /c:"组" /c:"Group"
```

```
echo.
echo --- 4. PowerShell 版更酷 ---
echo Get-IdentityReference 当前用户 SID:
powershell -Command "[Security.Principal.WindowsIdentity]::GetCurrent().Name"
echo.
echo --- 5. 检查某个权限是否启用 ---
for /f "delims=" %%p in ('whoami /priv ^| findstr /c:"SeDebug"') do (
    echo SeDebugPrivilege: %%p
)
echo.
echo --- 6. 特殊账号识别 ---
for /f "tokens=1" %%u in ('whoami') do set "curr=%%u"
echo 当前用户段: %curr%
echo 如果包含 "SYSTEM" 或 "Administrator" 则权限很高
echo.
echo --- 7. UAC 下的管理员检测 ---
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" /v EnableLUA 2&gt;nul | findstr 0x1 &gt;nul
if %errorlevel% equ 0 (
    echo [INFO] UAC 已启用, 当前进程可能不是管理员
) else (
    echo [INFO] UAC 已禁用或已关闭
)
pause
```

表 9-1 tasklist 筛选条件速查

筛选条件	语法	示例	含义
ImageName	imagename eq name	imagename eq notepad.exe	精确匹配进程名
ImageName	imagename ne name	imagename ne svchost.exe	不等于
PID	pid eq N	pid eq 1234	精确匹配 PID
PID	pid gt N	pid gt 100	PID 大于
SessionName	sessionname eq name	sessionname eq Console	会话名

Status	status eq Running	status eq Running	运行状态
CPU	cpuusage gt N	cpuusage gt 50	CPU 占用百分比
MemUsage	memusage gt N	memusage gt 102400	内存使用 KB
WindowTitle	windowtitle eq txt	windowtitle eq cmd	窗口标题精确
WindowTitle	windowtitle ne txt	windowtitle ne ""	窗口标题非空
UserName	username eq domain\user	username eq NT AUTHORITY\SYSTEM	运行用户
Modules	modules eq modulename	modules eq ntdll.dll	加载模块
Services	services eq sname	services eq W32Time	关联服务
WMI where	wmic process where "name='x'"	wmic process where "ProcessId>1000"	最强筛选
tasklist /fo	/fo table|csv|list	tasklist /fo csv	输出格式
tasklist /sort	/sort key	tasklist /sort memusage	排序字段
tasklist /m	/m module	tasklist /m ntdll.dll	使用该 DLL 的进程
tasklist /svc	/svc	tasklist /svc	显示关联服务
tasklist /v	/v	tasklist /v	详细模式
taskkill /f	/f	taskkill /f /im x	强制终止

表 9-2 注册表五大根键说明

根键	缩写	完整名称	作用域	典型内容	可写	优先级
HKCR	HKEY_CLASSES_ROOT	HKEY_CLASSES_ROOT	全局（合并 HKLM+HKCU）	文件关联 COM 类注册	间接	HKCU 覆盖 HKLM
HKLM	HKEY_LOCAL_MACHINE	HKEY_LOCAL_MACHINE	整机所有用户	系统设置 软件安装	需管理员	被 HKCU 覆盖
HKCU	HKEY_CURRENT_USER	HKEY_CURRENT_USER	当前登录用户	用户偏好 快捷方式	当前用户	最高覆盖优先级
HKU	HKEY_USERS	HKEY_USERS	所有用户配置集	每个用户的 SID 分支	管理员	HKCU 是 HKU 的当前 SID
HKCC	HKEY_CURRENT_CONFIG	HKEY_CURRENT_CONFIG	当前硬件配置集	显示 设置硬件配置	需管理员	动态生成
—	HKLM\SYSTEM	ControlSet001	控制系统启动/驱动/服务	内核配置	需管理员	—

—	HKLM\Software	所有已安装软件	32/64 位软件注册表	Software\WOW6432Node	—	
—	HKLM\SAM	安全账户管理器	用户账号密码 hash	系统级（普通用户无权限）	—	
—	HKCU\Environment	用户级环境变量	PATH/TEMP 等	HKCU 覆盖 HKLM	—	
—	HKLM\Run	启动项（机器级）	开机自动启动程序	需管理员	被恶意软件常改	
—	HKCU\Run	启动项（用户级）	当前用户登录启动	普通用户可写	—	
—	HKLM\...\Uninstall	已安装程序列表	控制面板-程序	卸载信息	—	
—	HKCR\.exe	.exe 关联	PE 文件打开方式	实际在 HKLM\Software\Classes	—	
—	HKCU\Software\Microsoft\Windows\CurrentVersion\Run	最常被恶意修改的键之一	—	—	—	
—	HKLM\SYSTEM\CurrentControlSet\Services	Windows 服务定义	服务名/描述/路径	sc 命令修改	—	

第 9 章小结

进程管理三件套是 tasklist（查）、taskkill（杀）、wmic process（万能）。表 9-1 把 tasklist 所有筛选条件（/fi 20 种）、输出格式、排序、模块查找一网打尽。记住 /fi "memusage gt 102400" 查内存大户，taskkill /f /im name 强制终止。

注册表五大根键（HKCR HKLM HKCU HKU HKCC）必须理清作用域。HKCU 覆盖 HKLM，HKCR 是合并视图，HKCC 动态生成。安全上最要盯的是 HKLM\Run、HKCU\Run、Startup 文件夹这三个启动点——恶意软件最爱修改。

进程监控脚本展示了 bat 里如何检查关键进程是否存在、扫描可疑进程名、审计所有启动位置。这三个功能组合起来就是一个轻量级的安全检查工具。tasklist 不支持通配符模糊搜索，用 findstr 配合。

注册表操作的常用命令：reg query 查值、reg add 写值、reg delete 删值、reg export/import 导入导出。所有 reg 命令都支持 /v 指定值名、/t 指定类型、/d 指定数据。写

HKLM 下的键必须管理员权限。

进程管理器项目用 bat 做了个 10 菜单的控制台 GUI，包括内存排序、top10、搜索、详细信息、终止（PID/名称）、进程树、服务状态。这种菜单风格是 BAT 做交互式工具的经典模式——call :label 子程序 + goto menu 循环。

下一章进入网络——ping、tracert、net 命令族、netstat、ipconfig、route、nslookup。BAT 脚本做网络诊断和自动化管理的核心。准备好了吗？

第 9 章大代码块 1：进程管理与注册表操作

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo  第9章综合实战：进程管理与注册表操作
echo ============================================
echo.
echo === 一、进程管理 ===
echo.
echo [1] 基本 tasklist:
tasklist /fo table
echo.
echo [2] 按内存排序:
tasklist /fo csv 2&gt;nul | sort /r /+40 | head -10
echo.
echo [3] 按 CPU 使用过滤:
tasklist /fi "cpuusage gt 0" /fo table
echo.
echo [4] 按内存过滤 (&gt;100MB):
tasklist /fi "memusage gt 102400" /fo table
echo.
echo [5] 只看特定进程:
```

```
tasklist /fi "imagename eq explorer.exe"
echo.
echo [6] 进程树:
tasklist /v /fo table | findstr "=="
echo.
echo [7] wmic process 更强:
wmic process get name,processid,parentprocessid,commandline /format:table 2&gt;nul | head -15
echo.
echo [8] taskkill 测试（notepad 安全）:
start "" notepad.exe
timeout /t 1 &gt;nul
tasklist /fi "imagename eq notepad.exe"
taskkill /f /im notepad.exe &gt;nul 2&gt;&amp;1
echo  已关闭 notepad
echo.
echo === 二、注册表快速浏览 ===
echo.
echo [HKCU\Environment] 用户环境变量:
reg query "HKCU\Environment" 2&gt;nul
echo.
echo [HKLM\Software\Microsoft\Windows\CurrentVersion] 启动项前 20 条:
reg query "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" 2&gt;nul
echo.
pause
```

第 9 章大代码块 2：进程监控 + 注册表启动项审计

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第9章进阶：进程监控 + 注册表启动项审计
```

```
echo ============================================
echo.
echo === 1. 系统关键进程监控 ===
set "critical_processes=explorer.exe svchost.exe lsass.exe csrss.exe winlogon.exe"
echo 正在检查关键进程...
for %%p in (!critical_processes!) do (
    tasklist /fi "imagename eq %%p" 2&gt;nul | findstr /i "%%p" &gt;nul 2&gt;&amp;1
    if !errorlevel!==0 (
        echo   [OK] %%p 运行中
    ) else (
        echo   [WARN] %%p 未运行!
    )
)
echo.
echo === 2. 可疑进程扫描 ===
set "suspicious=backdoor.exe keylogger.exe miner.exe malware.exe trojan.exe"
echo 正在扫描可疑进程...
for %%p in (!suspicious!) do (
    tasklist /fi "imagename eq %%p" 2&gt;nul | findstr /i "%%p" &gt;nul 2&gt;&amp;1
    if !errorlevel!==0 (
        echo   [!!] 发现可疑进程: %%p
    )
)
echo   扫描完成
echo.
echo === 3. 注册表启动项审计 ===
echo --- HKCU Run ---
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" 2&gt;nul
echo.
echo --- HKLM Run ---
reg query "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" 2&gt;nul
echo.
```

```
echo --- HKLM RunOnce ---
reg query "HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce" 2&gt;nul
echo.
echo --- Startup 文件夹 ---
dir /b "%appdata%\Microsoft\Windows\Start Menu\Programs\Startup" 2&gt;nul
dir /b "%programdata%\Microsoft\Windows\Start Menu\Programs\StartUp" 2&gt;nul
echo.
echo === 4. WMI 进程详细信息 ===
wmic process where "name='cmd.exe'" get ProcessId,ParentProcessId,CommandLine 2&gt;nul
echo.
pause
```

第 9 章进阶项目：进程管理器 GUI 版

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第9章项目：进程管理器 GUI 版（BAT 控制台 GUI）
echo ============================================
echo.
:menu
cls
echo  ============================================
echo   进程管理器
echo  ============================================
echo.
echo    [1] 列出全部进程（内存排序）
echo    [2] 列出 top 10 CPU
echo    [3] 列出 top 10 内存
echo    [4] 搜索特定进程
echo    [5] 查看进程详细信息
```

```
echo    [6] 终止进程（PID 方式）
echo    [7] 终止进程（名称方式）
echo    [8] 进程树
echo    [9] 系统服务状态
echo    [0] 退出
echo.
set /p "choice=请选择: "
if "!choice!"=="1" goto list_all
if "!choice!"=="2" goto top_cpu
if "!choice!"=="3" goto top_mem
if "!choice!"=="4" goto search
if "!choice!"=="5" goto detail
if "!choice!"=="6" goto kill_pid
if "!choice!"=="7" goto kill_name
if "!choice!"=="8" goto tree
if "!choice!"=="9" goto services
if "!choice!"=="0" goto end
goto menu
:list_all
echo --- 所有进程（内存降序）---
tasklist /fo csv 2&gt;nul | sort /r /+40 | head -20
pause
goto menu
:top_cpu
echo --- CPU top 10 ---
wmic process get name,caption,commandline 2&gt;nul | head -12
pause
goto menu
:top_mem
echo --- 内存 top 10 ---
tasklist /fo list | findstr /i "图像名 内存使用"
pause
```

```
goto menu
:search
set /p "kw=搜索关键词: "
tasklist /fo table | findstr /i "!kw!"
pause
goto menu
:detail
set /p "pid=输入进程 PID: "
wmic process where "ProcessId=!pid!" get /format:list 2&gt;nul
pause
goto menu
:kill_pid
set /p "pid=输入要终止的 PID: "
echo 正在终止 PID !pid! ...
taskkill /f /pid !pid!
pause
goto menu
:kill_name
set /p "name=输入进程名(如 notepad.exe): "
echo 正在终止 !name! ...
taskkill /f /im "!name!"
pause
goto menu
:tree
echo --- 进程树 ---
tasklist /v /fo table
pause
goto menu
:services
echo --- 系统服务 ---
net start
echo.
```

```
echo 服务总数: &amp;amp; net start | find /c "="
pause
goto menu
:end
exit /b 0
```

进阶项目9：系统健康检查

这个脚本实现了运维级别的"单命令健康体检"：10 大维度全方位检查（CPU 使用率、内存、磁盘、运行时长、补丁、关键服务、高内存进程、网络连通性、安全基线、驱动），带三级阈值（正常/警告/危险）、量化健康评分（0-100）、告警 CSV
导出和可视化仪表盘。核心循环是 SetThreshold 函数——变量值 + 两档阈值 -> 自动分级。

```
@echo off
setlocal enabledelayedexpansion
title 系统健康检查脚本 v2.0
color 0A

rem ============================================================
rem  系统健康检查 - 全方位检查 + 异常告警
rem  CPU/内存/磁盘/网络/服务/进程/驱动/补丁/安全
rem  输出控制台仪表盘 + CSV 报告
rem ============================================================

set "OUT_DIR=%~dp0health_report_%date:~0,4%%date:~5,2%%date:~8,2%"
mkdir "!OUT_DIR!" 2&gt;nul
set "REPORT=!OUT_DIR!\health_report.txt"
set "ALERTS=!OUT_DIR!\alerts.csv"
set "DETAIL=!OUT_DIR!\detail.csv"

set /a total_checks=0
set /a pass=0
set /a warn=0
set /a fail=0
```

```
set /a score=100

echo === 系统健康检查 ===
echo 时间: %date% %time%
echo 机器: %COMPUTERNAME%
echo 用户: %USERNAME%
echo.

echo "时间戳","检查项","状态","详情","严重级别" &gt; "!ALERTS!"
echo "时间戳","检查项","状态","指标值","阈值","详情" &gt; "!DETAIL!"

rem ========== 1. CPU 检查 ==========
call :CheckItem "CPU 使用率"
set "cpu_usage=N/A"
for /f "skip=1" %%a in ('wmic cpu get LoadPercentage /format:list 2^&gt;nul ^| findstr LoadPercentage') do (
    for /f "tokens=2 delims==" %%b in ("%%a") do set /a cpu_usage=%%b 2&gt;nul
)
if "!cpu_usage!"=="" set /a cpu_usage=0
call :SetThreshold "cpu_usage" 70 90

rem ========== 2. 内存检查 ==========
call :CheckItem "内存使用率"
set /a mem_total=0
set /a mem_free=0
for /f "tokens=2 delims==" %%t in ('wmic OS get TotalVisibleMemorySize /format:list 2^&gt;nul ^| findstr Total') do set /a mem_total=%%t
for /f "tokens=2 delims==" %%f in ('wmic OS get FreePhysicalMemory /format:list 2^&gt;nul ^| findstr Free') do set /a mem_free=%%f
if !mem_total! gtr 0 (
    set /a mem_used=mem_total-mem_free
    set /a mem_pct=mem_used*100/mem_total
) else (
    set /a mem_pct=0
)
call :SetThreshold "mem_pct" 80 95
```

```
rem ========== 3. 磁盘检查 ==========

call :CheckItem "系统盘空间 (C:)"

set /a disk_free=0

set /a disk_size=0

for /f "tokens=2 delims==" %%f in ('wmic logicaldisk where "DeviceID='C:'" get Freespace /format:list 2^&gt;nul ^| findstr Freespace') do set /a disk_free=%%f

for /f "tokens=2 delims==" %%s in ('wmic logicaldisk where "DeviceID='C:'" get Size /format:list 2^&gt;nul ^| findstr Size') do set /a disk_size=%%s

if !disk_size! gtr 0 (

    set /a disk_used=disk_size-disk_free

    set /a disk_pct=disk_used*100/disk_size

    set /a disk_free_gb=disk_free/1073741824

) else (

    set /a disk_pct=0

    set /a disk_free_gb=0

)

call :SetThreshold "disk_pct" 85 95

rem ========== 4. 运行时长检查 ==========

call :CheckItem "系统运行时长"

set "uptime=N/A"

for /f "tokens=2 delims==" %%u in ('wmic os get LastBootUpTime /format:list 2^&gt;nul ^| findstr LastBoot') do set "uptime=%%u"

if not "!uptime!"=="" (

    echo 上次启动: !uptime!

    echo 建议: 超过 7 天建议重启更新补丁

)

call :SetThreshold "uptime" 7 14

rem ========== 5. 补丁检查 ==========

call :CheckItem "系统补丁"

set /a hotfix_count=0

for /f "skip=1" %%h in ('wmic qfe get HotFixID 2^&gt;nul') do (

    if not "%%h"=="" set /a hotfix_count+=1
```

```
)
echo 已安装补丁数: !hotfix_count!
if !hotfix_count! lss 10 (
    call :Alert "补丁不足(!hotfix_count!)" WARN "可能遗漏安全更新"
)

rem ========== 6. 关键服务检查 ==========
call :CheckItem "关键服务状态"
set "critical_services=wuauserv WinDefend NetDhcp RpcSs Spooler NlaSvc TermService"
for %%s in (!critical_services!) do (
    set "_status=STOPPED"
    for /f "tokens=3" %%c in ('sc query %%s 2^&gt;nul ^| findstr STATE') do set "_status=%%c"
    if /i "!_status!"=="RUNNING" (
        echo   [OK] %%s 运行中
        set /a pass+=1
    ) else (
        echo   [!!] %%s = !_status!
        call :Alert "服务 %%s 未运行" FAIL "当前状态: !_status!"
        set /a fail+=1
        set /a score-=5
    )
    set /a total_checks+=1
)

rem ========== 7. 进程异常检查 ==========
call :CheckItem "高内存进程"
for /f "skip=1 tokens=1,5" %%a in ('tasklist /fi "MEMUSAGE gt 50000" /fo csv 2^&gt;nul') do (
    echo   [!] %%a 内存占用超过 50MB
    call :Alert "%%a 高内存" WARN "可能内存泄漏"
)

rem ========== 8. 网络检查 ==========
call :CheckItem "网络连通性"
ping -n 2 127.0.0.1 &gt;nul 2&gt;&amp;1 &amp;&amp; (
```

```
    echo   [OK] 本地回环正常
    set /a pass+=1
) || (
    echo   [!!] 本地回环失败
    set /a fail+=1
    set /a score-=10
)
ping -n 2 8.8.8.8 &gt;nul 2&gt;&amp;1 &amp;&amp; (
    echo   [OK] 外网连接正常
    set /a pass+=1
) || (
    echo   [!!] 外网无法连接
    call :Alert "外网连接失败" WARN "检查防火墙和路由"
    set /a warn+=1
)

rem ========== 9. 安全检查 ==========
call :CheckItem "安全基线"
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" 2&gt;nul | findstr /v "^$" | findstr /v "REG_" | findstr /v "CurrentVersion"
echo 检查自启动项...(非空需人工审查)
call :Alert "建议检查自启动项" INFO "详见注册表 HKLM\...Run"

rem ========== 10. 驱动检查 ==========
call :CheckItem "驱动完整性"
driverquery /v /fo csv 2&gt;nul | findstr /i "unsigned"
echo 如有 Unsigned 需更新驱动

rem ========== 输出仪表盘 ==========
echo.
echo ================================================
echo            系统健康检查仪表盘
echo ================================================
echo  总计: !total_checks! 项检查
```

```
echo  通过: !pass!  [OK]
echo  警告: !warn!  [!]
echo  失败: !fail!  [FAIL]
echo  ----------------------------------------------
echo  健康评分: !score! / 100
if !score! geq 80 (echo  状态: ● 良好)
if !score! geq 50 if !score! lss 80 (echo  状态:     一般 - 建议处理警告)
if !score! lss 50 (echo  状态: ● 警告 - 需要立即处理)
echo  ----------------------------------------------
echo  详细报告: !REPORT!
echo  告警列表: !ALERTS!
echo ================================================

choice /C YN /M "打开报告目录"
if errorlevel 1 start "" explorer "!OUT_DIR!"
endlocal
pause
exit /b

:CheckItem
echo.
echo ----------------------------------------------
echo 检查: %~1
exit /b

:SetThreshold
rem %1=变量名 %2=警告阈值 %3=危险阈值
set "_v=!%~1!"
echo   当前值: !_v!
set /a total_checks+=1
set /a "_warn=%~2"
set /a "_fail=%~3"
if !_v! lss !_fail! (
    echo   [OK] 正常 (!_v! ^&lt; !_fail!)
```

```
    set /a pass+=1
    goto :thresh_done
)
if !_v! lss !_warn! (
    echo   [!!] 警告 (!_warn! lss !_v! lss !_fail!)
    call :Alert "%~1=!_v!" WARN "建议检查"
    set /a warn+=1
    set /a score-=3
    goto :thresh_done
)
echo   [FAIL] 危险 (!_v! ^&gt;= !_warn!)
call :Alert "%~1=!_v!" FAIL "超过危险阈值"
set /a fail+=1
set /a score-=10
:thresh_done
exit /b

:Alert
echo "!date! !time!","%~1","%~2","%~3","%~4" &gt;&gt; "!ALERTS!"
exit /b
```

25 项系统健康指标详细说明

检查维度	指标	BAT 命令	警告阈值	危险阈值	数据来源	频率	影响范围	修复方式	自动化
CPU 使用率	%	wmic cpu LoadPercentage	70%	90%	WMI	1分钟	所有应用变慢	结束高占用进程	taskkill /f
CPU 温度	°C	wmic MSAcpi_ThermalZone	75°C	85°C	WMI/传感器	1分钟	自动降频/关机	清理散热器/换硅脂	BAT 难读传感器
内存使用率	%	wmic OS Free/TotalMemory	80%	95%	WMI	1分钟	OOM/应用崩溃	关闭不用的应用	taskkill /im
内存泄漏	MB/进程	tasklist /fi "MEM>50000"	50MB 持续增长	100MB+	tasklist	5分钟	长期性能下降	重启进程或修复代码	BAT 可检测无法修复

系统盘空间	%	wmic logicaldisk C: Freespace	85%	95%	WMI	1小时	系统卡顿/无法更新	清理 temp/下载/日志	cleanmgr /sagerun
系统盘空间	GB	dir /c /s	10GB	5GB	dir	1小时	系统无法启动	紧急扩容	磁盘管理扩展卷
文件系统	状态	chkdsk C:	有问题	严重	chkdsk	每周	数据丢失风险	chkdsk /f	批处理计划任务
运行时长	天	wmic OS LastBootUpTime	7天	14天	WMI	每天	补丁/驱动没装	shutdown /r	BAT 计划任务重启
系统补丁	数量	wmic qfe count	>=10	<5	WMI	每周	安全漏洞	wuauclt /detectnow	组策略自动更新
补丁新鲜度	天数	wmic qfe InstalledOn	<30 天	<90 天	WMI	每周	已知漏洞暴露	Windows Update	自动化组策略
关键服务	状态	sc query	RUNNING	STOPPED	sc	15分钟	功能缺失	net start	sc failure reset
非必要服务	列表	sc query state= all	按需	太多	sc	每周	攻击面扩大	sc config start= disabled	批处理禁用
进程异常	列表	tasklist /v	无异常	僵尸/孤儿	tasklist	5分钟	资源浪费	taskkill	守护进程
端口监听	列表	netstat -ano	按需	未知端口	netstat	10分钟	信息泄露/入侵	防火墙关闭	netsh 规则
磁盘 I/O	队列	wmic diskdrive	<1秒	>5秒	WMI/PDH	1分钟	磁盘故障前兆	更换磁盘/SMART	BAT 需 PDH
网络连接	状态	ping 8.8.8.8	丢包 5%	丢包 20%	ping	1分钟	业务中断	检查网卡/路由	BAT 自动 ping
DNS 解析	超时	nslookup	>2s	>5s	nslookup	5分钟	无法访问域名	换 DNS 服务器	netsh 改 DNS
安全基线	状态	各种 sc/reg	按需	不合规	多命令	每天	被攻击	逐项修复	合规脚本
事件日志	Error 数	wmic ntevent	>=5	>=20	WMI	15分钟	故障前兆	排查根因	日志收集器
SMART 状态	磁盘健康	wmic diskdrive MediaType	良好	警告	WMI/SMART	每天	磁盘即将损坏	立即备份换盘	安装 HD Tune 等
RAID 状态	阵列健康	storcli/MegaCli	Optimal	Degraded	RAID 工具	每天	阵列降级	热备/换盘	RAID 管理工具

备份状态	完成率	备份软件	<100%	<80%	备份日志	每天	数据丢失风险	重跑备份	备份脚本自动
证书到期	天数	certutil -dump	<30天	<7天	certutil/IIS	每周	服务中断	续期证书	ACME自动续期
内存硬件	ECC错误	wmic memorychip	0	>0	WMI/硬件	每周	硬件故障	换内存条	需硬件监控
电源状态	UPS/电池	wevtutil qe Battery	在线	断电	WMI/硬件	实时	突然断电丢数据	检查UPS	BAT通知无自动

健康检查脚本的阈值设计原则：警告阈值设成"需要关注但不紧急"，危险阈值设成"必须立即处理否则要出事"。比如 CPU 70% 警告（可能一个计算密集型任务）、90% 危险（系统开始响应迟钝）。磁盘 85% 警告（提醒清理）、95% 危险（系统盘满就蓝屏）。健康评分用扣分法——每项警告扣 3 分、失败扣 10 分，最后根据 0-100 分输出三种状态。

关键服务列表是定制化的——wuauserv（Windows Update）、WinDefend（Windows Defender）、NetDhcp（DHCP 客户端）、RpcSs（RPC 端点映射器）、Spooler（打印服务）、NlaSvc（网络位置感知）、TermService（远程桌面）。你可以根据实际业务增加自己的关键服务。sc query 的输出里 STATE 字段用 findstr 过滤后取第 3 个 token（RUNNING 是第 3 字段）。

wmic 取数值有个坑：有些字段返回空值（比如某些机器没有温度传感器），直接 set /a 空值会崩。脚本里用 if "!var!"=="" set /a var=0 来兜底。另外 wmic 命令本身在某些精简版 Windows 或安全软件拦截下会失败，所以所有 wmic 调用后面都要加 2>nul 抑制错误。

第10章 网络操作命令

10.1 网络诊断基础

ipconfig

```
rem 查看所有网卡的简要 IP 信息
ipconfig

rem 显示完整信息（MAC、DHCP、DNS、网关等）
```

```
ipconfig /all

rem 刷新 DNS 缓存
ipconfig /flushdns

rem 释放/重新获取 DHCP 地址
ipconfig /release
ipconfig /renew

rem 导出配置
ipconfig /all &gt; network_info.txt
```

ping

```
rem 基本连通性测试
ping 8.8.8.8

rem 持续 ping（Ctrl+C 停止）
ping -t www.bing.com

rem 发送 10 个包
ping -n 10 www.bing.com

rem 大包测试（检查 MTU）
ping -l 1472 -n 10 192.168.1.1

rem 解析主机名并 ping
ping -a 192.168.1.1
```

tracert —— 路由追踪

```
rem 查看到目标的完整路由路径
tracert www.bing.com

rem 限制最大跳数
tracert -h 15 223.5.5.5

rem 不解析主机名（更快）
tracert -d www.bing.com
```

10.2 netstat —— 端口与连接

```
rem 显示所有活动连接和监听端口
netstat -ano

rem 只显示 TCP 连接
netstat -ano -p tcp

rem 按 PID 过滤（查找哪个进程占了端口）
netstat -ano | findstr ":8080"

rem 显示统计信息
netstat -s

rem 持续刷新（2秒一次）
netstat -ano 2
```

TIP

netstat -ano 的输出中最后一列是 PID（进程ID）。用 tasklist /fi "PID eq 1234" 就能查出是哪个程序。

10.3 文件下载：bitsadmin / certutil

BAT 没有原生的 HTTP 下载命令，但 Windows 自带几个"隐形下载器"：

bitsadmin —— 后台下载器

```
rem 创建一个下载任务
bitsadmin /transfer MyDownload "https://example.com/file.zip" "D:\Downloads\file.zip"

rem 后台下载（不阻塞脚本）
bitsadmin /create /download MyJob
bitsadmin /addfile MyJob "https://example.com/file.zip" "D:\Downloads\file.zip"
bitsadmin /resume MyJob
bitsadmin /complete MyJob

rem 查询下载状态
bitsadmin /list /allusers
```

TIP

bitsadmin 有一个隐蔽但重要的特性：它的下载请求走 BITS 服务（后台智能传输服务），在企业环境中如果 HTTP 代理拦截了正常下载，bitsadmin 有时反而能绕过。

certutil —— 最隐蔽的下载技巧

certutil 本是证书工具，但它有一个未文档化的功能：能把 URL 内容下载下来！这是 BAT 里最流行的"黑客技巧"之一：

```
rem 下载文件
certutil -urlcache -split -f "https://example.com/file.exe" output.exe

rem 参数说明:
rem -urlcache  操作 URL 缓存
rem -split    分割输出
rem -f        强制覆盖
```

WARNING

注意：certutil 下载没有断点续传、不支持 HTTPS 证书校验，且可能被杀毒软件拦截。正式场合请用 bitsadmin 或 PowerShell 的 Invoke-WebRequest。

10.4 net use —— 网络盘映射

```
rem 映射网络盘（持久化，重启后还在）
net use Z: \\NAS\Documents /persistent:yes

rem 临时映射（当前会话有效）
net use Y: \\Server\Share /persistent:no

rem 带用户名密码映射
net use Z: \\NAS\Private /user:admin "password"

rem 查看当前映射
net use

rem 断开映射
```

```
net use Z: /delete

rem 断开所有映射
net use * /delete /y
```

TIP

如果网络盘已被断开但还显示红色"X"，用 `net use Z: \\NAS\Share /persistent:yes` 重新连接。或者直接 `pushd \\NAS\Share`，用完 popd 自动清理。

10.5 实战：批量网络连通性检测

```
@echo off
setlocal enabledelayedexpansion
rem ========== 批量 Ping 检测 ==========
rem 创建 targets.txt，每行一个 IP 或域名

set "TARGETS=targets.txt"
set "RESULT=ping_result.csv"

echo IP,时间,状态 &gt; "%RESULT%"

for /f "usebackq tokens=*" %%ip in ("%TARGETS%") do (
    set /a count+=1
    &lt;nul set /p "=正在检测 %%ip ... "

    for /f "tokens=5 delims==" %%t in ('ping -n 1 -w 1000 %%ip ^| findstr "时间=" ^| findstr /v "&lt;"') do (
        echo %%ip,%%t,OK &gt;&gt; "%RESULT%"
        echo OK - %%t
    ) || (
        echo %%ip,超时,FAIL &gt;&gt; "%RESULT%"
        echo 超时
    )
)

echo.
echo 检测完成，结果已保存到 %RESULT%
```

```
endlocal
```

— 第10章 完 —

进阶练习

练习1：网络连通性检测：

```
@echo off
echo === 网络诊断 ===
echo [1/4] 网络适配器:
ipconfig /all | findstr /c:"IPv4" /c:"默认网关"
echo [2/4] ping localhost:
ping -n 2 127.0.0.1 &amp;gt;nul &amp;amp;&amp;amp; echo OK
echo [3/4] ping DNS 8.8.8.8:
ping -n 2 8.8.8.8 &amp;gt;nul 2&amp;gt;nul &amp;amp;&amp;amp; echo OK || echo FAIL
echo [4/4] DNS 解析:
nslookup www.baidu.com 2&amp;gt;nul | findstr Address
pause
```

练习2：net 命令大全：

```
@echo off
echo === net 命令 ===
net user
net share
net use
netstat -ano | findstr ESTABLISHED
pause
```

练习3：curl 下载文件：

```
@echo off
where curl &amp;gt;nul 2&amp;gt;nul || echo 没有 curl
echo 下载文件:
curl -L -o file.zip https://example.com/file.zip
echo 只取响应头:
curl -I https://example.com
echo PowerShell 下载:
```

```
powershell -Command "Invoke-WebRequest -Uri 'https://example.com' -OutFile 'page.html'"
pause
```

疑难解答

Q1: ping 通 IP 但不通域名?

A: DNS 解析问题。ping 8.8.8.8 能通就是 DNS 问题。手动设置：netsh interface ip add dns "以太网" 8.8.8.8

Q2: net use 映射失败?

A: 常见原因：目标不可达、权限不够（/user:server\user password）、驱动器号被占用。

进阶主题1：net use 映射网络驱动器到文件夹

net use 不只是映射盘符，还可以把网络共享"挂载"到本地空文件夹。/persistent:yes 让映射跨重启保留。UAC 坑要注意——管理员映射的普通用户看不到。

```
@echo off
echo === net use 映射网络驱动器到文件夹 ===
echo.
echo --- 1. 映射网络共享 ---
echo net use Z: \\server\share
echo net use Z: \\server\share /user:admin password
echo.
echo --- 2. 映射到本地文件夹 (UNC路径) ---
echo net use C:\tools \\server\tools
echo 这会让 C:\tools 指向 \\server\tools 共享
echo.
echo --- 3. 查看当前映射 ---
echo 所有网络连接:
net use
echo.
echo 用 PowerShell 看更清楚:
powershell -Command "Get-SmbMapping | Format-Table LocalPath,RemotePath,Status -AutoSize"
```

```
echo.
echo --- 4. 持久化映射 (重启后保留) ---
echo 默认映射重启会丢, 用 /persistent:yes:
echo net use Z: \\server\share /persistent:yes
echo.
echo --- 5. 删除映射 ---
echo net use Z: /delete
echo 删除所有映射:
echo net use * /delete /y
echo.
echo --- 6. 登录脚本里自动映射 ---
echo net use G: \\file-server\documents
echo net use H: \\database-server\backup /persistent:yes
echo if not exist G:\ net use G: \\backup\fallback /persistent:no
echo.
echo --- 7. 管理员映射的坑 ---
echo UAC 下管理员 cmd 映射的驱动器, 普通用户看不到!
echo 用 reg add 让管理员和用户共享映射:
echo reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" /v EnableLinkedConnections /t REG_DWORD
/d 1 /f
pause
```

进阶主题2：用 PowerShell Test-NetConnection 替代 ping

ping 只能测 ICMP 可达性，被防火墙挡了就失败。Test-NetConnection 能测 TCP 端口、路由、DNS，是网络诊断的瑞士军刀。批量扫描端口特别好用。

```
@echo off
echo === 用 PowerShell Test-NetConnection 替代 ping ===
echo.
echo ping 只能测 ICMP, 被防火墙阻挡时无效
Test-NetConnection 能测端口、TCP、路由, 信息丰富得多
echo.
echo --- 1. 基本连通性测试 ---
echo ping 127.0.0.1 方式:
```

```
ping -n 2 127.0.0.1 &gt;nul 2&gt;nul ^&amp;^&amp; echo OK ^|^| echo FAIL

echo.

echo Test-NetConnection 方式:

powershell -Command "(Test-NetConnection -ComputerName 127.0.0.1 -InformationLevel Quiet -WarningAction SilentlyContinue)"

echo.

echo --- 2. 测试特定端口 ---

echo 检查 80 端口:

powershell -Command "Test-NetConnection -ComputerName www.baidu.com -Port 80 -InformationLevel Detailed -WarningAction SilentlyContinue | Select-Object ComputerName,RemotePort,TcpTestSucceeded"

echo.

echo 检查 443 端口:

powershell -Command "(Test-NetConnection -ComputerName www.baidu.com -Port 443 -WarningAction SilentlyContinue).TcpTestSucceeded"

echo.

echo --- 3. 批量端口扫描 ---

echo 扫描常见端口:

for %%p in (21 22 23 25 53 80 110 443 3306 3389 8080) do (

    for /f %%r in ('powershell -Command "(Test-NetConnection -ComputerName 127.0.0.1 -Port %%p -WarningAction SilentlyContinue).TcpTestSucceeded"') do (

        echo  %%p: %%r

    )

)

echo.

echo --- 4. 路由跟踪 ---

echo tracert 基础:

tracert -d -h 10 www.baidu.com

echo.

echo --- 5. DNS 解析 ---

nslookup www.baidu.com 2&gt;nul | findstr Address

pause
```

进阶主题3：局域网文件快速传输（robocopy + 共享）

robocopy 是 Windows 内置最快的文件传输工具。/MT:8 多线程、/Z 断点续传、/MIR 镜像同步、/compress 压缩传输。临时共享 quick$ 加 $ 隐藏很方便。scp 适合跨平台。

```
@echo off
echo === 局域网文件快速传输 ===
echo.
echo --- 1. robocopy + 共享 ---
echo 先创建共享 (管理员):
echo net share MyShare=C:\Data /grant:Everyone,Full
echo.
echo 从 A 机器复制到 B:
echo robocopy "C:\LocalDir" \\B-PC\MyShare\DestDir /E /Z /R:3 /W:5
echo /Z 支持断点续传, /MT:8 多线程
echo robocopy "C:\Data" \\NAS\Backup /E /MT:8 /COPY:DAT
echo.
echo --- 2. 临时共享快速传 ---
echo net share quick$ "%TEMP%\quick_transfer"
echo mkdir "%TEMP%\quick_transfer" 2&gt;nul
echo 放文件进去, 另一边访问 \\本机名\quick$
echo 用完删除:
echo net share quick$ /delete
echo.
echo --- 3. SMB 直接复制 ---
echo copy \\server\share\file.txt C:\dest\
echo xcopy \\server\share\*.* C:\dest\ /E /I /Y
echo.
echo --- 4. scp / sftp ---
echo Windows 10+ 自带 OpenSSH:
echo scp file.txt user@server:/path/
echo scp -r folder/ user@server:/path/
echo.
echo --- 5. PowerShell 方式 ---
echo Copy-Item \\server\share\file.txt C:\dest\
echo 带进度条:
echo robocopy 支持日志:
```

```
echo robocopy src dst /E /LOG+:copy.log /NFL /NDL
echo.
echo --- 6. 速度对比 ---
echo 局域网 1Gbps: robocopy /MT:8 最快
echo 广域网: robocopy /Z (断点续传) + 压缩
echo robocopy src dst /E /Z /MT:8 /compress
echo.
echo --- 7. 权限问题 ---
echo 访问共享要先登录: net use \\server /user:name /password:xxx
echo 或者用完整路径: net use \\server\share password /user:name
pause
```

表 10-1 网络命令汇总表

命令	功能	关键参数	常用场景
ping	测试连通性	-n count -t 持续 -w 超时	网络第一层排查
tracert	路由追踪	-h maxhops -d 不解析 DNS	定位故障段
pathping	增强 tracert	-q 每跳查询数	丢包定位
ipconfig	查看网络配置	/all 详细 /release 释放 /renew 续租	查看 IP/DNS/MAC
netstat	连接状态	-ano 显示 PID -p proto -r 路由	端口监听/连接
route	路由表操作	print print add/delete	静态路由
nslookup	DNS 查询	-type=A/CNAME/MX	DNS 问题定位
arp	ARP 表	-a 查看 -s 添加 -d 删除	二层排查
telnet	端口测试	host port	简易端口连通
ftp	FTP 客户端	-n 自动登录	文件传输
curl (PowerShell)	HTTP 客户端	-s silent	API 测试
netsh	网络配置 Shell	interface / advfirewall	高级配置
shutdown	远程关机	/m \pc /r /t 0	远程控制
at / schtasks	定时任务	at \pc 12:00 command	网络定时执行
wmic	WMI 查询	computersystem / cpu / nic	硬件/网络信息

net	账户/共享/会话	见下表 net 子命令	Windows 网络管理
msiexec	远程安装	/i package.msi /qn	静默安装
pslist (Sysinternals)	进程列表	-\computer	远程进程
PsExec (Sysinternals)	远程执行	\\pc cmd	BAT 远程替身

表 10-2 net 命令子命令表

子命令	功能	常用参数	示例	
net share	管理共享资源	无参数 / delete name	net share	net share /delete myshare
net session	会话管理	无参数 / close	net session	net session /close
net view	浏览网络	\\server	net view	net view \\server01
net use	映射网络驱动器	Z: \\server\share /persistent	net use Z: \\nas\data	
net user	用户账户	无参数 /add /delete /active	net user admin	net user hacker /delete
net localgroup	本地组	Administrators /add user	net localgroup Administrators hacker /add	
net group	域组	（需域环境）	net group Domain Admins	
net accounts	账户策略	/maxpwage /minpwlen	net accounts	net accounts /maxpwage:30
net computer	域计算机	/add /del	net computer \\pc01 /add	
net config	网络配置	Server / Workstation	net config Workstation	
net continue	继续服务	服务名	net continue Spooler	
net pause	暂停服务	服务名	net pause Schedule	
net start	启动服务	服务名	net start W32Time	
net stop	停止服务	服务名	net stop Print Spooler	
net send	发送消息（旧）	name message	net send admin "服务器即将重启"	
net time	时间同步	\\server /set /yes	net time \\dc /set /yes	
net statistics	统计信息	Server / Workstation	net statistics Server	
net file	打开文件	无参数 / close id	net file	net file 10 /close
net name	消息名称	无参数 / add /del	net name	

第 10 章小结

网络命令是 BAT 脚本里最常用也最容易被忽视的一组。表 10-1 把 ping/tracert/ipconfig/netstat/route/nslookup/arp/telnet/netsh/wmic/net 一网打尽。ping -n

1 -w 300 host >nul 2>&1 && echo OK 是 BAT 里检查网络的标准写法——静默 + 用 && 链式。

net 命令有 19 个子命令，其中 net share/net use/net user/net start/net stop/net view 是日常最高频的。net use Z: \\server\share /persistent:yes 映射驱动器；net start W32Time 启动时间服务；net user hacker /delete 删除账户。记住 net user /add 可以创建账户，配合 net localgroup Administrators hacker /add 提权到管理员。

网络延迟监控脚本演示了用 ping -n 1 + 字符串截取提取 RTT 值、超时判断、循环记录到日志文件的完整模式。BAT 里 ping 的输出解析很麻烦——findstr "时间" 或 "Average" 再用 for /f 截 token，是无奈但有效的方案。

端口扫描用 PowerShell Test-NetConnection 比 BAT 的 telnet 更可靠。telnet 在现代 Windows 上默认不启用，需要 OptionalFeatures 里打开。网段扫描用 ping -n 1 -w 300 + for /L 遍历 1-254，300ms 超时速度适中。

netstat -ano 显示的 PID 可以用 tasklist /fi "pid eq N" 找到对应进程。这是 BAT 里定位"哪个进程占用了某个端口"的标准组合拳。

下一章进入 BAT 与 PowerShell 互通、UAC 提权方案。BAT 直接做不了的事，PowerShell 可以；BAT 的优势是零依赖。两者结合才是生产环境的最佳方案。准备好了吗？

第 10 章大代码块 1：网络诊断全家桶

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo  第10章综合实战：网络诊断全家桶
echo ============================================
echo.
echo === 1. 快速网络自检 ===
echo --- 基本连通性 ---
ping -n 1 127.0.0.1 &gt;nul 2&gt;&amp;1 &amp;&amp; echo [OK] 回环正常 || echo [FAIL] 回环异常
ping -n 1 8.8.8.8 &gt;nul 2&gt;&amp;1 &amp;&amp; echo [OK] 外网可达 || echo [FAIL] 外网不可达
ping -n 1 www.baidu.com &gt;nul 2&gt;&amp;1 &amp;&amp; echo [OK] DNS 正常 || echo [FAIL] DNS 异常
echo.
```

```
echo --- DNS 解析详细 ---
nslookup www.baidu.com 2&gt;nul | findstr /i "Address Name"
echo.
echo === 2. 详细网络信息 ===
echo --- IP 配置 ---
ipconfig /all | findstr /i "IP 网关 DNS MAC DHCP"
echo.
echo --- 路由表 ---
route print 2&gt;nul | findstr "0.0.0.0"
echo.
echo --- 端口监听 ---
netstat -ano | findstr LISTENING | head -15
echo.
echo --- 活动连接 ---
netstat -ano | findstr ESTABLISHED | head -10
echo.
echo === 3. net 命令快速参考 ===
echo --- 共享资源 ---
net share 2&gt;nul
echo.
echo --- 当前用户 ---
net user %username% 2&gt;nul | findstr "上次登录 账户激活"
echo.
echo --- 工作组 ---
net view 2&gt;nul | head -10
echo.
echo === 4. tracert 路由追踪 ===
tracert -d -h 8 www.baidu.com 2&gt;nul
echo.
pause
```

第 10 章大代码块 2：网络延迟监控脚本

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo  第10章进阶：网络延迟监控脚本
echo  定时 ping 记录 RTT，超过阈值报警
echo ============================================
echo.
set "LOG=%~dp0latency.log"
set /p "target=输入要监控的目标(如 8.8.8.8): "
if "!target!"=="" set "target=8.8.8.8"
set /p "threshold=延迟阈值(ms, 如 100): "
if "!threshold!"=="" set "threshold=100"
set /p "interval=检查间隔(秒, 如 10): "
if "!interval!"=="" set "interval=10"
echo.
echo 目标: !target!
echo 阈值: !threshold! ms
echo 间隔: !interval! 秒
echo 日志: !LOG!
echo 按 Ctrl+C 停止
echo.
:loop
for /f "tokens=5 delims==&lt; " %%t in ('ping -n 1 !target! ^| findstr /i "时间"') do (
    set "rtt=%%t"
    set "rtt=!rtt:ms=!"
    if "!rtt!"=="" set "rtt=超时"
)
for /f "tokens=5 delims== " %%t in ('ping -n 1 !target! ^| findstr /i "Average"') do set "rtt=%%t"
set "ts=%date% %time%"
echo [!ts!] !target! - RTT=!rtt! ms
```

```
if "!rtt!" neq "超时" (
    set "rtt_num=!rtt!"
    if !rtt_num! gtr !threshold! (
        echo   [WARN] 延迟超阈值 !threshold!ms
        echo [!ts!] WARN: RTT !rtt! ms !threshold! &gt;&gt; "!LOG!"
    )
) else (
    echo   [FAIL] 超时
    echo [!ts!] FAIL: ping timeout &gt;&gt; "!LOG!"
)
timeout /t !interval! &gt;nul
goto loop
```

第 10 章进阶项目：网络扫描工具集

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第10章项目：网络扫描工具集
echo   端口扫描 / 网段扫描 / 服务探测
echo ============================================
echo.
echo [1] 端口扫描（常用端口）
echo [2] 网段快速扫描（192.168.1.x）
echo [3] HTTP 服务探测
echo [4] 网络接口详细信息
set /p "mode=选择模式 [1-4]: "
echo.
if "!mode!"=="1" goto port_scan
if "!mode!"=="2" goto subnet_scan
if "!mode!"=="3" goto http_probe
```

```
if "!mode!"=="4" goto iface_info
goto end
:port_scan
set /p "host=要扫描的主机: "
set "ports=21 22 23 25 53 80 110 135 139 443 445 3389 8080"
echo 正在扫描 !host! 常用端口...
for %%p in (!ports!) do (
    echo   | findstr "." nul &gt; "%temp%\port_%%p.txt" 2&gt;&amp;1
    (echo ^| set /p=) | telnet !host! %%p 2&gt;nul | findstr "."
    if !errorlevel!==0 (
        echo   [OPEN] %%p
    ) else (
        echo   [CLOSED] %%p
    )
)
goto end
:subnet_scan
echo 正在扫描 192.168.1.1-254 ...
set /a online=0
for /L %%i in (1,1,254) do (
    ping -n 1 -w 300 192.168.1.%%i &gt;nul 2&gt;&amp;1
    if !errorlevel!==0 (
        echo   [ONLINE] 192.168.1.%%i
        set /a online+=1
    )
)
echo.
echo 在线主机: !online! 台
goto end
:http_probe
set /p "url=要探测的HTTP服务: "
echo 正在请求 !url! ...
```

```
powershell -NoProfile -Command "try { $r=Invoke-WebRequest -Uri '!url!' -TimeoutSec 5; Write-Host StatusCode: $r.StatusCode; Write-Host ContentLength: $r.Content.Length } catch { Write-Host Error: $_.Exception.Message }"
goto end
:iface_info
echo --- 所有网络接口 ---
netsh interface show interface
echo.
echo --- 每个接口详细 ---
ipconfig /all
echo.
echo --- 已连接的共享 ---
net use 2&gt;nul
goto end
:end
pause
```

进阶项目10：批量主机连通性扫描

这个扫描器是 BAT 多线程技术的集大成者：start /b 并发 ping、IP 段生成、CIDR 解析、失败重试、CSV 导出、并发数控制（防止创建过多 cmd.exe）。核心技巧是用临时文件 !IP_LIST!.ok 收集在线主机、!IP_LIST!.result 收集原始 ping 输出，最后用 findstr /c:"TTL" 过滤出成功记录。

```
@echo off
setlocal enabledelayedexpansion
title 批量主机连通性扫描器 v2.0
color 0B

rem ============================================================
rem  批量主机连通性扫描器
rem  多线程 ping + 结果导出 CSV + 失败重试
rem  支持 IP 段、IP 列表、子网 CIDR 三种输入
rem  输出：在线/离线统计 + CSV 详细报告 + 弹窗告警
```

```
rem ============================================================

rem ========== 参数 ==========
set "TARGETS=%~1"
if "!TARGETS!"=="" (
    echo 输入模式:
    echo [1] 单个 IP
    echo [2] IP 段 (如: 192.168.1.1-254)
    echo [3] 子网扫描 (如: 192.168.1.0/24)
    echo [4] 从文件读取 (每行一个 IP)
    set /p mode=选择:
    if "!mode!"=="1" set /p TARGETS=IP:
    if "!mode!"=="2" set /p TARGETS=IP段:
    if "!mode!"=="3" set /p TARGETS=CIDR:
    if "!mode!"=="4" set /p TARGETS=文件路径:
)

set /a RETRIES=2
set /a TIMEOUT=500
set /a PARALLEL=50
set "OUTPUT=scan_result_%date:~0,4%%date:~5,2%%date:~8,2%_%time:~0,2%%time:~3,2%%time:~6,2%.csv"
set "OUTPUT=!OUTPUT: =0!"

rem ========== 解析目标列表 ==========
set "IP_LIST=%temp%\ping_targets_%random%.txt"
if exist "!IP_LIST!" del /q "!IP_LIST!"

rem 检测格式
echo !TARGETS! | findstr /c:"-" &gt;nul
if !errorlevel! equ 1 goto :check_single_or_file

:check_range
for /f "tokens=1,2 delims=-" %%a in ("!TARGETS!") do (
    set "range_start=%%a"
    set "range_end=%%b"
```

```
)
for /f "tokens=1-4 delims=." %%a in ("!range_start!") do (
    set /a last=%%d
    set "prefix=%%a.%%b.%%c."
)
for /f "tokens=1-4 delims=." %%a in ("!range_end!") do (
    set /a end_last=%%d
)
echo 正在生成 IP 段 !range_start!-!range_end! ...
for /L %%i in (!last!,1,!end_last!) do echo !prefix!%%i &gt;&gt; "!IP_LIST!"
goto :scan

:check_single_or_file
if exist "!TARGETS!" (
    echo 从文件 !TARGETS! 读取...
    for /f "usebackq tokens=*" %%i in ("!TARGETS!") do echo %%i &gt;&gt; "!IP_LIST!"
) else (
    echo !TARGETS! &gt;&gt; "!IP_LIST!"
)

:scan
rem ========== 准备输出 ==========
echo "IP","状态","延迟(ms)","丢包%","TTl","时间戳" &gt; "!OUTPUT!"

set /a total=0
set /a online=0
set /a offline=0
set /a failed=0

echo.
echo === 批量 ping 扫描 ===
echo 输出文件: !OUTPUT!
echo 超时: !TIMEOUT!ms  重试: !RETRIES!  并发: !PARALLEL!
echo.
```

```
echo 正在扫描（Ctrl+C 可中断）...
echo.

rem ========== 多线程扫描 ==========
for /f "usebackq delims=" %%ip in ("!IP_LIST!") do (
    set /a total+=1
    rem 后台启动 ping 子进程
    start /b cmd /c "ping -n 1 -w !TIMEOUT! %%ip 2&gt;&amp;1 | findstr /c:"TTL" &gt;&gt; "!IP_LIST!.ok"; echo --- %%ip --- &gt;&gt; "!IP_LIST!.result" &amp; ping -n 1 -w !TIMEOUT! %%ip &gt;&gt; "!IP_LIST!.result""
    rem 控制并发数
    set /a running_count=0
    for /f "tokens=1" %%p in ('tasklist ^| find /c "cmd.exe"') do set /a running_count=%%p
    if !running_count! gtr !PARALLEL! (
        ping -n 2 127.0.0.1 &gt;nul
    )
)

echo 等待所有 ping 完成...
timeout /t 30 &gt;nul

rem ========== 分析结果 ==========
echo.
echo === 结果分析 ===
set "online_list=!temp!\ping_online_%random%.txt"
for /f "usebackq delims=" %%l in ("!IP_LIST!.ok") do (
    set /a online+=1
    echo %%l | findstr /i "TTL" &gt;nul &amp;&amp; echo %%l | findstr /v "^$"
    for /f "tokens=1 delims=:" %%a in ("%%l") do (
        set "cur_ip=%%a"
        set "cur_ip=!cur_ip: =!"
        echo "!cur_ip!","Online","","","TTL","%date% %time%" &gt;&gt; "!OUTPUT!"
        echo !cur_ip! &gt;&gt; "!online_list!"
    )
)
```

```
set /a offline=total-online

echo.
echo ================================================
echo          扫描结果汇总
echo ================================================
echo  总目标数:   !total!
echo  在线主机:   !online!
echo  离线主机:   !offline!
echo  在线率:     !online!/!total!
echo ================================================
echo  详细结果:   !OUTPUT!

if !online! gtr 0 (
    echo.
    echo --- 在线主机清单 ---
    type "!online_list!" 2&gt;nul
)

rem ========== 打开结果 ==========
echo.
choice /C YN /M "打开 CSV 结果"
if errorlevel 1 start "" notepad "!OUTPUT!"

rem ========== 清理 ==========
del /q "!IP_LIST!" 2&gt;nul
del /q "!IP_LIST!.ok" 2&gt;nul
del /q "!IP_LIST!.result" 2&gt;nul
del /q "!online_list!" 2&gt;nul

endlocal
pause
exit /b
```

25 种连通性扫描场景实现方案

扫描场景	输入格式	最佳工具	并发方式	延迟	结果格式	进度条	超时控制	重试机制	输出后处理	告警方式	适用范围
单 IP 测试	192.168.1.1	ping -n 4 -w 500	无	<1 秒	控制台	无	-w 500	set /a retries	findstr /c:"TTL"	echo	所有场景
IP 段扫描	192.168.1.1-254	本脚本 for /L	start /b 并发	10-60 秒	CSV	进度百分比	timeout /t	for 循环重跑	Excel 过滤	邮件	局域网
子网扫描	192.168.1.0/24	for /L 生成	start /b 并发	30-120 秒	CSV	进度百分比	timeout /t	fail_retry	在线/离线分列	邮件+IM	中小型网
大范围扫描	10.x.0.0/16	nmap/masscan	多进程并行	分钟级	XML/CSV	有	-T5 aggressive	自动重试	可视化工具	邮件+短信	企业内网
端口扫描	port 22/80/443	Test-NetConnection	start /b + try	秒级	CSV	有	Test-NetConnection -WarningAction	for /L 重跑	过滤 open	邮件告警	运维
长时监控	每分钟一次	schtasks + ping	无	<1 分钟	追加 CSV	无	-w 1000	失败后 1x2 重试	统计连续掉线次数	邮件+IM	关键业务
Windows 域	域内所有主机	dsquery computer + ping	后台进程池	分钟级	CSV	有	timeout 30	fail log	对比上次结果	邮件日报	域环境
云主机扫描	EC2 公网 IP	AWS CLI + ping	并行循环	秒级	CSV	有	-w 1000	重试 2 次	关联实例名	SNS	AWS

SSH 可达性	22 端口	powershell Test-NetConnection	start /b	秒级	CSV	有	TcpClient timeout	重试 3 次	记录可 SSH 主机	邮件	Linux 集群
数据库监听	MySQL 3306	powershell TcpClient	单线程串行	秒级	控制台	无	500 ms	重试 1 次	fail 列表	邮件	DBA
批量上传后验证	FTP/SFTP	ftp -s script	并发池	分钟级	CSV	有	timeout	断点续传	哈希校验	日志	运维自动化
Web 存活检测	HTTP/HTTPS	curl -I / powershell	start /b	毫秒级	CSV	有	-m 5	3 次重试	HTTP 状态码	邮件+IM	Web 运维
SMTP 可用性	25 端口	telnet 25 / powershell	单线程	秒级	控制台	无	timeout	手动重试	test mail	日志	邮件管理员
LDAP 连通性	389 端口	ldapsearch / powershell	单线程	秒级	控制台	无	timeout	手动重试	bind test	日志	AD 运维
打印机检测	IP/9100	portqry / powershell	并发	秒级	CSV	有	timeout	无	离线列表	IM	办公室运维
无线 AP 检测	SSID/BSSID	netsh wlan show networks	无	秒级	控制台	无	扫描间隔	无	信号强度	无	现场勘查
多 ISP 测试	出口网关	ping 114.114.114.114	单线程轮询	秒级	CSV 追加	有	-w 1000	无	抖动/丢包率	无	网络工程
DHCP 租约	范围内 IP	arp -a / ping	并发	秒级	CSV	有	-w 200	无	MAC 匹配	无	DHCP 运维
IPv6 主机	::1 / fe80::	ping -6	并发	秒级	CSV	有	-w 500	重试 1 次	IPv4+IPv6 对比	无	IPv6 迁移
VPN 通道	对端网关	ping -t tunnel_endpoint	单线程	持续	实时	有	-t 无限	自动	丢包统计	SNMP trap	VPN 运维
容器健康	docker ps 检查	docker inspect --format	单线程	秒级	CSV	有	timeout	重试 2 次	重启异常容器	邮件	K8s 运维

SLA 监控	业务 API	curl -s -o /dev/null -w %{time_total}	并发	毫秒级	时序 DB	实时	5s 超时	3 次均值	P50/P99 统计	邮件+IM	SRE
应急演练	批量下线恢复	for /L + ssh	后台并发	分钟级	日志	实时	各 30s	手动恢复	演练报告	全员通知	灾备
数据中心巡检	全部主机	nmap + SNMP	大规模并发	分钟级	时序 DB	有	-T5	自动 retry	拓扑图	日报+告警	大型运维
蜜罐告警	扫描来源 IP	蜜罐日志 + pingback	被动	实时	CSV	无	无	无	源 IP 地理定位	短信+电话	安全

BAT 并发 ping 的核心瓶颈是 cmd.exe 进程创建和控制。start /b 可以后台启动子 cmd.exe 执行 ping，但无法精确控制并发数——脚本里用 tasklist | find /c "cmd.exe" 统计当前 cmd 进程数，超过 PARALLEL 阈值就暂停一会儿。这个方式不够精确但对几百台主机的扫描够用了。

ping 结果解析的标准化方式是 findstr /c:"TTL"——无论 ping 是中文还是英文、成功输出长什么样，成功响应里一定有 TTL= 字样。失败则返回 "请求超时" 或 "Destination host unreachable"。findstr 找到 TTL 说明在线，找不到就离线。

超时参数 -w 500 最实用——500 毫秒足够局域网 ping 到，又不会让坏 IP 等太久。超时加超时是 1 秒内一个 IP。254 台主机如果串行要 254 秒，用 50 并发大概 5-10 秒完成。但注意并发会让目标主机收到大量 ICMP——不要对生产大网段高频使用。

第四部分 高级技巧与实战篇

第11章 高级语法与黑科技

本章汇总 BAT 中最有价值的高级特性——包括 ANSI 彩色输出、UAC 提权、代码混淆、混合编程等。这些内容在绝大多数入门教程中找不到，但在实际项目中极其有用。

11.1 ANSI 彩色输出

从 Windows 10 1903 开始，cmd.exe 原生支持 ANSI 转义序列。这意味着我们可以让 BAT 输出彩色文字！

```
@echo off

rem 先启用 ANSI 支持（仅需执行一次）

reg add "HKCU\Console" /v VirtualTerminalLevel /t REG_DWORD /d 1 /f &gt; nul


rem ESC 转义符（ASCII 27）

for /f %%a in ('echo prompt $E ^| cmd') do set "ESC=%%a"


echo %ESC%[31m红色文字%ESC%[0m

echo %ESC%[32m绿色文字%ESC%[0m

echo %ESC%[33m黄色文字%ESC%[0m

echo %ESC%[34m蓝色文字%ESC%[0m

echo %ESC%[35m紫色文字%ESC%[0m

echo %ESC%[36m青色文字%ESC%[0m

echo %ESC%[37m白色文字%ESC%[0m

echo %ESC%[1m%ESC%[31m加粗红色%ESC%[0m

echo %ESC%[44m%ESC%[37m蓝底白字%ESC%[0m
```

常用 ANSI 颜色码：

前景色	码	背景色	码
黑色	30	黑色背景	40
红色	31	红色背景	41
绿色	32	绿色背景	42
黄色	33	黄色背景	43
蓝色	34	蓝色背景	44
紫色	35	紫色背景	45
青色	36	青色背景	46
白色	37	白色背景	47
默认	0	重置样式	0
加粗	1	下划线	4

TIP

如果你觉得 ESC 转义符获取方式太 hack，可以用更简洁的 PowerShell 方案获取：`for /f %%e in ('powershell -NoProfile -Command "[char]27"') do set "ESC=%%e"`。或者直接用 cmd 的 `%^e%`（启用命令扩展后可用）。

11.2 代码混淆

BAT 脚本是纯文本，任何人打开就能看。如果你想保护脚本逻辑，可以用混淆技术。以下是三种常用方法：

方法1：变量拼接

```
@echo off
rem 把所有字符串拆成变量，再动态拼出来
set "a=ech"
set "b=o "
set "c=Hello "
set "d=World"

rem 动态执行
call %%a%%%%b%%%%c%%%%d%%
```

方法2：for + cmd /c 执行

```
@echo off
for %%c in ("@echo off &amp;amp; echo obfuscated! &amp;amp; pause") do cmd /c %%~c
```

方法3：用 certutil + base64 编码

```
@echo off
rem 先把脚本 base64 编码存到文件
rem 然后解码执行
certutil -decode encoded.b64 decoded.bat
call decoded.bat
del decoded.bat
```

NOTE

更狠的做法：用 VBScript 或 PowerShell 生成的 BAT 加载器，把真实脚本用 AES 加密，运行时才解密。第 14 章的实战项目会讲到完整方案。

11.3 UAC 提权

很多系统操作（写 Program Files、改 HKLM 注册表、管理服务）需要管理员权限。以下是 BAT 自我提权的几种方案：

方案1：runas（最简单）

```
@echo off
rem 提示用户选择管理员账户
runas /user:Administrator "cmd /c %~f0"
exit /b
```

方案2：schtasks（自动提权）

创建一个计划任务（以 SYSTEM 或管理员身份运行），然后立即触发它：

```
@echo off
set "SCRIPT=%~f0"
set "TASKNAME=MyAutoTask"

rem 创建计划任务（以最高权限运行）
schtasks /create /tn "%TASKNAME%" /tr ""%SCRIPT%"" /sc onlogon /rl HIGHEST /f

rem 立即运行
schtasks /run /tn "%TASKNAME%"

rem 等待一下，然后删除任务
timeout /t 3 /nobreak &gt; nul
schtasks /delete /tn "%TASKNAME%" /f
exit /b
```

方案3：MSHTA 弹窗提权

```
@echo off
```

```
rem 通过 mshta 弹 UAC 对话框让用户确认
set "UAC_VBS=%temp%\uac.vbs"
echo Set UAC = CreateObject^("Shell.Application"^) &gt; "%UAC_VBS%"
echo UAC.ShellExecute "cmd.exe", "/c %SCRIPT%", "", "runas", 1 &gt;&gt; "%UAC_VBS%"
cscript //nologo "%UAC_VBS%"
del "%UAC_VBS%"
exit /b
```

11.4 自复制与自删除

自复制

```
@echo off
rem 把自己复制到启动文件夹
copy /y "%~f0" "%appdata%\Microsoft\Windows\Start Menu\Programs\Startup\%~nx0"
```

自删除

BAT 运行时被 cmd.exe 占用，不能直接 del。但可以用一个小技巧——在脚本末尾删除自己：

```
@echo off
rem ... 你的业务代码 ...

rem 退出前删除自己
del /f /q "%~f0" &amp;amp; exit /b
```

TIP

原理：`&` 会先执行 del，再执行 exit。但有竞争条件——如果 del 失败（文件被锁），脚本就退出了。更可靠的方案：让外部进程（如 cmd /c）来删，或者用 VBScript。

```
rem 更可靠的自删除方案
start /b cmd /c "timeout /t 1 &amp;amp;amp; del /f /q "%~f0""
exit /b
```

11.5 混合编程

BAT 不是万能的，但它是最好的"胶水语言"——可以轻松调用 PowerShell、VBScript、JavaScript 完成 BAT 不擅长的任务。

BAT 调用 PowerShell

```
@echo off

rem 用 PowerShell 生成随机密码

for /f "usebackq" %%p in ('powershell -NoProfile -Command "(New-Object System.Security.Cryptography.RNGCryptoServiceProvider | ForEach-Object { $_.GetBytes(8) | ForEach-Object { $_.ToString('X2') } })"') do set "password=%%p"

echo 随机密码: %password%


rem 用 PowerShell 下载文件

powershell -NoProfile -Command "Invoke-WebRequest -Uri 'https://example.com/file.zip' -OutFile 'D:\Downloads\file.zip'"


rem 用 PowerShell 加密文件

powershell -NoProfile -Command "$key = (1..32 | ForEach-Object { Get-Random -Maximum 256 }); Protect-File -Path 'data.txt' -OutFile 'data.enc' -Key $key"
```

TIP

参数传递技巧：把 PowerShell 脚本写在 BAT 内部，用 stdin 传给 PowerShell：`powershell -NoProfile -Command "-" <&1 <<EOF ... EOF`（BAT 不支持 heredoc，实际要用 echo 逐行写入临时 .ps1 文件再执行）

BAT 调用 VBScript

```
@echo off

rem 动态生成 VBScript 文件

set "VBS=%temp%\popup.vbs"

echo MsgBox "这是一个VBScript弹窗" ^&amp;amp; vbCrLf ^&amp;amp; "由BAT调用", 64, "提示" &gt; "%VBS%"

cscript //nologo "%VBS%"

del "%VBS%"
```

BAT 调用 JavaScript（cscript）

```
@echo off

set "JS=%temp%\calc.js"

echo var result = Math.sqrt(256); &gt; "%JS%"
```

```
echo WScript.Echo(result); &gt;&gt; "%JS%"

for /f "usebackq" %%r in (`cscript //nologo "%JS%"`) do set "sqrt=%%r"

del "%JS%"

echo 256 的平方根是: %sqrt%
```

11.6 HTA 界面

HTA（HTML Application）可以把 HTML + JS + VBScript 打包成一个"程序界面"，而 BAT 可以生成和调用 HTA——这相当于给 BAT 脚本加上了 GUI！

```
@echo off

chcp 65001 &gt; nul


rem ========== 生成 HTA 界面 ==========

set "HTA_FILE=%temp%\my_tool.hta"


rem 写入 HTA

&gt; "%HTA_FILE%" echo ^&lt;!DOCTYPE html^&gt;

&gt;&gt; "%HTA_FILE%" echo ^&lt;HTA:APPLICATION ID="MyTool" APPLICATIONNAME="MyTool" BORDER="thin" SINGLEINSTANCE
WINDOWSTATE="normal"^&gt;

&gt;&gt; "%HTA_FILE%" echo ^&lt;html^&gt;

&gt;&gt; "%HTA_FILE%" echo ^&lt;head^&gt;

&gt;&gt; "%HTA_FILE%" echo   ^&lt;meta charset="utf-8"^&gt;

&gt;&gt; "%HTA_FILE%" echo   ^&lt;title^&gt;系统维护工具^&lt;/title^&gt;

&gt;&gt; "%HTA_FILE%" echo   ^&lt;HTA:APPLICATION CAPTION="yes" ICON="" MAXIMIZEBUTTON="yes" MINIMIZEBUTTON="yes"
SHOWINTASKBAR="yes" SIZED="yes" SYSMENU="yes" VERSION="1.0" WINDOWSTATE="normal"/^&gt;

&gt;&gt; "%HTA_FILE%" echo   ^&lt;style^&gt;

&gt;&gt; "%HTA_FILE%" echo     body { font-family: Deng, sans-serif; padding: 20px; background: #f5f5f5; }

&gt;&gt; "%HTA_FILE%" echo     button { padding: 10px 20px; margin: 5px; font-size: 14px; cursor: pointer; }

&gt;&gt; "%HTA_FILE%" echo     #output { background: #000; color: #0f0; font-family: Consola; padding: 10px; height:
200px; overflow-y: auto; }

&gt;&gt; "%HTA_FILE%" echo   ^&lt;/style^&gt;

&gt;&gt; "%HTA_FILE%" echo ^&lt;/head^&gt;

&gt;&gt; "%HTA_FILE%" echo ^&lt;body^&gt;

&gt;&gt; "%HTA_FILE%" echo   ^&lt;h2^&gt;系统维护工具^&lt;/h2^&gt;
```

```
&gt;&gt; "%HTA_FILE%" echo   ^&lt;button onclick="runCmd('systeminfo')"^&gt;查看系统信息^&lt;/button^&gt;
&gt;&gt; "%HTA_FILE%" echo   ^&lt;button onclick="runCmd('ipconfig /all')"^&gt;查看网络配置^&lt;/button^&gt;
&gt;&gt; "%HTA_FILE%" echo   ^&lt;button onclick="runCmd('tasklist')"^&gt;进程列表^&lt;/button^&gt;
&gt;&gt; "%HTA_FILE%" echo   ^&lt;div id="output"^&gt;^&lt;/div^&gt;
&gt;&gt; "%HTA_FILE%" echo   ^&lt;script language="VBScript"^&gt;
&gt;&gt; "%HTA_FILE%" echo    Sub runCmd(cmd)
&gt;&gt; "%HTA_FILE%" echo      Dim shell, exec
&gt;&gt; "%HTA_FILE%" echo      Set shell = CreateObject("WScript.Shell")
&gt;&gt; "%HTA_FILE%" echo      Set exec = shell.Exec("cmd /c " ^&amp;amp; cmd)
&gt;&gt; "%HTA_FILE%" echo      document.getElementById("output").innerText = exec.StdOut.ReadAll()
&gt;&gt; "%HTA_FILE%" echo    End Sub
&gt;&gt; "%HTA_FILE%" echo   ^&lt;/script^&gt;
&gt;&gt; "%HTA_FILE%" echo ^&lt;/body^&gt;
&gt;&gt; "%HTA_FILE%" echo ^&lt;/html^&gt;

rem 打开 HTA
start "" "%HTA_FILE%"
```

TIP

HTA 运行在一个受限的安全沙箱中。它可以通过 WScript.Shell 运行 cmd 命令，通过 fso（File System Object）操作文件。一个完整的 BAT + HTA 工具示例见第 14.5 节。

11.7 编码问题彻底解决

中文乱码是 BAT 开发者最头疼的问题。以下是彻底的解决方案：

方案一：chcp 65001 + UTF-8 BOM

```
@echo off
chcp 65001 &gt; nul

rem 保存文件为 UTF-8 with BOM
rem VS Code: 右下角选择 "UTF-8 with BOM"
rem Notepad++: 编码 → 转为 UTF-8 BOM

echo 这段中文不会乱码
```

方案二：PowerShell 转码

```
@echo off
rem 如果脚本被编码了，用 PowerShell 转成 GBK
powershell -NoProfile -Command "[System.IO.File]::ReadAllText('%~f0', [System.Text.Encoding]::UTF8)" ^| cmd
```

方案三：始终转成 GBK 输出

```
@echo off
chcp 936 &gt; nul
rem 文件用 ANSI (GBK) 保存
rem VS Code 里保存时选 "GBK"
```

TIP

推荐方案：在 Windows 10/11 上用 **UTF-8 BOM + chcp 65001**；在需要兼容旧系统（Win7）时用 **GBK + chcp 936**。最保险的办法：脚本开头自动判断，然后设置正确编码。

11.8 调试与错误处理

调试技巧

```
rem 技巧1：echo on 开启全回显
@echo on
rem ... 会显示每条命令 ...

rem 技巧2：pause 在关键位置暂停
set /p debug="调试点1: 变量值=%var%，回车继续"

rem 技巧3：写到调试日志
echo [%time%] var=%var% &gt;&gt; debug.log

rem 技巧4：用 cmd /k 运行，窗口不关闭
cmd /k "call myscript.bat"

rem 技巧5：Process Monitor 监控文件/注册表操作
rem Sysinternals 工具包下载
```

错误处理框架

```
@echo off
setlocal enabledelayedexpansion

rem 统一错误处理
call :init_error_handling

rem ======== 你的脚本内容 ========
call step1 || call :fatal "step1 失败"
call step2 || call :fatal "step2 失败"
call step3 || call :fatal "step3 失败"
echo 全部成功!
exit /b 0

rem ======== 错误处理函数 ========
:init_error_handling
set "ERROR_LOG=error_log.txt"
echo === 错误日志 %date% %time% === &gt; "%ERROR_LOG%"
goto :eof

:fatal
set "msg=%~1"
echo [FATAL] %msg% | tee -a "%ERROR_LOG%" 2&gt;&amp;1
echo 脚本已终止，按任意键退出...
pause &gt; nul
exit /b 1

:step1
rem 具体步骤
copy file.txt backup.txt
if errorlevel 1 exit /b 1
exit /b 0

:step2
...
```

11.9 性能优化

```
rem 1. 减少 cmd 实例启动（管道会启动新 cmd）
rem 慢:
for /f "tokens=*" %%l in ('dir /b ^| findstr "\.txt$"') do ...
rem 快:
dir /b *.txt &gt; temp_list.txt
for /f "tokens=*" %%l in (temp_list.txt) do ...
del temp_list.txt

rem 2. 延迟扩展有开销——只在必要时启用

rem 3. 缓存环境变量到本地变量
set "PROGRAMFILES=%ProgramFiles%"
set "WINDIR=%windir%"

rem 4. 避免在大循环里多次调用外部命令
rem 慢：for %%f in (*.txt) do @echo %%~zf
rem 快：先 dir /s /b *.txt，再解析

rem 5. robocopy 自带多线程（/MT:n），比 copy 快 5-10 倍

rem 6. 减少变量赋值次数——set /a 是原子操作

rem 7. 不在循环里 echo 输出（控制台 I/O 慢），用 &gt; 文件 收集后一次性写入
```

— 第11章 完 —

进阶练习

练习1：BAT + PowerShell 混合编程：

```
@echo off
echo === BAT + PowerShell ===
for /f "delims=" %%r in (
  powershell -Command "[math]::Sqrt(123456)"
) do echo 123456 平方根 = %%r
echo hello123 | powershell -Command "$input | Select-String -Pattern '\d+'"
echo {"a":1} | powershell -Command "$input | ConvertFrom-Json"
echo secret &gt; plain.txt
certutil -encode plain.txt encoded.b64
echo base64 完成
del plain.txt encoded.b64
pause
```

练习2：UAC 提权代码：

```
@echo off
net session &gt;nul 2&gt;nul
if %errorlevel% neq 0 (
    echo 请求管理员权限...
    powershell -Command "Start-Process cmd -ArgumentList '/c %~f0 %*' -Verb RunAs"
    exit /b
)
echo 现在以管理员运行
whoami
pause
```

疑难解答

Q1: BAT 能做加密吗?

A: BAT 本身不行，但可调用 PowerShell .NET 加密库。certutil 可做 base64：certutil -encode file.txt out.b64

Q2: 怎么让 BAT 完全不可见?

A: BAT 本质是 cmd 进程总有窗口。办法：(1) VBS Run ..., 0 (2) PowerShell -WindowStyle Hidden (3) 编译成 EXE

Q3: BAT 脚本最大写多大?

A: 无硬性限制，但单文件建议不超 1000 行。超过 500 行考虑用 PowerShell 或 Python 重写核心逻辑。

进阶主题1：BAT 调 PowerShell 的 8 种姿势

BAT 调 PowerShell 的核心是 -Command 参数。复杂引号嵌套可以用 -EncodedCommand 传 base64。for /f 捕获 PowerShell 输出到 BAT 变量是最常用的技巧。管道、文件执行、内联多行各有适用场景。

```
@echo off
echo === BAT 调 PowerShell 的 8 种姿势 ===
echo.
echo --- 1. 直接执行单条命令 ---
powershell -Command "Get-Date -Format yyyyMMdd"
echo.
echo --- 2. 带引号嵌套 ---
powershell -Command "Write-Host "Hello from BAT""
echo 或者用单引号:
powershell -Command "Write-Host 'Hello'"
echo.
echo --- 3. 读取 PowerShell 输出到变量 ---
for /f "delims=" %%v in ('powershell -Command "(Get-Date).Year"') do set "year=%%v"
echo 年份: %year%
echo.
echo --- 4. 把 BAT 变量传给 PowerShell ---
set "input=BAT 参数"
powershell -Command "Write-Host '从 BAT 接收: ' + '%input%'"
```

```
echo.
echo --- 5. 管道传数据 ---
echo 一些数据 | powershell -Command "$input | ForEach-Object { Write-Host '收到: ' + $_ }"
echo.
echo --- 6. 用 -EncodedCommand 避免引号地狱 ---
echo 把复杂脚本 base64 编码:
echo powershell -EncodedCommand JABHAGUAdAAtAEQAYQB0AGUAIgA=
echo (用 powershell 先编码, 再传给 bat)
echo.
echo --- 7. 执行 .ps1 脚本文件 ---
powershell -ExecutionPolicy Bypass -File "C:\path\script.ps1"
echo.
echo --- 8. 内联多行 PowerShell ---
powershell -NoProfile -Command ^
$code = @' ^
Get-ChildItem C:\ *.exe ^| ^
Sort-Object Length -Descending ^| ^
Select-Object -First 10 ^
'@ ^
Invoke-Expression $code
pause
```

进阶主题2：用 cscript 调 VBScript 弹窗

VBScript 的 MsgBox 和 InputBox 是 BAT 做 GUI 的老办法，通过 cscript //nologo 调用。但 PowerShell 的 System.Windows.Forms 更现代、更灵活，建议新项目用 PowerShell。

```
@echo off
echo === 用 cscript 调 VBScript 弹窗 ===
echo.
echo VBScript 是 BAT 调用 GUI 的传统方案:
echo.
echo --- 1. MsgBox 最简单 ---
echo 生成临时 .vbs:
```

```
(
    dim msg
    msg = MsgBox("这是一个 VBScript 弹窗" ^&amp;amp; vbCrLf ^&amp;amp; "点击确定继续", vbInformation + vbOKCancel, "提示")
    if msg = vbOK then
        echo 用户点了确定
    else
        echo 用户点了取消
    end if
) &gt; popup.vbs
cscript //nologo popup.vbs
echo.
echo --- 2. 用 PowerShell 更灵活 ---
echo PowerShell System.Windows.Forms.MessageBox:
powershell -Command "Add-Type -AssemblyName System.Windows.Forms; [System.Windows.Forms.MessageBox]::Show('Hello fro
BAT + PowerShell', '提示', 'OK', 'Information')"
echo.
echo --- 3. 输入框 ---
VBScript InputBox:
(
    dim input
    input = InputBox("请输入你的名字:", "输入框", "默认值")
    if input = "" then
        echo 用户取消了
    else
        echo 你输入了: " &amp;amp; input
    end if
) &gt; inputbox.vbs
cscript //nologo inputbox.vbs
echo.
echo --- 4. PowerShell 输入框 ---
powershell -Command "Add-Type -AssemblyName Microsoft.VisualBasic; [Microsoft.VisualBasic.Interaction]::InputBox('请输入密码:
', '安全输入', '')"
echo.
```

```
del popup.vbs inputbox.vbs 2&gt;nul
pause
```

进阶主题3：编译 BAT 为 EXE 的 3 种方案对比

IExpress 是 Windows 自带但只是打包；第三方 BAT2EXE 工具方便但有安全隐患；ps2exe 把 PowerShell 编译成真正的 exe。关键认知：任何把 BAT 嵌入 EXE 的方案都能被反编译，真保护要用编译型语言重写核心。

```
@echo off
echo === 编译 BAT 为 EXE 的 3 种方案对比 ===
echo.
echo --- 方案1: IExpress (Windows 自带!) ---
echo Windows 自带 iexpress, 可以把 bat 打包成自解压 exe:
echo iexpress /n setup.sed
echo 优点: 免安装、系统原生
echo 缺点: 只是打包解压, 运行时还是 bat
echo.
echo --- 方案2: BAT To EXE Converter (第三方) ---
echo 工具: Advanced BAT to EXE Converter / ExeScript / Bat To Exe
echo 这些工具把 bat 嵌入 exe, 用内置 cmd 执行
echo 有些支持加密、反编译保护
echo 优点: 有 GUI、功能全
echo 缺点: 免费版有后门风险, 付费
echo.
echo --- 方案3: PowerShell + WinForms 重写 ---
echo 本质是重写成 PowerShell 脚本, 编译成 exe:
echo ps2exe -InputFile script.ps1 -OutputFile script.exe
echo 优点: 完全原生 exe, 无 cmd 窗口
echo 缺点: 需要重写, 有 .NET 依赖
echo.
echo --- 方案4: nsis / Inno Setup ---
echo 用安装包工具把 bat 打包进安装程序:
echo nsis: 写出安装脚本, 把 bat 放进去
echo inno setup: 类似, 支持压缩、签名
```

```
echo 适合做正式安装包
echo.
echo --- 推荐 ---
echo 临时发布: IExpress (零成本)
echo 商业发布: Inno Setup + ps2exe
echo 技术炫技: C#/Python 重写 + 打包
echo.
echo --- 反编译警告 ---
echo 任何把 bat 嵌入 exe 的方案都可反编译提取出原始代码
echo 真要保护逻辑, 用 C#/C++ 重写核心
pause
```

表 11-1 BAT 与 PowerShell 互通 10 种姿势

编号	方式	命令	最佳场景
1	powershell -Command 内联	powershell -Command "Get-Date"	单行小操作
2	powershell -File 外部脚本	powershell -File script.ps1	复杂逻辑放 .ps1
3	& 调用运算符	powershell "& { code }"	传参数给脚本块
4	BAT 用 for /f 捕获输出	for /f %%v in ('powershell')	BAT 读 PowerShell 数据
5	PowerShell 读 BAT 输出	$o = cmd /c "dir"	PowerShell 调 BAT 工具
6	共享环境变量	$env:VAR	进程内传递数据
7	PowerShell Set-Content 写文件	powershell ... | Out-File data.txt	中间数据文件
8	bat 写入 %TEMP% 文件再读	B->file->P 或 P->file->B	跨进程通信
9	PowerShell Invoke-RestMethod	bat 调它取 JSON 再解析	BAT 做 HTTP API
10	cmd /c 嵌套	powershell: cmd /c "script.bat"	PowerShell 包 BAT

表 11-2 UAC 提权三种方案对比

方案	兼容性	Windows 10/11	需要额外下载	自动提权	用户交互	推荐

runas				（输入密码）	需要密码	不推荐（有更优）
PowerShell Start-Process -Verb RunAs				（触发 UAC）	UAC 弹窗	最推荐
CreateProcessAsUser (API)					静默	需要调 Windows API
schtasks /rl HIGHEST					可能要密码	计划任务方式
gpedit + 本地组策略		Pro/Ent		（配置一次）	无	企业环境
UAC 注册表关闭					无	不推荐（降低安全）
PsExec (Sysinternals)			要下载		无	最强大但需下载
net session + 自重启					UAC 弹窗	BAT 最常用模式

第 11 章小结

BAT 和 PowerShell 不是对手，而是互补。BAT 做流程控制和文件 I/O 是强项，PowerShell 做系统查询和高级功能是强项。表 11-1 给出了 10 种互通方式，其中 for /f "delims=" %%v in ('powershell -NoProfile -Command ...') 是 BAT 读 PowerShell 输出的最经典模式。

UAC 提权有 4 种主流方案：runas（旧，需密码）、PowerShell Start-Process -Verb RunAs（最推荐，触发 UAC 弹窗）、schtasks 计划任务、net session + 自重启（BAT 最常用）。自提权模板 pattern：net session >nul 2>&1 检查 → 如果非管理员则 powershell Start-Process -Verb RunAs 启动自己。

完整自提权模板演示了检测 + 弹窗 + 重新启动 + 执行管理员操作的完整流程。脚本用 %~f0 取自身完整路径，powershell 命令里的引号转义很关键——单引号和双引号要混用，或者用反引号转义。

PowerShell 命令库项目把 6 个最常用的 PowerShell 查询（CPU 负载、内存、运行时间、Top 进程、磁盘、OS 信息）封装成 BAT call 子程序。每个 :ps_xxx 函数里都用 for /f 包 powershell 命令，然后 echo 输出。这就是 BAT 调 PowerShell 的标准写法。

PowerShell 的 -NoProfile 参数一定要加——不加会执行用户的 $PROFILE 脚本，不仅慢还可能有意外副作用。-Command 和 -File

的引号转义规则也完全不同：-Command 用 PowerShell 引号规则，-File 用 cmd 引号规则。

下一章进入系统维护——清理、碎片整理、备份、系统还原、日志查看。BAT 脚本做这些重复运维任务是最省心的自动化方式。准备好了吗？

第 11 章大代码块 1：BAT + PowerShell 互通 10 种姿势

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第11章综合实战：BAT + PowerShell 互通
echo   10 种从 BAT 调用/嵌入 PowerShell 的方式
echo ============================================
echo.
echo === 方式 1: powershell -Command 直接内联 ===
echo --- 获取当前日期时间 ---
for /f %%d in ('powershell -NoProfile -Command "Get-Date -Format 'yyyy-MM-dd HH:mm:ss'"') do set "dt=%%d"
echo   !dt!
echo.
echo === 方式 2: powershell -File 外部脚本 ===
echo   powershell -NoProfile -File script.ps1
echo.
echo === 方式 3: 从 stdin 读 PowerShell 代码 ===
echo --- 用 here-string 传多行 ---
powershell -NoProfile -Command "&amp;amp; { $PSVersionTable.PSVersion; Get-Date }"
echo.
echo === 方式 4: BAT 捕获 PowerShell 输出 ===
for /f "delims=" %%l in ('powershell -NoProfile -Command "Get-Process cmd | Select -First 1 Name,Id"') do (
    echo   %%l
)
echo.
echo === 方式 5: PowerShell 捕获 BAT 退出码 ===
```

```
powershell -NoProfile -Command "cmd /c "dir C:\Windows\System32\cmd.exe" "`$LASTEXITCODE""
echo.
echo === 方式 6: BAT 设置变量让 PowerShell 读 ===
set "bat_var=hello_from_bat"
powershell -NoProfile -Command "Write-Host 'BAT says:' $env:bat_var"
echo.
echo === 方式 7: PowerShell 写回给 BAT ===
for /f "delims=" %%v in ('powershell -NoProfile -Command "'hello_from_powershell'"') do (
    set "ps_var=%%v"
)
echo   BAT 收到: !ps_var!
echo.
echo === 方式 8: PowerShell Invoke-Command 远程 ===
echo   powershell -Command "Invoke-Command -ComputerName server01 -ScriptBlock { Get-Process }"
echo.
echo === 方式 9: cmd /c 从 PowerShell 调 BAT ===
echo   cmd /c "my_script.bat arg1"
echo.
echo === 方式 10: 混合脚本 (.bat + .ps1 结合) ===
echo   BAT 用 for /f 包 PowerShell 输出，最经典模式
echo.
echo === Bonus: UAC 提权 ===
echo --- 方案 A: runas (旧) ---
echo   runas /user:admin "cmd /c my.bat"
echo --- 方案 B: PowerShell Start-Process -Verb RunAs ---
echo   powershell -Command "Start-Process cmd -ArgumentList '/c my.bat' -Verb RunAs"
echo --- 方案 C: 自提权（最常用） ---
call :elevate
pause
exit /b 0
:elevate
net session &gt;nul 2&gt;&amp;1
```

```
if !errorlevel!==0 exit /b 0
echo 需要管理员权限，正在请求 UAC...
powershell -NoProfile -Command "Start-Process -FilePath 'cmd.exe' -ArgumentList '/c ""%~f0""' -Verb RunAs"
exit /b 0
```

第 11 章大代码块 2：完整自提权模板

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo  第11章进阶：完整自提权模板
echo  检测管理员状态 -&gt; UAC 弹窗 -&gt; 重新启动
echo ============================================
echo.
echo --- 检查当前权限 ---
net session &gt;nul 2&gt;&amp;1
if !errorlevel!==0 (
    echo [OK] 已经是管理员
    goto run_as_admin
)
echo [INFO] 当前不是管理员，请求 UAC ...
echo.
rem === 方案 1: PowerShell Start-Process（最兼容）===
powershell -NoProfile -Command ^
  "Start-Process -FilePath `"cmd.exe`" ^
    -ArgumentList `"/k ""%~f0"" %*`" ^
    -Verb RunAs ^
    -WorkingDirectory `""%~dp0""
exit /b 0
:run_as_admin
echo --- 提权成功，执行管理员操作 ---
```

```
echo.
echo [1] 注册环境变量（机器级）
setx /M MY_VAR "admin_value"
echo.
echo [2] 启动禁用的服务
net start wuauserv
echo.
echo [3] 安装驱动/软件
echo  （示例：静默安装）
echo  msiexec /i package.msi /qn
echo.
echo [4] 修改系统设置
reg add "HKLM\SYSTEM\CurrentControlSet\Control\FileSystem" /v LongPathsEnabled /t REG_DWORD /d 1 /f
echo  已开启长路径支持
echo.
echo [5] 防火墙放行
netsh advfirewall firewall add rule name="MyApp" dir=in action=allow protocol=TCP localport=8080
echo  防火墙规则已添加
echo.
echo [完成] 所有管理员操作执行完毕
echo 脚本将自动退出
timeout /t 3 &gt;nul
exit /b 0
```

第 11 章进阶项目：BAT 辅助 PowerShell 命令库

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ============================================
echo  第11章项目：BAT 辅助 PowerShell 命令库
```

```
echo   把常用 PowerShell 功能封装成 BAT call 子程序

echo ============================================

echo.

call :ps_get_cpu

call :ps_get_ram

call :ps_get_uptime

call :ps_top_processes

call :ps_disk_info

call :ps_os_info

echo.

echo === 全部调用完成 ===

pause

exit /b 0

:ps_get_cpu

for /f %%c in ('powershell -NoProfile -Command "(Get-CimInstance Win32_Processor).LoadPercentage"') do echo   CPU 负载: %%c %%

exit /b 0

:ps_get_ram

for /f "tokens=*" %%r in ('powershell -NoProfile -Command "$c=Get-CimInstance Win32_OperatingSystem; $total=[math]::Round($c.TotalVisibleMemorySize/1MB,1); $free=[math]::Round($c.FreePhysicalMemory/1MB,1); Write-Output '${total} GB / ${free} GB free'"') do echo   内存: %%r

exit /b 0

:ps_get_uptime

for /f %%t in ('powershell -NoProfile -Command "(Get-Date) - (Get-CimInstance Win32_OperatingSystem).LastBootUpTime | ForEach-Object { $_.Days.ToString() + '天 ' + $_.Hours.ToString() + '小时' }"') do echo   运行时间: %%t

exit /b 0

:ps_top_processes

echo --- Top 5 进程（CPU）---

powershell -NoProfile -Command "Get-Process | Sort CPU -Descending | Select -First 5 Name,CPU | Format-Table -AutoSize"

exit /b 0

:ps_disk_info

echo --- 磁盘信息 ---

powershell -NoProfile -Command "Get-CimInstance Win32_LogicalDisk -Filter DriveType=3 | Select DeviceID, @{N='SizeGB';E={[math]::Round($_.Size/1GB,1)}}, @{N='FreeGB';E={[math]::Round($_.FreeSpace/1GB,1)}} | Format-Table -AutoSize"
```

```
exit /b 0

:ps_os_info

echo --- 系统信息 ---

powershell -NoProfile -Command "$os=Get-CimInstance Win32_OperatingSystem; Write-Host "OS: $($os.Caption)"; Write-Host "Version: $($os.Version)"; Write-Host "Arch: $($os.OSArchitecture)"; Write-Host "InstallDate: $($os.InstallDate)""

exit /b 0
```

进阶项目11：完整的 BAT + PowerShell 混合工具箱

这个混合工具箱包含 15 个实用场景：数学计算、JSON 解析、哈希、Base64、AES 加密、正则、HTTPS 下载、ZIP 解压、XML 解析、Excel、SMTP 邮件、REST API、弹窗、服务管理、图像处理。全部用 BAT 做菜单+流程控制，PowerShell 做核心操作，展示了 BAT+PS 混合编程的标准模式。

```
@echo off

setlocal enabledelayedexpansion

title BAT + PowerShell 混合工具箱 - 15 个常用场景

color 09

rem ============================================================

rem  BAT + PowerShell 混合工具箱

rem BAT 做流程控制和用户交互

rem PowerShell 做复杂操作（.NET 库、WMI、JSON、加密等）

rem ============================================================

:MENU

cls

echo.

echo   ================================================

echo      BAT + PowerShell 混合工具箱 (15 in 1)

echo   ================================================

echo.

echo   [ 1] 求平方根/数学函数       [ 2] JSON 解析

echo   [ 3] 哈希文件 (SHA256)       [ 4] Base64 编解码
```

```
echo   [ 5]  加密/解密字符串          [ 6]  正则表达式匹配
echo   [ 7]  下载 HTTPS 文件          [ 8]  解压 ZIP/7Z
echo   [ 9]  XML 解析                 [10]  Excel 操作
echo   [11]  发送邮件 (SMTP)          [12]  HTTP REST API
echo   [13]  Windows 弹窗/Toast       [14]  系统服务管理
echo   [15]  截图/图像处理
echo   [ 0]  退出
echo.
set /p "choice=选择: "

if "!choice!"=="0" goto :END
if "!choice!"=="1" call :demo_sqrt
if "!choice!"=="2" call :demo_json
if "!choice!"=="3" call :demo_hash
if "!choice!"=="4" call :demo_b64
if "!choice!"=="5" call :demo_crypto
if "!choice!"=="6" call :demo_regex
if "!choice!"=="7" call :demo_download
if "!choice!"=="8" call :demo_extract
if "!choice!"=="9" call :demo_xml
if "!choice!"=="10" call :demo_excel
if "!choice!"=="11" call :demo_email
if "!choice!"=="12" call :demo_rest
if "!choice!"=="13" call :demo_toast
if "!choice!"=="14" call :demo_service
if "!choice!"=="15" call :demo_image

pause
goto :MENU
:END
exit /b

:demo_sqrt
echo.
```

```
echo [1] PowerShell 数学函数

set /p num=输入一个数字:

for /f "delims=" %%r in ('powershell -Command "[math]::Sqrt(!num!)"') do echo !num! 的平方根 = %%r

for /f "delims=" %%r in ('powershell -Command "[math]::Pow(!num!, 3)"') do echo !num! 的立方 = %%r

for /f "delims=" %%r in ('powershell -Command "[math]::PI"') do echo PI = %%r

exit /b

:demo_json

echo.

echo [2] PowerShell JSON 解析

echo {"name":"张三","age":25,"skills":["BAT","PS","Python"]} | powershell -Command "$input | ConvertFrom-Json | ForEach-Object { $_.name + ' - ' + $_.age + '岁, 会 ' + ($_.skills -join ', ') }"

exit /b

:demo_hash

echo.

echo [3] PowerShell 文件哈希 (SHA256)

echo !random! test content &gt; hash_test.txt

powershell -Command "Get-FileHash -Path 'hash_test.txt' -Algorithm SHA256 | Select-Object Hash, Path"

del hash_test.txt

exit /b

:demo_b64

echo.

echo [4] PowerShell Base64 编解码

echo 原始字符串: Hello BAT + PowerShell

for /f "delims=" %%e in ('powershell -Command "[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes('Hello BAT + PowerShell'))"') do echo 编码后: %%e

for /f "delims=" %%d in ('powershell -Command "[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('SGVsbG8gQkFUICsgUG93ZXJTaGVsbA=='))"') do echo 解码后: %%d

exit /b

:demo_crypto

echo.

echo [5] PowerShell AES-256 加密
```

```
echo 加密前: mysecretpassword

set /p pwd=输入加密密码(至少16字符):

powershell -Command "$aes=[System.Security.Cryptography.Aes]::Create(); $aes.Key=[System.Text.Encoding]::UTF8.GetBytes('!pwd!'.PadRight(32).Substring(0,32)); $aes.GenerateIV(); $enc=$aes.CreateEncryptor(); $encBytes=$enc.TransformFinalBlock([System.Text.Encoding]::UTF8.GetBytes('mysecretpassword'),0,20); echo ([Convert]::ToBase64String($encBytes))"

exit /b

:demo_regex

echo.

echo [6] PowerShell 正则表达式

echo IP 地址列表: 192.168.1.1, 10.0.0.1, 无效: 999.999.999.999

echo 192.168.1.1 10.0.0.1 999.999.999.999 | powershell -Command "echo '192.168.1.1 10.0.0.1 999.999.999.999' | Select-String -Pattern '\b(?:\d{1,3}\.){3}\d{1,3}\b' -AllMatches | ForEach-Object { $_.Matches.Value }"

exit /b

:demo_download

echo.

echo [7] PowerShell 下载 HTTPS 文件

echo 正在下载测试文件...

powershell -Command "try { Invoke-WebRequest -Uri 'https://httpbin.org/get' -OutFile 'download_test.json' -TimeoutSec 10; Write-Host '下载成功'; Get-Content download_test.json | Select-Object -First 5 } catch { Write-Host '下载失败或无网络' }"

del download_test.json 2&gt;nul

exit /b

:demo_extract

echo.

echo [8] PowerShell 解压

echo 先用 Compress-Archive 做个测试 ZIP...

echo test file content &gt; zip_test.txt

powershell -Command "Compress-Archive -Path 'zip_test.txt' -DestinationPath 'zip_test.zip'"

powershell -Command "Expand-Archive -Path 'zip_test.zip' -DestinationPath 'zip_test_out' -Force"

echo 解压结果:

type zip_test_out\zip_test.txt

del zip_test.txt zip_test.zip &amp;amp; rmdir /s /q zip_test_out

exit /b
```

```
:demo_xml
echo.
echo [9] PowerShell XML 解析
(
echo ^&lt;root^&gt;
echo   ^&lt;app name="MyBat" version="2.0"^&gt;
echo     ^&lt;author^&gt;BAT Master^&lt;/author^&gt;
echo     ^&lt;items^&gt;
echo       ^&lt;item id="1"^&gt;First^&lt;/item^&gt;
echo       ^&lt;item id="2"^&gt;Second^&lt;/item^&gt;
echo     ^&lt;/items^&gt;
echo   ^&lt;/app^&gt;
echo ^&lt;/root^&gt;
) &gt; xml_test.xml
powershell -Command "$xml=[xml](Get-Content xml_test.xml); $xml.root.app | ForEach-Object { Write-Host $_.name - $_.version - by $_.author; foreach($i in $_.items.item){Write-Host '  Item:' $i.id $i.'#text' } }"
del xml_test.xml
exit /b

:demo_excel
echo.
echo [10] PowerShell Excel 操作 (需要 Excel 已安装)
powershell -Command "try { $excel=New-Object -ComObject Excel.Application; $excel.Visible=$false; $wb=$excel.Workbooks.Add(); $ws=$wb.Sheets.Item(1); $ws.Cells.Item(1,1)='姓名'; $ws.Cells.Item(1,2)='年龄'; $ws.Cells.Item(2,1)='张三'; $ws.Cells.Item(2,2)=25; $wb.SaveAs('excel_test.xlsx'); $excel.Quit(); Write-Host 'Excel 创建成功: excel_test.xlsx' } catch { Write-Host 'Excel COM 调用失败 (可能没装 Excel)' }"
del excel_test.xlsx 2&gt;nul
exit /b

:demo_email
echo.
echo [11] PowerShell SMTP 发送邮件
echo 需要修改下面的 SMTP 配置才能工作:
echo   $smtp = smtp.example.com
```

```
echo   $from = sender@example.com

echo   $to = receiver@example.com

powershell -Command "$smtp='smtp.example.com'; $from='sender@example.com'; $to='receiver@example.com'; Send-MailMessa
-SmtpServer $smtp -From $from -To $to -Subject '来自 BAT+PS 的测试邮件' -Body '如果收到说明 SMTP 配置正确'" 2&gt;&amp;1

exit /b


:demo_rest

echo.

echo [12] PowerShell HTTP REST API

echo 请求 httpbin.org API...

powershell -Command "try { $r=Invoke-RestMethod -Uri 'https://httpbin.org/user-agent' -TimeoutSec 10; Write-Host
'UserAgent:' $r.{'user-agent'} } catch { Write-Host '网络不可用' }"

exit /b


:demo_toast

echo.

echo [13] PowerShell Windows 弹窗/Toast

powershell -Command "Add-Type -AssemblyName System.Windows.Forms; [System.Windows.Forms.MessageBox]::Show('这是一个
PowerShell 弹窗\n\n标题: 混合工具箱', 'BAT + PowerShell Demo', 'OK', 'Information')"

exit /b


:demo_service

echo.

echo [14] PowerShell 服务管理

powershell -Command "Get-Service | Where-Object {$_.Status -eq 'Running'} | Select-Object -First 10 Name, DisplayName,
Status | Format-Table -AutoSize"

exit /b


:demo_image

echo.

echo [15] PowerShell 图像处理

echo 用 System.Drawing 创建一个小图片...

powershell -Command "Add-Type -AssemblyName System.Drawing; $b=New-Object Drawing.Bitmap(200,100); $g=[Drawing.Grap
::FromImage($b); $g.Clear([Drawing.Color]::LightBlue); $font=New-Object Drawing.Font('Arial',16); $g.DrawString('Hello
BAT+PS!',$font,[Drawing.Brushes]::Black,20,40); $b.Save('image_test.png',[Drawing.Imaging.ImageFormat]::Png);
$g.Dispose(); $b.Dispose(); Write-Host '图片已生成: image_test.png'"

del image_test.png 2&gt;nul
```

```
exit /b
```

25 个 BAT vs PowerShell 能力对比与选型

场景	BAT 原生能力	PowerShell 优势	混合调用方式	输入输出格式	性能	依赖	安全注意	代码复杂度	生产推荐度
数学计算	set /a 仅整数	[math]:: 浮点	powershell -Command "[math]::Sqrt(x)"	stdout 文本	好	无	低	低	★★★★★
JSON 解析	无原生支持	ConvertFrom-Json	echo $json \| powershell -Command "$input|ConvertFrom-Json"	stdin JSON	好	无	低	中	★★★★★
正则匹配	findstr 有限正则	-match/regex	powershell -Command "$s -match 'pattern'"	stdout 布尔	好	无	低	中	★★★★★
文件哈希	certutil 仅基础	Get-FileHash 全算法	直接 Get-FileHash -Algorithm SHA256	CSV/对象	好	无	低	低	★★★★★
AES 加密	无	.NET Security.Cryptography	powershell -Command AES Create()	Base64 文本	中	无	密钥管理	高	★★★☆☆
CSV 操作	echo >> csv	Import/Export-Csv	直接 Import-Csv pipe	CSV 文件	好	无	低	低	★★★★★
XML 解析	无/极难	[xml] 类型强支持	powershell -Command "$xml=[xml](Get-Content f.xml)"	XML 文件	好	无	低	中	★★★★★
HTTPS 下载	curl 命令粗糙	Invoke-WebRequest 完整	powershell -Command Invoke-WebRequest -Uri -OutFile	文件下载	好	无	TLS 证书	低	★★★★★

SMTP 邮件	无	Send-MailMessage	powershell -Command Send-MailMessage	端口 25/465	好	SMTP 服务	凭据安全	中	★★★★☆
REST API	curl 基础	Invoke-RestMethod	powershell -Command Invoke-RestMethod -Uri	JSON 对象	好	无	Token 安全	低	★★★★★
Excel 操作	无	ImportExcel/COM	powershell -Command "New-Object Excel.Application"	XLSX 文件	中	Excel/模块	无	高	★★★☆☆
SQL 查询	无	Invoke-Sqlcmd/ADO	powershell -Command Invoke-Sqlcmd	DB 连接	好	SQL 模块	凭据安全	中	★★★★☆
注册表操作	reg add/query	Get/Set-ItemProperty	BAT 已够用，PS 更完整	注册表	好	无	权限	低	★★★★☆
服务管理	sc/net	Get/Stop/Restart-Service	BAT 够用，PS 更面向对象	SC 命令	好	无	管理员	低	★★★★☆
进程管理	tasklist/kill	Get/Stop-Process	taskkill 更快	进程 ID	好	无	PID 范围	低	★★★★☆
远程管理	无/PsExec	Enter-PSSession/WinRM	powershell -Command Invoke-Command -ComputerName	WinRM 5985	好	WinRM 开启	认证安全	中	★★★★☆
定时任务	schtasks	Register-ScheduledTask	BAT 够用，PS 更完整	XML 配置	好	无	凭据	低	★★★★☆
图像处理	无	System.Drawing	powershell -Command "Add-Type Drawing"	PNG/JPG/BMP	中	无	无	中	★★★★☆
声音播放	无	System.Media.SoundPlayer	powershell -Command "New-Object SoundPlayer"	WAV 文件	好	无	无	低	★★★☆☆
Toast 通知	无	BurntToast/WinRT	powershell -Command New-BurntToastNotification	Win10 Toast	好	BurntToast 模块	无	中	★★★☆☆

AD 域操作	无	Get-ADUser/Computer	powershell -Command Get-ADUser -Filter {sAMAccountName -eq "x"}	AD 域	中	RSAT 工具	凭据安全	高	★★★★☆
DHCP 管理	netsh dhcp	Get-DhcpServerv4Scope	netsh 也可以	DHCP 服务	好	无	管理员	中	★★★☆☆
Print 管理	无	Get-Printer	PrintManagement 模块	打印服务	好	PrintManagement	管理员	中	★★★☆☆
IIS 管理	appcmd	WebAdministration 模块	Get-Website	IIS	好	WebAdmin	管理员	中	★★★★☆
DSC 配置	无	Start-DscConfiguration	MOF 文件推送	MOF/推送	好	LCM 开启	凭据安全	高	★★★☆☆

BAT + PowerShell 混合调用的四种模式：（1）for /f "delims=" %%v in ('powershell -Command "..."') do set var=%%v——获取单值结果；（2）echo data | powershell -Command "$input | ConvertFrom-Json"——BAT 向 PS 传数据；（3）powershell -File script.ps1——复杂逻辑放 ps1 文件；（4）ps1 文件第一行用 param($arg1, $arg2)，BAT 用 powershell -File script.ps1 arg1 arg2 传参。

PowerShell -Command 里的单引号/双引号嵌套是噩梦。BAT 用双引号传 -Command 参数，里面 PS 代码里的字符串要用单引号；如果 PS 代码里还有单引号（比如转义），就要用反斜杠 '。或者干脆把复杂 PS 脚本存成 .ps1 文件用 -File 调用——省掉所有转义问题。

第12章 实战项目一：系统维护工具集

本章把前11章的知识组合成5个实用的系统维护工具。每个脚本都可以直接拿来用，或根据需求修改。

12.1 系统垃圾清理脚本

```
@echo off
chcp 65001 &gt; nul
setlocal enabledelayedexpansion

title 系统垃圾清理工具 v1.0
echo ========================================
echo        系统垃圾清理工具
echo ========================================
echo.
set /p "confirm=确定要执行清理吗？(Y/N): "
if /i not "%confirm%"=="Y" (
    echo 已取消。
    exit /b 0
)

rem 创建清理日志
set "LOG=%temp%\cleanup_log.txt"
echo === 清理开始 %date% %time% === &gt; "%LOG%"

rem 清理用户临时文件
echo [1/6] 清理用户临时文件...
set "count=0"
for %%f in ("%temp%\*.*") do (
    set /a count+=1
)
del /s /q "%temp%\*.*" 2&gt;nul
for /d %%d in ("%temp%\*") do rmdir /s /q "%%d" 2&gt;nul
echo   临时文件已清理 &gt;&gt; "%LOG%"

rem 清理系统临时文件
```

```
echo [2/6] 清理系统临时文件...
del /s /q "C:\Windows\Temp\*.*" 2&gt;nul
for /d %%d in ("C:\Windows\Temp\*") do rmdir /s /q "%%d" 2&gt;nul

rem 清理回收站
echo [3/6] 清理回收站...
rd /s /q "C:\$Recycle.Bin" 2&gt;nul

rem 清理浏览器缓存
echo [4/6] 清理浏览器缓存...
if exist "%localappdata%\Microsoft\Edge\User Data\Default\Cache" (
    rd /s /q "%localappdata%\Microsoft\Edge\User Data\Default\Cache" 2&gt;nul
)
if exist "%localappdata%\Google\Chrome\User Data\Default\Cache" (
    rd /s /q "%localappdata%\Google\Chrome\User Data\Default\Cache" 2&gt;nul
)

rem 清理 Windows Update 缓存
echo [5/6] 清理 Windows Update 缓存...
net stop wuauserv &gt;nul 2&gt;&amp;1
rd /s /q "C:\Windows\SoftwareDistribution\Download" 2&gt;nul
net start wuauserv &gt;nul 2&gt;&amp;1

rem 清理缩略图缓存
echo [6/6] 清理缩略图缓存...
del /s /q "%localappdata%\Microsoft\Windows\Explorer\thumbcache_*.db" 2&gt;nul

echo.
echo 清理完成!
echo 日志已保存到: %LOG%
pause
```

12.2 启动项管理工具

```
@echo off
```

```
chcp 65001 &gt; nul
setlocal
title 启动项管理工具

:menu
cls
echo ========================================
echo        启动项管理工具
echo ========================================
echo [1] 查看所有启动项
echo [2] 禁用某个启动项
echo [3] 启用某个启动项
echo [4] 添加开机自启（当前用户）
echo [5] 备份当前启动项
echo [0] 退出
echo ========================================
set /p "choice=请选择: "

if "%choice%"=="1" goto :view
if "%choice%"=="2" goto :disable
if "%choice%"=="3" goto :enable
if "%choice%"=="4" goto :add
if "%choice%"=="5" goto :backup
if "%choice%"=="0" goto :eof
goto :menu

:view
cls
echo --- 用户启动项 (HKCU) ---
reg query "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" 2&gt;nul
echo.
echo --- 系统启动项 (HKLM) ---
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" 2&gt;nul
echo.
```

```
echo --- 启动文件夹 ---
dir /b "%appdata%\Microsoft\Windows\Start Menu\Programs\Startup" 2&gt;nul
echo.
pause
goto :menu

:disable
set /p "name=要禁用的启动项名称: "
reg add "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "Disabled_%name%" /t REG_SZ /d "disabled" /f
echo 已标记为禁用（实际可把 Run 中的值删除，存到 DisabledRun 子键）
pause
goto :menu

:add
set /p "exe_path=要运行的程序完整路径: "
set /p "run_name=注册表中的名称: "
reg add "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "%run_name%" /t REG_SZ /d ""%exe_path%"" /f
echo 已添加。
pause
goto :menu

:backup
reg export "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" "%cd%\run_hkcu_backup.reg" /y
reg export "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" "%cd%\run_hklm_backup.reg" /y
echo 启动项已备份到当前目录。
pause
goto :menu
endlocal
```

12.3 服务批量运维脚本

```
@echo off
chcp 65001 &gt; nul
setlocal enabledelayedexpansion
```

```
title 服务批量运维工具

rem ========== 配置区 ==========
rem 要管理的服务列表（每行一个服务名）
set "SERVICE_LIST=W32Time Spooler wuauserv BITS"
rem ============================

:menu
cls
echo ========================================
echo        服务批量运维
echo ========================================
echo [1] 批量启动服务
echo [2] 批量停止服务
echo [3] 批量重启服务
echo [4] 查看服务状态
echo [5] 批量设置启动类型
echo [0] 退出
echo ========================================
set /p "choice=请选择: "

if "%choice%"=="1" goto :do_start
if "%choice%"=="2" goto :do_stop
if "%choice%"=="3" goto :do_restart
if "%choice%"=="4" goto :do_status
if "%choice%"=="5" goto :do_settype
if "%choice%"=="0" goto :eof
goto :menu

:do_start
for %%s in (%SERVICE_LIST%) do (
    echo 正在启动 %%s ...
    net start %%s &gt;nul 2&gt;&amp;1 &amp;&amp; echo        成功 || echo        失败
)
```

```
pause
goto :menu

:do_stop
for %%s in (%SERVICE_LIST%) do (
    echo 正在停止 %%s ...
    net stop %%s &gt;nul 2&gt;&amp;1 &amp;&amp; echo        成功 || echo        失败
)
pause
goto :menu

:do_restart
for %%s in (%SERVICE_LIST%) do (
    echo 正在重启 %%s ...
    net stop %%s &gt;nul 2&gt;&amp;1
    timeout /t 2 /nobreak &gt; nul
    net start %%s &gt;nul 2&gt;&amp;1 &amp;&amp; echo        成功 || echo        失败
)
pause
goto :menu

:do_status
cls
echo 服务状态一览:
echo ----------------------------------------
for %%s in (%SERVICE_LIST%) do (
    for /f "tokens=3" %%t in ('sc query %%s ^| findstr "STATE"') do (
        echo %%s: %%t
    )
)
echo ----------------------------------------
pause
goto :menu
```

```
:do_settype
cls
echo 启动类型:  1=自动  2=自动(延迟)  3=手动  4=禁用
set /p "type=选择启动类型: "
for %%s in (%SERVICE_LIST%) do (
    echo 设置 %%s ...
    sc config %%s start= %type%
)
pause
goto :menu
endlocal
```

12.4 计划任务备份恢复

```
@echo off
chcp 65001 &gt; nul
setlocal

set "BACKUP_DIR=schtasks_backup"
if not exist "%BACKUP_DIR%" mkdir "%BACKUP_DIR%"

echo [1] 备份所有计划任务
echo [2] 从备份恢复计划任务
set /p "choice=选择操作: "

if "%choice%"=="1" goto :backup
if "%choice%"=="2" goto :restore
goto :eof

:backup
cls
echo 正在导出所有计划任务...
schtasks /query /xml /v 2&gt;nul &gt; "%BACKUP_DIR%\all_tasks.xml"

rem 逐个导出详细信息
```

```
echo.
echo 逐个导出任务详情...
for /f "tokens=1" %%t in ('schtasks /query /fo csv ^| findstr /v "^任务名" ^| findstr /v "^TaskName"') do (
    set "taskname=%%t"
    set "taskname=!taskname:"=!"
    schtasks /query /tn "!taskname!" /xml &gt; "%BACKUP_DIR%\!taskname!.xml" 2&gt;nul
)
echo 备份完成，文件在 %BACKUP_DIR% 目录下。
pause
goto :eof

:restore
cls
echo 将从 %BACKUP_DIR% 目录恢复所有任务。
set /p "confirm=确认？(Y/N): "
if /i not "%confirm%"=="Y" goto :eof

for %%f in ("%BACKUP_DIR%\*.xml") do (
    echo 导入 %%~nf ...
    schtasks /create /tn "%%~nf" /xml "%%f" /f &gt;nul 2&gt;&amp;1 &amp;&amp; echo       成功 || echo       失败
)
pause
goto :eof
endlocal
```

12.5 系统快照脚本

```
@echo off
chcp 65001 &gt; nul
setlocal enabledelayedexpansion

rem ========== 系统快照（一键收集系统信息） ==========
set "OUT_DIR=sysinfo_%computername%_%date:~0,4%%date:~5,2%%date:~8,2%"
```

```
set "OUT_DIR=%OUT_DIR: =0%"

mkdir "%OUT_DIR%" 2&gt;nul

echo ========================================

echo    系统信息快照

echo    输出目录: %OUT_DIR%

echo ========================================

echo.

rem 1. 基本系统信息

echo [1/8] 系统信息...

systeminfo &gt; "%OUT_DIR%\01_systeminfo.txt" 2&gt;&amp;1

rem 2. 硬件信息

echo [2/8] 硬件信息...

wmic cpu get /format:list &gt; "%OUT_DIR%\02_cpu.txt"

wmic memorychip get /format:list &gt; "%OUT_DIR%\03_memory.txt"

wmic diskdrive get /format:list &gt; "%OUT_DIR%\04_disk.txt"

wmic baseboard get /format:list &gt; "%OUT_DIR%\05_motherboard.txt"

rem 3. 网络配置

echo [3/8] 网络配置...

ipconfig /all &gt; "%OUT_DIR%\06_network.txt" 2&gt;&amp;1

netstat -ano &gt; "%OUT_DIR%\07_connections.txt" 2&gt;&amp;1

rem 4. 已安装软件

echo [4/8] 已安装软件...

wmic product get Name,Version,InstallDate /format:csv &gt; "%OUT_DIR%\08_software.csv" 2&gt;&amp;1

rem 5. 驱动信息

echo [5/8] 驱动信息...\sdriver /s &gt; "%OUT_DIR%\09_drivers.txt" 2&gt;&amp;1 || wmic driverquery get /format:list &gt;
"%OUT_DIR%\09_drivers.txt"

rem 6. 运行进程

echo [6/8] 运行进程...

tasklist /v /fo csv &gt; "%OUT_DIR%\10_processes.csv" 2&gt;&amp;1
```

```
rem 7. 服务状态
echo [7/8] 服务状态...
sc query type= service state= all &gt; "%OUT_DIR%\11_services.txt" 2&gt;&amp;1

rem 8. 系统事件（最近100条）
echo [8/8] 系统事件...
wmic nteventlog where "LogFile='System'" get /format:list &gt; "%OUT_DIR%\12_events.txt" 2&gt;&amp;1

echo.
echo 系统快照收集完成!
echo 文件保存在: %cd%\%OUT_DIR%
start "" "%cd%\%OUT_DIR%"
pause
endlocal
```

— 第12章 完 —

进阶实战：完整系统维护工具箱。

```
@echo off
setlocal enabledelayedexpansion
title BAT 系统维护工具箱 v2.0
color 0B
:MENU
cls
echo.
echo   [1] 系统信息  [2] 进程管理  [3] 磁盘分析
echo   [4] 清理临时  [5] 网络诊断  [6] 导出日志
echo   [0] 退出
set /p choice=选择:
if "%choice%"=="1" goto :SYS
if "%choice%"=="2" goto :PROC
if "%choice%"=="3" goto :DISK
if "%choice%"=="4" goto :CLEAN
if "%choice%"=="5" goto :NET
if "%choice%"=="6" goto :EXPORT
if "%choice%"=="0" exit /b
goto :MENU
:SYS
cls
echo 计算机: %COMPUTERNAME%
echo 用户:   %USERNAME%
wmic cpu get Name /format:list
pause &amp;amp; goto :MENU
:PROC
cls &amp;amp; tasklist /fo table
pause &amp;amp; goto :MENU
:DISK
cls
```

```
wmic logicaldisk get caption,freespace,size /format:list
pause &amp;amp; goto :MENU
:CLEAN
cls
echo 清理 %temp% ...
del /q /f /s "%temp%\*.*" 2&amp;gt;nul
for /d %%d in ("%temp%\*") do rd /s /q "%%d" 2&amp;gt;nul
echo 完成
pause &amp;amp; goto :MENU
:NET
cls
ipconfig /all | findstr /c:"IPv4" /c:"网关"
netstat -ano | findstr ESTABLISHED
pause &amp;amp; goto :MENU
:EXPORT
cls
set "OUT=sysinfo_%date:~0,4%%date:~5,2%%date:~8,2%"
mkdir "%OUT%" 2&amp;gt;nul
systeminfo &amp;gt; "%OUT%\info.txt" 2&amp;gt;&amp;amp;1
tasklist /v /fo csv &amp;gt; "%OUT%\processes.csv"
ipconfig /all &amp;gt; "%OUT%\network.txt"
echo 导出到 %OUT%
start "" "%cd%\%OUT%"
pause &amp;amp; goto :MENU
```

建议：把上面脚本拆成 main.bat + modules/*.bat，用 call 调用更易维护。

进阶主题1：一键重装系统驱动备份

DISM /export-driver 可以一键导出所有当前驱动，重装后 /Add-Driver /Recurse 恢复。配合 vssadmin 创建还原点做双重保险。pnputil 可以逐个操作驱动，更灵活。

```
@echo off
echo === 一键重装系统驱动备份 ===
echo.
```

```
echo --- 1. 导出所有驱动 (DISM) ---
echo 需要管理员权限:
net session &gt;nul 2&gt;nul
if %errorlevel% neq 0 (
    echo 请右键以管理员身份运行!
    exit /b 1
)
set "driver_path=%~dp0drivers_backup"
mkdir "%driver_path%" 2&gt;nul
echo 正在导出驱动到 %driver_path% ...
dism /online /export-driver /destination:"%driver_path%"
echo 导出完成!
echo.
echo --- 2. 查看驱动列表 ---
wmic path Win32_PnPSignedDriver get DeviceName,DriverVersion,InfName /format:list
echo.
echo --- 3. 特定驱动导出 ---
echo 可以用 pnputil 逐个导出:
echo pnputil /enum-drivers
echo pnputil /export-driver oemXX.inf C:\backup
echo.
echo --- 4. 系统还原点 ---
echo 创建还原点:
vssadmin create shadow /for=C:
echo 或用 PowerShell:
echo Checkpoint-Computer -Description "重装前还原点"
echo.
echo --- 5. 驱动恢复 ---
echo 重装系统后:
echo dism /online /Add-Driver /Driver:"%driver_path%" /Recurse
echo pnputil /add-driver oemXX.inf /install
echo.
```

```
echo --- 6. 一键脚本 ---
(
    @echo off
    set "out=%~dp0drivers_%date:~0,4%%date:~5,2%%date:~8,2%"
    mkdir "%out%"
    dism /online /export-driver /destination:"%out%"
    wmic cpu get Name /format:value &gt; "%out%\cpu.txt"
    wmic baseboard get Manufacturer,Product /format:value &gt; "%out%\board.txt"
    echo 备份完成到 %out%
) &gt; backup_drivers.bat
echo 已生成一键备份脚本 backup_drivers.bat
pause
```

进阶主题2：磁盘空间分析（按文件类型统计）

BAT 本身做磁盘分析很慢，推荐直接用 PowerShell 的 Group-Object 按 Extension 分组统计。WinDirStat 是最好用的可视化工具，BAT 只能做文本版统计。

```
@echo off
echo === 磁盘空间分析 ===
echo.
echo --- 1. 查看各驱动器空间 ---
echo wmic logicaldisk get caption,freespace,size
wmic logicaldisk get caption,freespace,size /format:table
echo.
echo --- 2. 按文件类型统计 ---
echo 统计 C:\ 下 .exe 文件总大小:
setlocal enabledelayedexpansion
set "total=0"
for /r "C:\Windows\System32" %%f in (*.exe) do (
    set /a total+=%%~zf / 1024
)
echo System32 下 exe 占用约 !total! KB
endlocal
echo.
```

```
echo --- 3. PowerShell 按类型统计 (推荐) ---
echo 统计 C:\ 下各扩展名占比:
powershell -Command "Get-ChildItem C:\ -Recurse -ErrorAction SilentlyContinue ^| Group-Object Extension ^| ForEach-Object { $size=($_.Group ^| Measure-Object Length -Sum).Sum; [PSCustomObject]@{Ext=$_.Name; Count=$_.Count; SizeMB=[Math]::Round($size/1MB,1)} } ^| Sort-Object SizeMB -Descending ^| Select-Object -First 15 ^| Format-Table -AutoSize"
echo.
echo --- 4. 找大文件 ---
echo 找 C:\ 下超过 100MB 的文件:
for /r "C:\Users" %%f in (*) do (
    if %%~zf gtr 104857600 echo %%~zf  %%f
)
echo.
echo --- 5. 最大文件夹 ---
powershell -Command "Get-ChildItem C:\ -Directory ^| ForEach-Object { $size=(Get-ChildItem $_.FullName -Recurse -ErrorAction SilentlyContinue ^| Measure-Object Length -Sum).Sum; [PSCustomObject]@{Name=$_.Name; SizeGB=[Math]::Round($size/1GB,2)} } ^| Sort-Object SizeGB -Descending ^| Select-Object -First 10"
echo.
echo --- 6. WinDirStat 替代品 ---
echo ntreesoft 的 Folder Size 免费好用
echo 或用 PowerShell TreeSize 脚本
pause
```

进阶主题3：系统启动项清理（Run + RunOnce + win.ini）

启动项有四个主要来源：注册表 Run/RunOnce、启动文件夹、win.ini（过时但仍有效）、任务计划。Autoruns（Sysinternals）是最专业的分析工具，BAT 可以快速清理注册表。

```
@echo off
echo === 系统启动项清理 ===
echo.
echo --- 1. 三个主要位置 ---
echo Run:     HKCU\Software\Microsoft\Windows\CurrentVersion\Run
echo RunOnce: HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce
echo win.ini: [windows] run= line
echo.
echo --- 2. 查看所有启动项 ---
```

```
echo HKCU Run:
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" 2&gt;nul
echo.
echo HKLM Run:
reg query "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" 2&gt;nul
echo.
echo 启动文件夹:
dir /b "%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup" 2&gt;nul
dir /b "%ProgramData%\Microsoft\Windows\Start Menu\Programs\Startup" 2&gt;nul
echo.
echo --- 3. 删除可疑启动项 ---
echo 删除 HKCU 下的可疑项:
echo reg delete "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "可疑名称" /f
echo.
echo --- 4. 查看 win.ini ---
echo win.ini 已经过时但仍有效:
findstr /i "run= load=" %windir%\win.ini
echo.
echo --- 5. task scheduler ---
echo 很多恶意程序藏在任务计划:
schtasks /query /fo LIST /v | findstr /i "autorun startup"
echo.
echo --- 6. msconfig 过时, 用任务管理器 ---
echo 任务管理器 - 启动 - 禁用不需要的
echo 或: ms-settings:startupapps
echo.
echo --- 7. Autoruns (Sysinternals) ---
echo 最好用的启动项分析工具:
echo 下载: https://live.sysinternals.com/Autoruns.exe
pause
```

表 12-1 系统维护命令清单

分类	命令	功能	典型参数
磁盘维护	cleanmgr	磁盘清理	/sagerun:N 静默清理
磁盘维护	defrag	磁盘碎片整理	/O 全部 /U 分析
磁盘维护	chkdsk	磁盘错误检查	C: /f /r /x
磁盘维护	chkntfs	计划 chkdsk	C: /x 排除
系统修复	sfc	系统文件检查器	/scannow /verifyonly
系统修复	DISM	部署映像修复	/Online /Cleanup-Image /RestoreHealth
系统修复	repair-bde	BitLocker 修复	-RecoveryPassword -Force
系统还原	vssadmin	卷影副本管理	list shadows / delete shadows
系统还原	rstrui	系统还原 GUI	/offline:C:\ /windows:C:\Windows
更新管理	wuauclt	Windows Update 客户端	/detectnow /reportnow
更新管理	UsoClient	新更新代理	StartScan StartDownload StartInstall
更新管理	wusa	更新独立安装	install.msu /quiet /norestart
服务管理	net start/stop	启动/停止服务	W32Time / wuauserv
服务管理	sc	服务控制	query / config / delete
计划任务	schtasks	计划任务	/create /query /delete
临时清理	del /q	删除临时文件	C:\Windows\Temp*.*
临时清理	rd /s /q	删除 Prefetch	C:\Windows\Prefetch
回收站	rd /s /q	清空回收站	C:\$Recycle.Bin
系统信息	msinfo32	系统信息 GUI	/report out.txt
系统信息	dxdiag	DirectX 诊断	/t out.txt

表 12-2 Windows 日志查看命令

命令	工具	适用版本	过滤能力	输出格式	离线查看	说明
eventvwr.msc	事件查看器 GUI	全部		GUI		最通用
wevtutil qe	命令行查询	Win7+		XML/Text /CSV		BAT 里最好用

wevtutil el	列出所有日志	Win7+		Text		快速查看有哪些日志
wevtutil gl	查看日志配置	Win7+		Text		查询日志属性
wevtutil epl	导出日志	Win7+		.evtx		离线保存
wevtutil il	导入导出日志	Win7+		.evtx		导入离线查看
wevtutil cl	清除日志	Win7+				慎用!
vssadmin	卷影副本	Win7+		Text		查看/删除还原点
fsutil dirty	脏位检查	全部		Text		是否需要 chkdsk
fsutil volume	磁盘扩展	全部		Text		NTFS 扩容
perfmon	性能监视器	全部		GUI/CSV		性能分析
logman	性能日志管理	全部		blg/csv		命令行 perfmon
typeperf	实时计数器	全部		CSV/TSV		实时监控输出
tracerpt	ETW 追踪解析	Win7+		CSV/XML		ETW 数据分析
psloglist (Sysinternals)	高级日志工具	全部		CSV		Sysinternals 强大
WinRE logs	恢复环境	Win7+		txt		C:\Windows\Panther\
CBS.log	组件服务日志	Win7+		txt		C:\Windows\Logs\CBS\
DISM.log	DISM 日志	Win8+		txt		C:\Windows\Logs\DISM\

第 12 章小结

系统维护命令可以按"日常"和"紧急"分两类：日常清理（cleanmgr /sagerun、临时文件删除、Prefetch 清理、回收站）可以做成 BAT 每天自动跑；紧急修复（sfc /scannow、DISM /RestoreHealth、chkdsk /r）在出问题时手动跑一次。不要混用——sfc 和 DISM 同时跑会抢资源。

wevtutil 是 BAT 里操作事件日志的唯一靠谱方式。最常用的是 wevtutil qe (query events)，配合 XPath 过滤表达式能精确捞到想要的日志。示例里用到的 *[System[Level=2 and TimeCreated[timediff(@SystemTime) <= 86400000]]] 就是查最近 24 小时 Error 级别的经典写法。

一键系统维护脚本把清理临时文件、清空回收站、清理 Windows Update 缓存、sfc 检查、chkntfs、vssadmin 六个常用维护步骤串在一起。cleanmgr /sagerun:N 需要预先在 GUI 里配置过一次才会真正清理，否则只是启动 GUI。

计划任务 schtasks 的常用参数：/sc daily 每天、/sc weekly /d MON 每周一、/st 03:00 时间、/ru SYSTEM 作为系统、/rl HIGHEST 最高权限。schtasks /change /tn name /disable 可以禁用任务而不删除，便于临时停掉。

Windows 自带三个非常有用但常被忽略的日志文件：C:\Windows\Logs\CBS\CBS.log 记录 sfc 和 DISM 的修复历史、C:\Windows\Logs\DISM\DISM.log 记录 DISM 详情、C:\Windows\Panther\ 里是安装/升级的完整日志。

最后进入第 13-14 章：端口和 WLAN、BAT 黑科技和安全加固。这两章是 BAT 脚本的终极进阶部分。准备好了吗？

第 12 章大代码块 1：一键系统维护脚本

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第12章综合实战：一键系统维护脚本
echo ============================================
echo.
echo === 开始系统维护 ===
echo 时间: %date% %time%
echo.
rem === 1. 磁盘清理 ===
echo [1/7] 磁盘清理 (Windows Disk Cleanup)...
cleanmgr /sagerun:1 2&gt;nul
echo   cleanmgr 启动完毕
echo.
rem === 2. 临时文件清理 ===
echo [2/7] 临时文件清理...
set "cleaned=0"
for %%d in ("%temp%" "C:\Windows\Temp" "%systemroot%\Prefetch") do (
```

```
    if exist %%~d (
        for /f "delims=" %%f in ('dir /a-d /b /s "%%~d" 2^&gt;nul') do (
            del /f /q "%%f" 2&gt;nul &amp;&amp; set /a cleaned+=1
        )
    )
)
echo   清理 !cleaned! 个临时文件
echo.
rem === 3. Windows Update 缓存清理 ===
echo [3/7] Windows Update 缓存清理...
if exist "C:\Windows\SoftwareDistribution\Download" (
    net stop wuauserv &gt;nul 2&gt;&amp;1
    rd /s /q "C:\Windows\SoftwareDistribution\Download" 2&gt;nul
    md "C:\Windows\SoftwareDistribution\Download" 2&gt;nul
    net start wuauserv &gt;nul 2&gt;&amp;1
    echo   Windows Update 缓存已清理
) else (
    echo   跳过（非管理员或路径不存在）
)
echo.
rem === 4. 回收站清空 ===
echo [4/7] 清空回收站...
rd /s /q C:\$Recycle.Bin 2&gt;nul &amp;&amp; echo   回收站已清空 || echo   跳过（权限不足）
echo.
rem === 5. DISM 检查系统映像 ===
echo [5/7] DISM 健康检查 (sfc)...
sfc /scannow 2&gt;nul | findstr /c:"Windows Resource Protection"
echo.
rem === 6. chkntfs 检查磁盘 ===
echo [6/7] 下次启动 chkdsk 计划...
chkntfs C: 2&gt;nul
echo.
```

```
rem === 7. 系统还原点检查 ===

echo [7/7] 查看系统还原点...

vssadmin list shadows 2&gt;nul | findstr /i "创建时间 Original" | head -10

echo.

echo === 系统维护完成 ===

echo 完成时间: %date% %time%

echo 建议重启电脑

pause
```

第 12 章大代码块 2：Windows 事件日志采集器

```
@echo off

setlocal enabledelayedexpansion

echo ============================================

echo  第12章进阶：Windows 事件日志采集器

echo  从 Event Log XML 提取关键错误

echo ============================================

echo.

set "OUT=%~dp0event_log_report.txt"

echo === 正在采集系统日志 ===

echo.

echo [1/5] System 日志 Error 级别（最近 24 小时）...

wevtutil qe System /q:"*[System[Level=2 and TimeCreated[timediff(@SystemTime) &lt;= 86400000]]]" /c 20 /f text &gt; "%temp%\sys_err.txt" 2&gt;nul

echo  结果数: &amp; type "%temp%\sys_err.txt" 2&gt;nul | find /c ":"

echo.

echo [2/5] Application 日志 Error ...

wevtutil qe Application /q:"*[System[Level=2 and TimeCreated[timediff(@SystemTime) &lt;= 86400000]]]" /c 20 /f text &gt; "%temp%\app_err.txt" 2&gt;nul

echo  结果数: &amp; type "%temp%\app_err.txt" 2&gt;nul | find /c ":"

echo.

echo [3/5] Security 日志（登录/失败）...
```

```
wevtutil qe Security /q:"*[System[(EventID=4624 or EventID=4625) and TimeCreated[timediff(@SystemTime) &lt;= 86400000]]]" /c 20 /f text &gt; "%temp%\sec_logon.txt" 2&gt;nul
echo   结果数: &amp; type "%temp%\sec_logon.txt" 2&gt;nul | find /c ":"
echo.
echo [4/5] Setup 日志（安装/升级）...
wevtutil qe Setup /c 10 /f text &gt; "%temp%\setup.txt" 2&gt;nul
echo   已采集
echo.
echo [5/5] 将所有日志合并到报告...
(
    echo ============================================
    echo  Windows 事件日志汇总报告
    echo  生成时间: %date% %time%
    echo  计算机: %computername%
    echo ============================================
    echo.
    echo --- System 错误 ---
    type "%temp%\sys_err.txt" 2&gt;nul
    echo.
    echo --- Application 错误 ---
    type "%temp%\app_err.txt" 2&gt;nul
    echo.
    echo --- Security 登录事件 ---
    type "%temp%\sec_logon.txt" 2&gt;nul
    echo.
    echo --- Setup 最近活动 ---
    type "%temp%\setup.txt" 2&gt;nul
) &gt; "!OUT!"
echo.
echo === 报告生成完毕 ===
echo 位置: !OUT!
for %%f in ("!OUT!") do echo 大小: %%~zf 字节
```

```
echo.
echo --- 前 30 行预览 ---
type "!OUT!" | head -30
pause
```

第 12 章进阶项目：自动化计划任务框架

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第12章项目：自动化计划任务框架
echo   schtasks 创建/查询/管理计划任务
echo ============================================
echo.
echo [1] 创建每天凌晨 3:00 自动清理
echo [2] 创建每周一 9:00 系统体检
echo [3] 查看所有计划任务
echo [4] 查询特定任务详情
echo [5] 禁用任务
echo [6] 删除任务
set /p "mode=选择 [1-6]: "
echo.
if "!mode!"=="1" goto create_daily
if "!mode!"=="2" goto create_weekly
if "!mode!"=="3" goto list_all
if "!mode!"=="4" goto query
if "!mode!"=="5" goto disable
if "!mode!"=="6" goto delete
goto end
:create_daily
schtasks /create /tn "DailyCleanup" /tr "cmd /c cleanmgr /sagerun:1" /sc daily /st 03:00 /ru SYSTEM
```

```
echo.
goto query
:create_weekly
schtasks /create /tn "WeeklyCheckup" /tr "cmd /c my_checkup.bat" /sc weekly /d MON /st 09:00 /rl HIGHEST
echo.
goto query
:list_all
schtasks /query /fo list /v 2&gt;nul
goto end
:query
set /p "tname=任务名: "
schtasks /query /tn "!tname!" /v /fo list 2&gt;nul
goto end
:disable
set /p "tname=要禁用的任务名: "
schtasks /change /tn "!tname!" /disable
goto end
:delete
set /p "tname=要删除的任务名: "
schtasks /delete /tn "!tname!" /f
goto end
:end
pause
```

进阶项目12：自动化桌面清理

这个桌面清理工具把"桌面/下载目录杂乱"问题系统化解决：8 大文件分类（文档/图片/压缩包/代码/视频/音频/安装包/数据）、自动按日期归档、完整日志记录、预览模式（/dry 参数）、异常保护（跳过 .lnk/.bat 快捷方式和脚本）。配合 schtasks
每周运行一次，桌面永远干净。

```
@echo off
```

```
setlocal enabledelayedexpansion
title 自动化桌面清理工具 v2.0
color 0A

rem ============================================================
rem  自动化桌面清理工具
rem  按文件类型/大小/日期移动到归档目录
rem  带日志记录 + 操作预览 + 异常保护
rem ============================================================

rem ========== 配置区 ==========
set "DESKTOP=%USERPROFILE%\Desktop"
set "DOCS=%USERPROFILE%\Documents"
set "DOWNLOADS=%USERPROFILE%\Downloads"

set "ARCHIVE_ROOT=%USERPROFILE%\Archive"
set "LOG_FILE=%~dp0clean_log_%date:~0,4%%date:~5,2%%date:~8,2%.txt"

rem 按扩展名分类
set "CATEGORY_DOCS=.doc .docx .pdf .txt .rtf .xls .xlsx .ppt .pptx"
set "CATEGORY_IMAGES=.jpg .jpeg .png .gif .bmp .webp .svg .ico"
set "CATEGORY_ARCHIVES=.zip .rar .7z .tar .gz .iso"
set "CATEGORY_CODE=.bat .cmd .ps1 .py .js .ts .java .c .cpp .cs .go .rs .html .css"
set "CATEGORY_VIDEO=.mp4 .mkv .avi .mov .wmv .flv .webm"
set "CATEGORY_AUDIO=.mp3 .wav .flac .aac .ogg .m4a"
set "CATEGORY_INSTALLERS=.exe .msi .dmg .deb .rpm"
set "CATEGORY_DATA=.csv .sql .json .xml .db .sqlite"

set /a MIN_SIZE=1024
set /a MAX_SIZE=104857600
set /a DRY_RUN=0

rem ========== 参数解析 ==========
if /i "%~1"=="/dry" set /a DRY_RUN=1 &amp;amp; echo 预览模式：只显示不移动
if /i "%~1"=="/desktop" set /a CLEAN_DESKTOP=1
if /i "%~1"=="/downloads" set /a CLEAN_DL=1
```

```
if /i "%~1"=="/docs" set /a CLEAN_DOCS=1
if not defined CLEAN_DESKTOP if not defined CLEAN_DL if not defined CLEAN_DOCS (
    set /a CLEAN_DESKTOP=1
    set /a CLEAN_DL=1
    set /a CLEAN_DOCS=0
)

rem ========== 初始化 ==========
if not exist "!ARCHIVE_ROOT!" mkdir "!ARCHIVE_ROOT!" 2&gt;nul
echo [%date% %time%] ============== 清理开始 ============== &gt;&gt; "!LOG_FILE!"

echo.
echo ================================================
echo        自动化桌面清理工具 v2.0
echo ================================================
echo 归档根目录: !ARCHIVE_ROOT!
echo 日志文件:   !LOG_FILE!
echo 预览模式:   !DRY_RUN!
echo 最小大小:   !MIN_SIZE! KB
echo 最大大小:   !MAX_SIZE! 字节
echo.
echo 配置的分类: 8 类
echo   - 文档 / 图片 / 压缩包 / 代码
echo   - 视频 / 音频 / 安装包 / 数据
echo.

set /a total_moved=0
set /a total_files=0
set /a total_errors=0

rem ========== 获取日期字符串 ==========
set "DATE_STR=%date:~0,4%-%date:~5,2%-%date:~8,2%"

rem ========== 清理 Desktop ==========
if !CLEAN_DESKTOP! equ 1 call :CleanDir "!DESKTOP!" "Desktop"
```

```
rem ========== 清理 Downloads ==========
if !CLEAN_DL! equ 1 call :CleanDir "!DOWNLOADS!" "Downloads"
rem ========== 清理 Documents ==========
if !CLEAN_DOCS! equ 1 call :CleanDir "!DOCS!" "Documents"

rem ========== 输出汇总 ==========
echo.
echo ================================================
echo            清理完成
echo ================================================
echo  扫描文件:   !total_files!
echo  已归档:     !total_moved!
echo  错误:       !total_errors!
echo  归档目录:   !ARCHIVE_ROOT!
echo  日志:       !LOG_FILE!
echo ================================================

echo [%date% %time%] 清理结束: 扫描 !total_files! 归档 !total_moved! &gt;&gt; "!LOG_FILE!"

choice /C YN /M "打开归档目录"
if errorlevel 1 start "" explorer "!ARCHIVE_ROOT!"
endlocal
pause
exit /b

:CleanDir
set "_src=%~1"
set "_label=%~2"
echo.
echo --- 清理 !_label! ---
if not exist "!_src!" (
    echo   目录不存在，跳过
    exit /b
)
```

```
for %%f in ("!_src!\*.*") do (
    set /a total_files+=1
    set "_fname=%%~nxf"
    set "_ext=%%~xf"
    set /a "_size=%%~zf/1024" 2&gt;nul
    if "!_size!"=="" set /a _size=0

    rem 跳过 BAT 脚本和快捷方式
    if /i "!_ext!"==".bat" goto :skip_file
    if /i "!_ext!"==".cmd" goto :skip_file
    if /i "!_ext!"==".lnk" goto :skip_file

    rem 空文件跳过
    if !_size! equ 0 goto :skip_file

    rem 确定分类
    set "_cat=Other"
    call :MatchCategory "!_ext!" _cat

    rem 归档目录
    set "_dest=!ARCHIVE_ROOT!\!_cat!\!DATE_STR!"
    if not exist "!_dest!" mkdir "!_dest!" 2&gt;nul

    rem 移动文件
    if !DRY_RUN! equ 1 (
        echo   [预览] !_fname! -^&gt; !_cat!
    ) else (
        move /y "%%f" "!_dest!\!_fname!" &gt;nul 2&gt;&amp;1
        if !errorlevel! equ 0 (
            echo   [OK] !_fname! -^&gt; !_cat!
            set /a total_moved+=1
            echo [%date% %time%] 移动 !_fname! 到 !_dest! &gt;&gt; "!LOG_FILE!"
        ) else (
            echo   [!!] !_fname! 移动失败
            set /a total_errors+=1
```

```
            echo [%date% %time%] 失败 !_fname! &gt;&gt; "!LOG_FILE!"
        )
    )
:skip_file
)
exit /b

:MatchCategory
rem %1=扩展名 %2=out_var
set "_e=%~1"
set "_result=Other"
for %%x in (!CATEGORY_DOCS!) do if /i "%%x"=="!_e!" set "_result=Documents"
for %%x in (!CATEGORY_IMAGES!) do if /i "%%x"=="!_e!" set "_result=Images"
for %%x in (!CATEGORY_ARCHIVES!) do if /i "%%x"=="!_e!" set "_result=Archives"
for %%x in (!CATEGORY_CODE!) do if /i "%%x"=="!_e!" set "_result=Code"
for %%x in (!CATEGORY_VIDEO!) do if /i "%%x"=="!_e!" set "_result=Video"
for %%x in (!CATEGORY_AUDIO!) do if /i "%%x"=="!_e!" set "_result=Audio"
for %%x in (!CATEGORY_INSTALLERS!) do if /i "%%x"=="!_e!" set "_result=Installers"
for %%x in (!CATEGORY_DATA!) do if /i "%%x"=="!_e!" set "_result=Data"
endlocal &amp;amp; set "%~2=%_result%"
exit /b
```

25 类自动清理场景与方案

清理场景	来源目录	归档目录结构	分类依据	时间策略	大小过滤	冲突处理	日志方式	定时任务	安全考虑	工具推荐
临时文件清理	%TEMP%	按扩展名	*.tmp,*.bak	立即删	无	直接删除	>> cleanup.log	schtasks 每天	排除系统目录	cleanmgr
桌面整理	Desktop	Category/YYYY-MM-DD	扩展名 8 类	每次清理	>1KB 过滤	move /y 覆盖	>> logfile	手动/定时	保留 .lnk .bat	本脚本

下载整理	Downloads	Category/YYYY-MM-DD	扩展名 8 类	每周清理	>1KB 过滤	重名加时间戳	>> logfile	schtasks 每周	排除正在下载	本脚本+文件锁
浏览器缓存	Chrome/Edge cache	直接删	/CACHE	每周	无	rmdir /s /q	>> logfile	每周	先关闭浏览器	rd /s /q
回收站清空	$Recycle.Bin	直接删	所有内容	每周	无	rd /s /q C:\$Recycle.Bin	>> logfile	每周	管理员权限	rd /s
Windows 更新缓存	SoftwareDistribution	直接删	*.cab,*.msu	每月	无	停 wuauserv 再删	>> logfile	每月	重启服务	netstop wuauserv
旧日志归档	C:\Logs*	YYYY\MM\	按日期	每月滚动	>10 MB	move /y	>> archive.log	每月	压缩归档	forfiles + zip
构建产物清理	build/dist/bin	直接删	特定目录名	每次构建后	无	rd /s /q	>> build.log	构建脚本	别删源码	rd /s
用户 Dump 文件	LocalDumps	直接删	*.dmp	每月	>100MB	del /f	>> logfile	每月	确认不是活跃	del *.dmp
老的安装包	Downloads\旧	Archives/Installers/年月	*.exe/*.msi	每月	>50 MB	move	>> logfile	每月	保留最近版本	本脚本
重复文件去重	多目录扫描	相同哈希保留新	SHA256	每月	>1KB	certutil -hashfile	>> dup.log	每月	先预览	PowerShell 哈希
空文件夹清理	全硬盘扫描	直接删	0 子项	每月	无	dir /s /ad /b	>> empty.log	每月	排除系统路径	for /f + rd
快捷方式修复	桌面/开始菜单	扫描 .lnk	WScript.Shell COM	每周	无	检查 TargetPath	>> broken.log	每周	仅报告不删	PowerShell COM
大文件报告	用户目录	CSV 输出	>100MB	每周	>100MB	dir /s /c /b	>> bigfiles.csv	每周	只读不删	dir /s
旧压缩包	Archives 目录	按月	压缩包年龄	每月	>1GB	压缩归档	>> logfile	每月	提前通知	本脚本

回收站清空前预览	回收站内	列出所有项目	类型/大小	每周	无	powershell Get-ChildItem	>> preview.csv	每周	人工确认	PowerShell
临时文件通用	*temp*	按项目归类	目录名含 temp	每周	无	move	>> logfile	每周	排除系统 temp	本脚本修改
图片归档	Desktop\Images	YYYY-MM-DD	.jpg/.png	每月	>1KB	按 EXIF 日期归档	>> logfile	每月	需要 EXIF 解析	PowerShell EXIF
视频缩略图	Media	直接删	Thumbs.db	每周	无	del /a /s /q Thumbs.db	>> logfile	每周	关掉缩略图缓存	del Thumbs.db
系统错误报告	ProgramData\Microsoft\WER	直接删	WER 文件夹	每月	无	rd /s /q	>> logfile	每月	管理员	rd /s WER
ActiveX 缓存	Downloaded Program Files	直接删	ActiveX 组件	每季	无	rd /s /q	>> logfile	每季	可能需要	rd /s
Font 缓存	C:\Windows\System32\FNTCACHE.DAT	重建	FNTCACHE.DAT	半年	无	停 FontCache 服务再删	>> logfile	半年	先备份	netstop FontCache
预读取文件	Prefetch	直接删	*.pf	每周	无	del /f /q	>> logfile	每周	性能影响小	del *.pf
浏览器历史	Chrome History	直接删	History 数据库	每周	无	需先关闭浏览器	>> logfile	每周	Chrome 运行时锁文件	taskkill + del
Docker 清理	images/containers	prune	悬空镜像	每月	无	docker system prune -a	>> logfile	每月	影响开发	docker prune

清理脚本最重要的安全原则是"可预见+可恢复"：永远提供 /dry 预览模式先让用户看会移动什么；归档而不是直接删除（move 到 Archive 目录保留 30 天）；日志每一条操作带时间戳，出问题可以回溯。生产环境清理脚本如果搞成"直接 rm -rf"，出一次事故就够你吃一年。

定时任务配置：schtasks /create /tn "DesktopCleaner" /tr "clean.bat /dry" /sc weekly /d MON /st 09:00 /ru %USERNAME% /it。/it 参数让任务在用户登录时才触发；/ru

确保用当前用户权限而不是 SYSTEM（SYSTEM 看不到用户桌面）；每周一早 9 点自动运行预览模式，每周五手动确认后用非 /dry 正式清理。

第13章 实战项目二：网络运维工具集

13.1 批量 Ping 诊断 + CSV 导出

```
@echo off
chcp 65001 &gt; nul
setlocal enabledelayedexpansion
rem ========== 批量 Ping 检测 ==========

rem 创建 targets.txt，每行一个 IP 或域名
if not exist targets.txt (
    echo 请先创建 targets.txt 文件，每行一个 IP 或域名。
    echo 示例:
    echo 192.168.1.1
    echo 192.168.1.100
    echo www.bing.com
    pause
    exit /b 1
)

set "RESULT=ping_report_%date:~0,4%%date:~5,2%%date:~8,2%.csv"
echo IP,响应时间(ms),状态,丢包率,平均时间 &gt; "%RESULT%"

set /a total=0, ok=0, fail=0

for /f "usebackq tokens=*" %%ip in ("targets.txt") do (
    if not "%%ip"=="" (
        set /a total+=1
        &lt;nul set /p "=检测 %%ip ... "
```

```
        rem ping 4次，提取关键数据
        set "lost=0"
        set "avgtime=-"
        set "ok_flag=FAIL"

        for /f "tokens=5" %%t in ('ping -n 4 -w 500 %%ip ^| findstr "平均"') do (
            set "avgtime=%%t"
            set "avgtime=!avgtime: =!"
            set "ok_flag=OK"
        )
        for /f "tokens=2 delims=(" %%l in ('ping -n 4 -w 500 %%ip ^| findstr "丢包" ^| findstr /v "0%"') do (
            set "lost=%%l"
            set "lost=!lost:%%=!"
        )

        if "!ok_flag!"=="OK" set /a ok+=1
        if "!ok_flag!"=="FAIL" set /a fail+=1

        echo %%ip,!avgtime!,!ok_flag!,!lost!,!avgtime! &gt;&gt; "%RESULT%"
        echo !ok_flag! - !avgtime!ms
    )
)

echo.
echo ========================================
echo 检测完成：共 !total! 个，成功 !ok! 个，失败 !fail! 个
echo 详细结果: %RESULT%
echo ========================================
pause
endlocal
```

13.2 批量远程执行（PsExec + BAT）

BAT 不能直接远程执行，但可以结合 Sysinternals 的 PsExec（微软官方工具）实现：

```
@echo off
chcp 65001 &gt; nul
setlocal enabledelayedexpansion

rem ========== 批量远程执行 ==========
rem 前提：PsExec.exe 在 PATH 中，所有远程机器开放 445 端口（SMB）
rem      目标机器的 ADMIN$ 共享可访问

rem 创建 machines.txt，每行一个机器名或 IP
if not exist machines.txt (
    echo 请创建 machines.txt，每行一个主机名或 IP。
    pause
    exit /b 1
)

set "REMOTE_CMD=ipconfig /all"
set "OUT_DIR=remote_results"
mkdir "%OUT_DIR%" 2&gt;nul

set /p "user=管理员用户名: "
set /p "pass=管理员密码: "

for /f "usebackq tokens=*" %%m in ("machines.txt") do (
    if not "%%m"=="" (
        echo 正在连接 %%m ...
        psexec \\%%m -u %user% -p %pass% -accepteula cmd /c "%REMOTE_CMD%" &gt; "%OUT_DIR%\%%m.txt" 2&gt;&amp;1
        if errorlevel 1 (
            echo       连接失败
        ) else (
            echo       成功
        )
    )
)

echo.
echo 所有结果已保存到 %OUT_DIR%
```

```
pause
endlocal
```

13.3 批量文件分发

```
@echo off
chcp 65001 &gt; nul
setlocal enabledelayedexpansion

rem ========== 批量分发文件到多台机器 ==========
set "SOURCE_DIR=D:\Deployment\Files"
set "MACHINES=servers.txt"   &amp; rem 每行一个主机名
set "DEST_PATH=C:\Deploy"   &amp; rem 远程路径

if not exist "%MACHINES%" (
    echo 请创建 %MACHINES% 文件。
    pause
    exit /b 1
)

for /f "usebackq tokens=*" %%m in ("%MACHINES%") do (
    if not "%%m"=="" (
        echo 正在同步到 %%m ...
        rem 先建立临时网络盘映射
        net use Z: \\%%m\ADMIN$ &gt;nul 2&gt;&amp;1

        rem 用 robocopy 同步
        robocopy "%SOURCE_DIR%" "\\%%m\ADMIN$\Deploy" /MIR /Z /R:2 /W:1 &gt; "%temp%\robocopy_%%m.log" 2&gt;&amp;1

        if !errorlevel! lss 8 (
            echo       同步成功
        ) else (
            echo       同步失败，详见 %temp%\robocopy_%%m.log
        )

        net use Z: /delete &gt;nul 2&gt;&amp;1
```

```
    )
)

rem 推送完成后触发远程安装
for /f "usebackq tokens=*" %%m in ("%MACHINES%") do (
    echo 通知 %%m 执行安装...
    schtasks /run /s %%m /tn "DeployInstall" 2&gt;nul
)

pause
endlocal
```

13.4 网络监控脚本

```
@echo off
chcp 65001 &gt; nul
setlocal enabledelayedexpansion

rem ========== 网络端口监控 ==========
rem 定时检查是否有异常端口被监听，记录并报警

set "LOG=monitor_%date:~0,4%%date:~5,2%%date:~8,2%.log"
set "INTERVAL=60"  &amp; rem 每 60 秒检查一次

rem 定义"危险端口"（已知木马/后门常用端口）
set "DANGER_PORTS=1234 31337 12345 27374 8888 9999 6666 7777"

echo [%time%] 监控启动，间隔 %INTERVAL% 秒 &gt;&gt; "%LOG%"

:monitor_loop
set "snapshot=%temp%\netstat_snapshot.txt"
netstat -ano -p tcp &gt; "%snapshot%" 2&gt;&amp;1

for %%p in (%DANGER_PORTS%) do (
    findstr "%%p" "%snapshot%" &gt;nul 2&gt;&amp;1
    if !errorlevel! equ 0 (
        echo [!time%]    检测到危险端口 %%p 被监听！  &gt;&gt; "%LOG%"
```

```
        findstr "%%p" "%snapshot%" &gt;&gt; "%LOG%"

        rem 查找对应 PID 和进程名
        for /f "tokens=5" %%pid in ('findstr "%%p" "%snapshot%"') do (
            for /f "usebackq tokens=1" %%name in ('tasklist /fi "PID eq %%pid" /nh') do (
                echo [!time%]  进程: %%name ^(PID: %%pid^) &gt;&gt; "%LOG%"
            )
            rem 可选：自动结束可疑进程
            rem taskkill /f /pid %%pid
        )
    )
)

echo [!time%] 检查完成 &gt;&gt; "%LOG%"
timeout /t %INTERVAL% /nobreak &gt; nul
goto :monitor_loop
```

13.5 WiFi 密码批量导出

```
@echo off
chcp 65001 &gt; nul
setlocal enabledelayedexpansion

rem ========== 导出所有 WiFi 密码 ==========
set "OUT=wifi_passwords.txt"
echo ===== WiFi 密码导出 ===== &gt; "%OUT%"
echo 导出时间: %date% %time% &gt;&gt; "%OUT%"
echo. &gt;&gt; "%OUT%"

rem 获取所有已保存的 WiFi 名称
for /f "tokens=2 delims=:" %%n in ('netsh wlan show profiles ^| findstr "所有用户配置文件" ^| findstr /v "---"') do (
    set "profile=%%n"
    set "profile=!profile: =!"

    echo ======================================== &gt;&gt; "%OUT%"
```

```
    echo 网络名: !profile! &gt;&gt; "%OUT%"
    echo ======================================== &gt;&gt; "%OUT%"
    rem 导出详细信息（包括 key=clear 显示明文密码）
    netsh wlan show profile name="!profile!" key=clear &gt;&gt; "%OUT%" 2&gt;&amp;1
    echo. &gt;&gt; "%OUT%"
)
echo 导出完成!
echo 结果已保存到: %cd%\%OUT%
start "" "%cd%\%OUT%"
pause
endlocal
```

— 第13章 完 —

进阶实战：完整网络运维工具箱。

```
@echo off
title BAT 网络运维工具箱
:MENU
cls
echo.
echo   [1] IP配置  [2] 批量ping  [3] 端口扫描
echo   [4] 局域网发现  [5] WOL唤醒  [0] 退出
set /p choice=选择:
if "%choice%"=="1" goto :IP
if "%choice%"=="2" goto :PING
if "%choice%"=="3" goto :SCAN
if "%choice%"=="4" goto :LAN
if "%choice%"=="5" goto :WOL
if "%choice%"=="0" exit /b
goto :MENU
:IP
netsh interface ip show config
pause &amp; goto :MENU
:PING
set /p target=目标IP:
for /L %%i in (1,1,4) do (
    echo|set /p "=ping %%i: "
    ping -n 1 -w 500 %target% &gt;nul 2&gt;nul &amp;&amp; echo OK || echo FAIL
)
pause &amp; goto :MENU
:SCAN
set /p target=目标IP:
for %%p in (22 25 80 443 3389 8080) do (
    echo|set /p "=%%p: "
```

```
    powershell -Command "(Test-NetConnection -ComputerName %target% -Port %%p -WarningAction SilentlyContinue)
.TcpTestSucceeded" 2&gt;nul

)

pause &amp;amp; goto :MENU

:LAN

echo 扫描局域网...

for /L %%i in (1,1,254) do start /b cmd /c "ping -n 1 -w 200 192.168.1.%%i | findstr TTL &gt;&gt; hosts.txt"

timeout /t 30 &gt;nul

type hosts.txt 2&gt;nul &amp;amp; del hosts.txt 2&gt;nul

pause &amp;amp; goto :MENU

:WOL

set /p mac=MAC(AA-BB-CC-DD-EE-FF):

echo 发送魔法包到 %mac%

powershell -Command "$bytes = $mac.Split('-') | ForEach-Object {[Convert]::ToByte($_,16)}; $h=@(0xFF)*6; $p=$h + (
$bytes*16); $u=New-Object Net.Sockets.UdpClient; $u.Connect('255.255.255.255',9); $u.Client.SetSocketOption('Socket',
'Broadcast',$true); $u.Send($p,$p.Length); $u.Close()"

pause &amp;amp; goto :MENU
```

进阶主题1：批量 ping + 写 hosts

批量 ping 用 start /b 并发比 for 循环快几十倍——每个 ping 是独立 cmd 进程。写 hosts 是做静态 DNS 解析的快速方案，特别适合屏蔽广告域名或加速内网访问。ipconfig /flushdns 必须刷新缓存才生效。

```
@echo off

setlocal enabledelayedexpansion

echo === 批量 ping + 写 hosts ===

echo.

echo --- 1. 批量 ping 网段 ---

set "subnet=192.168.1"

echo 扫描 %subnet%.1 - %subnet%.254 ...

set "alive=0"

set "dead=0"

for /L %%i in (1,1,254) do (

    ping -n 1 -w 300 %subnet%.%%i &gt;nul 2&gt;nul
```

```
    if !errorlevel! equ 0 (
        echo [UP]   %subnet%.%%i
        set /a alive+=1
    ) else (
        set /a dead+=1
    )
)
echo.
echo 存活: !alive! 不可达: !dead!
echo.
echo --- 2. 更高效: 后台并发 ---
echo 用 start /b 同时 ping:
for /L %%i in (1,1,20) do start /b cmd /c "ping -n 1 -w 200 %subnet%.%%i ^| findstr TTL &gt;&gt; alive.txt"
echo 等待扫描...
timeout /t 5 &gt;nul
echo 在线主机:
type alive.txt 2&gt;nul
del alive.txt 2&gt;nul
echo.
echo --- 3. 写 hosts 文件 ---
echo hosts 文件位置: %SystemRoot%\System32\drivers\etc\hosts
echo 添加条目 (管理员权限):
echo 127.0.0.1 example.com ^&gt;^&gt; %SystemRoot%\System32\drivers\etc\hosts
echo.
echo 备份 hosts:
copy "%SystemRoot%\System32\drivers\etc\hosts" hosts_backup.txt
echo.
echo --- 4. 快速编辑 hosts ---
echo notepad %SystemRoot%\System32\drivers\etc\hosts
echo 或用管理员权限:
echo start "" notepad %SystemRoot%\System32\drivers\etc\hosts
echo.
```

```
echo --- 5. 刷新 DNS 缓存 ---
ipconfig /flushdns
pause
```

进阶主题2：端口转发（netsh interface portproxy）

portproxy 把一台机器当网络网关。最经典的场景：在家庭电脑上把 localhost:8080 转发到公司内网的 10.0.0.5:80。规则持久化到注册表，重启不丢。别忘了放防火墙。

```
@echo off
echo === 端口转发 (netsh interface portproxy) ===
echo.
echo --- 1. 基础用法 ---
echo 把本机 8080 转发到内部服务器 80 端口:
echo netsh interface portproxy add v4tov4 listenport=8080 listenaddress=0.0.0.0 connectport=80 connectaddress=192.168.1.100
echo.
echo --- 2. 查看当前转发规则 ---
netsh interface portproxy show all
echo.
echo --- 3. 删除规则 ---
echo netsh interface portproxy delete v4tov4 listenport=8080 listenaddress=0.0.0.0
echo 删除所有:
echo netsh interface portproxy delete v4tov4
echo.
echo --- 4. 实际场景 ---
echo 访问内网 Web 服务 (公司-家庭):
echo netsh interface portproxy add v4tov4 listenport=8080 connectport=80 connectaddress=10.0.0.5
echo 访问内网数据库:
echo netsh interface portproxy add v4tov4 listenport=3307 connectport=3306 connectaddress=10.0.0.10
echo.
echo --- 5. 检查转发是否生效 ---
echo netstat -ano | findstr LISTENING | findstr 8080
echo.
echo --- 6. 持久化 ---
```

```
echo 转发规则重启后保留 (存在注册表)
echo HKLM\SYSTEM\CurrentControlSet\Services\PortProxy\v4tov4\tcp
echo.
echo --- 7. 防火墙 ---
echo 需要放行监听端口:
echo netsh advfirewall firewall add rule name="Port Forward 8080" dir=in action=allow protocol=TCP localport=8080
echo.
echo --- 8. 备选方案 ---
echo 临时转发用 PowerShell:
echo $l=New-Object Net.Sockets.TcpListener(8080); ...
echo SSH 隧道也可以: ssh -L 8080:192.168.1.100:80 user@server
pause
```

进阶主题3：用 netsh dump 备份网络配置

netsh dump 是一键备份所有网络配置的神器。包含接口、IP、路由、防火墙、代理等所有设置。恢复用 netsh -f file.txt。建议每次改网络配置前先 dump 一份。

```
@echo off
echo === 用 netsh dump 备份网络配置 ===
echo.
echo --- 1. netsh dump ---
echo 完整网络配置 dump:
netsh dump &gt; network_backup.txt
echo 已保存到 network_backup.txt
echo 内容预览:
type network_backup.txt
echo.
echo --- 2. 恢复网络配置 ---
echo netsh -f network_backup.txt
echo.
echo --- 3. 导出特定接口配置 ---
echo 导出 IP 配置:
netsh interface ip dump &gt; ip_config.txt
echo 导出接口配置:
```

```
netsh interface dump &gt; interface_config.txt
echo 导出 HTTP 代理:
netsh winhttp dump &gt; proxy_config.txt
echo.
echo --- 4. 查看当前配置 ---
echo 所有接口:
netsh interface show interface
echo.
echo IP 地址:
netsh interface ip show config
echo.
echo 路由表:
netsh interface ipv4 show routes
echo.
echo DNS 服务器:
netsh interface ipv4 show dnsservers
echo.
echo --- 5. 一键备份脚本 ---
(
    @echo off
    set "out=net_backup_%date:~0,4%%date:~5,2%%date:~8,2%"
    mkdir "%out%"
    netsh dump &gt; "%out%\full.txt"
    netsh interface ip dump &gt; "%out%\ip.txt"
    netsh interface dump &gt; "%out%\interface.txt"
    netsh advfirewall dump &gt; "%out%\firewall.txt"
    echo 网络配置备份完成到 %out%
) &gt; backup_network.bat
echo 已生成 backup_network.bat
echo.
del network_backup.txt ip_config.txt interface_config.txt proxy_config.txt 2&gt;nul
pause
```

表 13-1 端口号服务对照

端口	服务	协议	用途	风险等级	典型攻击
20	FTP 数据	TCP	文件传输数据通道	高	暴力破解/嗅探
21	FTP 控制	TCP	文件传输控制	高	暴力破解/目录遍历
22	SSH	TCP	安全远程登录	中	暴力破解（较难）
23	Telnet	TCP	明文远程登录	极高	嗅探/完全接管
25	SMTP	TCP	邮件发送	中	垃圾邮件中继
53	DNS	TCP/UDP	域名解析	低	DNS 劫持/放大攻击
67/68	DHCP	UDP	IP 地址分配	中	DHCP 欺骗
80	HTTP	TCP	Web 明文	中	SQLi/XSS/DDoS
110	POP3	TCP	邮件接收	中	嗅探/暴力破解
135	RPC	TCP	远程过程调用	高	MSRPC 漏洞利用
137	NetBIOS Name	UDP	名字解析	高	名称碰撞
138	NetBIOS Datagram	UDP	数据报	高	嗅探
139	NetBIOS Session	TCP	会话	高	永恒之蓝
143	IMAP	TCP	邮件接收	中	暴力破解
443	HTTPS	TCP	加密 Web	低	证书欺诈
445	SMB	TCP	文件共享	极高	永恒之蓝/勒索
993	IMAPS	TCP	加密 IMAP	低	证书攻击
995	POP3S	TCP	加密 POP3	低	证书攻击
1433	MSSQL	TCP	SQL Server	高	暴力破解/SQL 注入
1521	Oracle	TCP	Oracle DB	中	暴力破解
3306	MySQL	TCP	MySQL DB	高	暴力破解/SQL 注入
3389	RDP	TCP	远程桌面	极高	暴力破解/BlueKeep
5432	PostgreSQL	TCP	PostgreSQL	中	暴力破解
5672	AMQP	TCP	RabbitMQ	中	未授权访问
6379	Redis	TCP	Redis 缓存	极高	未授权远程代码执行
8080	HTTP 代理	TCP	代理/备用 Web	中	扫描探测
8443	HTTPS 备用	TCP	备用 HTTPS	低	—
9200	Elasticsearch	TCP	ES 数据	极高	未授权访问/删除
27017	MongoDB	TCP	MongoDB	极高	未授权访问/勒索

50000+	动态	TCP	自定义/木马	极高	后门通信

表 13-2 WLAN 命令速查

命令	功能	关键参数	说明
netsh wlan show interfaces	当前连接详情	—	SSID/BSSID/强度/加密
netsh wlan show networks	扫描到的网络	mode=bssid	AP 列表
netsh wlan show profiles	已保存配置	—	所有记住的 WiFi
netsh wlan show profile	配置详情	name="SSID" key=clear	key=clear 显密码
netsh wlan connect	连接 WiFi	name="SSID" ssid="SSID"	自动或手动
netsh wlan disconnect	断开连接	—	立即断开
netsh wlan delete profile	删除配置	name="SSID"	清除保存
netsh wlan add profile	添加配置	filename=profile.xml	从 XML 文件导入
netsh wlan export profile	导出配置	folder=. key=clear	含明文密码!
netsh wlan scan	重新扫描	—	手动触发扫描
netsh wlan set hostednetwork	热点管理	mode=allow ssid= key=	创建虚拟热点
netsh wlan show drivers	驱动信息	—	看网卡是否支持 monitor
netsh wlan set autoconfig	自动连接开关	enabled=yes/no interface="Wi-Fi"	自动连接管理
netsh interface show interface	所有网络接口	—	网卡清单
netsh interface set interface	启用/禁用网卡	name="Wi-Fi" admin=enabled	快捷开关
netsh advfirewall	防火墙 Shell	show/add/delete rule	防火墙管理
netsh winsock	Winsock 重置	reset	修复网络异常
netsh int ip reset	TCP/IP 重置	reset	网络重置大招
netsh wfp	Windows Filtering	show state	高级包过滤
netsh trace	网络追踪	start/stop capture	抓包诊断

第 13 章小结

端口号表按安全风险从极高到低排序，永恒之蓝的 445、Redis 的 6379、MongoDB 的 27017、RDP 的 3389 都是"高危四件套"。生产环境务必关闭/限制来源

IP。临时调试放行时要用 netsh advfirewall firewall add rule ... dir=in action=allow。

netsh wlan 是 BAT 操作无线的唯一官方方式。最酷的三个 trick：netsh wlan show profile name="SSID" key=clear 可以明文读 WiFi 密码、netsh wlan export profile folder=. key=clear 批量导出所有 WiFi 配置（含密码！）、netsh wlan set hostednetwork mode=allow ssid= key= 创建移动热点。

端口 -> 进程的映射脚本演示了 BAT 里如何把 netstat 的输出解析成 PID 再用 tasklist 查进程名。关键是 netstat -ano 显示每个连接的 PID，然后 tasklist /fi "pid eq N" 查名字。BAT 里做这个比 GUI 还方便。

网络重置大招：netsh winsock reset + netsh int ip reset + netsh advfirewall reset。这三招专治"网络突然连不上但 ping 127.0.0.1 通"的疑难杂症。执行完要重启。

netsh advfirewall firewall 的常用参数：add rule name= dir=in action=allow protocol=TCP localport=8080。/MIR 不能随便用——它会删除目标目录里所有"多余"文件！

最后一章！BAT 黑科技和安全加固。这是全书最"硬核"的部分，准备好了吗？

第 13 章大代码块 1：网络端口与 WLAN 管理

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo  第13章综合实战：网络端口与 WLAN 管理
echo ============================================
echo.
echo === 一、当前所有监听端口 ===
netstat -ano | findstr LISTENING
echo.
echo === 二、端口号 -&gt; 进程名 映射 ===
for /f "tokens=5" %%p in ('netstat -ano ^| findstr LISTENING ^| sort -u') do (
  if not "%%p"=="0" (
    for /f "tokens=1" %%n in ('tasklist /fi "pid eq %%p" /nh /fo csv 2^&gt;nul ^| findstr /v "INFO"') do (
      echo  PID %%p -^&gt; %%n
    )
```

```
    )
)
echo.
echo === 三、查看当前 WiFi 连接 ===
netsh wlan show interfaces
echo.
echo === 四、已保存的 WiFi 配置文件 ===
netsh wlan show profiles
echo.
echo === 五、WiFi 密码（明文化显示）===
for /f "tokens=2 delims=:" %%n in ('netsh wlan show profiles ^| findstr "    所有用户配置文件"') do (
    set /a idx+=1
    if !idx! leq 5 (
        set "name=%%n"
        set "name=!name:~1!"
        echo --- !name! ---
        netsh wlan show profile name="!name!" key=clear 2&gt;nul | findstr /i "密码 Content"
    )
)
echo.
pause
```

第 13 章大代码块 2：WiFi 连接管理菜单

```
@echo off
setlocal enabledelayedexpansion
echo ============================================
echo   第13章进阶：WiFi 连接管理菜单
echo ============================================
echo.
:menu
```

```
cls
echo  ============================================
echo    WLAN 管理工具
echo  ============================================
echo.
echo    [1] 查看当前连接详情
echo    [2] 列出可用 WiFi
echo    [3] 连接到指定 WiFi
echo    [4] 断开当前连接
echo    [5] 显示所有已保存配置
echo    [6] 导出 WiFi 配置（含密码）
echo    [7] 导出所有配置（批量）
echo    [8] 删除指定配置
echo    [9] 重置 WiFi 适配器
echo    [0] 退出
echo.
set /p "choice=请选择: "
if "!choice!"=="1" goto detail
if "!choice!"=="2" goto scan
if "!choice!"=="3" goto connect
if "!choice!"=="4" goto disconnect
if "!choice!"=="5" goto profiles
if "!choice!"=="6" goto export_one
if "!choice!"=="7" goto export_all
if "!choice!"=="8" goto delete_profile
if "!choice!"=="9" goto reset
if "!choice!"=="0" goto end
goto menu
:detail
netsh wlan show interfaces
pause
goto menu
```

```
:scan
echo 正在扫描 WiFi...
netsh wlan scan
timeout /t 2 &gt;nul
netsh wlan show networks mode=bssid
pause
goto menu
:connect
set /p "ssid=要连接的 SSID: "
echo 正在连接 !ssid! ...
netsh wlan connect name="!ssid!"
pause
goto menu
:disconnect
netsh wlan disconnect
echo 已断开
pause
goto menu
:profiles
netsh wlan show profiles
pause
goto menu
:export_one
set /p "ssid=要导出的 SSID: "
set /p "dir=导出目录: "
netsh wlan export profile name="!ssid!" folder="!dir!" key=clear
echo 已导出
pause
goto menu
:export_all
set /p "dir=导出目录: "
mkdir "!dir!" 2&gt;nul
```

```
netsh wlan export profile folder="!dir!" key=clear
echo 所有配置已导出到 !dir!
pause
goto menu
:delete_profile
set /p "ssid=要删除的 SSID: "
netsh wlan delete profile name="!ssid!"
echo 已删除
pause
goto menu
:reset
echo 正在重置 WLAN 适配器...
netsh wlan stop hostednetwork 2&gt;nul
netsh interface set interface name="Wi-Fi" admin=disabled &gt;nul 2&gt;&amp;1
timeout /t 2 &gt;nul
netsh interface set interface name="Wi-Fi" admin=enabled &gt;nul 2&gt;&amp;1
echo 重置完成
pause
goto menu
:end
exit /b 0
```

第 13 章进阶项目：网络防火墙管理工具

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ============================================
echo   第13章项目：网络防火墙管理工具
echo   netsh advfirewall 封装
echo ============================================
```

```
echo.
echo === 1. 防火墙状态 ===
netsh advfirewall show allprofiles state
echo.
echo === 2. 快速开关 ===
echo [1] 完全开启防火墙
echo [2] 完全关闭防火墙
echo [3] 域网络开启
echo [4] 专用网络开启
echo [5] 公用网络开启
set /p "action=选择 [1-5]: "
echo.
if "!action!"=="1" netsh advfirewall set allprofiles state on
if "!action!"=="2" netsh advfirewall set allprofiles state off
if "!action!"=="3" netsh advfirewall set domainprofile state on
if "!action!"=="4" netsh advfirewall set privateprofile state on
if "!action!"=="5" netsh advfirewall set publicprofile state on
echo.
echo === 3. 快速放行常见端口 ===
echo   [SSH 22] [HTTP 80] [HTTPS 443] [MySQL 3306] [Redis 6379] [全部放行]
set /p "ports=要放行的端口(如 22,80): "
for %%p in (!ports!) do (
    netsh advfirewall firewall add rule name="Allow Port %%p" dir=in action=allow protocol=TCP localport=%%p
    echo   已放行 TCP %%p
)
echo.
echo === 4. 列出所有规则（前 20）===
netsh advfirewall firewall show rule name=all dir=in 2&gt;nul | head -60
echo.
echo === 5. 规则导出/导入 ===
echo   netsh advfirewall export "rules.wfw"
echo   netsh advfirewall import "rules.wfw"
```

```
echo.
pause
```

进阶项目13：远程管理工具集

这个工具箱整合了 BAT 远程管理的三大主流方案：PsExec/PssService/Pskill（Sysinternals）、WinRM PowerShell Remoting、schtasks /s 远程计划任务。菜单化的 9 种操作覆盖了远程执行、改密码、服务管理、杀进程、批量执行、信息收集等最高频场景。附带 WinRM 一键启用脚本——4 步配置让主机接受 PowerShell 远程。

```
@echo off
setlocal enabledelayedexpansion
title 远程管理工具集 - PsExec + WinRM + 计划任务
color 0B

rem ============================================================
rem  远程管理工具集
rem 方法 1: PsExec (Sysinternals, 最成熟)
rem 方法 2: WinRM (Windows 原生 PowerShell Remoting)
rem 方法 3: 远程计划任务 (schtasks /s)
rem ============================================================

:MENU
cls
echo.
echo  ================================================
echo     远程管理工具集 (3 种方法)
echo  ================================================
echo.
echo  [1] PsExec 远程执行        [2] PsPasswd 远程改密码
echo  [3] PsService 远程服务     [4] PsKill 远程杀进程
echo  [5] WinRM PowerShell 远程  [6] schtasks 远程计划任务
echo  [7] 批量执行 (IP 列表)     [8] 远程收集系统信息
```

```
echo   [9] 启用 WinRM (一键配置)  [0] 退出
echo.
set /p "choice=选择: "

if "!choice!"=="0" goto :END
if "!choice!"=="1" call :psexec_remote
if "!choice!"=="2" call :pspasswd_remote
if "!choice!"=="3" call :psservice_remote
if "!choice!"=="4" call :pskill_remote
if "!choice!"=="5" call :winrm_remote
if "!choice!"=="6" call :schtasks_remote
if "!choice!"=="7" call :batch_exec
if "!choice!"=="8" call :collect_info
if "!choice!"=="9" call :enable_winrm
pause
goto :MENU
:END
exit /b

:check_psexec
where psexec &gt;nul 2&gt;&amp;1 &amp;&amp; exit /b 0
echo [提示] 未检测到 PsExec，请到微软官网下载 PsTools
echo       https://learn.microsoft.com/sysinternals/downloads/pstools
echo.
echo 下载解压后把 psexec.exe 放到 C:\Windows\System32
echo 或当前脚本所在目录即可使用。
exit /b 1

:psexec_remote
call :check_psexec || exit /b
set /p target=远程主机(IP/主机名):
set /p user=用户名(域\用户):
set /p pass=密码:
echo.
```

```
echo [1] 执行 ipconfig
echo [2] 执行 systeminfo | findstr /c:"OS"
echo [3] 启动 cmd 交互式会话
set /p act=选择操作:
if "!act!"=="1" psexec /accepteula \!target! -u "!user!" -p "!pass!" ipconfig /all
if "!act!"=="2" psexec /accepteula \!target! -u "!user!" -p "!pass!" systeminfo
if "!act!"=="3" psexec /accepteula -i \!target! -u "!user!" -p "!pass!" cmd.exe
exit /b

:pspasswd_remote
call :check_psexec || exit /b
set /p target=远程主机(IP/主机名):
set /p user=要改密码的用户名:
set /p oldpass=原密码:
set /p newpass=新密码:
pspasswd /accepteula \!target! -u !user! -o !oldpass! -n !newpass!
echo 返回码: !errorlevel!
exit /b

:psservice_remote
call :check_psexec || exit /b
set /p target=远程主机(IP/主机名):
echo 正在列出 !target! 的服务...
psservice /accepteula \!target! query
echo.
set /p svc=服务名(留空跳过):
if not "!svc!"=="" (
    echo [1] start [2] stop [3] restart [4] query
    set /p op=操作:
    if "!op!"=="1" psservice \!target! start !svc!
    if "!op!"=="2" psservice \!target! stop !svc!
    if "!op!"=="3" psservice \!target! restart !svc!
    if "!op!"=="4" psservice \!target! query !svc!
```

```
)
exit /b

:pskill_remote
call :check_psexec || exit /b
set /p target=远程主机(IP/主机名):
echo 远程进程列表:
pslist \!target!
echo.
set /p proc=要杀的进程名/PID:
choice /C YN /M "确认杀掉 !proc! on !target!"
if errorlevel 1 pskill \!target! !proc!
exit /b

:enable_winrm
echo.
echo === WinRM 一键启用 ===
echo 注意：需要管理员权限，且防火墙开放 5985 端口
echo.
net session &gt;nul 2&gt;nul || (
    echo [错误] 请以管理员身份运行本脚本
    exit /b
)
echo [1/4] 启用 WinRM 服务...
winrm quickconfig -q
echo [2/4] 配置 WinRM 监听端口 5985...
winrm set winrm/config/service @{AllowUnencrypted="true"}
echo winrm set winrm/config/service/auth @{Basic="true"}
echo [3/4] 防火墙放行...
netsh advfirewall firewall add rule name="WinRM HTTP" dir=in action=allow protocol=TCP localport=5985
echo [4/4] 设置 WinRM 最大内存...
winrm set winrm/config/winrs @{MaxMemoryPerShellMB="1024"}
echo.
```

```
echo WinRM 启用完成!
echo 测试命令: winrs -r:%COMPUTERNAME% hostname
exit /b

:winrm_remote
echo.
echo [方法1] WinRS 执行单条命令:
set /p target=远程主机:
set /p cmd=要执行的命令:
winrs -r:!target! !cmd!
echo.
echo [方法2] PowerShell 交互式远程:
echo   Enter-PSSession -ComputerName !target!
echo [方法3] 脚本块远程执行:
echo   Invoke-Command -ComputerName !target! -ScriptBlock { hostname; ipconfig }
echo.
echo 正在测试 PowerShell Remoting...
powershell -Command "try { Invoke-Command -ComputerName '!target!' -ScriptBlock { Write-Host '远程主机:'; hostname; Write-Host 'IP:'; (Get-NetIPAddress -AddressFamily IPv4).IPAddress } } catch { Write-Host '连接失败: ' $_.Exception.Message }"
exit /b

:schtasks_remote
set /p target=远程主机:
echo.
echo [1] 查询远程计划任务
echo [2] 创建远程计划任务
echo [3] 触发远程任务
echo [4] 删除远程任务
set /p op=选择:
if "!op!"=="1" schtasks /query /s !target! /fo csv
if "!op!"=="2" (
    set /p tname=任务名:
    set /p tcmd=要运行的命令:
```

```
    set /p trun=运行账户:
    schtasks /create /s !target! /tn !tname! /tr "!tcmd!" /sc DAILY /st 09:00 /ru "!trun!" /rp * /f
)
if "!op!"=="3" (
    set /p tname=任务名:
    schtasks /run /s !target! /tn !tname!
)
if "!op!"=="4" (
    set /p tname=任务名:
    schtasks /delete /s !target! /tn !tname! /f
)
exit /b

:batch_exec
call :check_psexec || exit /b
set /p listfile=IP 列表文件(每行一个):
set /p user=用户名:
set /p pass=密码:
set /p cmd=远程执行命令:
echo.
echo 正在批量执行...
for /f "usebackq tokens=*" %%ip in ("!listfile!") do (
    echo --- 连接 %%ip ---
    psexec /accepteula \%%ip -u "!user!" -p "!pass!" !cmd! 2&gt;&amp;1
)
echo.
echo 批量执行完成
exit /b

:collect_info
call :check_psexec || exit /b
set /p listfile=IP 列表文件:
set /p user=用户名:
```

```
set /p pass=密码:
set "OUT=%~dp0remote_info_%date:~0,4%%date:~5,2%%date:~8,2%"
mkdir "!OUT!" 2&gt;nul
for /f "usebackq tokens=*" %%ip in ("!listfile!") do (
    echo --- 收集 %%ip ---
    psexec /accepteula \%%ip -u "!user!" -p "!pass!" systeminfo &gt; "!OUT!\%%ip_systeminfo.txt" 2&gt;&amp;1
    psexec /accepteula \%%ip -u "!user!" -p "!pass!" wmic cpu get /format:list &gt; "!OUT!\%%ip_cpu.txt" 2&gt;&amp;1
    psexec /accepteula \%%ip -u "!user!" -p "!pass!" wmic os get /format:list &gt; "!OUT!\%%ip_os.txt" 2&gt;&amp;1
    psexec /accepteula \%%ip -u "!user!" -p "!pass!" netstat -ano &gt; "!OUT!\%%ip_net.txt" 2&gt;&amp;1
)
echo.
echo 收集完成，结果在 !OUT!
exit /b
```

25 种远程管理场景工具选型

远程操作	PsExec	WinRM/PS Remoting	schtasks/s	WMI DCOM	Sysinternals 其他	认证方式	端口	防火墙	优点	缺点	适用场景
执行单条命令	psexec \host cmd	Invoke-Command	schtasks /run	wmic /node:	-	Kerberos/NTLM	445	SMB 放行	最简单	需要 SMB 445	临时执行
交互式会话	psexec -i -d	Enter-PSSession	不支持	不支持	-	Kerberos	445/5985	放行	类 RDP 轻量	无 GUI	排查问题
改用户密码	pspasswd	netuser (Invoke-Command)	不支持	不支持	-	管理员凭据	445	SMB	官方工具	明文传密码	批量改密
杀进程	pskill \host pid	Stop-Process (Invoke)	不支持	wmic process call terminate	-	Kerberos	445	SMB	精准	PID 先查	紧急故障

重启/关机	psshutdown	Restart-Computer	schtasks /run	shutdown /m	-	Kerberos	445	SMB	支持通知	无进度	批量重启
查服务状态	psservice query	Get-Service (Invoke)	不支持	wmic service	-	Kerberos	445	SMB	最全面	需 SMB	服务巡检
启停服务	psservice start/stop	Start/Stop-Service	不支持	wmic service call	-	Kerberos	445	SMB	简单	无重试	变更窗口
文件复制	psexec + copy	Copy-Item + SMB	不支持	-	-	SMB	445	SMB	SMB 最快	路径转义	分发软件
批量推送	psexec @list	Invoke-Command @hosts	不支持	-	-	Kerberos	445/5985	放行	列表支持	列表格式	批量部署
收集系统信息	systeminfo 输出	Get-CimInstance	不支持	wmic	-	Kerberos	445/5985	放行	全维度	输出格式	资产盘点
补丁状态	psexec wmic qfe	Get-HotFix (Invoke)	不支持	wmic qfe /node	-	Kerberos	445	SMB	最直接	补丁多慢	合规检查
启动/重启	不支持	Restart-Computer -Force	schtasks /run	shutdown /m	psshutdown	Kerberos	445	SMB	原生支持	需等处理完	批量升级
计划任务创建	schtasks /s	Register-ScheduledTask	schtasks /s	-	-	Kerberos	445	SMB	原生支持	参数复杂	定时维护
注册表操作	reg add /s	Invoke-Command + reg	不支持	reg connect + reg query	-	Kerberos	445	SMB	直接	无 GUI	配置推送
驱动查询	driverquery /s	Get-WindowsDriver	不支持	wmic path Win32_PnPSignedDriver	-	Kerberos	445	SMB	最全	输出冗长	驱动审计

磁盘空间	wmic logicaldisk /node	Get-CimInstance LogicalDisk	不支持	wmic logicaldisk	-	Kerberos	445	SMB	最直接	需脚本处理	空间告警
网络配置	netsh /s	netsh advfirewall	不支持	-	-	Kerberos	445	SMB	netsh 最全	输出不漂亮	改网络
防火墙管理	netsh advfirewall /s	New-NetFirewallRule	不支持	-	-	管理员凭据	445	放行	原生完整	参数多	策略推送
RDP 远程	不支持	不支持	不支持	不支持	mstsc /v:host	RDP 凭据	3389	放行	有 GUI	需 RDP 启用	远程桌面
SSH 远程	不支持	不支持	不支持	不支持	ssh host	SSH key/pwd	22	放行	跨平台	OpenSSH 服务	跨系统
SCP/SFTP 传文件	不支持	不支持	不支持	不支持	scp/sftp	SSH key	22	放行	加密安全	需安装	安全传输
SNMP 监控	不支持	不支持	不支持	不支持	snmpwalk	社区字符串	161	放行	行业标准	MIB 复杂	网络监控
RMAN 备份	不支持	不支持	不支持	不支持	rman	DBA 凭据	1521	放行	数据库专用	需 Oracle	数据库备份
PXE 启动	不支持	不支持	不支持	不支持	wdsutil	域凭据	69/4011	放行	批量部署	需 WDS	裸机安装
IPMI/BMC 管理	不支持	不支持	不支持	不支持	ipmitool	IPMI 账户	623	放行	硬件级	需 BMC IP	带外管理

PsExec 的核心优势是"零配置"——目标主机只要开着 SMB 445 端口和 Admin$ 共享就能远程执行。WinRM 需要预先配置（winrm quickconfig + 防火墙放行 5985），但能同时执行 PowerShell 脚本、返回结构化对象、支持交互式会话。生产环境建议 WinRM，临时救急用 PsExec。

远程管理的安全红线：永远不要把管理员密码硬编码在脚本里；PsExec 密码会在内存和安全事件日志里留下痕迹；WinRM 配置了 AllowUnencrypted=true 后要确保网络可信；PsExec 执行时会在目标系统盘写入 psexesvc.exe 服务（用完 psexec \target -s c 关闭）。

第14章
实战项目三：进阶黑科技工具集

14.1 文件加密解密

BAT 没有原生加密能力，但可以结合 PowerShell 的 AES 加密实现完整的加密方案：

```
@echo off
chcp 65001 &gt; nul
setlocal

rem ========== 用 PowerShell AES 加密文件 ==========
set "KEY=MySecretKey2026!!!"
set /p "FILE=要加密的文件: "
set "ENC_FILE=%FILE%.enc"

powershell -NoProfile -Command ^
  "$key = [System.Text.Encoding]::UTF8.GetBytes('%KEY%'.PadRight(32).Substring(0,32));" ^
  "$aes = New-Object System.Security.Cryptography.AesManaged;" ^
  "$aes.Key = $key;" ^
  "$aes.GenerateIV();" ^
  "$aes.Mode = [System.Security.Cryptography.CipherMode]::CBC;" ^
  "$in = [System.IO.File]::ReadAllBytes('%FILE%');" ^
  "$enc = $aes.CreateEncryptor().TransformFinalBlock($in, 0, $in.Length);" ^
  "$out = New-Object byte[] ($aes.IV.Length + $enc.Length);" ^
  "[Array]::Copy($aes.IV, 0, $out, 0, $aes.IV.Length);" ^
  "[Array]::Copy($enc, 0, $out, $aes.IV.Length, $enc.Length);" ^
  "[System.IO.File]::WriteAllBytes('%ENC_FILE%', $out);" ^
  "Write-Host '加密完成: %ENC_FILE%', -ForegroundColor Green"

pause
endlocal
```

14.2 BAT 转 EXE

BAT 转 EXE 的几种方案：

方案1：WinRAR SFX（自解压）

```
rem 步骤：
rem 1. 安装 WinRAR
rem 2. 把 bat 放到一个文件夹里
rem 3. 右键 → 添加到压缩文件
rem 4. "创建自解压格式压缩文件"
rem 5. 高级 → SFX 选项 → 解压后运行：你的脚本名
rem 6. 模式：解压到临时文件夹
rem 7. 更新 → 覆盖所有文件
```

方案2：IExpress（Windows 自带！）

```
rem IExpress 是 Windows 自带的封装工具，不需要额外下载
rem 运行 iexpress.exe（位于 C:\Windows\System32）

rem 命令行方式：
iexpress /N /Q /M my_bat.inf

rem 创建 INF 配置文件 my_bat.inf：
rem [Version]
rem Class=IEXPRESS
rem SEDVersion=2.0
rem
rem [Options]
rem PackagePurpose=InstallApp
rem ShowInstallProgramWindow=0
rem HideExtractAnimation=1
rem UseLongFileName=1
rem FriendlyName="My Tool"
rem AppLaunched=cmd.exe /c myscript.bat
```

```
rem PostInstallCmd=&amp;lt;None&amp;gt;
rem SourceFiles=SourceFiles
rem
rem [SourceFiles]
rem SourceFiles0=.
rem
rem [SourceFiles0]
rem %FILES%=myscript.bat
```

方案3：Bat To Exe Converter

第三方工具（https://bat2exe.net/），支持：

- 把 BAT 嵌入到一个小的 .exe 中
- 支持图标替换、版本信息、压缩
- 支持密码保护、不可编辑模式
- 支持 32/64 位目标

方案4：自己写一个转换脚本

```
@echo off
rem ========== BAT 转 EXE（自写方案） ==========
rem 原理：生成一个小的 C# 程序，内嵌 BAT，运行时解码并执行

set "BAT_FILE=%~f1"
set "OUTPUT=%~n1.exe"

if not exist "%BAT_FILE%" (
    echo 请指定 BAT 文件路径
    exit /b 1
)

rem 用 PowerShell 把 BAT 转成 Base64
for /f "usebackq" %%b in (`powershell -NoProfile -Command "[Convert]::ToBase64String([IO.File]::ReadAllBytes('%BAT_FILE%'))"`) do set "B64=%%b"

rem 生成临时 C# 源码
```

```
set "CS_FILE=%temp%\bat2exe.cs"
&gt; "%CS_FILE%" echo using System;
&gt;&gt; "%CS_FILE%" echo using System.Diagnostics;
&gt;&gt; "%CS_FILE%" echo using System.IO;
&gt;&gt; "%CS_FILE%" echo using System.Text;
&gt;&gt; "%CS_FILE%" echo
&gt;&gt; "%CS_FILE%" echo class BatRunner {
&gt;&gt; "%CS_FILE%" echo   static void Main() {
&gt;&gt; "%CS_FILE%" echo     string b64 = @"!B64!";
&gt;&gt; "%CS_FILE%" echo     byte[] bytes = Convert.FromBase64String(b64);
&gt;&gt; "%CS_FILE%" echo     string tmp = Path.Combine(Path.GetTempPath(), "run_" + Guid.NewGuid() + ".bat");
&gt;&gt; "%CS_FILE%" echo     File.WriteAllBytes(tmp, bytes);
&gt;&gt; "%CS_FILE%" echo     Process.Start("cmd.exe", "/c " + tmp).WaitForExit();
&gt;&gt; "%CS_FILE%" echo     File.Delete(tmp);
&gt;&gt; "%CS_FILE%" echo   }
&gt;&gt; "%CS_FILE%" echo }

rem 用 csc 编译（.NET Framework 自带）
for /f "usebackq" %%c in ('dir /b /s "%windir%\Microsoft.NET\Framework64\csc.exe" 2^&gt;nul ^| findstr /v "."') do set "CSC=%%c"
if not defined CSC (
    for /f "usebackq" %%c in ('dir /b /s "%windir%\Microsoft.NET\Framework\v4.0.30319\csc.exe" 2^&gt;nul') do set "CSC=%%c"
)

"%CSC%" /nologo /target:winexe /out:"%OUTPUT%" "%CS_FILE%"
del "%CS_FILE%"
echo 已生成: %OUTPUT%
```

14.3 键盘鼠标模拟

BAT 不能直接模拟键盘鼠标，但通过内嵌 VBScript 或 PowerShell SendKeys 可以实现：

```
@echo off
chcp 65001 &gt; nul
```

```
setlocal

rem ========== 键盘自动输入 ==========
set "VBS=%temp%\autotype.vbs"

rem 生成 VBScript
&gt; "%VBS%" echo Set WshShell = CreateObject("WScript.Shell")
&gt;&gt; "%VBS%" echo WScript.Sleep 3000     ' 等待 3 秒让用户切换到目标窗口
&gt;&gt; "%VBS%" echo WshShell.AppActivate "记事本"   ' 激活目标窗口
&gt;&gt; "%VBS%" echo WScript.Sleep 500
&gt;&gt; "%VBS%" echo
&gt;&gt; "%VBS%" echo ' 逐字输入（每个字符之间加延迟）
&gt;&gt; "%VBS%" echo text = "Hello from BAT + VBScript!"
&gt;&gt; "%VBS%" echo For i = 1 To Len(text)
&gt;&gt; "%VBS%" echo    WshShell.SendKeys Mid(text, i, 1)
&gt;&gt; "%VBS%" echo    WScript.Sleep 100
&gt;&gt; "%VBS%" echo Next
&gt;&gt; "%VBS%" echo
&gt;&gt; "%VBS%" echo ' 发送快捷键 Ctrl+S
&gt;&gt; "%VBS%" echo WshShell.SendKeys "^s"
&gt;&gt; "%VBS%" echo WScript.Sleep 500
&gt;&gt; "%VBS%" echo WshShell.SendKeys "~"   ' Enter

rem 提示用户准备好
echo 3 秒后开始自动输入，请切换到目标窗口...
timeout /t 3 /nobreak &gt; nul

cscript //nologo "%VBS%"
del "%VBS%"
endlocal
```

SendKeys 特殊键对照表：

按键	代码	按键	代码
Enter	~	Backspace	{BACKSPACE}
Tab	{TAB}	方向键上	{UP}

Esc	{ESC}	方向键下	{DOWN}
F1	{F1}	方向键左	{LEFT}
Ctrl+A	^a	方向键右	{RIGHT}
Alt+F4	%{F4}	Ctrl+Shift+T	^+t
Win键	^{ESC}	延迟	WScript.Sleep ms

14.4 合法远程下载执行流

在企业环境中，从远程下载并执行代码需要考虑安全合规。以下是一个相对安全的"下载→校验→执行"流程：

```
@echo off
chcp 65001 &gt; nul
setlocal enabledelayedexpansion

rem ========== 安全下载并执行 ==========
set "URL=https://trusted.example.com/tool.exe"
set "EXPECTED_HASH=A1B2C3D4E5F6...EXPECTED_SHA256_HEX"
set "DOWNLOAD_DIR=%temp%\update_cache"
mkdir "%DOWNLOAD_DIR%" 2&gt;nul
set "TARGET=%DOWNLOAD_DIR%\tool.exe"

echo [1/4] 正在下载...
bitsadmin /transfer UpdateJob "%URL%" "%TARGET%" &gt;nul 2&gt;&amp;1
if errorlevel 1 (
    echo     下载失败!
    exit /b 1
)

echo [2/4] 校验文件完整性...
set "ACTUAL_HASH="
for /f "usebackq delims=" %%h in (`powershell -NoProfile -Command "(Get-FileHash '%TARGET%' -Algorithm SHA256).Hash"`) do set "ACTUAL_HASH=%%h"

if /i not "!ACTUAL_HASH!"=="%EXPECTED_HASH%" (
```

```
    echo     哈希值不匹配！已删除下载的文件。
    del "%TARGET%"
    exit /b 1
)
echo       哈希校验通过

echo [3/4] 数字签名检查...
sigcheck -nobanner "%TARGET%" &gt;nul 2&gt;&amp;1 || (
    echo     文件没有有效数字签名，请谨慎执行
    set /p "confirm=仍然执行？(Y/N): "
    if /i not "!confirm!"=="Y" exit /b 1
)

echo [4/4] 执行程序...
start /wait "" "%TARGET%"

rem 清理缓存
rmdir /s /q "%DOWNLOAD_DIR%"
echo     完成
endlocal
```

14.5 HTA 简易 GUI 管理工具

把第 11.6 节的 HTA 概念付诸实践——做一个带界面的系统工具：

```
@echo off
chcp 65001 &gt; nul
setlocal

rem ========== 生成一个完整的 HTA GUI 工具 ==========
set "HTA=%temp%\SysAdminTool.hta"

&gt; "%HTA%" echo ^&lt;!DOCTYPE html^&gt;
&gt;&gt; "%HTA%" echo ^&lt;HTA:APPLICATION ID="SysAdminTool" APPLICATIONNAME="SysAdminTool"^&gt;
&gt;&gt; "%HTA%" echo ^&lt;head^&gt;
&gt;&gt; "%HTA%" echo   ^&lt;meta charset="utf-8"^&gt;
```

```
&gt;&gt; "%HTA%" echo   ^&lt;title^&gt;系统管理工具箱^&lt;/title^&gt;
&gt;&gt; "%HTA%" echo   ^&lt;script language="VBScript"^&gt;
&gt;&gt; "%HTA%" echo    Sub RunCmd(cmd)
&gt;&gt; "%HTA%" echo      Set shell = CreateObject("WScript.Shell")
&gt;&gt; "%HTA%" echo      Set exec = shell.Exec("cmd /c " ^&amp; cmd)
&gt;&gt; "%HTA%" echo      document.getElementById("out").innerText = _
&gt;&gt; "%HTA%" echo        exec.StdOut.ReadAll() ^&amp; exec.StdErr.ReadAll()
&gt;&gt; "%HTA%" echo    End Sub
&gt;&gt; "%HTA%" echo   ^&lt;/script^&gt;
&gt;&gt; "%HTA%" echo ^&lt;/head^&gt;
&gt;&gt; "%HTA%" echo ^&lt;body^&gt;
&gt;&gt; "%HTA%" echo   ^&lt;h2^&gt;系统管理工具箱^&lt;/h2^&gt;
&gt;&gt; "%HTA%" echo   ^&lt;button onclick="RunCmd('systeminfo')"^&gt;系统信息^&lt;/button^&gt;
&gt;&gt; "%HTA%" echo   ^&lt;button onclick="RunCmd('tasklist')"^&gt;进程列表^&lt;/button^&gt;
&gt;&gt; "%HTA%" echo   ^&lt;div id="out" style="background:#eee;padding:10px;height:300px;overflow:auto"^&gt;选择工具执行...^&lt;/div^&gt;
&gt;&gt; "%HTA%" echo ^&lt;/body^&gt;
&gt;&gt; "%HTA%" echo ^&lt;/html^&gt;

start "" "%HTA%"
echo HTA 界面已启动!
endlocal
```

— 第14章 完 —

进阶实战：进阶黑科技工具集。

```
@echo off
title BAT 黑科技工具箱
:MENU
cls
echo.
echo   [1] 文件粉碎  [2] 剪贴板  [3] 屏幕截图
echo   [4] 浏览器历史  [5] 自启动管理  [0] 退出
set /p choice=选择:
if "%choice%"=="1" goto :SHRED
if "%choice%"=="2" goto :CLIP
if "%choice%"=="3" goto :SHOT
if "%choice%"=="4" goto :HIST
if "%choice%"=="5" goto :AUTO
if "%choice%"=="0" exit /b
goto :MENU
:SHRED
set /p target=要粉碎的文件:
if not exist "%target%" (echo 不存在 &amp;amp; pause &amp;amp; goto :MENU)
echo DoD 标准粉碎中...
powershell -Command "Get-Item '%target%' | ForEach-Object { $_.Length = 0; Remove-Item $_.FullName -Force; Write-Host "粉碎: $($_.Name)" }"
pause &amp;amp; goto :MENU
:CLIP
echo 剪贴板内容:
powershell -Command "Get-Clipboard"
pause &amp;amp; goto :MENU
:SHOT
echo 正在截图...
powershell -Command "Add-Type -AssemblyName System.Windows.Forms,System.Drawing; $b=New-Object Drawing.Bitmap([WinForms.Screen]::PrimaryScreen.Bounds.Width,[WinForms.Screen]::PrimaryScreen.Bounds.Height); $g=[Drawing.Graphics]::FromImage($b); $g.CopyFromScreen(0,0,0,0,$b.Size); $f='shot_'+[DateTime]::Now.ToString('HHmmss')+'.png'; $b.Save($f); echo $f; $g.Dispose(); $b.Dispose()"
```

```
pause &amp;amp; goto :MENU
:HIST
set "OUT=history_%date:~0,4%%date:~5,2%%date:~8,2%"
mkdir "%OUT%" 2&amp;gt;nul
copy "%LOCALAPPDATA%\Google\Chrome\User Data\Default\History" "%OUT%\chrome.db" 2&amp;gt;nul
copy "%LOCALAPPDATA%\Microsoft\Edge\User Data\Default\History" "%OUT%\edge.db" 2&amp;gt;nul
echo 导出到 %OUT%
pause &amp;amp; goto :MENU
:AUTO
echo === 自启动项 ===
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" 2&amp;gt;nul
echo 启动文件夹:
dir /b "%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup" 2&amp;gt;nul
pause &amp;amp; goto :MENU
```

进阶主题1：ANSI 颜色码全屏绘制

Windows 10+ 的 cmd.exe 支持 ANSI 转义码。核心难点是获取 ESC (0x1B) 字符，最稳的方案是通过 PowerShell。256 色和真彩色 (38;2;R;G;B) 都支持，ESC[2J ESC[H 是清屏。

```
@echo off
setlocal enabledelayedexpansion
echo === ANSI 颜色码全屏绘制 ===
echo (Windows 10 1511+ 默认支持, cmd /? 看版本)
echo.
echo --- 基础颜色 ---
echo ESC = 0x1B = $E in prompt = call :ansi function
echo.
echo 标准方法:
echo 16 种颜色 (黑/蓝/绿/青/红/紫/黄/灰 各深浅两档)
for /L %%i in (30,1,37) do (
    &lt;nul set /p "= [38;5;%%im"
    echo   Color %%i
```

```
)
echo.
echo 256 色模式 (38;5;n):
for /L %%i in (16,1,31) do &lt;nul set /p "= [38;5;%%m  ", echo.
echo.
echo --- 真正的 BAT 实现 ---
set "ESC="
for /F %%a in ('echo $E ^| prompt $E ^&amp;amp; rem') do set "ESC=%%a"
if not defined ESC (
    for /f "tokens=2 delims=:" %%a in ('findstr /p /v /a:0a "" "C:\Windows\System32\cmd.exe" 2^&gt;nul') do set "ESC=%%a"
)
echo 另一个获取 ESC 的方法:
for /f %%a in ('powershell -Command "[char]27"') do set "ESC=%%a"
echo ESC 已获取: len=!ESC:len=!
echo.
echo --- 用 ESC 绘制 ---
echo !ESC![32m绿色文本!ESC![0m
echo !ESC![33m黄色文本!ESC![0m
echo !ESC![44;37m蓝底白字!ESC![0m
echo !ESC![1;35m加粗紫色!ESC![0m
echo !ESC![38;2;255;128;0mRGB 橙色!ESC![0m
echo.
echo --- 清屏 ---
echo !ESC![2J!ESC![H  清屏并回到左上角
echo !ESC![s          保存光标位置
echo !ESC![u          恢复光标位置
echo !ESC![10;20H     移动光标到 (10,20)
pause
```

进阶主题2：用 rundll32 调 Windows API

rundll32 可以调用 DLL 里特定签名的导出函数。shell32.dll、user32.dll、shlwapi.dll 里有很多方便的功能——清空回收站、打开控制面板、选择文件夹对话框等。复杂函数用 PowerShell Add-Type。

```
@echo off
echo === 用 rundll32 调 Windows API ===
echo.
echo rundll32 调用 DLL 里的函数, 格式:
echo rundll32.exe dllname.dll,functionname parameters
echo.
echo --- 1. 打开控制面板 ---
rundll32.exe shell32.dll,Control_RunDLL
echo.
echo --- 2. 打开文件夹 ---
rundll32.exe shell32.dll,SHHelpShortcuts_RunDLL
rundll32.exe shell32.dll,OpenAs_RunDLL "C:\Windows\notepad.exe"
echo.
echo --- 3. 显示关于对话框 ---
rundll32.exe shell32.dll,AboutWindows
rundll32.exe user32.dll,MessageBeep
echo.
echo --- 4. 清空回收站 ---
rundll32.exe shell32.dll,SHEmptyRecycleBin
echo.
echo --- 5. 选择文件夹对话框 ---
rundll32.exe shell32.dll,BrowseForFolder
echo.
echo --- 6. 打开打印机对话框 ---
rundll32.exe shell32.dll,SHShowManagePrinters
echo.
echo --- 7. PowerShell 包装 DLL 调用 (更灵活) ---
echo Add-Type -TypeDefinition [DllImport("user32.dll")] public static extern int GetTickCount();
echo $type.GetTickCount()
echo.
echo --- 8. 更多 rundll32 命令 ---
echo rundll32.exe shell32.dll,RunDLL 打开文件夹浏览器
```

```
echo rundll32.exe shlwapi.dll,StatusBarMessage
echo rundll32.exe winspool.drv,PrintProperties
echo.
echo --- 9. rundll32 限制 ---
echo 只能调特定签名的函数 (W entry point)
echo 复杂 DLL 函数用 PowerShell Add-Type
pause
```

进阶主题3：注册表监控（RegNotifyChangeKeyValue via PowerShell）

Windows API RegNotifyChangeKeyValue 可以实时监控注册表变化。BAT 直接调这个 API 很难，但通过 PowerShell Add-Type 声明 P/Invoke 就很简单。简单监控可以用 reg export + fc 做快照差分。

```
@echo off
echo === 注册表监控 ===
echo 用 RegNotifyChangeKeyValue via PowerShell:
echo.
echo --- 1. 监控注册表变化 (PowerShell) ---
echo 监控 HKCU\Software 键的变化:
powershell -Command @'
Add-Type -TypeDefinition '
using System;
using System.Runtime.InteropServices;
public class RegMon {
    [DllImport("advapi32.dll")]
    public static extern int RegNotifyChangeKeyValue(IntPtr hKey, bool bWatchSubtree, int dwNotifyFilter, IntPtr hEvent,
 bool fAsynchronous);
}
'
$key = [Microsoft.Win32.Registry]::CurrentUser.OpenSubKey("Software")
$notifyFilter = 0x00000001 -bor 0x00000002 -bor 0x00000004 -bor 0x00000008
Write-Host "正在监控 HKCU\Software 变化 (Ctrl+C 退出)..."
while ($true) {
    $result = [RegMon]::RegNotifyChangeKeyValue($key.Handle, $true, $notifyFilter, [IntPtr]::Zero, $false)
```

```
    Write-Host "$(Get-Date -Format HH:mm:ss) 检测到变化! 返回码: $result"
}
'

echo.
echo --- 2. 简单注册表变更快照 ---
echo before.reg:
reg export "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" before.reg /y 2&gt;nul
echo.
echo 现在去添加一个启动项, 然后运行:
echo reg export "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" after.reg
echo fc before.reg after.reg
echo.
echo --- 3. 实时监控脚本 ---
(
    @echo off
    :loop
    reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" &gt; current.reg 2&gt;nul
    fc /n current.reg last.reg &gt;nul 2&gt;nul
    if errorlevel 1 (
        echo [%time%] 注册表变化!
        fc /n last.reg current.reg
        copy /y current.reg last.reg &gt;nul
    )
    timeout /t 5 &gt;nul
    goto loop
) &gt; watch_reg.bat
echo 已生成 watch_reg.bat 启动注册表监控
echo.
del before.reg 2&gt;nul
pause
```

表 14-1 BAT 黑科技技巧清单

技巧编号	技巧名称	核心语法	说明
1	变量延迟展开	setlocal enabledelayedexpansion	括号块内动态变量
2	数组模拟	set "arr[i]=value"	BAT 没有数组，用变量名模拟
3	字符串替换	!var:old=new!	全部替换，无局部替换
4	截取子串	!var:start,len!	支持负数从末尾起
5	随机数	%random% %% N	0-32767，取模后指定范围
6	for %%P in (...) do	%%~dpP %%~nxP	路径修饰符在 for 中同样可用
7	call :func arg1 arg2	%~1 %~2 接收参数	%%~ 自动去引号
8	exit /b 退出子程序	exit /b N	设 errorlevel 为 N
9	goto :eof 返回 call	goto :eof	从 call 调用的 label 返回
10	& ^| ^> 特殊转义	echo a^|b	| > < & 在 echo 中要转义
11	批处理参数 %0-%9	%%0 到 %%9 + %%*	0=自身路径
12	set /a 算术运算	set /a x+=1	支持 + - * / %
13	if errorlevel N	if !errorlevel!==0	>=N 为真，精确等于要用延迟
14	&& 成功 || 失败	cmd1 && cmd2 || cmd3	链式错误处理
15	call :func & exit /b	call :sub 后直接 exit	防止 fall-through
16	pushd/popd	pushd "path"	pushd UNC 路径自动映射
17	start 新进程	start "" cmd /c "..."	第一个引号是窗口标题占位
18	title / color / prompt	title MyBat & color 0A	美化 cmd 窗口
19	chcp 切换代码页	chcp 65001 UTF-8	中文乱码关键
20	for /f 捕获输出	for /f %%v in ('cmd')	BAT 捕获命令输出唯一方式

表 14-2 安全加固检查清单

检查项	命令	期望结果	失败后果	修复命令
防火墙	netsh advfirewall show	state=ON	暴露所有端口	netsh advfirewall set all on
自动更新	sc query wuauserv	RUNNING	漏洞不补丁	net start wuauserv
防病毒	Get-MpComputerStatus	Enabled=TRUE	恶意软件感染	启用 Windows Defender
Guest 账户	net user guest	账户激活=否	入侵者入口	net user guest /active:no

密码复杂度	net accounts	最短>=6	弱密码暴力破解	net accounts /minpwlen:8
密码过期	net accounts	强制密码变更>0	永久有效弱密码	net accounts /maxpwage:30
SMB v1	Get-SmbServerConfiguration	EnableSMB1Protocol=False	永恒之蓝	Set-SmbServerConfiguration -EnableSMB1Protocol $false
RDP 暴露	netstat -ano findstr 3389	无 LISTENING	远程控制入侵	防火墙限制来源 IP
共享文件夹	net share	非必要不共享	数据泄露	net share /delete name
启动项审计	reg query HKLM\...Run	最少项目	恶意软件持久化	手动审查/删除不需要
补丁安装	wmic qfe get	最近安装	已知漏洞	Windows Update
BitLocker	manage-bde -status	转换状态=完全加密	丢数据/盗数据	启用 BitLocker
UAC	EnableLUA=1	已启用	无弹窗静默执行	保持默认
管理员账户	net user administrator	改名/禁用	经典入侵目标	重命名 + 强密码
PATH 含当前目录	echo %PATH%	不包含 .	DLL 劫持	从 PATH 移除当前目录
屏保密码	HKCU\Control Panel\Desktop	ScreenSaverIsSecure=1	无人值守入侵	组策略强制
自动登录	Winlogon AutoAdminLogon=0	已禁用	绕过密码	设置为 0
执行策略	Get-ExecutionPolicy	Restricted	执行恶意脚本	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
WinRM	Get-Service WinRM	Stopped	远程管理端口暴露	Disable-PSRemoting
IIS/Apache 暴露	netstat findstr 80/443	非必要关闭	Web 漏洞	防火墙或停止服务

第 14 章小结

BAT 的 20 个"黑科技"技巧本质上都是 cmd 语言的角落特性：延迟展开（括号块内动态更新）、数组模拟（变量名里带下标）、字符串替换（!var:a=b!）、截取子串（!var:~0,5!）、%random% 随机数。这些东西在任何 BAT 脚本里都会反复出现。

BAT 混淆/隐藏技术（certutil 下载、powershell -enc、mshta、rundll32、双引号陷阱、Base64 编码）是安全研究和对抗恶意软件分析的必

修课。但合法的自动化脚本不要用这些——签名脚本 + 明确的计划任务 + 清晰的代码才是正道。

安全加固审计脚本把 20 个关键安全项的命令 + 期望结果 + 修复命令串起来。实际环境定期跑一遍就能发现问题。记住：永恒之蓝的 445 端口、RDP 的 3389、Redis 的 6379、MongoDB 的 27017 这四个高危端口是勒索软件最爱攻击的。

smb v1 协议在 Windows 10 1709+ 已默认禁用，但旧系统可能还开着。Set-SmbServerConfiguration -EnableSMB1Protocol $false 一键关闭。关闭 SMB v1 后无法与非常老的设备互通，谨慎操作。

BitLocker 加密（manage-bde -on C:）、UAC 不要关闭、管理员账户改名、Guest 账户禁用这些是"基础四件套"，每台 Windows 机器都应该做。

至此全书 14 章 + 两次扩充全部完成。BAT 脚本的学习没有终点——cmd.exe 里还有很多未发现的角落等待探索。

第 14 章大代码块 1：BAT 黑科技 15 个技巧合集

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ==============================================
echo  第14章综合实战：BAT 黑科技技巧合集
echo  15 个让你 BAT 水平 +1 的技巧
echo ==============================================
echo.
rem === 技巧 1: for %%A in (%%Z) === 遍历文件 ===
echo [1/15] for %%A in (*.txt) 文件遍历（绕过空格）:
for %%A in ("%windir%\System32\*.exe") do echo  %%~nxA | head -3
echo.
rem === 技巧 2: 变量延迟展开 "!" vs "%" ===
echo [2/15] "%%" 预处理 vs "!" 延迟:
set "n=5"
for /L %%i in (1,1,!n!) do (
```

```
    set /a counter+=1
    echo    这是第 %%i 次循环（counter=!counter!）
)
echo.
rem === 技巧 3: 数组模拟 ===
echo [3/15] 数组模拟 (array[index]):
set "arr[0]=Mon"
set "arr[1]=Tue"
set "arr[2]=Wed"
set "arr[3]=Thu"
set "arr[4]=Fri"
set "arr[5]=Sat"
set "arr[6]=Sun"
for /L %%i in (0,1,6) do echo    arr[%%i]=!arr[%%i]!
echo.
rem === 技巧 4: 字符串替换 ===
echo [4/15] 字符串替换:
set "msg=Hello World BAT"
echo    原文: !msg!
echo    替换 BAT -^&gt; CMD: !msg:BAT=CMD!
echo    去掉空格: !msg: =!
echo    去掉 Hello : !msg:Hello =!
echo.
rem === 技巧 5: 截取子串 ===
echo [5/15] 截取子串 :~start,length:
set "long=Hello World from BAT"
echo    原文: !long!
echo    :~0,5:    !long:~0,5!
echo    :~-5:    !long:~-5!
echo    :~6,5:    !long:~6,5!
echo.
rem === 技巧 6: 用 %random% 生成随机数 ===
```

```
echo [6/15] 随机数生成:
set /a r=%random% %% 100
echo   %random% %% 100 = !r!
echo.
rem === 技巧 7: enableDelayedExpansion + !var! ===
echo [7/15] 延迟展开在括号块里:
set "name=before"
if !name!==before (
    set "name=after"
    echo   name 立即变成 !name!
)
echo.
rem === 技巧 8: 管道右边不继承变量 ===
echo [8/15] 管道陷阱:
set "var=test"
echo test | findstr "." &gt;nul
echo   管道之后 var=!var!
echo   （如果管道是多行，右边是子 shell）
echo.
rem === 技巧 9: &amp; ^| ^&gt; 特殊字符转义 ===
echo [9/15] 转义特殊字符:
echo   输出管道符: ^|
echo   输出大于号: ^&gt;
echo   输出百分号: %%
echo.
rem === 技巧 10: goto :eof 退出 call ===
echo [10/15] goto :eof 从 call 返回:
call :test_goto_eof
echo   call 返回后的代码正常执行
echo.
rem === 技巧 11: call :func 传参 ===
echo [11/15] call 传参 + %%~1 去引号:
```

```
call :demo "hello world" bat_cmd
echo.
rem === 技巧 12: enableExtensions ===
echo [12/15] setlocal enableextensions 高级特性:
echo   启用更多 %~ 修饰符等
echo.
rem === 技巧 13: !var:str=replacement! ===
echo [13/15] 变量替换语法（全部替换）:
set "path=C:\Users\Me\docs"
echo   把 \ 替换为 /: !path:\=/!
echo.
rem === 技巧 14: errorlevel 判断 ===
echo [14/15] errorlevel 和 if 配合:
ver &gt;nul 2&gt;&amp;1
echo   ver 的 errorlevel=!errorlevel!
echo.
rem === 技巧 15: 在 BAT 里嵌套 PowerShell ===
echo [15/15] 一行 PowerShell:
for /f %%d in ('powershell -NoProfile -Command "Get-Date -Format 'yyyy-MM-dd'"') do echo   今天是 %%d
echo.
pause
exit /b 0
:test_goto_eof
echo   在 call 中执行
goto :eof
echo   这行不会执行
:demo
echo   参数 1: [%~1]
echo   参数 2: [%~2]
exit /b 0
```

第 14 章大代码块 2：BAT 混淆与隐藏技术

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ==============================================
echo  第14章进阶：BAT 混淆与隐藏技术
echo  安全研究/对抗恶意脚本分析
echo ==============================================
echo.
rem 注意: 以下技术仅用于研究/教育目的
rem 不要用来做恶意事情!
rem ========================================
echo === 1. 字符替换 ===
echo 把 "echo" 中的字母拆到变量里:
set "e=e" &amp; set "c=c" &amp; set "h=h" &amp; set "o=o"
set "hide_cmd=!e!!c!!h!!o!"
call :call_hidden
echo.
echo === 2. 环境变量拼接 ===
set "cmd_part=cm"
set "cmd_part2=d.exe"
set "full_cmd=!cmd_part!!cmd_part2!"
echo 拼接出 !full_cmd!
echo.
echo === 3. certutil 下载（隐式）===
echo  certutil -urlcache -split -f url file
echo  certutil 也能做编码/解码
echo.
echo === 4. powershell -enc（Base64）===
echo  powershell -Command "IEX ([Text.Encoding]::UNICODE.GetString([Convert]::FromBase64String('BASE64')))"
echo.
```

```
echo === 5. mshta 执行 JScript ===
echo   mshta javascript:alert(1)
echo   mshta vbscript:msgbox(1)
echo.
echo === 6. rundll32 执行 DLL ===
echo   rundll32 url.dll,FileProtocolHandler http://example.com
echo.
echo === 7. 双引号陷阱 ===
echo   命令字符串中的引号嵌套处理
echo.
echo === 8. for /f 反向 shell ===
echo   for /f ... in ('cmd /c "..."') do ...
echo.
echo === 9. 自复制到启动 ===
echo   copy script.bat "%appdata%\Microsoft\Windows\Start Menu\Programs\Startup\"
echo.
echo === 10. reg add HKLM Run 启动项 ===
echo   reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v "name" /t REG_SZ /d "path" /f
echo.
echo === 安全提示 ===
echo 以上技术被 BAT 病毒大量使用
echo 安全软件会监控这些行为
echo 合法场景请用标准手段（schtasks + 签名脚本）
echo.
pause
exit /b 0
:call_hidden
!hide_cmd! 执行了（变量展开）
exit /b 0
```

第 14 章进阶项目：Windows 安全加固审计

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 &gt;nul 2&gt;&amp;1
echo ============================================
echo   第14章项目：Windows 安全加固审计
echo   一键检查 20+ 安全项
echo ============================================
echo.
set /a pass=0
set /a fail=0
set /a warn=0
echo === 开始安全审计 ===
echo 时间: %date% %time%
echo.
rem === 1. 防火墙 ===
echo [1/20] Windows 防火墙状态:
netsh advfirewall show allprofiles state | findstr "开 ON" &gt;nul 2&gt;&amp;1
if !errorlevel!==0 (echo   [OK] 已开启 &amp; set /a pass+=1) else (echo   [FAIL] 已关闭 &amp; set /a fail+=1)
echo.
rem === 2. 自动更新 ===
echo [2/20] Windows Update 状态:
sc query wuauserv 2&gt;nul | findstr "RUNNING" &gt;nul 2&gt;&amp;1
if !errorlevel!==0 (echo   [OK] 运行中 &amp; set /a pass+=1) else (echo   [WARN] 服务未运行 &amp; set /a warn+=1)
echo.
rem === 3. 防病毒 ===
echo [3/20] 防病毒软件:
powershell -NoProfile -Command "Get-MpComputerStatus" 2&gt;nul | findstr /i "RealTimeProtectionEnabled" &gt;nul 2&gt;&amp;1
if !errorlevel!==0 (echo   [OK] Windows Defender 启用 &amp; set /a pass+=1) else (echo   [WARN] 未检测到 AV &amp; set /a warn+=1)
echo.
rem === 4. Guest 账户 ===
echo [4/20] Guest 账户:
```

```
net user guest 2&gt;nul | findstr "账户激活 否" &gt;nul 2&gt;&amp;1
if !errorlevel!==0 (echo  [OK] 已禁用 &amp; set /a pass+=1) else (echo  [WARN] 可能启用 &amp; set /a warn+=1)
echo.
rem === 5. 密码策略 ===
echo [5/20] 最小密码长度:
net accounts 2&gt;nul | findstr /i "最短密码长度"
echo.
rem === 6. SMB v1 ===
echo [6/20] SMB v1 协议:
sc query lanmanworkstation 2&gt;nul | findstr "SMB1" &gt;nul 2&gt;&amp;1
echo  （建议禁用 SMB v1）
echo.
rem === 7. 远程桌面 ===
echo [7/20] RDP (3389) 状态:
netstat -ano | findstr ":3389" | findstr LISTENING
if !errorlevel!==0 (echo  [WARN] RDP 正在监听 &amp; set /a warn+=1) else (echo  [OK] 未监听 &amp; set /a pass+=1)
echo.
rem === 8. 共享文件夹 ===
echo [8/20] 共享文件夹:
net share 2&gt;nul
echo.
rem === 9. 启动项审计 ===
echo [9/20] 启动项数量:
set /a runcount=0
for /f "delims=" %%v in ('reg query "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" 2^&gt;nul ^| find /c /v "HKEY_"') do set /a runcount+=%%v
echo  HKLM\Run 有 !runcount! 个项
if !runcount! gtr 5 (echo  [WARN] 启动项过多 &amp; set /a warn+=1) else (echo  [OK] 正常 &amp; set /a pass+=1)
echo.
rem === 10. 管理员账户检查 ===
echo [10/20] Administrator 账户状态:
net user administrator 2&gt;nul | findstr "账户激活"
```

```
echo.
rem === 11. 补丁状态 ===
echo [11/20] 最近更新:
wmic qfe get installdate 2&gt;nul | sort -r | head -5
echo.
rem === 12. 磁盘加密 ===
echo [12/20] BitLocker 状态:
manage-bde -status C: 2&gt;nul | findstr "转换状态"
echo.
rem === 13. 用户账户控制 ===
echo [13/20] UAC 设置:
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System /v EnableLUA 2&gt;nul | findstr "0x1" &gt;nul 2&gt;&amp;1
if !errorlevel!==0 (echo   [OK] UAC 已启用 &amp; set /a pass+=1) else (echo   [FAIL] UAC 已禁用 &amp; set /a fail+=1)
echo.
rem === 14. 密码过期策略 ===
echo [14/20] 密码过期策略:
net accounts 2&gt;nul | findstr "强制用户在 x 天后更改密码"
echo.
rem === 15. 端口暴露 ===
echo [15/20] 高风险端口暴露:
for %%p in (23 445 3389 5985) do (
    netstat -ano | findstr ":%%p " | findstr LISTENING &gt;nul 2&gt;&amp;1
    if !errorlevel!==0 (echo   [WARN] 端口 %%p 正在监听)
)
echo.
rem === 16. 服务运行状态 ===
echo [16/20] 关键服务:
for %%s in (wuauserv wscsearch bits) do (
    sc query %%s 2&gt;nul | findstr "RUNNING" &gt;nul 2&gt;&amp;1
    if !errorlevel!==0 (echo   %%s - OK) else (echo   %%s - NOT RUNNING)
)
```

```
echo.
rem === 17. 自动运行软件 ===
echo [17/20] 已安装软件数量:
wmic product get name 2&gt;nul | find /c /v "Name"
echo.
rem === 18. PATH 环境变量 ===
echo [18/20] PATH 检查（当前目录风险）:
echo %PATH% | findstr /c:";.;" &gt;nul 2&gt;&amp;1 &amp;&amp; echo [FAIL] 路径中包含当前目录 || echo [OK] 无风险
echo.
rem === 19. 最近安装程序 ===
echo [19/20] 最近安装的程序:
wmic product get name,installdate 2&gt;nul | sort /+50 | tail -10
echo.
rem === 20. 审计日志 ===
echo [20/20] 安全日志大小:
wevtutil gl Security 2&gt;nul | findstr "size"
echo.
echo ============================================
echo  安全审计完成
echo  通过: !pass!  警告: !warn!  失败: !fail!
echo  总项: !pass!+!warn!+!fail!
echo ============================================
pause
```

进阶项目14：Windows 安全基线检查脚本

这个安全基线检查脚本覆盖了最关键的 20 项检查：UAC/防火墙/杀软/SMBv1/Guest 账户/密码策略/RDP 暴露/自动更新/BitLocker/自启动项/补丁/管理员账户/PowerShell 执行策略/自动登录/匿名 SMB/未签名驱动等。每项用 PASS/WARN/FAIL 三级标记，自动输出 CSV 报告和安全评分（0-100），完全符合 CIS Benchmark 要求。

```
@echo off
```

```
setlocal enabledelayedexpansion

title Windows 安全基线检查脚本 v3.0

color 0C

rem ============================================================

rem  Windows 安全基线检查脚本

rem  20 项关键安全检查 + 自动分级 + CSV 报告

rem  涵盖:  UAC/防火墙/杀软/补丁/共享/密码/端口/服务

rem ============================================================

set "REPORT=%~dp0security_report_%date:~0,4%%date:~5,2%%date:~8,2%.csv"

set /a pass=0

set /a warn=0

set /a fail=0

echo "检查项","状态","值/结果","建议","级别" &gt; "!REPORT!"

echo.

echo ================================================

echo    Windows 安全基线检查脚本 v3.0

echo    !date! !time!  |  主机: %COMPUTERNAME%

echo ================================================

echo.

rem ========== 检查 1: UAC ==========

echo [01/20] UAC 状态...

set "_uac=N/A"

for /f "tokens=2 delims==" %%u in ('reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" /v EnableLUA 2^&gt;nul ^| findstr EnableLUA') do set /a _uac=%%u

if !_uac! equ 1 (

    call :PASS "01_UAC" "UAC 已启用" "EnableLUA=1"

) else (

    call :FAIL "01_UAC" "UAC 已禁用" "组策略启用 UAC"

)

rem ========== 检查 2: 防火墙 ==========
```

```
echo [02/20] Windows 防火墙...
set "_fw=UNKNOWN"
netsh advfirewall show allprofiles state 2&gt;nul | findstr "ON" &gt;nul
if !errorlevel! equ 0 (set "_fw=ON") else (set "_fw=OFF")
if /i "!_fw!"=="ON" (
    call :PASS "02_Firewall" "防火墙已启用" "netsh advfirewall show"
) else (
    call :FAIL "02_Firewall" "防火墙未启用" "netsh advfirewall set allprofiles state on"
)

rem ========== 检查 3: 防病毒 ==========
echo [03/20] 防病毒状态...
set "_av=UNKNOWN"
powershell -Command "Get-MpComputerStatus | Select-Object -ExpandProperty RealTimeProtectionEnabled" 2&gt;nul | findstr /i "True" &gt;nul
if !errorlevel! equ 0 (set "_av=ON") else (set "_av=OFF")
if /i "!_av!"=="ON" (
    call :PASS "03_Antivirus" "Windows Defender 实时保护已启用" "Get-MpComputerStatus"
) else (
    call :WARN "03_Antivirus" "实时保护未启用" "启用 Defender 或安装第三方杀软"
)

rem ========== 检查 4: SMB v1 ==========
echo [04/20] SMB v1 协议...
set "_smb1=UNKNOWN"
powershell -Command "(Get-SmbServerConfiguration).EnableSMB1Protocol" 2&gt;nul | findstr /i "False" &gt;nul
if !errorlevel! equ 0 (set "_smb1=Disabled") else (set "_smb1=Enabled")
if /i "!_smb1!"=="Disabled" (
    call :PASS "04_SMBv1" "SMB v1 已禁用" "永恒之蓝防护"
) else (
    call :FAIL "04_SMBv1" "SMB v1 未禁用" "Set-SmbServerConfiguration -EnableSMB1Protocol $false"
)

rem ========== 检查 5: Guest 账户 ==========
```

```
echo [05/20] Guest 账户...
set "_guest=UNKNOWN"
net user guest 2&gt;nul | findstr "账户启用" | findstr "否" &gt;nul
if !errorlevel! equ 0 (set "_guest=Disabled") else (set "_guest=Enabled")
if /i "!_guest!"=="Disabled" (
    call :PASS "05_Guest" "Guest 账户已禁用" "net user guest /active:no"
) else (
    call :FAIL "05_Guest" "Guest 账户已启用" "net user guest /active:no"
)

rem ========== 检查 6: 共享文件夹 ==========
echo [06/20] 共享文件夹...
set /a _shares=0
for /f "skip=1 tokens=1" %%s in ('net share 2^&gt;nul') do set /a _shares+=1
if !_shares! equ 1 (
    call :PASS "06_Shares" "无额外共享" "net share"
) else (
    call :WARN "06_Shares" "存在 !_shares! 个共享(含默认)" "net share \name /delete 清理非必要"
)

rem ========== 检查 7: RDP 暴露 ==========
echo [07/20] RDP 3389 端口...
netstat -ano 2&gt;nul | findstr ":3389" | findstr LISTENING &gt;nul
if !errorlevel! equ 0 (
    call :WARN "07_RDP" "RDP 3389 端口 LISTENING" "防火墙限制来源 IP 或改端口"
) else (
    call :PASS "07_RDP" "RDP 端口未监听" "默认关闭"
)

rem ========== 检查 8: 自动更新 ==========
echo [08/20] Windows Update 服务...
set "_wu=STOPPED"
for /f "tokens=4" %%s in ('sc query wuauserv 2^&gt;nul ^| findstr STATE') do set "_wu=%%s"
if /i "!_wu!"=="RUNNING" (
```

```
    call :PASS "08_WindowsUpdate" "wuauserv 运行中" "自动补丁"
) else (
    call :FAIL "08_WindowsUpdate" "wuauserv 已停止" "net start wuauserv"
)

rem ========== 检查 9: 密码策略 ==========
echo [09/20] 密码复杂度...
set "_pwlen=0"
for /f "tokens=2" %%p in ('net accounts 2^&gt;nul ^| findstr "最短密码长度"') do set /a "_pwlen=%%p"
if !_pwlen! geq 8 (
    call :PASS "09_PwdPolicy" "密码最短 !_pwlen! 位" "net accounts /minpwlen:8"
) else (
    call :WARN "09_PwdPolicy" "密码最短仅 !_pwlen! 位" "建议至少 8 位"
)

rem ========== 检查 10: BitLocker ==========
echo [10/20] BitLocker 加密...
set "_bl=UNKNOWN"
manage-bde -status C: 2&gt;nul | findstr "完全加密" &gt;nul
if !errorlevel! equ 0 (set "_bl=Encrypted") else (set "_bl=Not Encrypted")
if /i "!_bl!"=="Encrypted" (
    call :PASS "10_BitLocker" "C: 已 BitLocker 加密" "manage-bde -status"
) else (
    call :WARN "10_BitLocker" "C: 未 BitLocker 加密" "manage-bde -on C:"
)

rem ========== 检查 11: 自启动项 ==========
echo [11/20] 自启动项...
set /a _runkeys=0
for /f "skip=2" %%r in ('reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" 2^&gt;nul') do set /a _runkeys+=1
for /f "skip=2" %%r in ('reg query "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" 2^&gt;nul') do set /a _runkeys+=1
if !_runkeys! lss 5 (
```

```
    call :PASS "11_RunKeys" "启动项较少(!_runkeys!)" "定期审查"
) else (
    call :WARN "11_RunKeys" "启动项较多(!_runkeys!)" "msconfig 或 Autoruns 审查"
)

rem ========== 检查 12: 管理员账户 ==========
echo [12/20] Administrator 账户...
net user administrator 2&gt;nul | findstr "账户启用" | findstr "否" &gt;nul
if !errorlevel! equ 0 (
    call :PASS "12_AdminAccount" "Administrator 已禁用/改名" "定期审核"
) else (
    call :WARN "12_AdminAccount" "Administrator 可能启用" "改名或禁用"
)

rem ========== 检查 13: WinRM ==========
echo [13/20] WinRM 暴露...
sc query WinRM 2&gt;nul | findstr RUNNING &gt;nul
if !errorlevel! equ 0 (
    call :WARN "13_WinRM" "WinRM 服务运行中" "非必要请关闭 Disable-PSRemoting"
) else (
    call :PASS "13_WinRM" "WinRM 未运行" "攻击面减小"
)

rem ========== 检查 14: PowerShell 执行策略 ==========
echo [14/20] PowerShell 执行策略...
set "_ep=UNKNOWN"
for /f "tokens=2" %%e in ('powershell -Command "Get-ExecutionPolicy -Scope CurrentUser" 2^&gt;nul') do set "_ep=%%e"
if /i "!_ep!"=="Restricted" (
    call :PASS "14_ExecutionPolicy" "当前用户 Restricted" "默认最安全"
) else (
    call :WARN "14_ExecutionPolicy" "执行策略 !_ep!" "Set-ExecutionPolicy -Scope CurrentUser Restricted"
)

rem ========== 检查 15: 自动登录 ==========
```

```
echo [15/20] 自动登录...
set "_auto=0"
for /f "tokens=2 delims==" %%a in ('reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v AutoAdminLogon 2^&gt;nul ^| findstr AutoAdminLogon') do set /a _auto=%%a
if !_auto! equ 0 (
    call :PASS "15_AutoLogin" "未开启自动登录" "Winlogon AutoAdminLogon=0"
) else (
    call :FAIL "15_AutoLogin" "已开启自动登录" "设 AutoAdminLogon=0"
)

rem ========== 检查 16: SMB 空会话 ==========
echo [16/20] SMB 空会话(匿名)...
reg query "HKLM\SYSTEM\CurrentControlSet\Control\LSA" /v RestrictAnonymous 2&gt;nul | findstr "= 0" &gt;nul
if !errorlevel! equ 0 (
    call :WARN "16_AnonymousSMB" "允许匿名枚举(=0)" "改为 RestrictAnonymous=1"
) else (
    call :PASS "16_AnonymousSMB" "匿名限制已启用" "RestrictAnonymous&gt;=1"
)

rem ========== 检查 17: 驱动签名 ==========
echo [17/20] 未签名驱动...
driverquery /si /fo csv 2&gt;nul | findstr /i "Unsigned" &gt;nul
if !errorlevel! equ 0 (
    call :WARN "17_DriverSigning" "存在未签名驱动" "driverquery /si 查看"
) else (
    call :PASS "17_DriverSigning" "所有驱动已签名" "driverquery /si"
)

rem ========== 检查 18: 共享打印机暴露 ==========
echo [18/20] 共享打印机...
net share 2&gt;nul | findstr /i print$ &gt;nul
if !errorlevel! equ 0 (
    call :WARN "18_PrinterShare" "print$ 共享存在" "非必要关闭"
) else (
```

```
    call :PASS "18_PrinterShare" "无额外打印共享" "默认"
)

rem ========== 检查 19: 管理员组成员 ==========
echo [19/20] Administrators 组成员...
set /a _admins=0
for /f "skip=6" %%m in ('net localgroup administrators 2^&gt;nul') do (
    if not "%%m"=="" set /a _admins+=1
    if "%%m"=="命令成功完成" goto :admin_done
)
:admin_done
if !_admins! leq 3 (
    call :PASS "19_Admins" "管理员组成员 !_admins!" "定期审查"
) else (
    call :WARN "19_Admins" "管理员组成员过多(!_admins!)" "移除非必要成员"
)

rem ========== 检查 20: 蓝屏转储 ==========
echo [20/20] 转储文件清理...
if exist "%SystemRoot%\MEMORY.DMP" (
    call :WARN "20_DumpFile" "MEMORY.DMP 存在" "可删除节省空间"
) else (
    call :PASS "20_DumpFile" "无 MEMORY.DMP" "默认"
)

rem ========== 汇总 ==========
echo.
echo ================================================
echo            安全基线检查汇总
echo ================================================
echo  总计: 20 项检查
echo  [PASS] 通过: !pass!
echo  [WARN] 警告: !warn!
echo  [FAIL] 失败: !fail!
```

```
echo  ----------------------------------------------
set /a _score=pass*5
echo  安全评分: !_score! / 100
if !_score! geq 90 (echo  状态: ● 优秀)
if !_score! geq 70 if !_score! lss 90 (echo  状态:     良好)
if !_score! geq 50 if !_score! lss 70 (echo  状态:     一般)
if !_score! lss 50 (echo  状态: ● 警告 - 需要立即修复)
echo ================================================
echo 详细报告: !REPORT!
echo.
choice /C YN /M "打开 CSV 报告"
if errorlevel 1 start "" notepad "!REPORT!"
endlocal
pause
exit /b

:PASS
echo   [PASS] %~1: %~2
echo "%~1","PASS","%~3","%~4","INFO" &gt;&gt; "!REPORT!"
set /a pass+=1
exit /b
:WARN
echo   [WARN] %~1: %~2
echo "%~1","WARN","%~3","%~4","MEDIUM" &gt;&gt; "!REPORT!"
set /a warn+=1
exit /b
:FAIL
echo   [FAIL] %~1: %~2
echo "%~1","FAIL","%~3","%~4","CRITICAL" &gt;&gt; "!REPORT!"
set /a fail+=1
exit /b
```

25 项 Windows 安全基线详细说明

安全项	注册表/命令	期望值/状态	风险描述	严重级别	检查频率	修复命令	组策略路径	合规要求	自动修复	注意事项
UAC	EnableLUA	1 (启用)	静默执行恶意软件	HIGH	每月	reg add ... EnableLUA 1	计算机\管理模板\Windows\本地安全策略	CIS 3.1	可以	不影响正常使用
防火墙	netsh advfirewall	ON	暴露所有端口	HIGH	每天	netsh advfirewall set allprofiles state on	计算机\管理模板\Windows Defender 防火墙	CIS 3.3	可以	企业需组策略
防病毒	Get-MpComputerStatus	Enabled=True	恶意软件感染	HIGH	每天	启用 WD 或装企业杀软	计算机\管理模板\Windows Defender	CIS 8.1	可以	不建议关闭
SMBv1	EnableSMB1Protocol	False	永恒之蓝/勒索	CRITICAL	每周	Set-SmbServerConfiguration -EnableSMB1Protocol $false	计算机\管理模板\网络\LAN 管理器	CIS 9.7	可以	影响老设备
Guest 账户	net user guest	激活=否	匿名入侵	HIGH	每月	net user guest /active:no	计算机\本地策略\用户权限分配	CIS 4.9	可以	域环境影响大
密码复杂度	net accounts	最短>=8, 需复杂度	弱密码暴力破解	HIGH	每月	net accounts /minpwlen:12 /maxpwage:90	计算机\本地策略\安全选项	CIS 4.1	可以	用户抱怨
密码过期	net accounts	强制变更<=90天	永久密码泄露后长期有效	MEDIUM	每月	net accounts /maxpwage:90	域策略	CIS 4.2	可以	VPN 用户需注意
RDP 暴露	netstat :3389	非 LISTENING	远程桌面暴力破解	MEDIUM	每周	防火墙限制来源 IP	计算机\管理模板\Windows\终端服务	CIS 9.2	可以	影响远程工作

自动更新	wuauserv 状态	RUNNING	补丁不安装	HIGH	每周	net start wuauserv	计算机\管理模板\Windows\Windows Update	CIS 3.4	可以	域环境用 WSUS
共享文件夹	net share	非必要不共享	数据泄露/勒索	MEDIUM	每月	net share share_name /delete	计算机\本地策略\网络访问	CIS 9.6	部分可以	业务需共享
BitLocker	manage-bde -status	完全加密	硬盘丢失/数据泄露	CRITICAL	每月	manage-bde -on C:	计算机\管理模板\Windows 组件\BitLocker	CIS 2.2	需要 TPM	影响性能
自启动项	reg query Run	最少项目	恶意软件持久化	HIGH	每周	手动审查/Autoruns	计算机\管理模板\系统\登录	CIS 3.5	不自动	谨慎清理
补丁安装	wmic qfe get	最近有安装	已知漏洞利用	CRITICAL	每周	wuauclt /detectnow	Windows Update 组策略	CIS 3.4	可以	重启影响
管理员账户	net user administrator	改名/禁用	经典入侵目标	HIGH	每月	net user administrator /active:no	计算机\本地策略\安全选项	CIS 4.8	可以	域有默认
PATH 安全	echo %PATH%	不含 . 或相对路径	DLL 劫持	MEDIUM	每月	从 PATH 移除 .	组策略	CIS 1.7	可以	影响某些脚本
自动登录	AutoAdminLogon	0 (禁用)	绕过登录密码	CRITICAL	每月	reg add ... AutoAdminLogon 0	计算机\管理模板\Windows\系统\登录	CIS 1.5	可以	用户体验下降
PowerShell EP	Get-ExecutionPolicy	Restricted	执行恶意脚本	MEDIUM	每月	Set-ExecutionPolicy Restricted	计算机\管理模板\Windows PowerShell	CIS 2.1	可以	影响管理脚本
匿名 SMB	RestrictAnonymous	1 或 2	匿名枚举用户/共享	HIGH	每月	reg add ... RestrictAnonymous 1	计算机\本地策略\安全选项	CIS 9.8	可以	影响老工具

转储文件	MEMORY.DMP	无	包含敏感内存数据	MEDIUM	每月	del %SystemRoot%\MEMORY.DMP	任务计划	CIS 2.5	可以	排障需要时保留
驱动签名	driverquery /si	全 Signed	加载恶意驱动	HIGH	每季度	更新驱动或启用驱动签名强制	计算机\本地策略\安全选项	CIS 2.6	部分	影响硬件兼容性
Administrators 组	net localgroup administrators	<=5 成员	权限过大横向	HIGH	每月	net localgroup administrators del user	计算机\本地策略\受限组	CIS 4.4	部分	小心误删
WinRM 暴露	sc query WinRM	Stopped (按需)	远程管理攻击面	MEDIUM	每月	Disable-PSRemoting	计算机\管理模板\Windows Remoting	CIS 9.4	可以	影响运维效率
SMTP 匿名	Exchange/本地 SMTP	禁止匿名	垃圾邮件中继	MEDIUM	每月	Exchange 中心管理	IIS SMTP 设置	CIS 5.1	可以	影响外发
蓝屏自动重启	SystemFailure	0 等待手动	无法分析蓝屏	LOW	半年	sysdm.cpl 高级\启动和故障恢复	组策略	无	可以	服务器建议改
屏幕保护密码	ScreenSaverIsSecure	1	无人值守访问	MEDIUM	每月	组策略强制启用	用户配置\管理模板\控制面板\个性化	CIS 4.5	可以	移动端有 Pin

安全基线检查脚本必须做成"只读"——只能检测不能自动修复（除非 /auto 参数），因为修复操作可能影响业务。比如关闭 SMB v1 会导致老打印机无法使用、强制 BitLocker 加密需要物理接触输入恢复密钥、禁用 Guest 账户会影响某些老旧软件。检测脚本的职责是"发现问题"，修复需要人工评估。

CSV 报告是检查脚本最有价值的输出——可以定期跑、对比变化、做合规审计。建议每月跑一次脚本，把报告归档到安全文件夹，年底能作为 CIS 等保审计的直接证据。BAT 用 echo "字段1","字段2" > file.csv 写 CSV 很简单，Excel 打开也不会乱码（全用英文表头）。

安全评分计算建议用加权——CRITICAL 级 FAIL 扣 10 分、HIGH 扣 5 分、MEDIUM 扣 3 分，PASS 加 2 分。这样 SMBv1 关闭（CRITICAL）和普通自启动项告警（MEDIUM）对评分的影响不一样。脚本里用简单的 pass*5 计算，如果要加权可以改成 FAIL_CRITICAL_COUNT * 10 + FAIL_HIGH * 5 这种形式。

附录A Windows 批处理核心命令速查表

本附录按功能分类，汇总 BAT 批处理中最常用的命令。每个命令附带基本语法和简短说明。

A.1 流程控制

命令	说明	示例
`call`	调用外部 BAT 或内部子程序（:label）	call other.bat / call :sub
`goto`	跳转到指定标签	goto :error
`if`	条件判断（4种模式）	if errorlevel 1 / if exist file
`for`	循环（/D /R /L /F 四种模式）	for /L %%i in (1,1,10) do
`exit`	退出脚本	exit /b [code]
`pause`	暂停等待按键	pause

A.2 变量与环境

命令	说明	示例
`set`	定义/查看/删除变量	set "var=value" / set var=
`setlocal`	创建局部变量作用域	setlocal enabledelayedexpansion
`endlocal`	结束局部作用域	endlocal
`shift`	移动 %1~%9 参数	shift /n
`echo`	输出文本/控制回显	@echo off / echo text

A.3 文件操作

命令	说明	示例
`copy`	复制文件	copy file1 file2
`xcopy`	高级复制（支持目录树）	xcopy /E /I src dest

`robocopy`	企业级文件复制/同步	robocopy src dest /MIR
`move`	移动/重命名	move old new
`del`	删除文件	del /f /q file
`ren`	重命名	ren old new
`type`	显示文件内容	type file.txt
`fc`	文件比较	fc /i file1 file2
`more`	分页显示	more +10 file

A.4 目录操作

命令	说明	示例
`cd`	切换目录	cd /d D:\path
`md`	创建目录	md path\to\dir
`rd`	删除目录	rd /s /q dir
`dir`	列出目录内容	dir /s /b *.txt
`tree`	目录树可视化	tree /f
`pushd`	压入目录栈并跳转	pushd \\NAS\Share
`popd`	弹出目录栈	popd

A.5 系统管理

命令	说明	示例
`tasklist`	列出进程	tasklist /fi "pid eq 1234"
`taskkill`	结束进程	taskkill /f /im app.exe
`start`	启动程序	start /wait "" app.exe
`sc`	服务控制	sc create / query / start
`schtasks`	计划任务	schtasks /create /tn "Name" /tr script
`reg`	注册表操作	reg add / query / delete
`wmic`	WMI 系统查询	wmic os get /format:list
`systeminfo`	系统信息快照	systeminfo
`net`	网络/用户/共享	net user / net share / net use

`netsh`	网络配置	netsh interface ip set address

A.6 网络工具

命令	说明	示例
`ping`	连通性测试	ping -n 10 -t host
`tracert`	路由追踪	tracert -d host
`ipconfig`	IP 配置	ipconfig /all /flushdns
`netstat`	网络连接	netstat -ano -p tcp
`findstr`	正则文本搜索	findstr /i "error warn" file
`find`	简单文本搜索	find /c "text" file
`sort`	排序	sort /r file
`bitsadmin`	后台下载	bitsadmin /transfer name url dest
`certutil`	证书工具（隐式下载）	certutil -urlcache -split -f url

A.7 批处理专用

命令	说明	示例
`rem`	注释	rem comment
`::`	注释（hack写法）	:: comment
`errorlevel`	上一条命令退出码	if errorlevel 1
`%var%`	预处理阶段变量展开	%computername%
`!var!`	执行阶段变量展开（需启用）	!counter!
`%~dp0`	脚本自身目录	%~dp0data
`%~nx1`	参数文件名+后缀	%%~nx1

附录B cmd.exe 环境变量大全

B.1 cmd 内置特殊变量

变量	含义	类型
`%cd%`	当前工作目录	动态
`%date%`	当前日期	动态（格式依赖系统设置）
`%time%`	当前时间	动态（格式依赖系统设置）
`%random%`	0~32767 随机数	动态
`%errorlevel%`	上一条命令退出码	动态
`%cmdextversion%`	命令扩展版本号	常量（值=2）
`%_cmdvarset%`	命令行参数（shift后变化）	动态
`%^e%`	ESC 转义符	动态（cmd 1903+）
`%=C:%`	C盘当前目录	动态

B.2 Windows 系统环境变量

变量	典型值	含义
`%SystemRoot% / %windir%`	C:\Windows	Windows 安装目录
`%SystemDrive%`	C:	系统驱动器号
`%System%`	C:\Windows\System32	系统目录
`%ProgramFiles%`	C:\Program Files	64位程序目录
`%ProgramFiles(x86)%`	C:\Program Files (x86)	32位程序目录
`%USERPROFILE%`	C:\Users\Harry	当前用户主目录
`%APPDATA%`	C:\Users\Harry\AppData\Roaming	用户应用数据
`%LOCALAPPDATA%`	C:\Users\Harry\AppData\Local	本地应用数据
`%TEMP% / %TMP%`	C:\Users\Harry\AppData\Local\Temp	临时文件目录
`%COMPUTERNAME%`	DESKTOP-HARRY	计算机名
`%USERNAME%`	Harry	当前用户名
`%NUMBER_OF_PROCESSORS%`	8	CPU 逻辑核心数

%OS%	Windows_NT	操作系统类型（固定值）
%PATH%	C:\Windows\System32;...	命令搜索路径
%PATHEXT%	.COM;.EXE;.BAT;.CMD;.VBS;...	可执行文件后缀列表

B.3 变量优先级

从高到低：

1. 当前 cmd 会话中 set 定义的变量（最高）
2. 用户环境变量（注册表 HKCU\Environment）
3. 系统环境变量（注册表 HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Environment）
4. cmd 内置变量（%cd%、%time% 等总是动态的，无法覆盖）

附录C 批处理常见问题与疑难解答

Q1: BAT 脚本里中文乱码怎么办?

确保文件保存为 UTF-8 with BOM，脚本开头加 `chcp 65001 > nul`。在 VS Code 右下角选择 "UTF-8 with BOM" 保存编码。如果是老系统（Win7），改用 GBK 编码 + chcp 936。

Q2: for 循环里变量值不变怎么办?

需要启用变量延迟扩展：脚本开头加 `setlocal enabledelayedexpansion`，然后在循环内用 `!var!` 代替 `%var%`。

Q3: 路径里有空格导致命令执行失败?

所有含空格的路径必须用**双引号包裹**：`copy "C:\Program Files\a.txt" "D:\target"`。此外 cd 跨盘符时必须加 `/d`：`cd /d "D:\My Folder"`。

Q4: echo 输出空行写 echo. 还是 echo: 还是 echo(?

这三种都可以输出空行。推荐用 `echo.` 或 `echo(`——它们比 echo: 更可靠。注意 echo. 如果当前目录下有个叫 "echo" 的文件会失败，但这种情况极其罕见。

Q5: if 里用 %errorlevel% 还是 errorlevel?

两种都可以但有区别：**if errorlevel 1** 检查退出码 >= 1；**if %errorlevel%==1** 检查等于 1。注意 %errorlevel% 会在预处理阶段展开，在括号块内可能得到旧值——这种情况要用 if !errorlevel!==1 或直接用 errorlevel N。

Q6: 怎么在 BAT 里获取管理员权限?

方案1：`runas /user:Administrator "cmd /c %~f0"`；方案2：用 VBScript ShellExecute 以 runas 方式启动 cmd；方案3：创建高权限的计划任务然后立即运行。详见第 11.3 节。

Q7: BAT 能 sleep 吗?

BAT 没有原生 sleep 命令，但有多种替代：`timeout /t 5 /nobreak`（推荐）、`ping -n 6 127.0.0.1 > nul`（通用但不精确）、或调用 PowerShell 的 Start-Sleep。

Q8: 怎么让 BAT 窗口关闭前停留一会儿?

脚本最后加 `pause`。如果用 exit 关闭整个 cmd，可以在 BAT 开头加：`if /i "%1"=="_pause" goto :main & cmd /k "%~f0" _pause & exit`。

Q9: 如何获取脚本自身所在目录?

用 `%~dp0`（注意末尾有反斜杠）。推荐保存到变量：`set "SCRIPT_DIR=%~dp0"`，之后用 `%SCRIPT_DIR%data\file.txt` 拼接路径。

Q10: 怎么把 BAT 变成 EXE?

三种主流方案：WinRAR SFX 自解压、IExpress（Windows 自带）、Bat To Exe Converter 等第三方工具。第 14.2 节提供了完整方案对比和一个自写的 C# 编译方案。

Q11: for /f 的 delims 和 tokens 怎么配合?

delims 指定用什么字符分隔行（默认空格和 Tab），tokens 指定取第几段（从 1 开始）。tokens=1,2,3 会把第 1/2/3 段分别赋给 %%a、%%b、%%c；tokens=1-3 效果相同但更短。tokens=* 表示整行赋给第一个变量。详见第 5.5 节。

Q12: 怎么判断文件是否被占用（正在使用）?

没有直接命令。变通方案：尝试用 >> 追加写入一行（echo test>> file），成功则没被占用，失败（errorlevel 1）则被占用。或者用 handle.exe（Sysinternals 工具）查找锁定文件的进程。

Q13: BAT 脚本之间怎么互相传参?

调用方：`call sub.bat "arg1" "arg2"`；被调用方用 `%1 %2 %*` 接收。如果需要传递大量变量，用 setlocal/endlocal + 共享文件 或 管道（但管道会丢失变量，不推荐）。

Q14: 怎么检测自己是否以管理员身份运行?

方案1：尝试写入 %windir%（失败则不是管理员）；方案2：查询管理员组 SID（S-1-5-32-544）；方案3：最简单——执行一个需要管理员权限的命令看 errorlevel。

Q15: 如何输出带颜色的文字?

Windows 10 1903+ 支持 ANSI 转义序列，详见第 11.1 节。老系统只能通过 VBScript（cscript 只能输出黑白）或 PowerShell 的 Write-Host -ForegroundColor。

Q16: rem 和 :: 有什么区别?

rem 是正规的命令；:: 是 hack（把 :: 当作无效标签）。在括号块、for 的 () 内部，:: 可能导致解析错误——此时必须用 rem。稳妥起见，一律用 rem。

Q17: BAT 怎么操作注册表?

用 reg 命令：`reg add / query / delete / export / import`。操作 HKLM 分支需要管理员权限。详见第 9.4 节。

Q18: 怎么从 BAT 里下载文件?

推荐用 bitsadmin（企业级、支持断点续传）；或 PowerShell Invoke-WebRequest；不推荐 certutil（隐蔽但可能被杀软拦截）。详见第 10.3 节。

Q19: 批处理文件最大多大?

技术上没有硬性限制。实际中 1-2MB 完全没问题。但文件越大，cmd 解析预扫描阶段开销越大。建议超过 5000 行时考虑用 PowerShell 或 Python 重写。

Q20: 为什么我的 BAT 运行一闪而过?

通常有三个原因：① 路径或文件名中有特殊字符导致 cmd 找不到文件；② 脚本执行出错后没有 pause；③ exit 关闭了 cmd 窗口。调试方法：在 cmd 里手动运行（cmd /k "call myscript.bat"），或在脚本末尾加 pause。

附录D BAT 开发工具与资源

D.1 编辑器推荐

编辑器	适用平台	BAT 支持	说明
VS Code	全平台	插件	免费、插件丰富、Git 集成好。安装 "Batch Script Support"
Notepad++	Windows	内建	经典轻量编辑器，自带 BAT 语法高亮
Sublime Text	全平台	插件	优雅的商业编辑器，需装 Batch Script 插件
TextPad	Windows	内建	老牌 Windows 文本编辑器
Notepad (Win11)	Windows 11	内建	新版记事本支持多标签、UTF-8 BOM

D.2 BAT 调试工具

- **echo on**：在脚本开头加 echo on 显示每一条执行的命令
- **pause**：在关键位置暂停看变量值
- **cmd /k "call script.bat"**：让窗口不关闭
- **Process Monitor**（Sysinternals）：监控文件、注册表、网络操作
- **Process Explorer**（Sysinternals）：查看进程详情、句柄、DLL
- **PES Explorer**：查看 PE 文件（.exe/.dll）结构

D.3 BAT 转 EXE 工具

工具	类型	说明
WinRAR SFX	商业软件	最通用的方案，不需要额外工具
IExpress	Windows 自带	在 System32 目录里，完全免费
Bat To Exe Converter	免费软件	bat2exe.net，支持加密、压缩、图标替换

Advanced BAT to EXE Converter	商业软件	功能最全，支持多种加密方式
Bat2Exe Pro	免费	开源项目，支持自定义图标
C# 自编译方案	DIY	见第 14.2 节

D.4 在线学习资源

- **Microsoft Learn - Windows Commands**（官方）：learn.microsoft.com/windows-server/administration/windows-commands
- **Rob van der Woude 的批处理页面**（权威）：robvanderwoude.com/batchfiles.php
- **Dostips**（入门）：dostips.com/commandlinescripts
- **Tutorialspoint Batch Script**：tutorialspoint.com/batch_script
- **SS64**（完整命令参考）：ss64.com/nt
- **批处理之家（中文社区）**：bbs.bathome.net

附录E 示例代码全集索引

本书所有章节中出现的可运行代码示例，以下按章节索引。所有代码均已在 Windows 10/11 环境下测试通过。

章节	脚本名	功能简述
2.7	`sysinfo.bat`	系统信息查询工具（第一个完整脚本）
4.5	`calc_demo.bat`	set /a 数值计算演示
4.6	`str_ops.bat`	字符串替换/截取/长度计算
5.3	`menu.bat`	goto + call 菜单程序
5.5	`for_f_demo.bat`	for /F 解析文件和命令输出
7.2	`copy_demo.bat`	copy /xcopy /robocopy 对比
7.4	`robocopy_sync.bat`	robocopy 镜像同步
7.8	`daily_backup.bat`	每日自动备份脚本
8.6	`project_structure.bat`	批量创建项目目录结构

9.9	`autorun_setup.bat`	三种开机自启配置方案
10.5	`ping_batch.bat`	批量 Ping 检测 + CSV 导出
11.1	`ansi_colors.bat`	ANSI 彩色输出演示
11.3	`uac_elevate.bat`	UAC 提权三种方案
11.6	`hta_gui.bat`	BAT 生成 HTA GUI 界面
12.1	`system_cleanup.bat`	系统垃圾清理工具
12.2	`startup_manager.bat`	启动项管理工具
12.3	`batch_service.bat`	服务批量运维工具
12.5	`sys_snapshot.bat`	系统快照一键收集
13.2	`remote_exec.bat`	批量远程执行（PsExec）
13.3	`batch_deploy.bat`	批量文件分发
13.4	`net_monitor.bat`	网络端口监控脚本
13.5	`wifi_export.bat`	WiFi 密码批量导出
14.1	`aes_encrypt.bat`	PowerShell AES 文件加密
14.2	`bat2exe.bat`	BAT 转 EXE（自写 C# 方案）
14.3	`auto_type.bat`	VBScript 键盘自动输入
14.4	`secure_download.bat`	安全下载-校验-执行流程
14.5	`admin_hta.bat`	完整的 HTA 系统管理 GUI 工具

运行环境要求：Windows 7 SP1+（推荐 Windows 10/11），部分高级功能需要 Windows 10 1903+。

反馈渠道：欢迎在各大 BAT 论坛（批处理之家 bathome.net、GitHub Issues 等）反馈问题。

版权声明：本书为原创教程，代码示例可自由使用、修改、分发。禁止将本书内容直接用于商业出版物。

感谢阅读《Windows BAT 批处理完全教程》！

2026